

Visualization 101 2026

Data Visualization In R

Boris Shor

2026-01-01

Table of contents

1	Visualization 101 2026	8
1.1	Schedule	8
1.2	Setup	9
1.3	Files	9
2	Start Here: Workflow, Quarto, And RStudio	10
2.1	Learning Goals	10
2.2	The Course Workflow	10
2.3	Plain R Scripts	10
2.4	The Console	11
2.5	R, RStudio, Posit Cloud, And Quarto	11
2.6	Project Files And Portable Paths	12
2.7	The RStudio Layout	12
2.8	Source Mode And Visual Mode	12
2.9	Markdown Basics	13
2.10	Code Chunks	13
2.11	Chunk Names And Navigation	14
2.12	Chunk Options	14
2.13	Rendering	15
2.14	A First Reproducible Example	15
2.15	Reading Output Carefully	16
2.16	Short Exercise	16
2.17	Extras: RStudio Settings	17
2.18	Extras: Output Formats	17
	2.18.1 Presentation Demo	18
	2.18.2 Paper Or Report Demo	18
2.19	What Comes Next	19
3	Data Import And The Tidyverse	20
3.1	Learning Goals	20
3.2	Load Packages	20
3.3	Importing Local Data	20
3.4	Tibbles And First Inspection	21
3.5	Base R Import And Inspection	23

3.6	Other Local File Types	23
3.6.1	Excel	23
3.6.2	Stata	23
3.6.3	SPSS	24
3.6.4	SAS	24
3.7	Wide And Long Data	24
3.8	Core Tidyverse Verbs	25
3.9	<code>filter()</code> : Keep Rows	26
3.10	<code>select()</code> : Keep Columns	27
3.11	<code>mutate()</code> : Create Variables	27
3.12	<code>group_by()</code> And <code>summarize()</code> : Aggregate Data	28
3.13	Readable Pipelines	29
3.14	Joins	30
3.15	Short Exercise	30
3.16	Extras: Google Sheets	31
3.17	Extras: Census, GSS, And State Policy Data	31
3.18	Extras: Qualtrics And Canvas	34
3.19	What Comes Next	34
4	Core ggplot2	35
4.1	Learning Goals	35
4.2	Load Packages	35
4.3	Data For This Chapter	35
4.4	Comparing Base R Graphics And <code>ggplot2</code>	36
4.5	The Grammar Of Graphics	37
4.6	The Base Layer: <code>ggplot()</code> And <code>aes()</code>	38
4.7	Add A Geom Layer	38
4.8	Saving A Plot Object	39
4.9	Mapping Versus Setting	40
4.10	Smoothers	43
4.11	Log Scales	45
4.12	Labels With <code>labs()</code>	46
4.13	Themes	47
4.14	Color Palettes	50
4.15	Putting It All Together	54
4.16	Short Exercise	55
4.17	Extras: Continuous Color	56
4.18	Extras: Shape And Size	57
4.19	Extras: External Themes And Rich Text	58
4.20	What Comes Next	61
5	Separating and Comparing Data	62
5.1	Why separate data?	62

5.2	Start with one plot	62
5.3	Color as separation	63
5.4	Facets as small multiples	65
5.5	Reordering facets	66
5.6	Free scales	67
5.7	Direct labels	68
5.8	<code>facet_grid()</code>	70
5.9	Patchwork	71
5.10	Extra: State Policy Snapshot	72
5.11	Exercise	77
6	Distributions	78
6.1	Why distributions matter	78
6.2	Histograms	78
6.3	Comparing histograms	81
6.4	Density plots	82
6.5	Boxplots	85
6.6	Ridge plots	87
6.7	Distributions in Gapminder	88
6.8	Choosing among histograms, densities, and boxplots	91
6.9	Exercise 1: histogram	91
6.10	Exercise 2: density plot	92
6.11	Exercise 3: boxplot	93
6.12	Extra: distribution intervals with <code>ggdist</code>	93
6.13	Extra: worked exercise variants	94
6.14	Extra: heatmaps for dense two-variable distributions	98
7	Lines, Bars, and Annotation	100
7.1	Line charts	100
7.2	Working with dates	103
7.3	Highlighting trends	106
7.4	Bar charts	110
7.5	<code>geom_col()</code> versus <code>geom_bar()</code>	113
7.6	Dodged bars	114
7.7	Bar chart example: OECD life expectancy gap	116
7.8	Exercise	117
7.9	Extra: slopegraphs	118
8	From Question to Final Figure	122
8.1	The workflow	122
8.2	Question	122
8.3	Inspect the data	123
8.4	First rough plot	124

8.5	Check one state first	125
8.6	Compare a small set of states	126
8.7	Try color on all states	127
8.8	Build gray context with labels	128
8.9	Highlight one state	129
8.10	Add points only where they help	131
8.11	Add a direct label	132
8.12	Compare Michigan to the average state	133
8.13	Final line figure	135
8.14	Exporting	137
8.15	Checklist for final figures	137
8.16	Exercise	138
8.17	Extra: summarize total change	138
8.18	Extra: alternative question	140
9	Interactivity And Dashboards	142
9.1	Learning Goals	142
9.2	Where This Fits	142
9.3	Setup	142
9.4	From Static To Interactive	143
9.5	Custom Hover Text	145
9.6	Interactive Time Series	147
9.7	Linked Highlighting	148
9.8	Toolbar Control	149
9.9	Highcharter	150
9.10	Highcharter Line Charts	152
9.11	Highcharter Range Selector	153
9.12	Highcharter Column Chart	155
9.13	Highcharter Heatmap	156
9.14	Dashboards As Output	158
9.14.1	Baby Names Dashboard	158
9.14.2	Cigarette Tax Dashboard	159
9.15	Exercise	159
9.16	Closing	159
10	Tables	160
10.1	A first frequency table	160
10.2	Cleaner crosstabs with <code>janitor</code>	161
10.3	Three-way tables	164
10.4	Publication-style data tables with <code>tinytable</code>	165
10.5	Summary tables with <code>modelsummary</code>	167
10.6	Colored correlation tables with <code>corrr</code>	169
10.7	Exporting tables	170

10.8	Short exercise	171
10.9	Extra: table package choices	171
10.10	Extra: interactive tables with DT	172
11	Visualizing Models	174
11.1	A first model: predicting price directly	174
11.2	OLS on log price	179
11.3	Coefficient plots	181
11.4	A manual coefficient plot	182
11.5	Tidy model output with broom	184
11.6	Prediction plots	185
11.7	Comparing nested models	188
11.8	Interactions	191
11.9	Logit: binary outcomes	194
11.10	Marginal and adjusted predictions	196
11.11	Slopes and marginal effects	197
11.12	Comparing plots	199
12	Mapping	200
12.1	A blank US map	200
12.2	State and county maps	201
12.3	Understanding map data	202
12.4	FIPS codes	203
12.5	A state choropleth	204
12.6	Focused regional maps	205
12.7	County choropleths	208
12.8	Joining your own data	209
12.9	Choosing map colors	211
12.10	Short exercise	212
12.11	Extra: ACS data with tidycensus	213
12.12	Extra: equal-area state tile maps with statebins	214
12.13	Extra: world maps with tmap	214
12.13.1	Zooming into a region	216
13	Agentic AI and Next Steps	218
13.1	What AI can help with	218
13.2	Web chatbots	219
13.3	IDE integration	219
13.4	RStudio-focused AI with gander	220
13.5	Programmatic AI with ellmer	221
13.6	Local and cloud models	221
13.7	Agentic coding tools	222
13.8	A safe AI workflow	223

13.9 How to ask better questions	223
13.10 Reviewing AI output	224
13.11 Privacy, attribution, and reproducibility	224
13.12 AI for visualization critique	224
13.13 AI for debugging Quarto projects	225
13.14 Next steps after the course	226
13.15 Short exercise	226
13.16 Final checklist	227

1 Visualization 101 2026

This book is the working text for a five-day course on data visualization in R. It is designed to be read, run, edited, rendered, and reused. The goal is not only to make attractive graphs, but to make visual choices that clarify an argument, reveal a pattern, and leave a reproducible record of how the result was made.

Visualization is a sequence of choices. A plot emphasizes some comparisons and pushes others into the background. It chooses what counts as the main unit, what deserves color or space, what should be labeled directly, what can remain implicit, and what should be excluded because it distracts from the point. The course therefore treats visualization as part of analysis rather than as decoration added at the end.

R is the statistical language used throughout the course. RStudio Desktop and Posit Cloud are the interfaces used to write and run code. The tidyverse is the central programming framework: it supplies a readable grammar for importing, reshaping, summarizing, and plotting data. `ggplot2`, the tidyverse visualization package, is the main plotting system.

Quarto is the document system used for the course files. A Quarto document can contain prose, code, output, figures, tables, citations, and links in one place. The same source document can produce HTML, PDF, slides, dashboards, and reports. When the data or code changes, the outputs can be regenerated from the source instead of manually updated across Word documents, spreadsheets, and slide decks.

Literate programming also supports replication. Code and interpretation live together, which makes it easier to return to a project after a long break, share work with collaborators, and identify exactly how a result was produced. Writing the explanation forces decisions to be made explicit while they are still fresh.

1.1 Schedule

Day	Main Focus
1	Workflow, Quarto, data import, and core <code>ggplot2</code>
2	Tidy data, wrangling, themes, scales, and visual separation

Day	Main Focus
3	Distributions, lines, bars, annotation, and applied workflow
4	Applied workflow, interactivity, dashboards, tables, and output
5	Model visualization, mapping, and a two-session block on agentic AI and wrap-up

Each day has five instructional sessions. The written chapters contain more material than the minimum live path so that faster pacing, review, and post-course study are supported.

1.2 Setup

The Posit Cloud project should already include the packages needed for the main course. The file `install_packages.R` automates installation if the materials are moved to a local computer. The file `install_extra_packages.R` installs optional packages used only in extra sections.

Run `check_setup.R` to verify that the main packages and data files are available.

1.3 Files

The main book chapters are the numbered `.qmd` files in this project. Data files are under `Data/`. Standalone examples are under `Demos/`. Short exercises and solutions are under `Exercises/`.

The course examples use files included with the project, such as `Data/gapminder/gapminder.csv`.

2 Start Here: Workflow, Quarto, And RStudio

2.1 Learning Goals

By the end of this chapter, the main goals are:

- open and navigate the course project
- understand the relationship among R, RStudio, Posit Cloud, and Quarto
- run code chunks inside a `.qmd` file
- render a Quarto document to HTML
- use source mode, visual mode, and the document outline intentionally

2.2 The Course Workflow

This course uses R for computation, RStudio for the working interface, and Quarto for the documents that combine prose, code, and output. The goal is not only to make individual plots. The goal is to build a reproducible workflow where the text explains the question, the code performs the work, and the output records what happened.

During class, most work will happen inside `.qmd` files like this one. A Quarto file is a plain-text document that can contain headings, paragraphs, lists, links, code chunks, tables, figures, and notes about interpretation. When Quarto renders the file, it runs the code chunks and combines the results with the surrounding text.

This is different from working only in the console. The console is useful for quick tests, but it does not automatically preserve the reasoning around the command. A Quarto document keeps the analysis in order: what question was being asked, what data were used, what code was run, what output appeared, and what conclusion followed.

2.3 Plain R Scripts

The simplest kind of R source file is a plain R script. A plain R script uses the `.R` extension and contains only R code. It is useful for setup tasks, repeated checks, helper functions, and other code that does not need surrounding prose or rendered output.

This project includes a few plain R scripts:

```
install_packages.R  
install_extra_packages.R  
check_setup.R
```

The file `check_setup.R` is a good example. It checks whether the main packages and course data files are available. Opening that file and clicking **Source** runs the whole script. The result is a quick setup check, not a rendered document.

Use `.R` scripts when the goal is to run code directly. Use `.qmd` files when the goal is to combine explanation, code, and output.

2.4 The Console

The console is the fastest place to test a command, inspect an object, or rerun something without rendering a document.

Two console habits are especially helpful:

- the up arrow cycles through recent commands
- `Ctrl + R` searches command history in many RStudio setups

The console is not a replacement for a source file. If a command becomes part of the analysis, move it into a `.qmd` or `.R` file so it is preserved.

2.5 R, RStudio, Posit Cloud, And Quarto

These tools have different jobs:

- R is the programming language that reads data, transforms it, estimates models, and makes graphics.
- RStudio is the interface where you edit files, run code, inspect objects, view plots, and render documents.
- Posit Cloud is a browser-based version of the RStudio workflow that avoids many local installation problems.
- Quarto is the publishing system that turns `.qmd` source files into HTML, PDF, Word, slides, dashboards, and books.

R Markdown was the predecessor used in earlier versions of this course. Quarto keeps the same basic idea, but it is now the preferred format for this version of the workshop. If you have used `.Rmd` files before, the transition should feel familiar: a `.qmd` file still mixes text and code. The main difference is that Quarto uses a more general and consistent document system.

2.6 Project Files And Portable Paths

The examples in this course assume that the project has been opened in RStudio or Posit Cloud. Files are organized inside the project in folders such as `Data/`, `Demos/`, and `Exercises/`.

The data examples use file names like these:

```
Data/gapminder/gapminder.csv
Data/penguins/penguins.csv
Demos/presentation-demo.qmd
Exercises/04-scatterplot-solution.R
```

Avoid file names that depend on a personal Desktop, Downloads folder, or another location on one person's computer.

Keeping files inside the project matters because the course should work in Posit Cloud, on another computer, or after being archived for later use.

2.7 The RStudio Layout

RStudio usually has four main panes:

- the source pane, where `.qmd` and `.R` files are edited
- the console, where commands can be typed and run directly
- the environment and history pane, where current objects and previous commands are listed
- the files, plots, packages, help, and viewer pane

The exact pane arrangement can be changed under Global Options. The important habit is to know which pane is doing which job. The source pane is for durable work. The console is for quick tests. The environment pane shows what objects currently exist. The viewer pane is often where rendered HTML output appears.

2.8 Source Mode And Visual Mode

RStudio can edit Quarto documents in source mode or visual mode.

Source mode shows the underlying Markdown and code exactly as written. It is the most transparent mode for learning because it makes headings, chunks, chunk options, links, and lists visible.

Visual mode is a more word-processor-like interface. It can be helpful for editing prose, tables, footnotes, and links. It is still editing the same `.qmd` file, but it hides some of the raw syntax while you write.

Both modes are useful. During this course, source mode is usually the clearest mode for understanding how the document works. Visual mode can be helpful later when revising narrative text.

2.9 Markdown Basics

Quarto uses Markdown for ordinary text formatting. A few patterns cover most needs:

```
# First-level heading

## Second-level heading

Regular paragraph text.

- a bullet point
- another bullet point

**bold text**

*italic text*

[Quarto website] (https://quarto.org)
```

The number of `#` symbols controls the heading level. Headings also create the document outline, which makes long lessons easier to navigate.

2.10 Code Chunks

An R code chunk starts with three backticks and `{r}`. A useful chunk also has a short name after `r`, then ends with three backticks.

In the source file, the opening line would be ````{r ch02-code-chunks-1}` and the closing line would be `````. The R code between those lines could be:

```
1 + 1
```

The code inside the chunk can be run interactively, and it can also be run during rendering. In RStudio, you can run a chunk with the green play button or with keyboard shortcuts. You can insert a new chunk from the menu or by typing the chunk delimiters yourself.

If that chunk runs, the result would be:

```
[1] 2
```

Chunks can contain more than one line. For example:

```
x <- 10
y <- 25
x + y
```

The assignment operator `<-` stores a value in an object. In the example above, `x` stores 10 and `y` stores 25. After those objects exist, R can use them in later commands.

2.11 Chunk Names And Navigation

Most real code chunks in these course files have a name. The name appears in the opening line of the chunk. For example, this opening line names a chunk `ch02-first-calculation`:

```
# opening line in the source file:
# ```{r ch02-first-calculation}
```

Chunk names make long documents easier to navigate. In RStudio, the document outline can show both section headings and code chunks, so a descriptive chunk name makes it easier to jump to a specific part of the file. Chunk names also make error messages easier to interpret because Quarto can report which chunk failed.

Chunk names should be short, unique, and descriptive. A name such as `ch04-first-scatterplot` is more useful than a name such as `chunk1`.

2.12 Chunk Options

Chunk options control how code behaves during rendering. In Quarto, options are often written as special comment lines at the top of the chunk.

This chunk runs but hides the code:

```
[1] "2026-05-15"
```

This chunk displays code without running it:

```
Sys.Date()
```

`eval: false` is important for examples that require credentials, internet access, private files, or intentional manual action. The code remains visible as a template, but it does not run automatically.

2.13 Rendering

Rendering turns a source `.qmd` file into an output document. For this course, HTML is the primary output because it is easy to view in a browser and works well for code, figures, tables, and links.

When you render a document, Quarto starts from the top, runs the chunks in order, and inserts the output where it belongs. If a chunk fails, rendering stops. That is useful because it exposes broken code early.

The render button in RStudio is the easiest way to start. Open the file and click **Render** to create the output document. PDF output is also useful for papers, handouts, and print-ready reports, but it depends on a working TeX installation. HTML will be the most reliable output during the workshop.

2.14 A First Reproducible Example

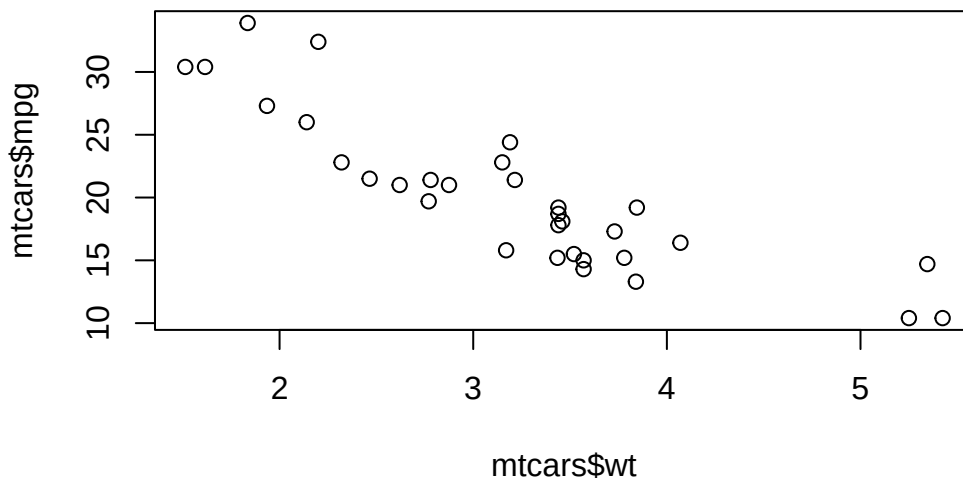
The built-in `mtcars` dataset is available in base R, so it is useful for a first test that does not depend on course data files.

```
head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

This command creates a simple base R plot:

```
plot(mtcars$wt, mtcars$mpg)
```



The plot is not polished, but it confirms that R is running, code chunks are executing, and graphical output appears in the document.

2.15 Reading Output Carefully

A rendered document is not automatically correct just because it renders. Rendering answers one question: did the code run from top to bottom? You still have to ask whether the data are the right data, whether the variables mean what you think they mean, whether missing values were handled correctly, and whether the plot supports the claim being made.

This course will return to that distinction throughout the week. Reproducibility is necessary, but it is not enough. A reproducible mistake is still a mistake.

2.16 Short Exercise

Create a new code chunk below this paragraph. In that chunk:

1. store the number 100 in an object named `start_value`
2. store the number 25 in an object named `change_value`
3. subtract `change_value` from `start_value`

```
# Write your code here.
```

2.17 Extras: RStudio Settings

A few RStudio settings can make workshop work more comfortable:

- turn off restoring `.RData` into the workspace at startup
- set saving the workspace on exit to “Never”
- turn on rainbow parentheses if available
- use the document outline for long `.qmd` files
- choose a source editor font and size that are comfortable for several hours of reading

These settings do not change the analysis, but they reduce friction. The main principle is that the project files should define the work, not a hidden workspace image from a previous session.

2.18 Extras: Output Formats

Quarto can produce many output formats from similar source files:

- HTML documents and books
- PDF reports
- Word documents
- RevealJS slides
- dashboards
- websites

The same analysis can therefore be adapted for different audiences. A rough exploration may become a class exercise. A refined figure may become part of a report. A report may become a short presentation. Quarto is useful because those outputs can remain connected to the same source code and data workflow.

2.18.1 Presentation Demo

Quarto can render slides. A presentation file uses a different YAML header:

```
---
title: "Presentation Demo"
format:
  revealjs:
    theme: simple
---
```

The course includes a standalone revealjs demo at `Demos/presentation-demo.qmd`. The main difference from a book chapter is that second-level headings become slides. Code chunks work the same way, but slides usually hide more code and show more finished output.

```
## Slide Title

Slide text goes here.
```

2.18.2 Paper Or Report Demo

A paper-style Quarto document emphasizes prose, citations, figures, tables, and cross-references. The course includes `Demos/paper-example.qmd` and `Demos/paper-example.bib`. The `.bib` file stores citation records; the `.qmd` file cites them with keys such as `[@wickham_ggplot2_2016]`.

A minimal paper-style header looks like this:

```
---
title: "Paper Demo"
author: "Name"
bibliography: paper-example.bib
format:
  html: default
  pdf: default
---
```

Paper-style files usually keep more prose visible than slides or dashboards. They also make more use of citations, captions, footnotes, and cross-references.

2.19 What Comes Next

The next chapter begins the data workflow: importing files, inspecting tibbles, reshaping data, and using the core tidyverse verbs that prepare data for visualization.

3 Data Import And The Tidyverse

3.1 Learning Goals

By the end of this chapter, the main goals are:

- read local data included with the project
- recognize the difference between a data frame and a tibble
- inspect a dataset before plotting it
- distinguish wide data from long data
- reshape data with `pivot_longer()` and `pivot_wider()`
- use the core tidyverse verbs for common data preparation tasks
- recognize which import examples should not run automatically because they require credentials or external services

3.2 Load Packages

The **tidyverse** is a collection of packages for importing, transforming, and visualizing data. Loading it makes packages such as **readr**, **dplyr**, **tidyr**, **ggplot2**, **stringr**, and **tibble** available.

```
library(tidyverse)
library(skimr)
```

3.3 Importing Local Data

Many projects begin with a local CSV file. CSV files are plain-text tables where columns are separated by commas. They are common because nearly every spreadsheet, statistical package, and database can export them.

The preferred tidyverse function is `read_csv()` from the **readr** package. Compared with base R's `read.csv()`, it usually prints more clearly, guesses column types more consistently, and returns a tibble.

The course data live under `Data/`. The example below reads the Gapminder CSV file included with the project.

```
gap <- read_csv("Data/gapminder/gapminder.csv", show_col_types = FALSE)
```

```
gap
```

```
# A tibble: 1,704 x 6
```

	country	continent	year	lifeExp	pop	gdpPercap
	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	Afghanistan	Asia	1952	28.8	8425333	779.
2	Afghanistan	Asia	1957	30.3	9240934	821.
3	Afghanistan	Asia	1962	32.0	10267083	853.
4	Afghanistan	Asia	1967	34.0	11537966	836.
5	Afghanistan	Asia	1972	36.1	13079460	740.
6	Afghanistan	Asia	1977	38.4	14880372	786.
7	Afghanistan	Asia	1982	39.9	12881816	978.
8	Afghanistan	Asia	1987	40.8	13867957	852.
9	Afghanistan	Asia	1992	41.7	16317921	649.
10	Afghanistan	Asia	1997	41.8	22227415	635.

```
# i 1,694 more rows
```

The object name `gap` now points to a table in memory. The assignment operator `<-` reads as “store the result on the right in the object named on the left.” Object names should be short enough to type but descriptive enough to understand later.

3.4 Tibbles And First Inspection

A tibble is the tidyverse version of a data frame. It is still a rectangular table with rows and columns, but it prints in a more controlled way. A tibble tells you the number of rows and columns, shows variable types, and avoids flooding the console with every row of a large dataset.

Start every new dataset with a quick inspection:

```
glimpse(gap)
```

```
Rows: 1,704
```

```
Columns: 6
```

```
$ country   <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", ~
```

```
$ continent <chr> "Asia", "Asia", "Asia", "Asia", "Asia", "Asia", "Asia", "Asia"
$ year <dbl> 1952, 1957, 1962, 1967, 1972, 1977, 1982, 1987, 1992, 1997, ~
$ lifeExp <dbl> 28.801, 30.332, 31.997, 34.020, 36.088, 38.438, 39.854, 40.8~
$ pop <dbl> 8425333, 9240934, 10267083, 11537966, 13079460, 14880372, 12~
$ gdpPercap <dbl> 779.4453, 820.8530, 853.1007, 836.1971, 739.9811, 786.1134, ~
```

```
skim(gap)
```

Table 3.1: Data summary

Name	gap
Number of rows	1704
Number of columns	6
Column type frequency:	
character	2
numeric	4
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
country	0	1	4	24	0	142	0
continent	0	1	4	8	0	5	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
year	0	1	1979.50	17.27	1952.00	1965.75	1979.50	1993.25	2007.0	
lifeExp	0	1	59.47	12.92	23.60	48.20	60.71	70.85	82.6	
pop	0	1	29601212	33615789	60011.00	2793664.00	23595.50	585221.75	18683096.0	
gdpPer-cap	0	1	7215.33	9857.45	241.17	1202.06	3531.85	9325.46	113523.1	

`glimpse()` comes from `dplyr`, which is loaded as part of the tidyverse. It gives a compact structural view of the data. `skim()` comes from the `skimr` package and gives a broader summary, including missingness and descriptive statistics. These checks are not just housekeeping. They help identify the unit of observation, variable types, unexpected missing values, and suspicious ranges before those problems enter a plot.

3.5 Base R Import And Inspection

Base R can also import data. The syntax is common in older examples and in some online answers:

```
gap_base <- read.csv("Data/gapminder/gapminder.csv")
str(gap_base)
head(gap_base)
```

3.6 Other Local File Types

CSV is only one possible format. Social science projects often use Excel, Stata, SPSS, or SAS files. The syntax below should be treated as a template. These chunks are non-executing unless the named files are present and the relevant packages are installed.

3.6.1 Excel

The function `read_excel()` comes from the `readxl` package.

```
library(readxl)

gap_excel <- read_excel(
  path = "Data/gapminder/gapminder.xlsx",
  sheet = "gapminder"
)

gap_excel
```

3.6.2 Stata

The functions for Stata, SPSS, and SAS files below come from the `haven` package.

```
library(haven)

state_legislatures <- read_dta("Data/state_policy/shor_mccarty_state_data.dta")

state_legislatures
```

3.6.3 SPSS

The example uses a small SPSS file that is installed with the **haven** package.

```
library(haven)

path <- system.file("examples", "iris.sav", package = "haven")
survey_data <- read_sav(path)

survey_data
```

3.6.4 SAS

The example uses a small SAS file that is installed with the **haven** package.

```
library(haven)

path <- system.file("examples", "iris.sas7bdat", package = "haven")
admin_data <- read_sas(path)

admin_data
```

3.7 Wide And Long Data

Data can be organized in more than one shape. A *wide* dataset has one row per unit and a separate column for each measurement (`gdpPercap`, `lifeExp`). A *long* dataset stacks the measurement names into one column and the values into another. Wide data is often easier for people to read; long data is often easier for `ggplot2` and group-wise summaries because the measurement name itself can be mapped to color, facets, or groups.

`pivot_longer()` and `pivot_wider()` from `tidyr` move data between the two shapes.

```
wide_data <- tibble(
  country = c("Afghanistan", "Brazil"),
  year = c(2007, 2007),
  gdpPercap = c(974.6, 9065.8),
  lifeExp = c(43.8, 72.4)
)

long_data <- wide_data |>
```



```

pivot_longer(
  cols = c(gdpPercap, lifeExp),
  names_to = "variable",
  values_to = "value"
)

long_data

```

```

# A tibble: 4 x 4
  country      year variable  value
  <chr>      <dbl> <chr>    <dbl>
1 Afghanistan 2007 gdpPercap 975.
2 Afghanistan 2007 lifeExp  43.8
3 Brazil       2007 gdpPercap 9066.
4 Brazil       2007 lifeExp  72.4

```

```

long_data |>
  pivot_wider(names_from = variable, values_from = value)

```

```

# A tibble: 2 x 4
  country      year gdpPercap lifeExp
  <chr>      <dbl>    <dbl>    <dbl>
1 Afghanistan 2007      975.     43.8
2 Brazil       2007     9066.     72.4

```

The pipe operator `|>` means “take the result so far and send it into the next function.” It lets a sequence of data steps read in the same order as the logic of the analysis.

3.8 Core Tidyverse Verbs

Most data preparation before plotting uses a small set of verbs:

- `filter()` keeps rows
- `select()` keeps columns
- `mutate()` creates or modifies variables
- `group_by()` defines groups
- `summarize()` collapses rows into summaries
- joins combine information from two tables

These verbs come from `dplyr`, which is loaded as part of the tidyverse. They are useful because they correspond to ordinary analytical questions. Which cases belong in this plot? Which variables matter? Do we need a new rate, logged value, or category? What is the summary by group? Does another table contain information that should be merged in?

3.9 `filter()`: Keep Rows

`filter()` keeps observations that satisfy logical conditions. The operator `==` tests exact equality. The operator `%in%` tests whether a value belongs to a set.

```
gap |>
  filter(year == 2007)
```

```
# A tibble: 142 x 6
  country    continent year lifeExp      pop gdpPercap
  <chr>      <chr>    <dbl>  <dbl>    <dbl>    <dbl>
1 Afghanistan Asia      2007   43.8  31889923    975.
2 Albania    Europe    2007   76.4   3600523   5937.
3 Algeria    Africa    2007   72.3  33333216   6223.
4 Angola     Africa    2007   42.7  12420476   4797.
5 Argentina  Americas  2007   75.3  40301927  12779.
6 Australia  Oceania   2007   81.2  20434176  34435.
7 Austria    Europe    2007   79.8   8199783   36126.
8 Bahrain    Asia      2007   75.6   708573    29796.
9 Bangladesh Asia      2007   64.1 150448339   1391.
10 Belgium   Europe    2007   79.4  10392226  33693.
# i 132 more rows
```

```
gap |>
  filter(continent %in% c("Americas", "Europe"))
```

```
# A tibble: 660 x 6
  country    continent year lifeExp      pop gdpPercap
  <chr>      <chr>    <dbl>  <dbl>    <dbl>    <dbl>
1 Albania Europe    1952   55.2  1282697   1601.
2 Albania Europe    1957   59.3  1476505   1942.
3 Albania Europe    1962   64.8  1728137   2313.
4 Albania Europe    1967   66.2  1984060   2760.
5 Albania Europe    1972   67.7  2263554   3313.
6 Albania Europe    1977   68.9  2509048   3533.
```

```

7 Albania Europe      1982    70.4 2780097    3631.
8 Albania Europe      1987    72   3075321    3739.
9 Albania Europe      1992    71.6 3326498    2497.
10 Albania Europe      1997    73.0 3428038    3193.
# i 650 more rows

```

Use `==` for one value. Use `%in%` when several values are acceptable.

3.10 `select()`: Keep Columns

`select()` keeps variables. This is useful when a dataset has many columns but the next step only needs a few.

```

gap |>
  select(country, continent, year, lifeExp, gdpPercap)

```

```

# A tibble: 1,704 x 5
  country      continent year lifeExp gdpPercap
  <chr>        <chr>    <dbl>  <dbl>    <dbl>
1 Afghanistan Asia      1952    28.8     779.
2 Afghanistan Asia      1957    30.3     821.
3 Afghanistan Asia      1962    32.0     853.
4 Afghanistan Asia      1967    34.0     836.
5 Afghanistan Asia      1972    36.1     740.
6 Afghanistan Asia      1977    38.4     786.
7 Afghanistan Asia      1982    39.9     978.
8 Afghanistan Asia      1987    40.8     852.
9 Afghanistan Asia      1992    41.7     649.
10 Afghanistan Asia      1997    41.8     635.
# i 1,694 more rows

```

Rows are observations. Columns are variables. `filter()` works on rows; `select()` works on columns.

3.11 `mutate()`: Create Variables

`mutate()` creates new variables or rewrites existing ones. In the next example, `round()` makes GDP per capita easier to read, and `log10()` creates a transformed variable that will be useful for plotting.

```
gap |>
  mutate(
    gdp_pc_round = round(gdpPercap),
    log_gdp_pc = log10(gdpPercap)
  ) |>
  select(country, gdpPercap, gdp_pc_round, log_gdp_pc)
```

```
# A tibble: 1,704 x 4
  country      gdpPercap gdp_pc_round log_gdp_pc
  <chr>         <dbl>         <dbl>     <dbl>
1 Afghanistan  779.           779       2.89
2 Afghanistan  821.           821       2.91
3 Afghanistan  853.           853       2.93
4 Afghanistan  836.           836       2.92
5 Afghanistan  740.           740       2.87
6 Afghanistan  786.           786       2.90
7 Afghanistan  978.           978       2.99
8 Afghanistan  852.           852       2.93
9 Afghanistan  649.           649       2.81
10 Afghanistan 635.           635       2.80
# i 1,694 more rows
```

`round(gdpPercap)` rounds each value in the `gdpPercap` column. `log10(gdpPercap)` takes the base-10 logarithm. The original variable remains unless you explicitly overwrite it.

3.12 `group_by()` And `summarize()`: Aggregate Data

Many plots need data at a different level than the imported file. For example, the Gapminder data are country-year observations, but we may want a continent-level summary.

```
continent_summary <- gap |>
  group_by(continent) |>
  summarize(
    median_life_exp = median(lifeExp, na.rm = TRUE),
    median_gdp_pc = median(gdpPercap, na.rm = TRUE),
    n_countries = n_distinct(country)
  )

continent_summary
```

```
# A tibble: 5 x 4
  continent median_life_exp median_gdp_pc n_countries
  <chr>          <dbl>          <dbl>          <int>
1 Africa          47.8          1192.           52
2 Americas        67.0          5466.           25
3 Asia            61.8          2647.           33
4 Europe          72.2          12082.          30
5 Oceania         73.7          17983.           2
```

`na.rm = TRUE` tells functions such as `median()` to ignore missing values. Without it, one missing value can make a summary missing.

3.13 Readable Pipelines

Long pipelines are easier to read when each verb gets its own line and arguments are indented consistently.

```
gap |>
  filter(year == 2007) |>
  mutate(gdp_pc_round = round(gdpPercap)) |>
  group_by(continent) |>
  summarize(
    median_life_exp = median(lifeExp, na.rm = TRUE),
    median_gdp_pc = median(gdp_pc_round, na.rm = TRUE)
  ) |>
  arrange(desc(median_life_exp))
```

```
# A tibble: 5 x 3
  continent median_life_exp median_gdp_pc
  <chr>          <dbl>          <dbl>
1 Oceania          80.7          29810
2 Europe           78.6          28054
3 Americas         72.9           8948
4 Asia             72.4           4471
5 Africa           52.9           1452
```

The line breaks are not just style. They make the data workflow visible. Each line is one transformation.

3.14 Joins

Joins combine two tables using one or more key variables. A common pattern is to keep all rows from the main table and add matching columns from a smaller lookup table. That is a `left_join()`, another `dplyr` function.

```
region_notes <- tibble(
  continent = c("Africa", "Americas", "Asia", "Europe", "Oceania"),
  example_note = c(
    "Many countries, large range of outcomes",
    "Includes North, Central, and South America",
    "Large population range",
    "High median life expectancy in many examples",
    "Small number of countries in many teaching datasets"
  )
)

continent_summary |>
  left_join(region_notes, by = "continent")
```

```
# A tibble: 5 x 5
  continent median_life_exp median_gdp_pc n_countries example_note
  <chr>          <dbl>          <dbl>      <int> <chr>
1 Africa          47.8          1192.         52 Many countries, large ran~
2 Americas        67.0          5466.         25 Includes North, Central, ~
3 Asia           61.8          2647.         33 Large population range
4 Europe          72.2         12082.         30 High median life expectan~
5 Oceania         73.7         17983.          2 Small number of countries~
```

Rows without a match in the lookup table would remain, but the new variables would be NA. That behavior is usually safer than silently dropping observations.

3.15 Short Exercise

Use the built-in `mtcars` dataset. Write a pipeline that:

1. converts `mtcars` to a tibble with row names stored as a column named `car`
2. groups by `cyl`
3. calculates average `mpg` and maximum `wt`
4. returns one row per value of `cyl`

```
# Write your code here.
```

3.16 Extras: Google Sheets

Google Sheets can be useful when data are collaboratively maintained. These examples require authentication and should not run automatically in a rendered course book.

```
library(google Sheets4)

sheet_data <- read_sheet(
  ss = "PASTE-GOOGLE-SHEET-ID-OR-URL-HERE",
  sheet = "Sheet1"
)

sheet_data
```

```
library(google Sheets4)

ss <- gs4_create("Visualization 101 2026 Example Sheet")

mtcars |>
  as_tibble(rownames = "car") |>
  sheet_write(ss = ss, sheet = "mtcars")
```

3.17 Extras: Census, GSS, And State Policy Data

Some data sources are best accessed through specialized packages. These examples are templates, not automatically running chunks.

The `tidycensus` package wraps the U.S. Census Bureau's APIs. The two most common entry points are the American Community Survey (`get_acs()`) and the decennial census (`get_decennial()`). A free Census API key is required. Request one at https://api.census.gov/data/key_signup.html, then register it once on your machine with `census_api_key("YOUR_KEY", install = TRUE)`. After that, the key is available to every R session and the line can be removed from teaching files.

```
library(tidycensus)

#census_api_key("2c5ba48be91062db570c7fc5fa49dbca03306c33", install = TRUE)
```

`load_variables()` lists the variables available in a given dataset and year. Caching the result avoids re-downloading the table.

```
acs_vars <- load_variables(2020, "acs5", cache = TRUE) |>
  distinct(concept, .keep_all = TRUE)

acs_vars
```

If `load_variables()` errors with a JSON parse message like `invalid char in json text. <html>`, the Census Bureau returned an HTML error page instead of the variable table. That usually means the requested year is not yet published for that dataset, the Census server is temporarily down, or the request was rate-limited. Try a slightly older year (for ACS 5-year, two years behind the current year is usually safe), re-run the call after a minute, or check <https://www.census.gov/data/developers/data-sets.html> for the dataset's release schedule.

The `variables` argument of `get_acs()` accepts named variable codes. Naming the entries gives readable column names instead of raw codes like `B19013_001`. `output = "wide"` puts each variable in its own column, which is usually easier to plot from than the default long format.

```
acs_raw <- get_acs(
  geography = "congressional district",
  variables = c(
    median_income = "B19013_001",
    total_pop = "B01003_001"
  ),
  year = 2022,
  survey = "acs5",
  output = "wide"
)

acs_raw
```

The `GEOID` column is a concatenation of geographic identifiers. For congressional districts it combines the two-digit state FIPS code with the two-digit district number. Splitting it with `str_sub()` gives joinable columns.

```
acs <- acs_raw |>
  mutate(
    fips = str_sub(GEOID, 1, 2),
    cd = str_sub(GEOID, 3, 4)
  )
```



```
acs
```

State-level summaries are simpler because the `GE0ID` is just the state FIPS code. The wide format makes it easy to compare one variable against another.

```
acs_state <- get_acs(  
  geography = "state",  
  variables = c(  
    median_income = "B19013_001",  
    median_rent = "B25064_001"  
  ),  
  year = 2022,  
  survey = "acs5",  
  output = "wide"  
)  
  
acs_state |>  
  summarize(  
    n_states = n(),  
    cor_income_rent = cor(median_incomeE, median_rentE, use = "complete.obs")  
  )
```

Note that the wide variable names get an E suffix for estimates and an M suffix for the margin of error. Both are usually returned. For most teaching purposes the E columns are enough; in published work the M columns should be acknowledged.

The decennial census uses `get_decennial()` instead. It returns counts rather than estimates and does not need a `survey` argument.

```
pop_2020 <- get_decennial(  
  geography = "state",  
  variables = c(total_pop = "P1_001N"),  
  year = 2020,  
  sumfile = "p1"  
)  
  
pop_2020
```

3.18 Extras: Qualtrics And Canvas

Survey platforms and learning management systems are also possible sources of data. The main point is that API-based imports should be treated as retrieval steps, not as code that runs every time a teaching document renders.

```
library(qualtRics)

raw_survey <- fetch_survey(
  surveyID = "YOUR-SURVEY-ID",
  label = FALSE,
  convert = FALSE
)
```

```
library(rcanvas)

courses <- get_course_list()
```

3.19 What Comes Next

The next chapter uses these data ideas to build plots. `ggplot2` works best when the data are already in a clear rectangular form and the variables needed for the plot have been inspected, selected, transformed, or summarized.

4 Core ggplot2

4.1 Learning Goals

By the end of this chapter, the main goals are:

- compare base R graphics with `ggplot2`
- understand the grammar of graphics at a practical level
- build plots from `ggplot()`, `aes()`, and geoms
- use the `+` operator to build a visualization pipeline
- distinguish mapped aesthetics from fixed aesthetics
- add scatterplot layers, smoothers, log scales, and labels
- apply built-in themes and color palettes

4.2 Load Packages

`ggplot2` is part of the tidyverse. The `scales` package is useful for readable axis labels. `RColorBrewer` provides named color palettes.

```
library(tidyverse)
library(scales)
library(RColorBrewer)
```

4.3 Data For This Chapter

The main examples use the local Gapminder country-year dataset in `Data/gapminder/gapminder.csv`.

```
gap <- read_csv("Data/gapminder/gapminder.csv", show_col_types = FALSE)

gap_2007 <- gap |>
  filter(year == max(year, na.rm = TRUE))

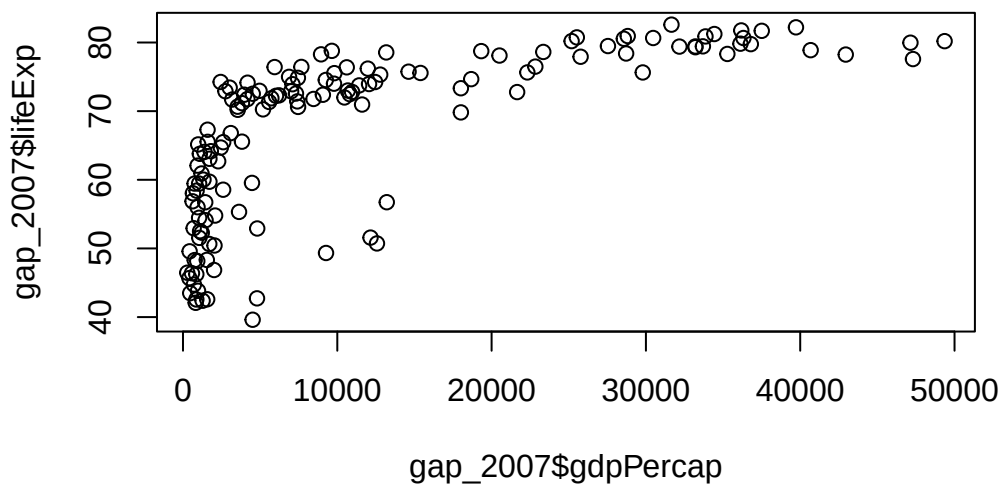
gap_2007
```

```
# A tibble: 142 x 6
  country    continent year lifeExp      pop gdpPercap
  <chr>      <chr>    <dbl>  <dbl>    <dbl>    <dbl>
1 Afghanistan Asia      2007   43.8  31889923    975.
2 Albania    Europe    2007   76.4   3600523   5937.
3 Algeria    Africa    2007   72.3  33333216   6223.
4 Angola     Africa    2007   42.7  12420476   4797.
5 Argentina  Americas  2007   75.3  40301927  12779.
6 Australia  Oceania   2007   81.2  20434176  34435.
7 Austria    Europe    2007   79.8   8199783  36126.
8 Bahrain    Asia      2007   75.6    708573  29796.
9 Bangladesh Asia      2007   64.1 150448339   1391.
10 Belgium   Europe    2007   79.4  10392226  33693.
# i 132 more rows
```

4.4 Comparing Base R Graphics And ggplot2

Base R has plotting functions built in. They are useful for quick checks and appear in many older examples.

```
plot(gap_2007$gdpPercap, gap_2007$lifeExp)
```



That plot works, but the labels are not very informative, the x-axis is hard to read, and customization quickly becomes a matter of remembering many arguments.

A base R histogram shows a similar pattern:

```
hist(gap_2007$lifeExp, breaks = 8)
```



The problem is not that base R cannot make plots. The problem is that complex plots become difficult to revise. `ggplot2` gives us a more structured way to build visualizations.

4.5 The Grammar Of Graphics

A `ggplot2` figure is built in layers. Every plot needs three minimum elements before any layer can sit on top of them:

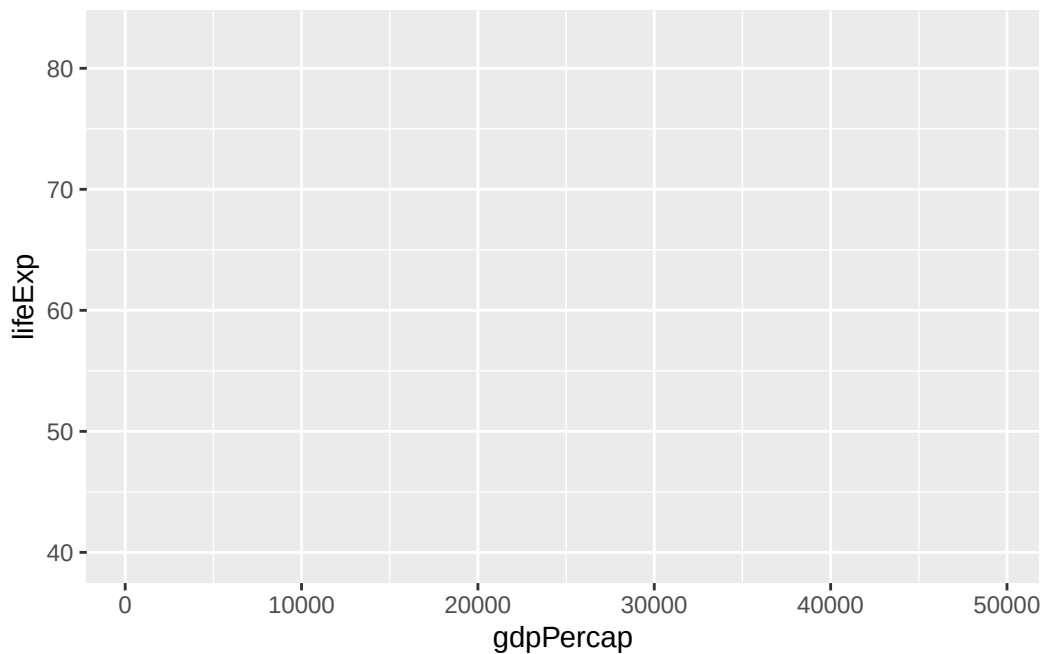
- data: the table being plotted
- mappings: the links between variables and visual features
- geoms: the visual marks, such as points, lines, bars, or histograms

This separation is what makes `ggplot2` powerful. You can change the data, the mapping, or the geom without rewriting the whole plot from scratch.

4.6 The Base Layer: `ggplot()` And `aes()`

The `ggplot()` function starts a plot. The `aes()` function defines aesthetic mappings. Aesthetic mappings connect variables in the data to visual properties such as x-position, y-position, color, size, shape, or transparency.

```
ggplot(  
  data = gap_2007,  
  mapping = aes(x = gdpPercap, y = lifeExp)  
)
```

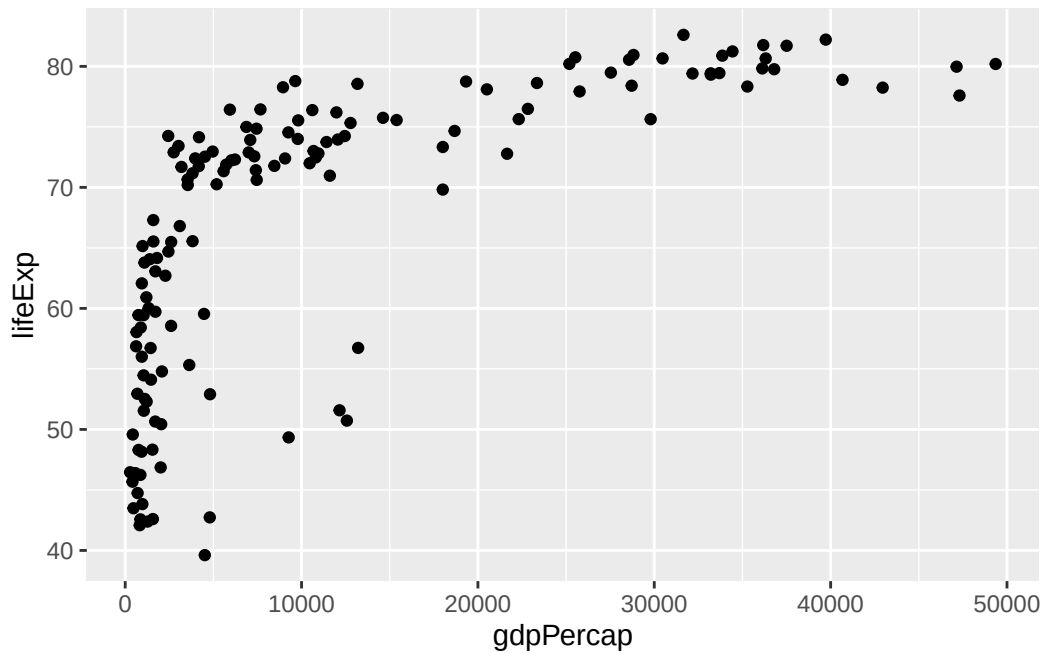


This is the **base layer**. The data and mappings are set, the axes are drawn, but nothing is plotted yet because no geom has been added. Every later layer — points, lines, smoothers, labels, scales — sits on top of this base.

4.7 Add A Geom Layer

`geom_point()` draws points. Because each row in `gap_2007` is a country, each point is a country.

```
ggplot(
  data = gap_2007,
  mapping = aes(x = gdpPercap, y = lifeExp)
) +
  geom_point()
```



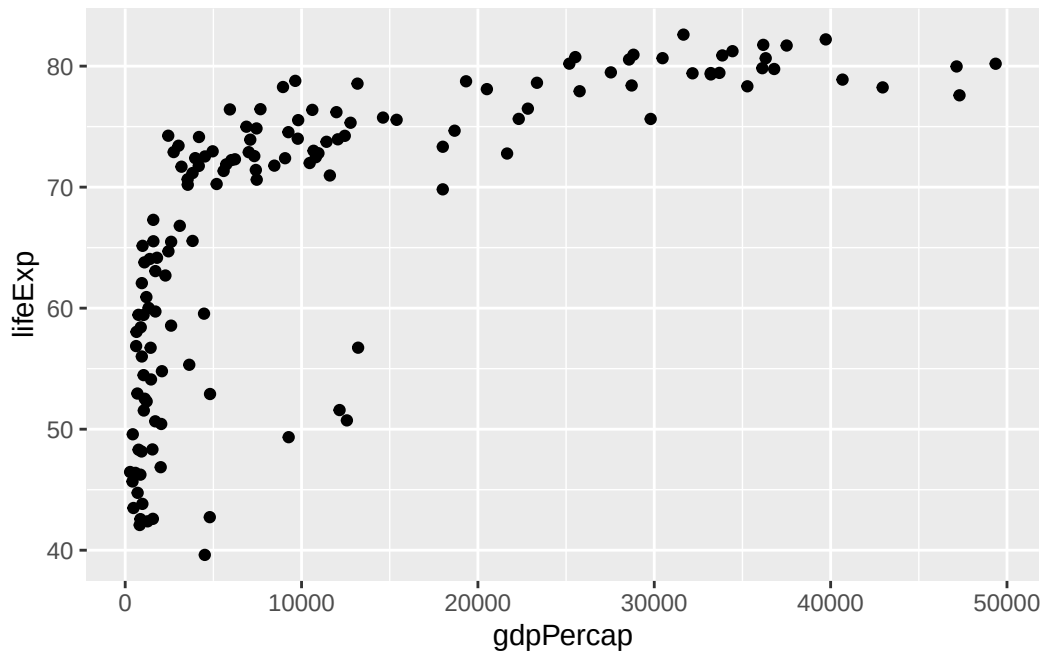
The + operator means “add the next plot component.” In `ggplot2`, line breaks usually put + at the end of the line so R knows the plot is not finished.

4.8 Saving A Plot Object

It is often useful to store the base plot and add layers later.

```
p_gap <- ggplot(
  data = gap_2007,
  mapping = aes(x = gdpPercap, y = lifeExp)
)

p_gap +
  geom_point()
```



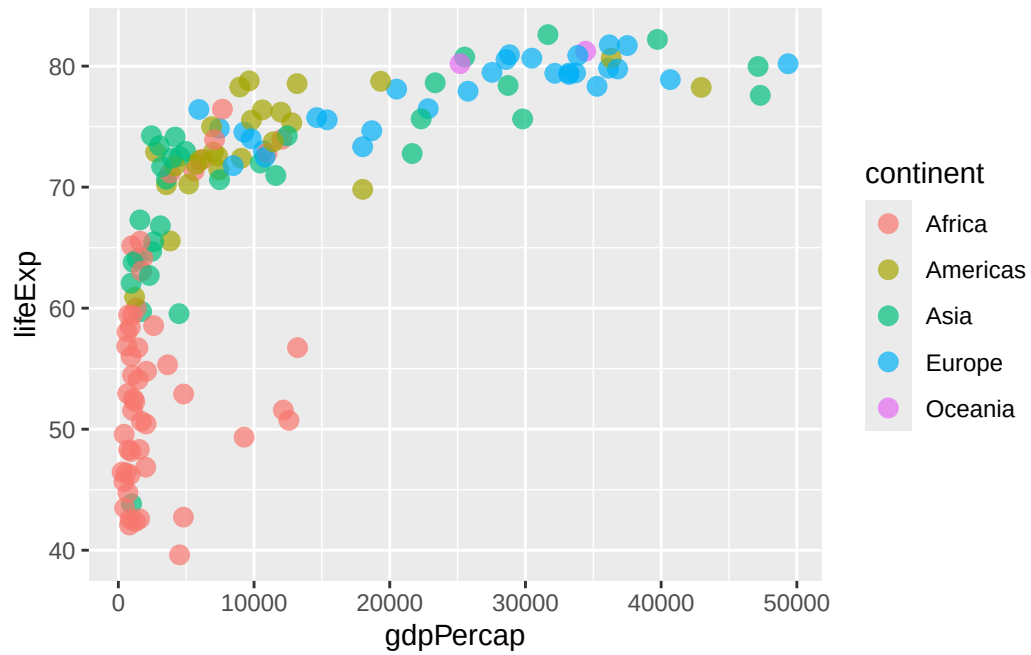
The object `p_gap` stores the data and default mappings. It does not lock the plot into one final form. We can keep adding layers.

4.9 Mapping Versus Setting

One of the most important `ggplot2` distinctions is the difference between mapping and setting.

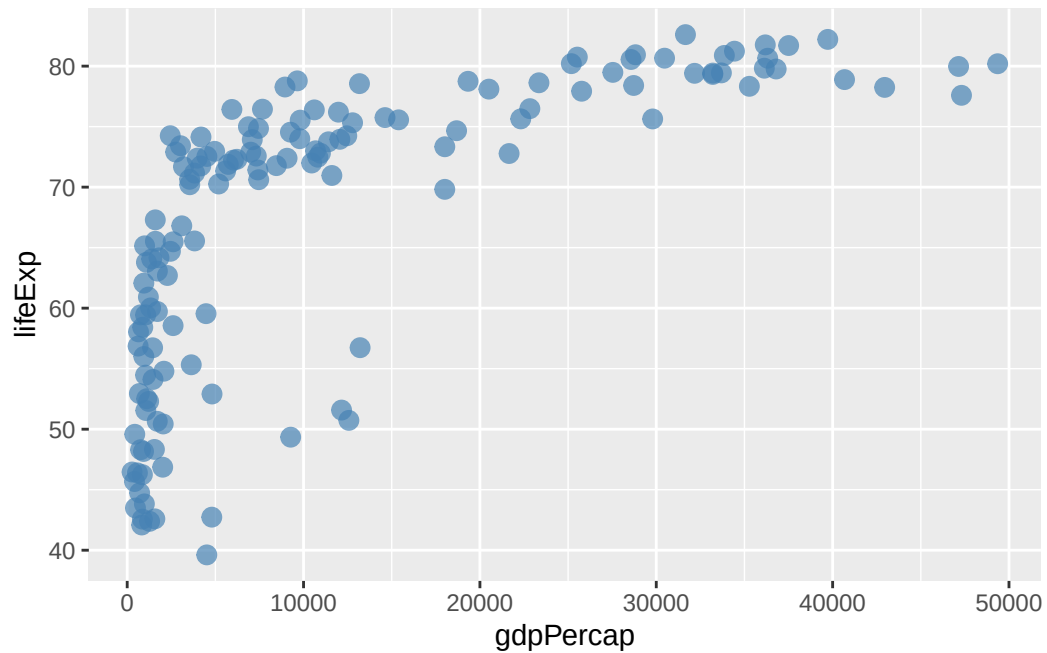
When an aesthetic is mapped, it goes inside `aes()` and represents data. In the next plot, color means continent.

```
p_gap +  
  geom_point(aes(color = continent), size = 3, alpha = 0.7)
```

When an aesthetic is set, it stays fixed and goes outside `aes()`. In the next plot, all points are the same color. The color does not encode information.

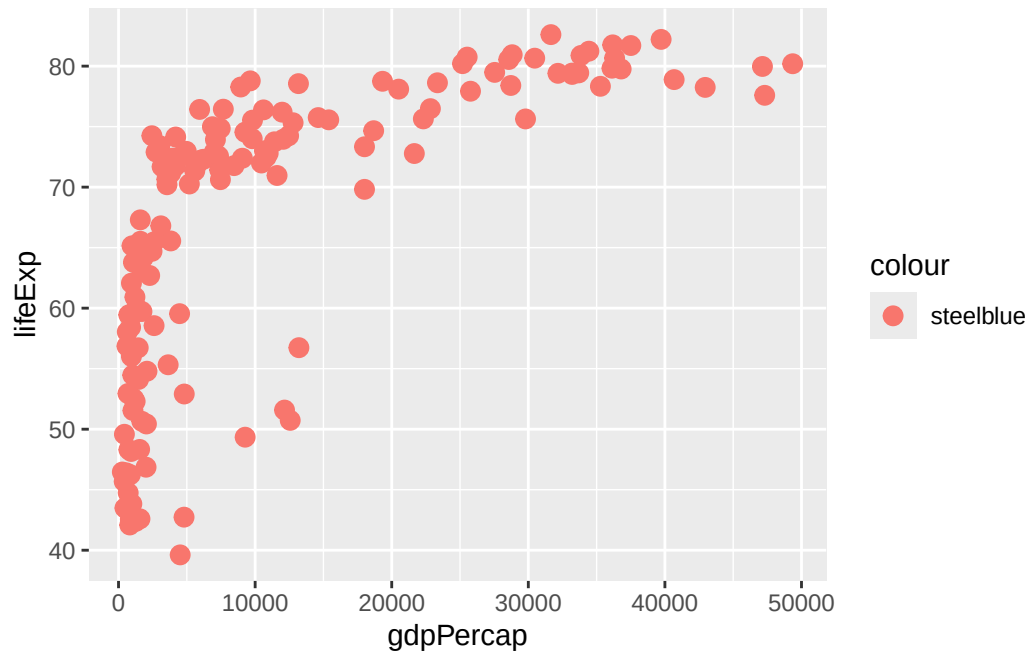
```
p_gap +  
  geom_point(color = "steelblue", size = 3, alpha = 0.7)
```



This distinction matters because viewers interpret visual differences. If color varies because it is mapped to a variable, it carries meaning. If color is fixed, it is a design choice.

A common mistake is to put a fixed color inside `aes()`:

```
p_gap +  
  geom_point(aes(color = "steelblue"), size = 3)
```



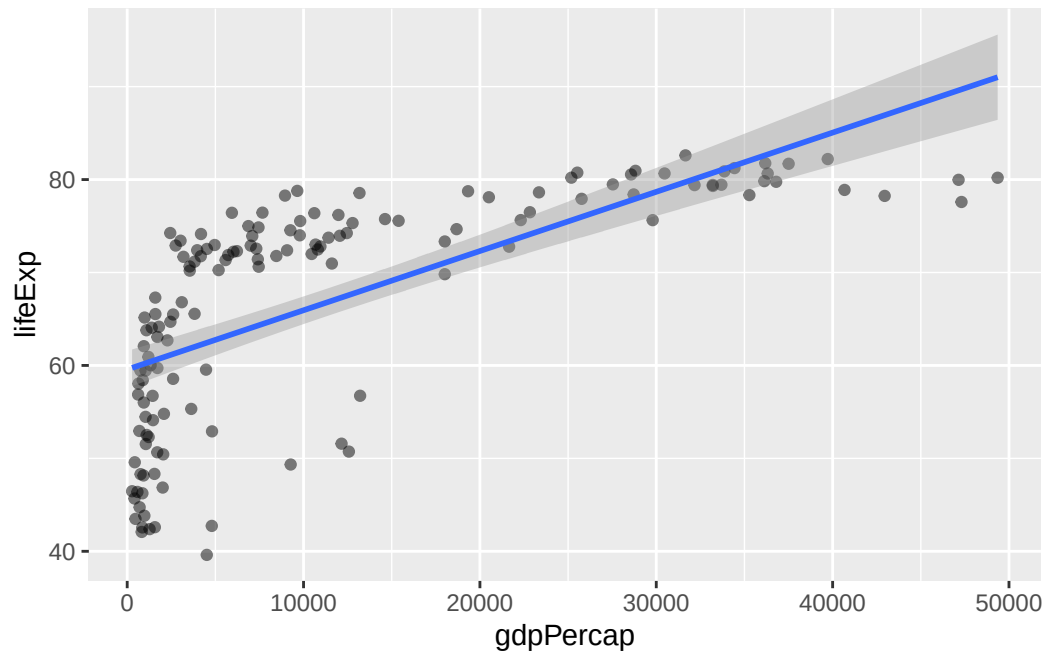
In that plot, "steelblue" is treated like a one-value variable, not as a fixed color instruction. The result is a legend that does not help the reader. The fix is to move fixed settings outside `aes()`.

4.10 Smoothers

Scatterplots show individual observations. Smoothers add a summary of the relationship.

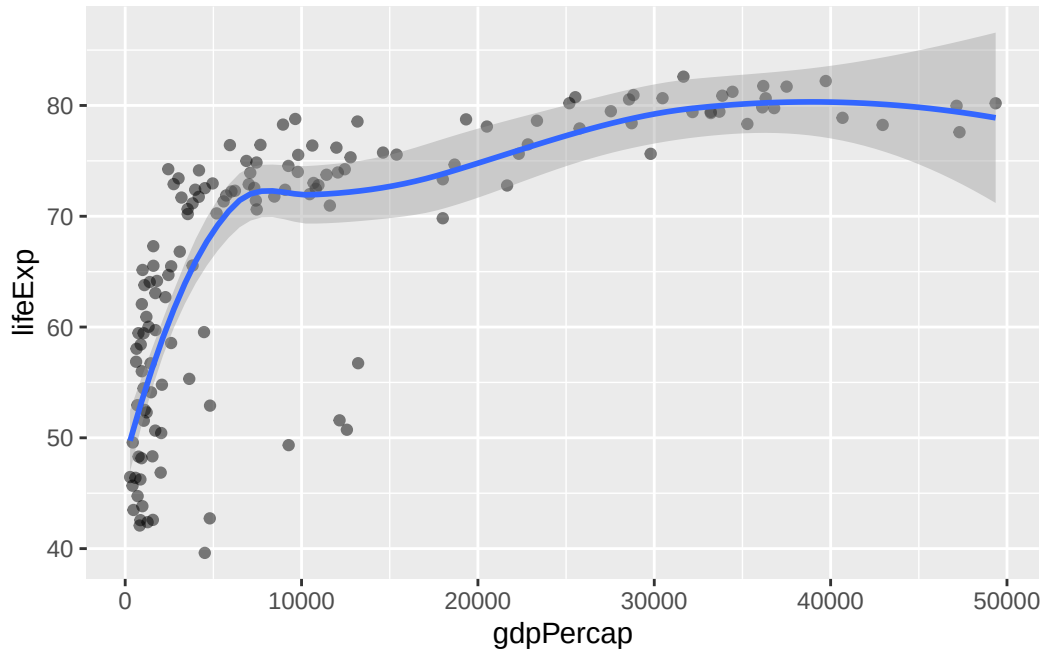
A linear smoother fits a straight line:

```
p_gap +  
  geom_point(alpha = 0.5) +  
  geom_smooth(method = "lm", formula = y ~ x)
```



A loess smoother is more flexible:

```
p_gap +  
  geom_point(alpha = 0.5) +  
  geom_smooth(method = "loess", formula = y ~ x)
```



Neither smoother is automatically correct. A straight line is easy to interpret but may miss curvature. A loess curve can reveal bends in the data but is more descriptive than explanatory. The right choice depends on the purpose of the plot.

4.11 Log Scales

GDP per capita is usually highly skewed. A few very high values can compress most observations into the left side of the plot. A log scale spreads the lower and middle values out while keeping the order intact.

```
p_gap +  
  geom_point(alpha = 0.6) +  
  scale_x_log10(labels = label_dollar())
```

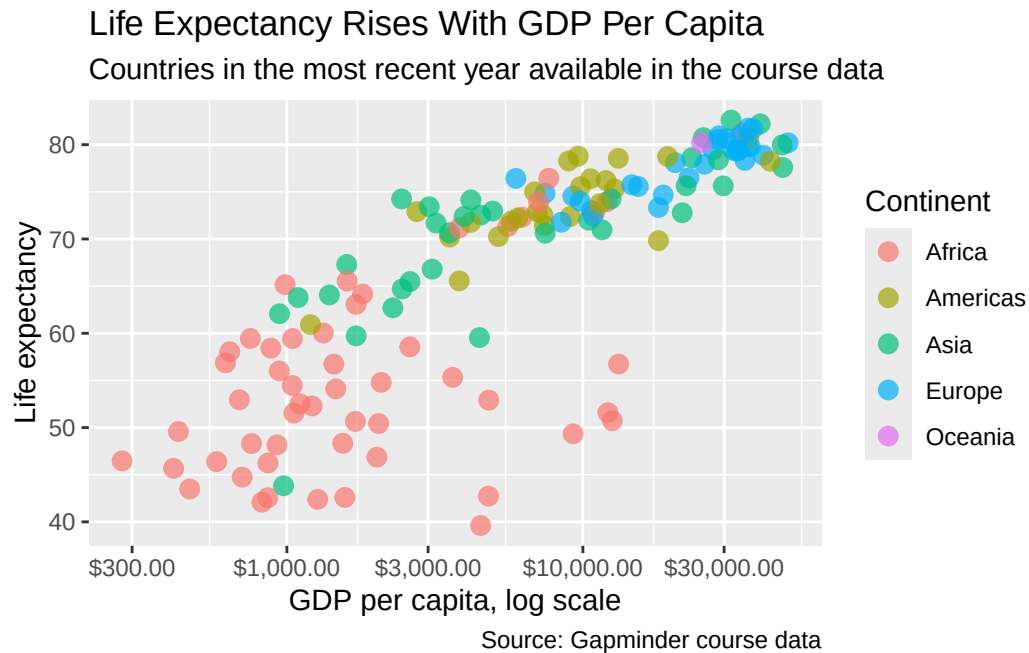


The command `scale_x_log10()` changes the x-axis, not the underlying data object. The argument `labels = label_dollar()` formats the axis as dollars instead of scientific notation.

4.12 Labels With `labs()`

Plots should be understandable outside the code chunk. `labs()` adds titles, subtitles, axis labels, legend labels, and captions.

```
p_gap +
  geom_point(aes(color = continent), alpha = 0.7, size = 3) +
  scale_x_log10(labels = label_dollar()) +
  labs(
    title = "Life Expectancy Rises With GDP Per Capita",
    subtitle = "Countries in the most recent year available in the course data",
    x = "GDP per capita, log scale",
    y = "Life expectancy",
    color = "Continent",
    caption = "Source: Gapminder course data"
  )
```

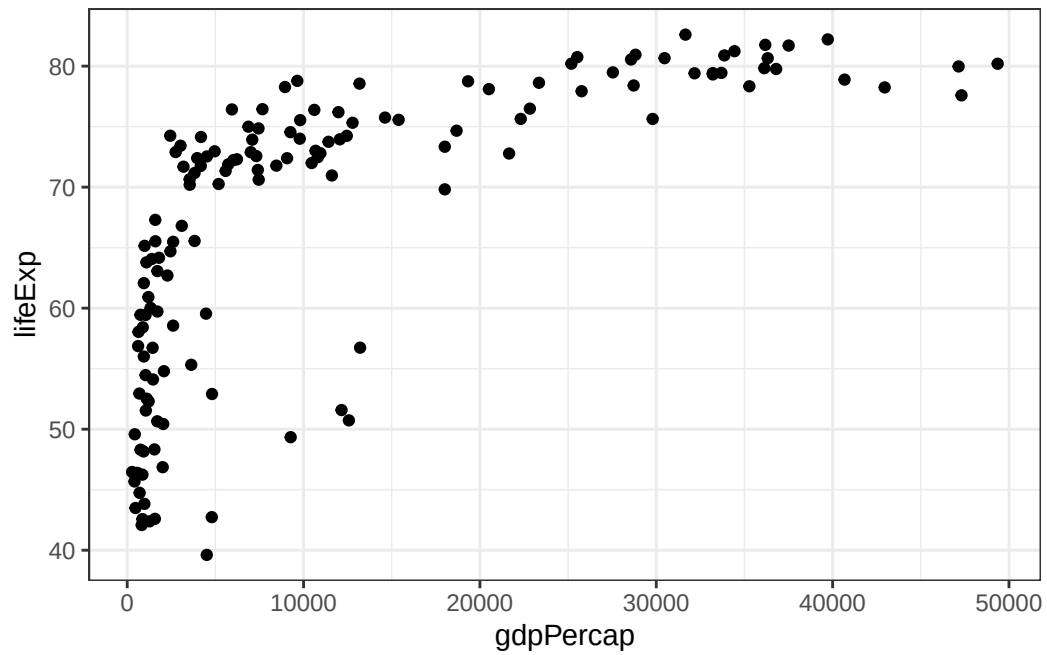


The title tells the reader the main subject. The subtitle adds context. Axis labels state the meaning of the plotted variables. The legend title clarifies the mapped aesthetic. The caption is a natural place for data source information.

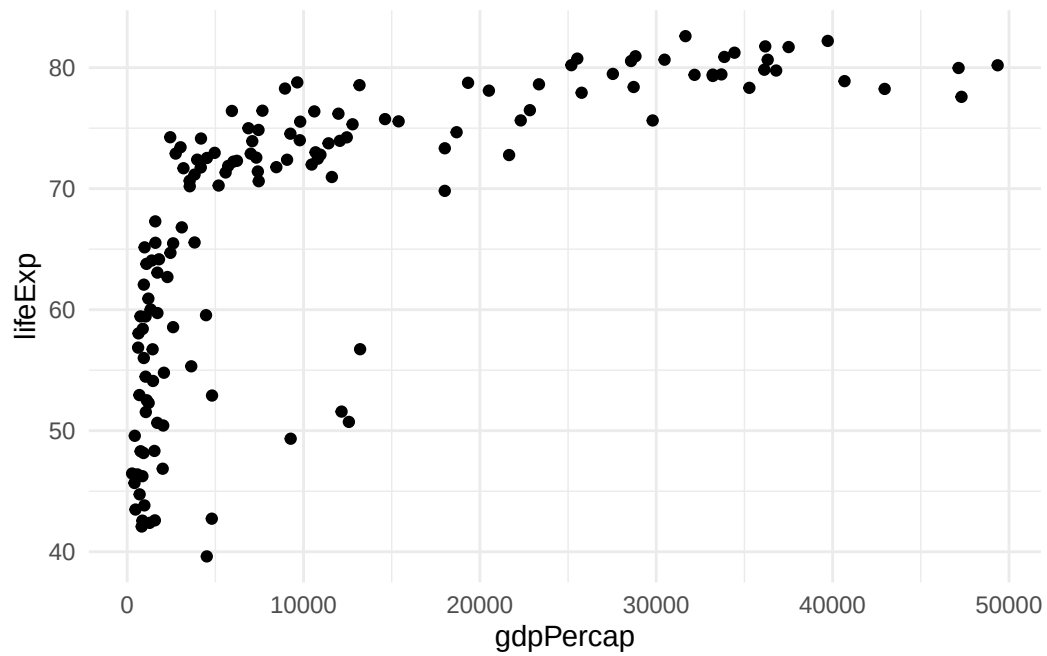
4.13 Themes

Themes control the non-data parts of the plot: background, grid lines, axis text, title placement, legend placement, and similar elements. A theme does not change the data being shown, but it can change how easy the plot is to read.

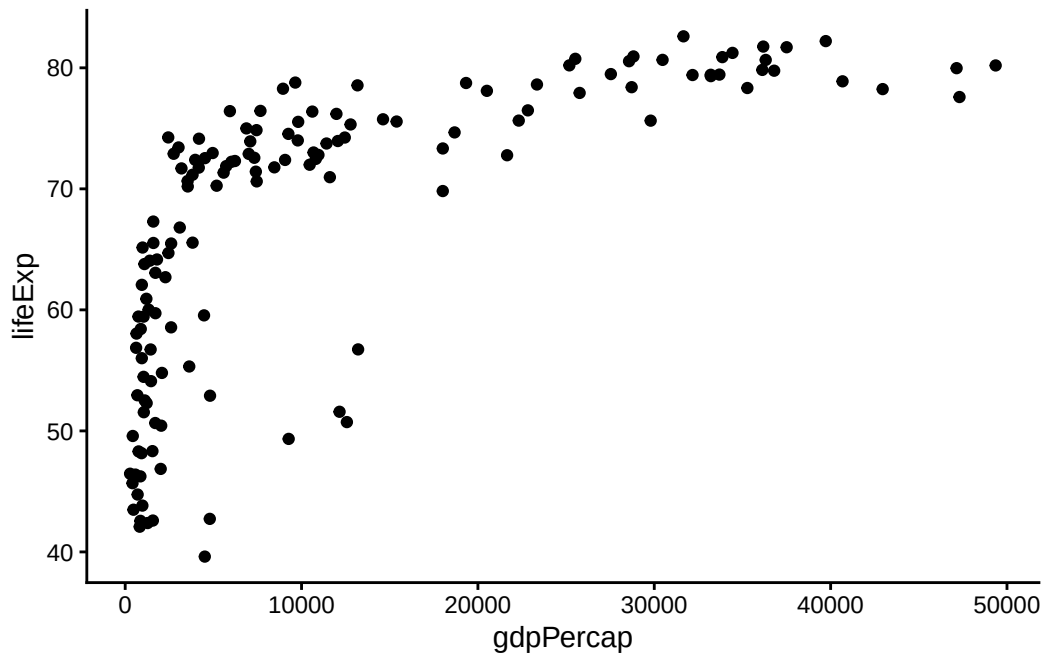
```
p_gap + geom_point() + theme_bw()
```



```
p_gap + geom_point() + theme_minimal()
```

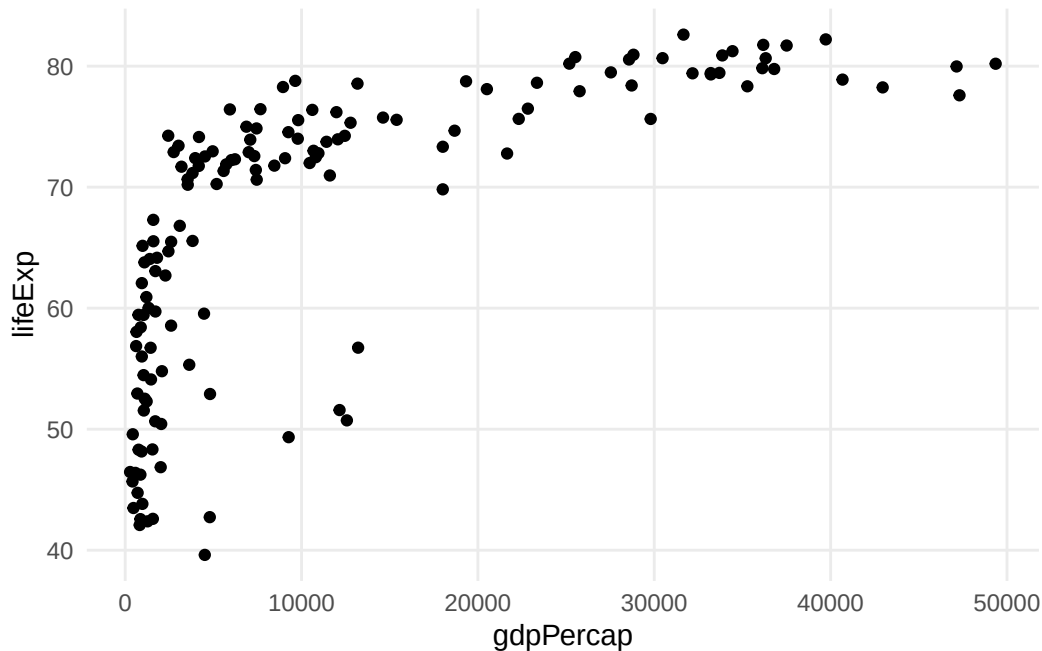



```
p_gap + geom_point() + theme_classic()
```



Built-in themes are good starting points. Individual elements can then be adjusted with `theme()`.

```
p_gap +
  geom_point() +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 16, face = "bold"),
    plot.subtitle = element_text(size = 11),
    axis.title = element_text(size = 11),
    panel.grid.minor = element_blank()
  )
```



The most important rule is that the theme should support the comparison. A visually dramatic theme that makes the data harder to read is not helping.

4.14 Color Palettes

Default colors are fine for early drafts. For more intentional figures, choose palettes that match the type of data.

- Qualitative palettes distinguish categories with no natural order.
- Sequential palettes show ordered values from low to high.
- Diverging palettes show distance from a meaningful midpoint.

Start from a plot that maps a categorical variable to color:

```
p_gap_color <- p_gap +
  geom_point(aes(color = continent), alpha = 0.75, size = 2) +
  scale_x_log10(labels = label_dollar()) +
  labs(
    x = "GDP per capita, log scale",
    y = "Life expectancy",
    color = "Continent"
  ) +
```

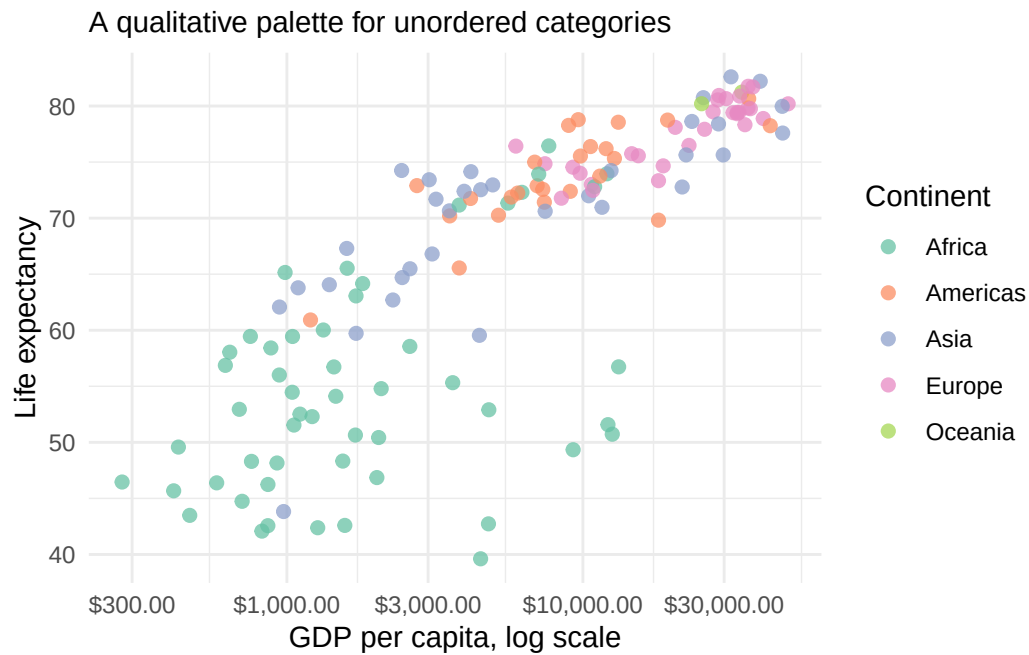
```
theme_minimal()
```

```
p_gap_color
```



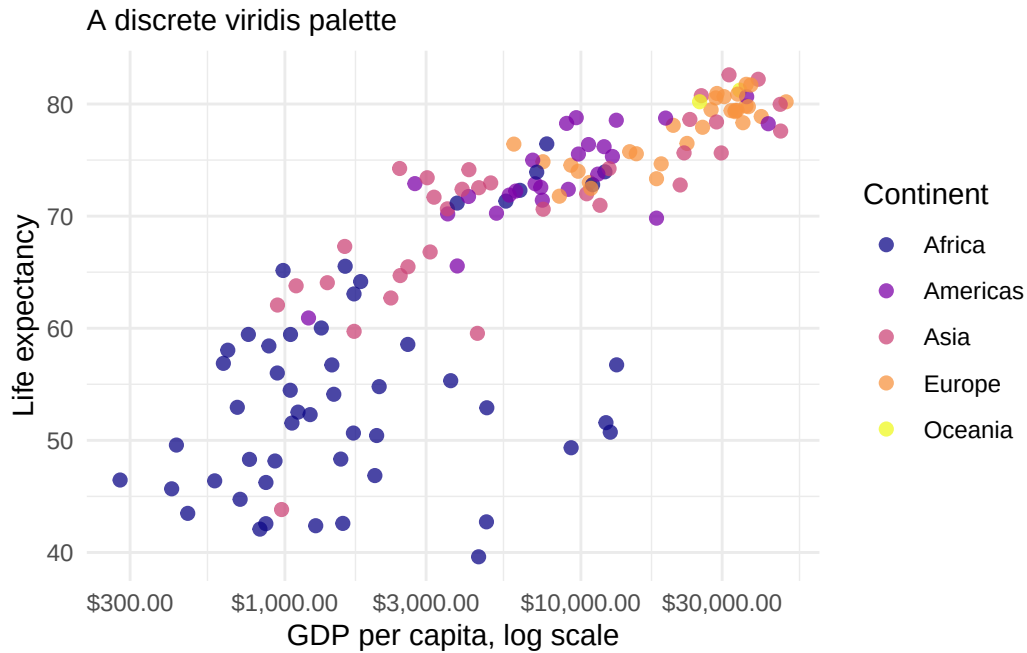
RColorBrewer provides many named palettes.

```
p_gap_color +  
  scale_color_brewer(palette = "Set2") +  
  labs(subtitle = "A qualitative palette for unordered categories")
```



The viridis scales are also useful because they are designed to be readable in color and when converted to grayscale.

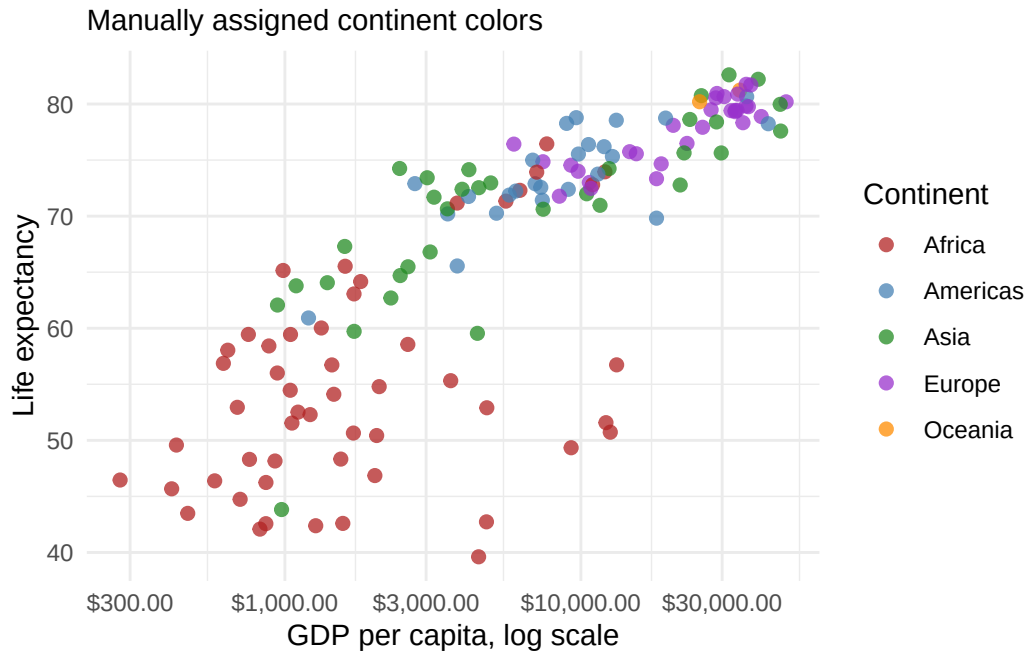
```
p_gap_color +  
  scale_color_viridis_d(option = "plasma") +  
  labs(subtitle = "A discrete viridis palette")
```



Manual color scales are useful when specific categories should keep specific colors across several plots.

```
continent_colors <- c(
  "Africa" = "firebrick",
  "Americas" = "steelblue",
  "Asia" = "forestgreen",
  "Europe" = "darkorchid",
  "Oceania" = "darkorange"
)

p_gap_color +
  scale_color_manual(values = continent_colors) +
  labs(subtitle = "Manually assigned continent colors")
```



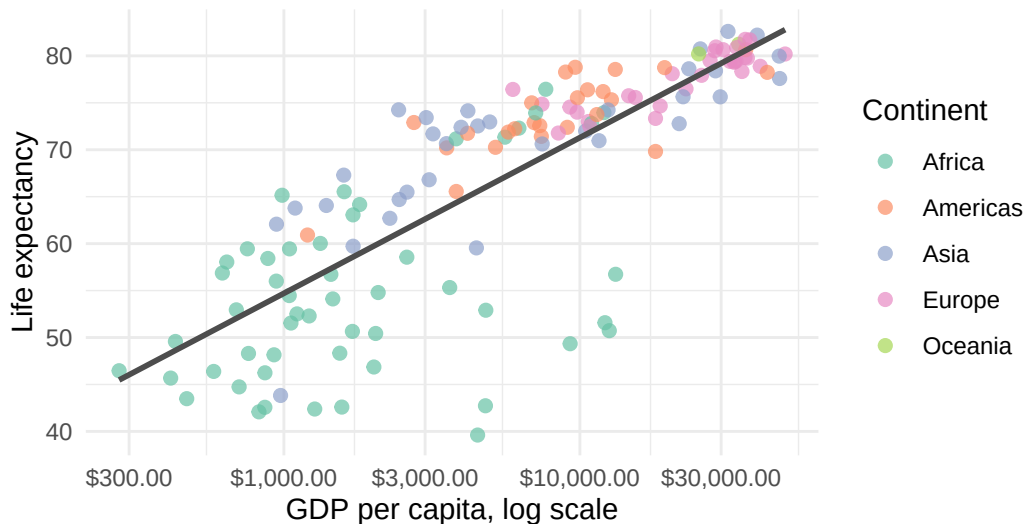
4.15 Putting It All Together

Each piece of this chapter has been introduced as one layer at a time. The full plot stacks them in order: base layer, point geom, smoother, scale, labels, palette, theme.

```
ggplot(
  data = gap_2007,
  mapping = aes(x = gdpPercap, y = lifeExp)
) +
  geom_point(aes(color = continent), alpha = 0.7, size = 2) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE, color = "gray30") +
  scale_x_log10(labels = label_dollar()) +
  scale_color_brewer(palette = "Set2") +
  labs(
    title = "Life Expectancy Rises With GDP Per Capita",
    subtitle = "Countries in the most recent year available in the course data",
    x = "GDP per capita, log scale",
    y = "Life expectancy",
    color = "Continent",
    caption = "Source: Gapminder course data"
  ) +
  theme_minimal()
```

Life Expectancy Rises With GDP Per Capita

Countries in the most recent year available in the course data



Source: Gapminder course data

Read the code from the top:

- `ggplot(...)` sets the base layer with data and mappings.
- `geom_point(...)` draws each country as a point and maps continent to color.
- `geom_smooth(...)` adds a single linear summary across all continents.
- `scale_x_log10(...)` transforms the x-axis so the lower values are easier to read.
- `scale_color_brewer(...)` swaps the default palette for a qualitative one.
- `labs(...)` adds the title, subtitle, axis labels, legend title, and caption.
- `theme_minimal()` controls the non-data styling.

If the plot needs to change, the line responsible for that change is usually easy to identify.

4.16 Short Exercise

The built-in `mtcars` dataset is always available. Convert it to a tibble first so the car names become a column:

```
cars_tbl <- mtcars |>
  as_tibble(rownames = "car")
```

Using `cars_tbl`, build a scatterplot with:

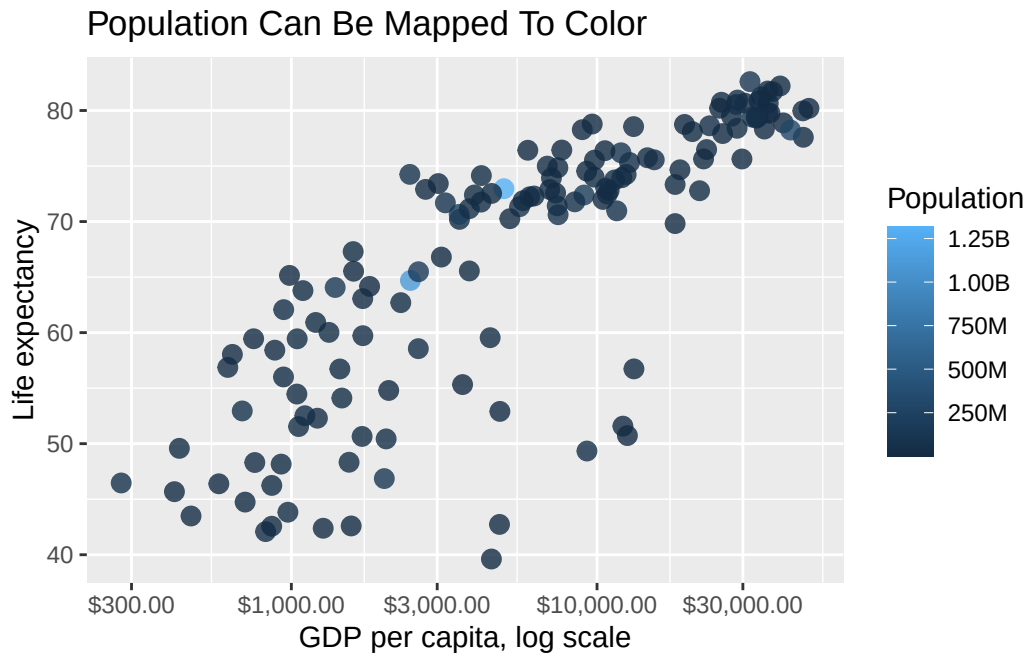
1. `hp` on the x-axis
2. `mpg` on the y-axis
3. `cyl` mapped to color
4. a linear smoother with `se = FALSE`
5. readable labels with `labs()`

```
# Write your code here.
```

4.17 Extras: Continuous Color

Color can represent a continuous variable as well as a category. In the next plot, color represents population.

```
p_gap +  
  geom_point(aes(color = pop), size = 3, alpha = 0.8) +  
  scale_x_log10(labels = label_dollar()) +  
  scale_color_continuous(labels = label_number(scale_cut = cut_short_scale())) +  
  labs(  
    title = "Population Can Be Mapped To Color",  
    x = "GDP per capita, log scale",  
    y = "Life expectancy",  
    color = "Population"  
  )
```

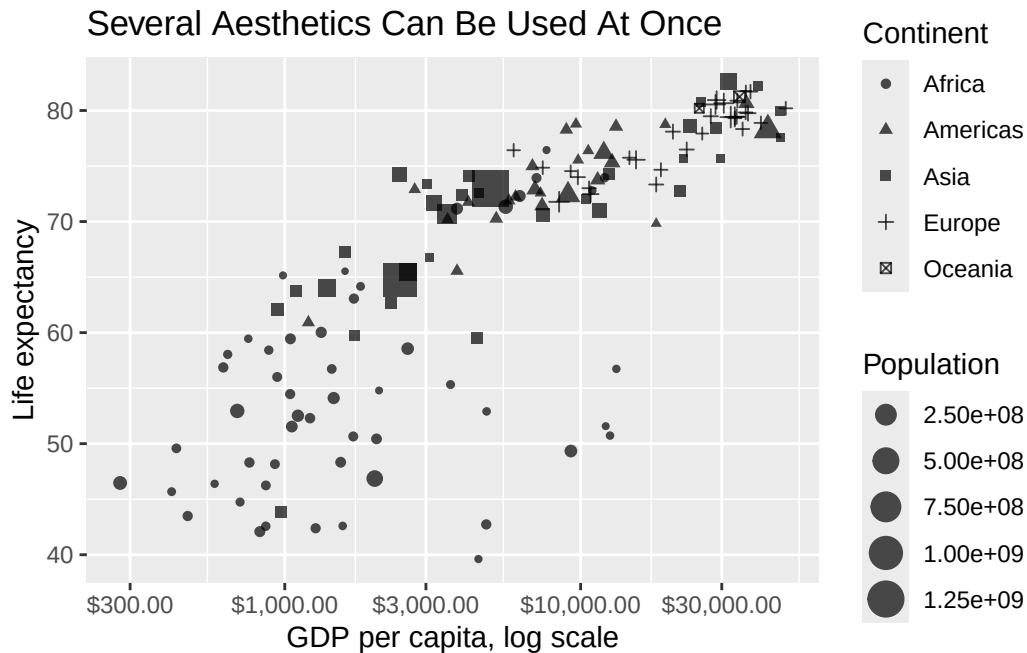



This is possible, but it is not always the best choice. Continuous color scales can be hard to read precisely. Use them when the added variable helps interpretation rather than simply making the plot look more complex.

4.18 Extras: Shape And Size

Other aesthetics can also be mapped or set. Shape works best for a small number of categories. Size can work for quantities, but area comparisons are difficult for readers.

```
ggplot(gap_2007, aes(x = gdpPercap, y = lifeExp)) +
  geom_point(aes(shape = continent, size = pop), alpha = 0.7) +
  scale_x_log10(labels = label_dollar()) +
  labs(
    title = "Several Aesthetics Can Be Used At Once",
    x = "GDP per capita, log scale",
    y = "Life expectancy",
    shape = "Continent",
    size = "Population"
  )
```



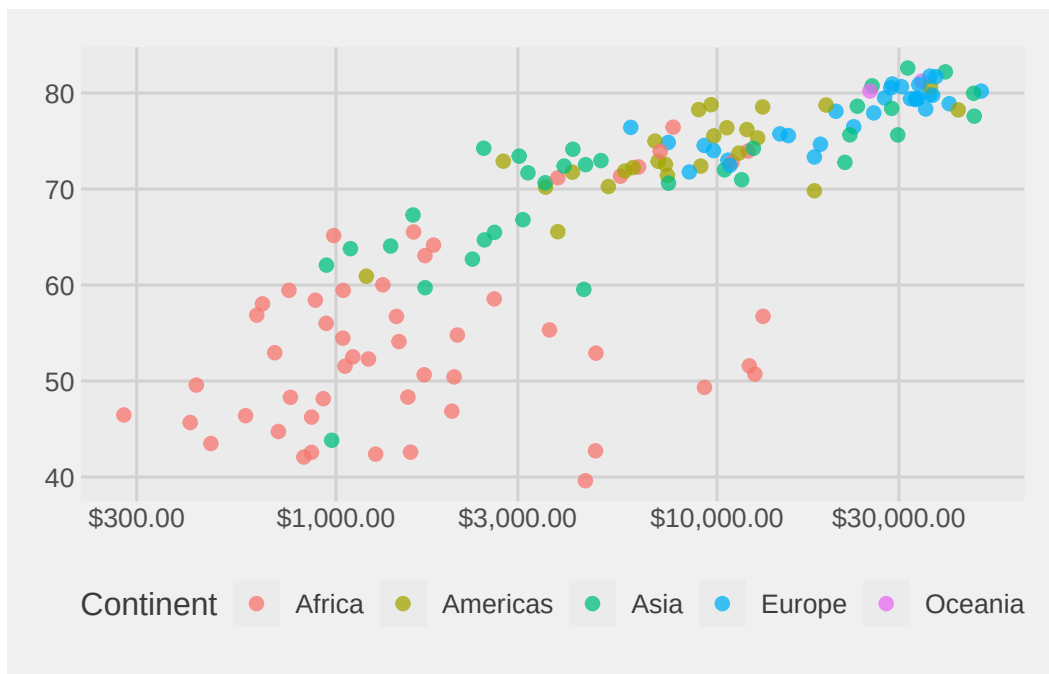
Adding more aesthetics is not automatically an improvement. Each additional encoding asks the reader to do more work. The best plot is usually the one that makes the intended comparison clear with the fewest necessary encodings.

4.19 Extras: External Themes And Rich Text

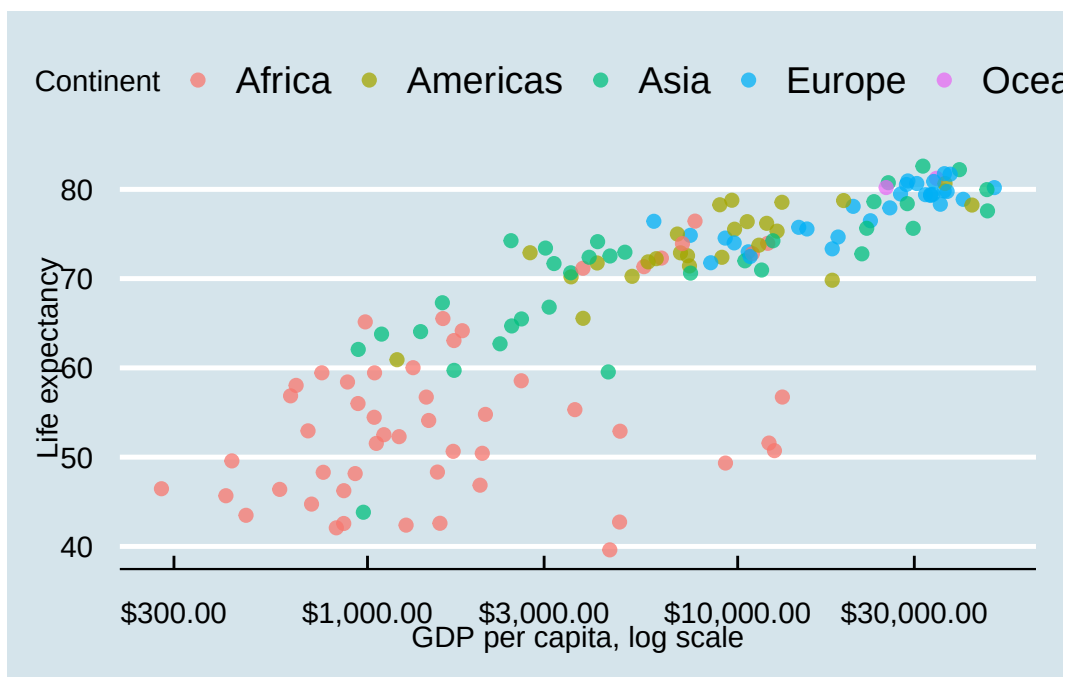
The packages `ggthemes`, `hrbrthemes`, and `ggtext` provide additional styling options. These are useful when a plot needs to match a publication, organization, or report style.

`ggthemes` includes themes that imitate familiar publication and media styles.

```
p_gap_color +
  ggthemes::theme_fivethirtyeight()
```

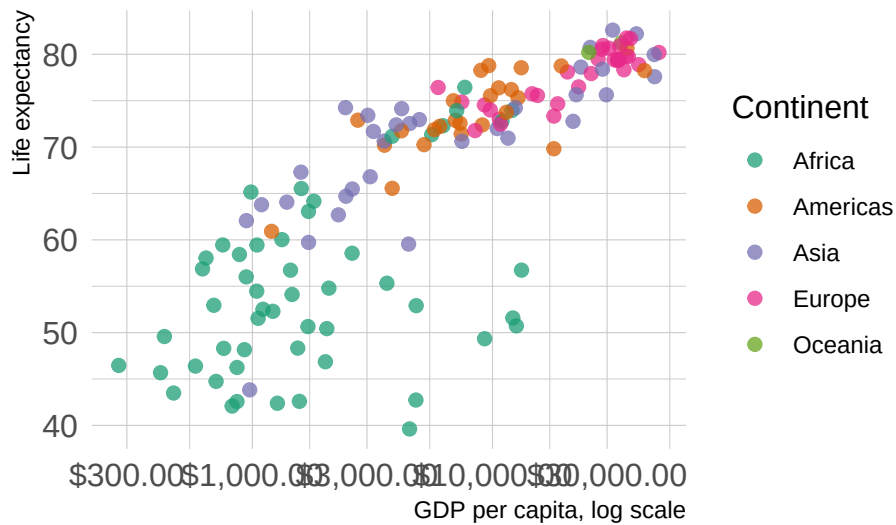


```
p_gap_color +
  ggthemes::theme_economist()
```



The `hrbrthemes` package is especially useful for clean report-style plots.

```
p_gap_color +
  scale_color_brewer(palette = "Dark2") +
  hrbrthemes::theme_ipsum(base_family = "sans")
```

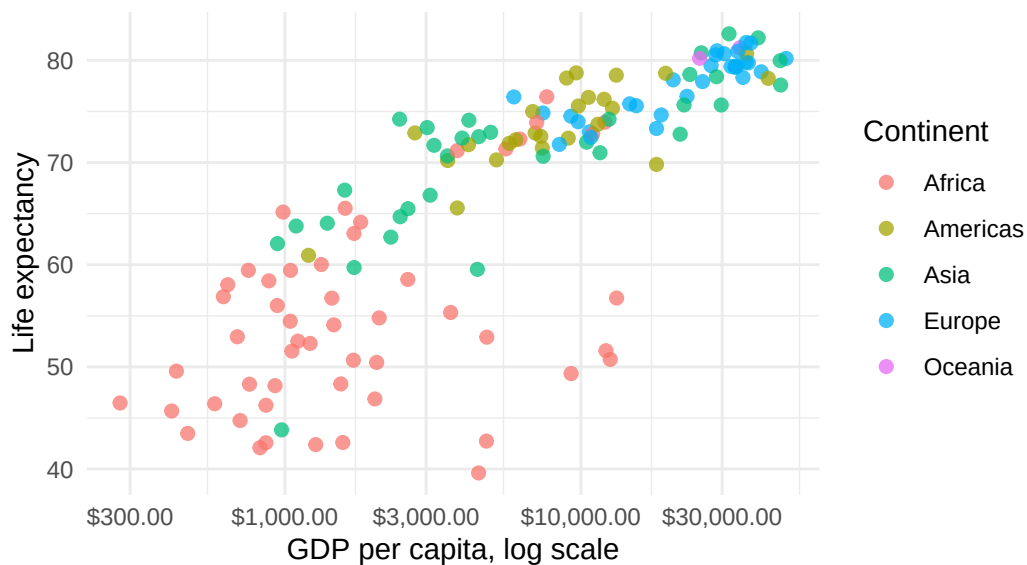


`ggtext` allows limited Markdown and HTML-like formatting inside plot text. This is useful for emphasizing one phrase in a title or coloring part of a subtitle.

```
p_gap_color +
  labs(
    title = "Economic Growth and **Life Expectancy**",
    subtitle = "Countries in <span style='color:darkorange;'>2007</span>"
  ) +
  theme_minimal() +
  theme(
    plot.title = ggtext::element_markdown(),
    plot.subtitle = ggtext::element_markdown()
  )
```

Economic Growth and Life Expectancy

Countries in 2007



Style should refine a figure, not rescue an unclear comparison.

4.20 What Comes Next

The next chapter moves from basic plot construction to visual comparison: separating groups with color, facets, small multiples, free scales, direct labels, and combining plots with `patchwork`. The focus shifts from making a plot work to making a comparison easier to see.

5 Separating and Comparing Data

5.1 Why separate data?

Many plots become more useful when we separate the data into meaningful groups. The relationship between two variables might look simple overall, but the pattern can differ across continents, years, regions, parties, income groups, or other subgroups.

There are two common `ggplot2` strategies:

- Map a grouping variable to an aesthetic such as `color`, `shape`, `fill`, or `linetype`.
- Use facets to create small multiples: repeated panels that show the same plot for different subsets of the data.

Both approaches are useful. The question is not which one is always better. The question is which one makes the comparison easier.

5.2 Start with one plot

We will use Gapminder data to compare GDP per capita and life expectancy. First, filter to one year so that each point is one country.

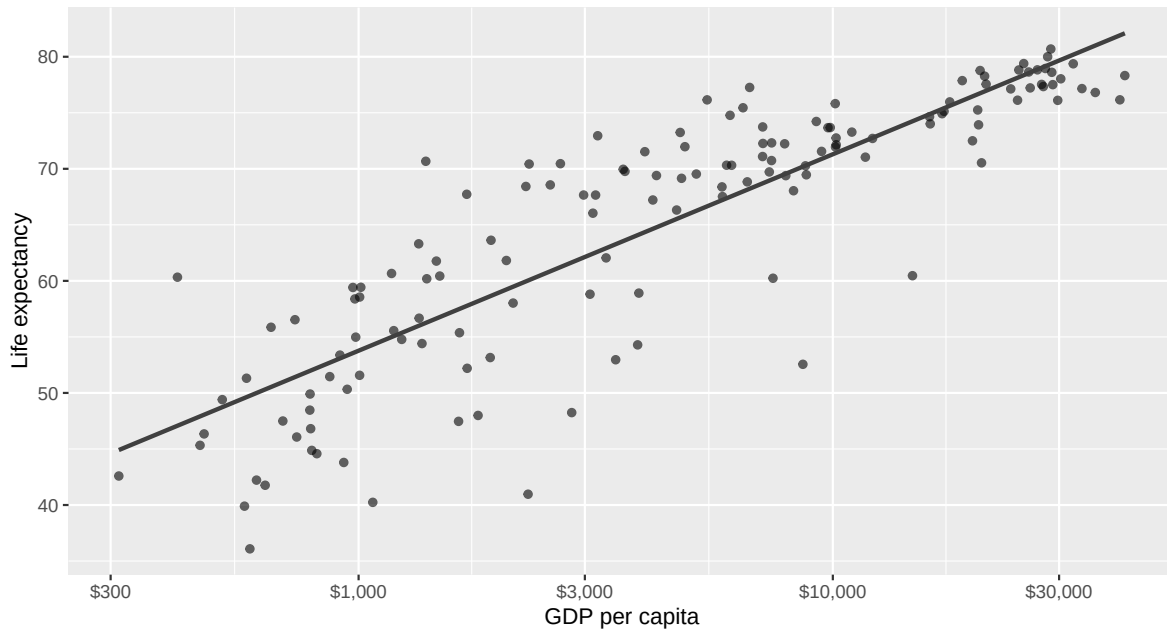
```
gap_1997 <- gapminder |>
  filter(year == 1997)

base_gap <- ggplot(
  data = gap_1997,
  mapping = aes(x = gdpPercap, y = lifeExp)
) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE, color = "gray25") +
  scale_x_log10(labels = dollar_format(accuracy = 1)) +
  scale_y_continuous(breaks = seq(20, 90, by = 10)) +
  labs(
    title = "Economic Growth and Life Expectancy",
    subtitle = "Countries in 1997",
  )
```

```
x = "GDP per capita",
y = "Life expectancy",
caption = "Source: Gapminder."
)
```

base_gap

Economic Growth and Life Expectancy
Countries in 1997



Source: Gapminder.

The log scale on the x-axis helps because GDP per capita is very skewed. Without it, the lower-income countries would be compressed into a small part of the plot.

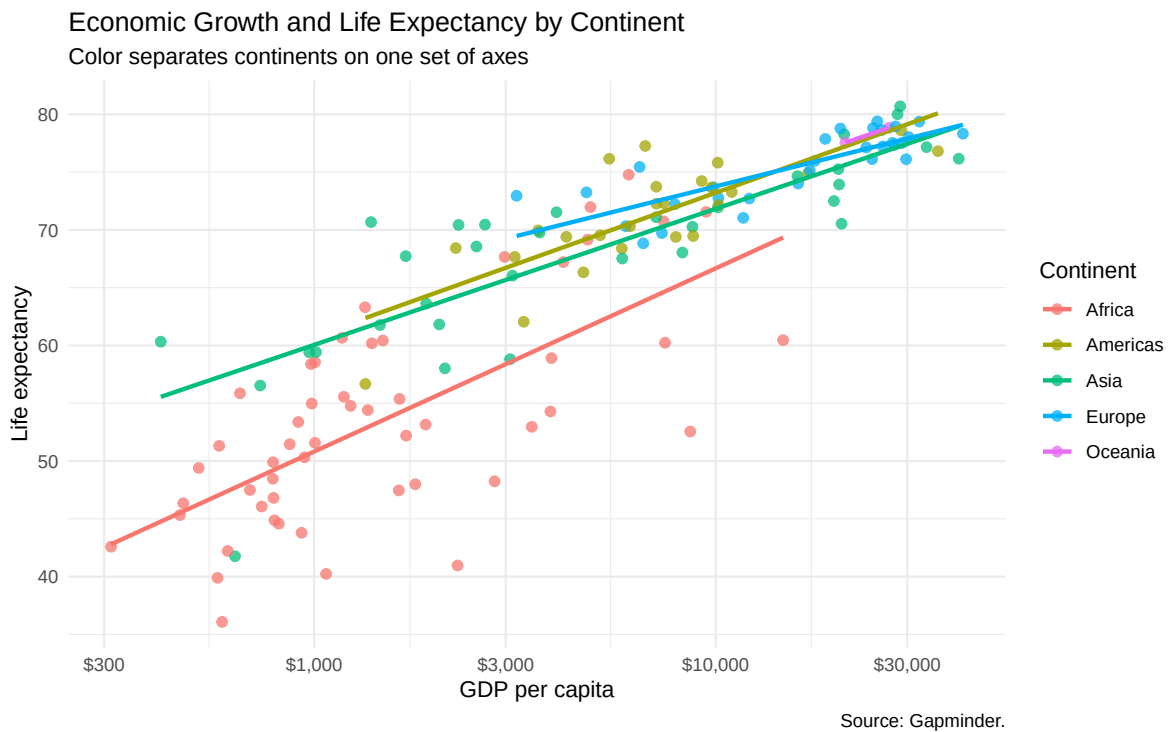
5.3 Color as separation

One way to separate groups is to map a variable to color inside `aes()`.

```
gap_color <- ggplot(
  data = gap_1997,
  mapping = aes(x = gdpPercap, y = lifeExp, color = continent)
) +
  geom_point(alpha = 0.75, size = 2) +
```

```
geom_smooth(method = "lm", formula = y ~ x, se = FALSE) +
scale_x_log10(labels = dollar_format(accuracy = 1)) +
labs(
  title = "Economic Growth and Life Expectancy by Continent",
  subtitle = "Color separates continents on one set of axes",
  x = "GDP per capita",
  y = "Life expectancy",
  color = "Continent",
  caption = "Source: Gapminder."
) +
theme_minimal()

gap_color
```

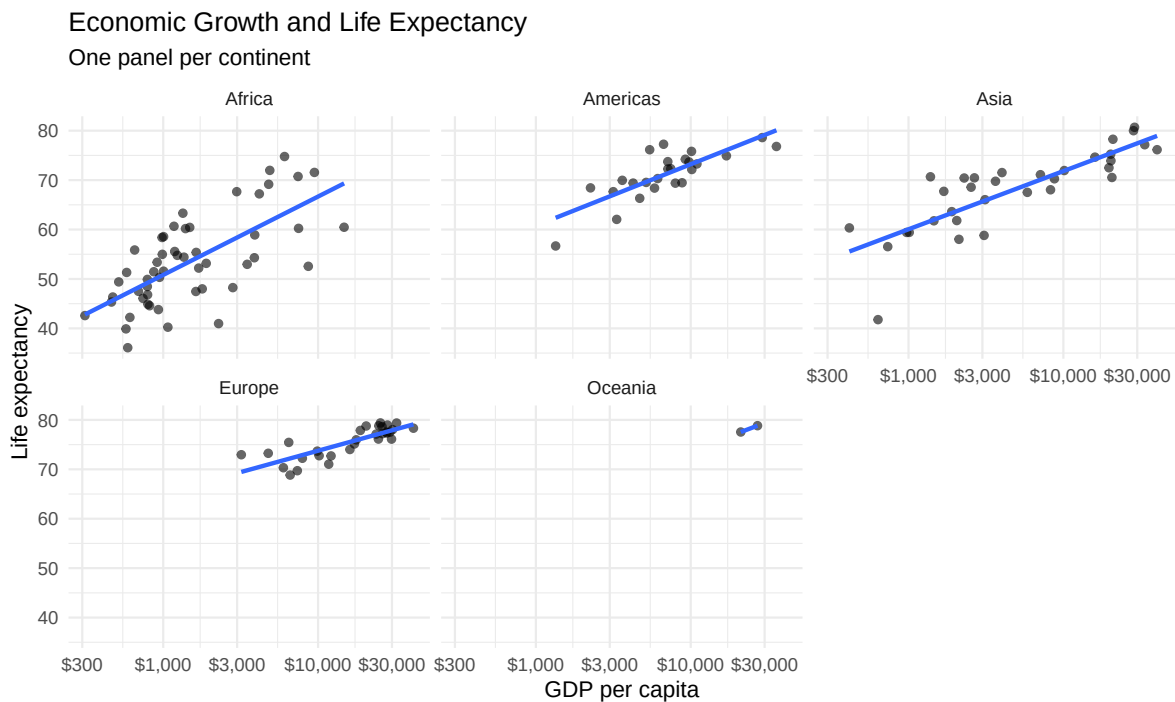


Because `continent` is mapped inside `aes()`, `ggplot2` treats color as part of the data mapping and creates a legend. If we wrote `geom_point(color = "steelblue")`, every point would be steel blue and no legend would be needed.

5.4 Facets as small multiples

Faceting creates one panel for each level of a variable. This gives each group its own space.

```
ggplot(data = gap_1997, mapping = aes(x = gdpPercap, y = lifeExp)) +  
  geom_point(alpha = 0.6) +  
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE) +  
  scale_x_log10(labels = dollar_format(accuracy = 1)) +  
  facet_wrap(~ continent) +  
  labs(  
    title = "Economic Growth and Life Expectancy",  
    subtitle = "One panel per continent",  
    x = "GDP per capita",  
    y = "Life expectancy",  
    caption = "Source: Gapminder."  
  ) +  
  theme_minimal()
```



Source: Gapminder.

The default shared scales make comparisons across panels fair. Africa and Europe are plotted on the same x and y ranges, so differences in location and spread are visible.

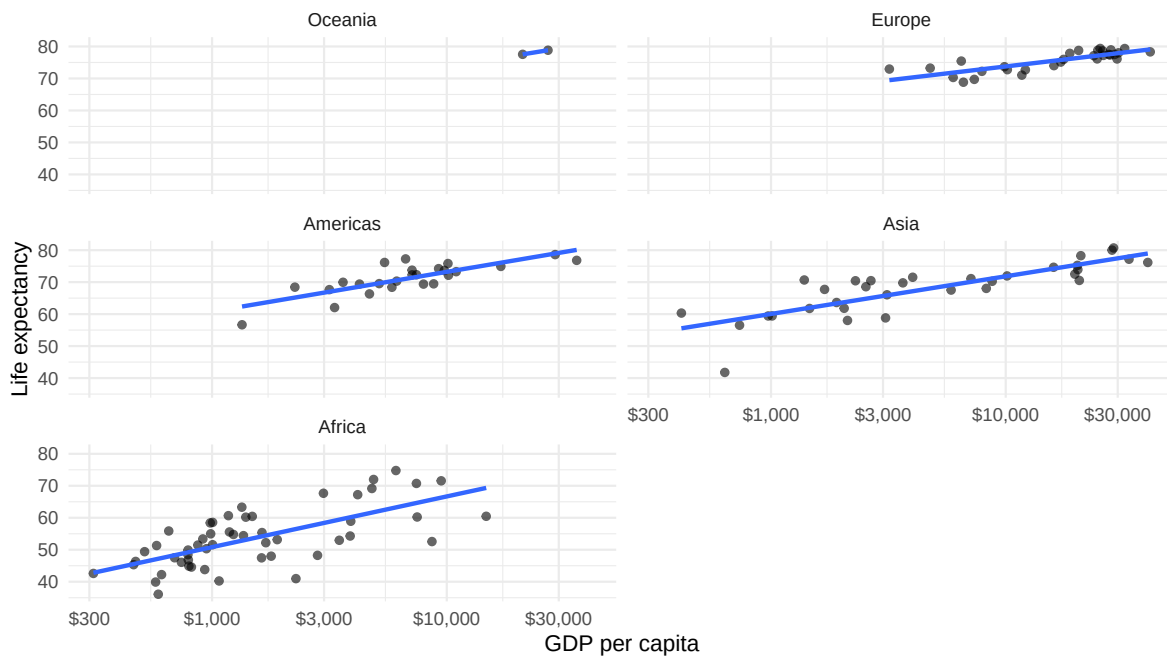
5.5 Reordering facets

Facet order is not always meaningful by default. We can reorder the continent variable by average life expectancy.

```
gap_1997_ordered <- gap_1997 |>
  mutate(
    continent = fct_reorder(
      .f = continent,
      .x = lifeExp,
      .fun = mean,
      .desc = TRUE
    )
  )

ggplot(data = gap_1997_ordered, mapping = aes(x = gdpPercap, y = lifeExp)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE) +
  scale_x_log10(labels = dollar_format(accuracy = 1)) +
  facet_wrap(~ continent, ncol = 2) +
  labs(
    title = "Facets Ordered by Average Life Expectancy",
    x = "GDP per capita",
    y = "Life expectancy",
    caption = "Source: Gapminder."
  ) +
  theme_minimal()
```

Facets Ordered by Average Life Expectancy



Source: Gapminder.

`fct_reorder()` changes the order of the factor levels. That order is then used by `facet_wrap()`.

5.6 Free scales

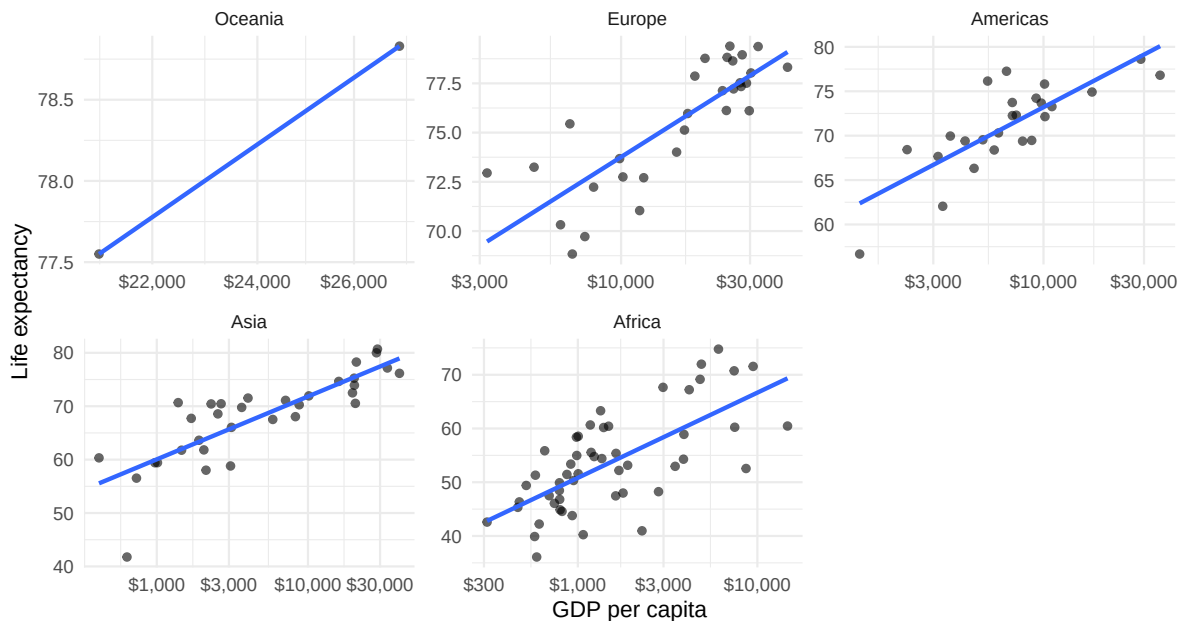
Sometimes fixed scales make within-group patterns hard to see. Free scales let each panel use its own axis range.

```
ggplot(data = gap_1997_ordered, mapping = aes(x = gdpPercap, y = lifeExp)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE) +
  scale_x_log10(labels = dollar_format(accuracy = 1)) +
  facet_wrap(~ continent, scales = "free") +
  labs(
    title = "Free Scales Reveal Within-Continent Detail",
    subtitle = "Use with care: cross-panel comparisons become harder",
    x = "GDP per capita",
    y = "Life expectancy",
    caption = "Source: Gapminder."
  )
```

```
) +  
theme_minimal()
```

Free Scales Reveal Within-Continent Detail

Use with care: cross-panel comparisons become harder



Source: Gapminder.

Free scales are not wrong, but they answer a different question. Fixed scales emphasize comparison across groups. Free scales emphasize detail within each group.

5.7 Direct labels

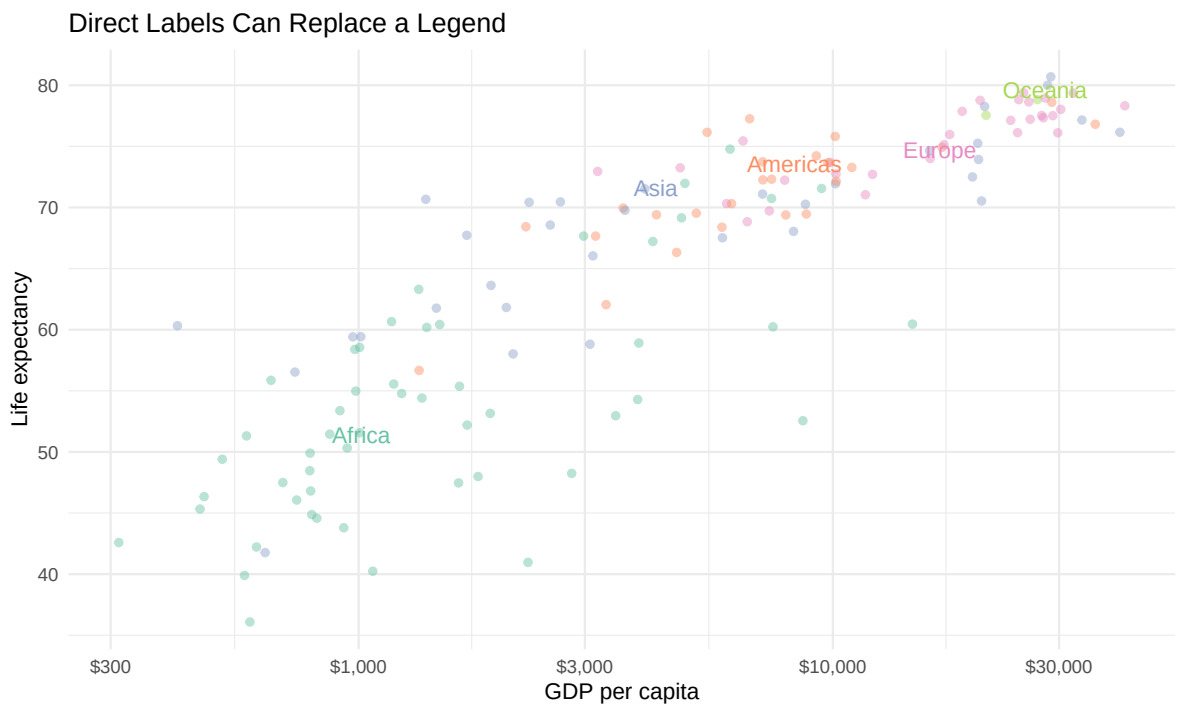
Legends require the reader to look back and forth between the data and the key. Direct labels can reduce that work.

```
continent_summary <- gap_1997 |>  
  group_by(continent) |>  
  summarize(  
    gdpPercap = median(gdpPercap),  
    lifeExp = median(lifeExp)  
  )  
  
ggplot(data = gap_1997, mapping = aes(x = gdpPercap, y = lifeExp, color = continent)) +
```

```

geom_point(alpha = 0.45, show.legend = FALSE) +
geom_text_repel(
  data = continent_summary,
  mapping = aes(label = continent),
  size = 4,
  show.legend = FALSE
) +
scale_x_log10(labels = dollar_format(accuracy = 1)) +
scale_color_brewer(palette = "Set2") +
labs(
  title = "Direct Labels Can Replace a Legend",
  x = "GDP per capita",
  y = "Life expectancy",
  caption = "Source: Gapminder."
) +
theme_minimal()

```



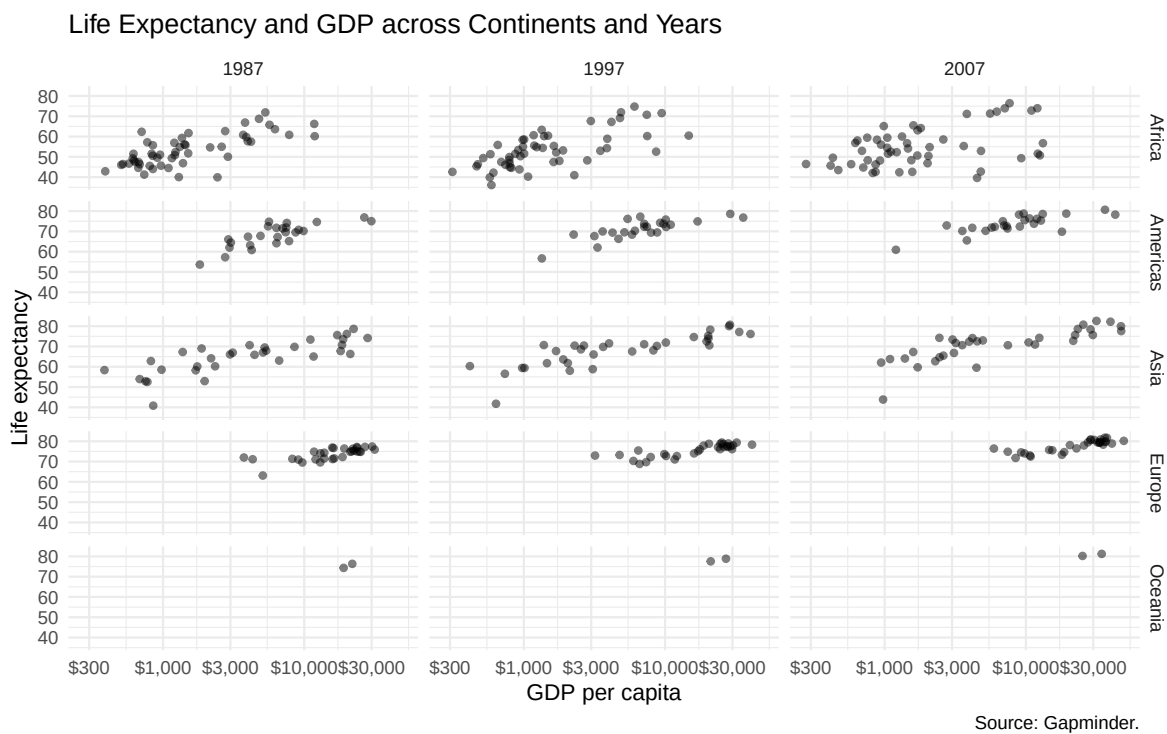
Direct labels work best when there are not too many groups and there is enough space for the labels.

5.8 facet_grid()

`facet_wrap()` is usually the easiest choice for one grouping variable. `facet_grid()` is useful when the panel layout has rows and columns with substantive meaning.

```
gap_recent <- gapminder |>
  filter(year %in% c(1987, 1997, 2007))

ggplot(data = gap_recent, mapping = aes(x = gdpPercap, y = lifeExp)) +
  geom_point(alpha = 0.5, size = 1.3) +
  scale_x_log10(labels = dollar_format(accuracy = 1)) +
  facet_grid(continent ~ year) +
  labs(
    title = "Life Expectancy and GDP across Continents and Years",
    x = "GDP per capita",
    y = "Life expectancy",
    caption = "Source: Gapminder."
  ) +
  theme_minimal()
```



Read `facet_grid(continent ~ year)` as “continent defines the rows, year defines the columns.”

5.9 Patchwork

Facets repeat the same plot structure across groups. Sometimes you want to combine different plots into one figure. The `patchwork` package lets you assemble separate `ggplot` objects.

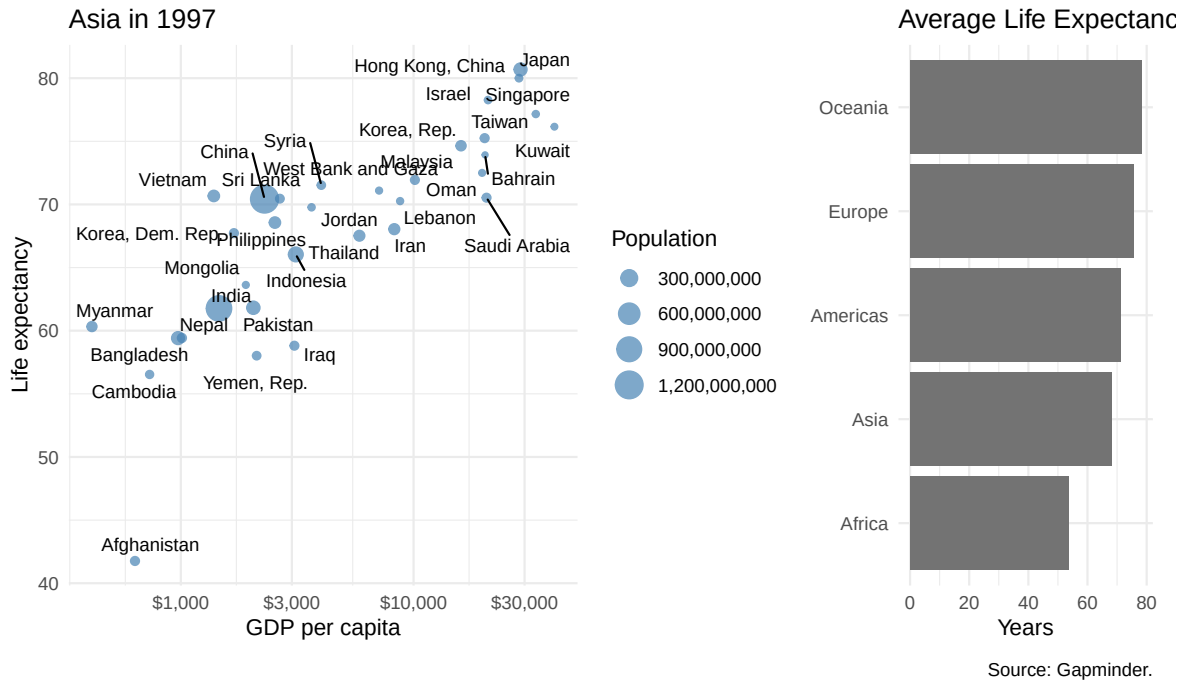
```
asia_scatter <- gap_1997 |>
  filter(continent == "Asia") |>
  ggplot(mapping = aes(x = gdpPercap, y = lifeExp)) +
  geom_point(aes(size = pop), color = "steelblue", alpha = 0.7) +
  geom_text_repel(aes(label = country), size = 3, max.overlaps = 12) +
  scale_x_log10(labels = dollar_format(accuracy = 1)) +
  scale_size_continuous(labels = comma) +
  labs(
    title = "Asia in 1997",
    x = "GDP per capita",
    y = "Life expectancy",
    size = "Population"
  ) +
  theme_minimal()

continent_bar <- gap_1997 |>
  group_by(continent) |>
  summarize(avg_life_exp = mean(lifeExp)) |>
  ggplot(mapping = aes(x = reorder(continent, avg_life_exp), y = avg_life_exp)) +
  geom_col(fill = "gray45") +
  coord_flip() +
  labs(
    title = "Average Life Expectancy",
    x = NULL,
    y = "Years"
  ) +
  theme_minimal()

asia_scatter + continent_bar +
  plot_layout(widths = c(2, 1)) +
  plot_annotation(
    title = "Two Views of Gapminder Data",
```

```
caption = "Source: Gapminder."
)
```

Two Views of Gapminder Data



Source: Gapminder.

5.10 Extra: State Policy Snapshot

State-level political and policy data work well for comparison examples. This section uses a small local file from the course `Data/` folder.

The inspection function `glimpse()` comes from `dplyr`, which is loaded as part of the `tidyverse`.

```
state_policy <- read_csv(
  "Data/state_policy/state_data_sorted.csv",
  show_col_types = FALSE
) |>
mutate(
  union_pct = 100 * union,
  religion_pct = 100 * religion,
  region = str_to_title(cregion)
)
```

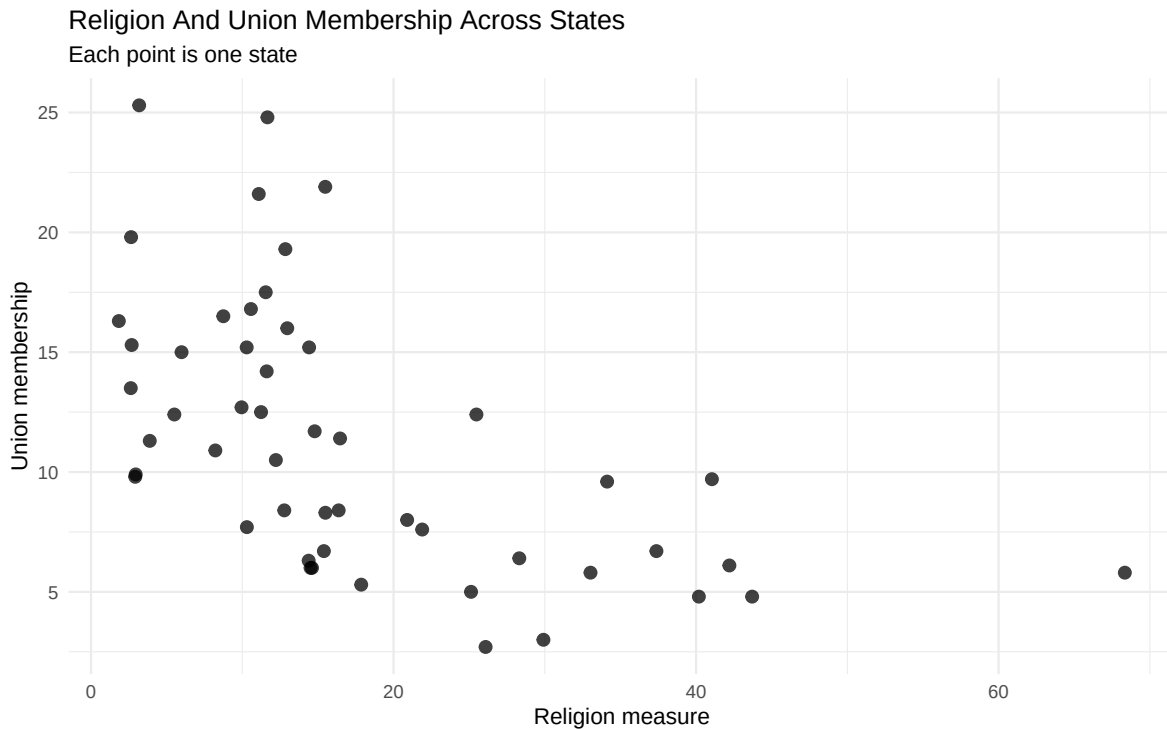


```
glimpse(state_policy)
```

```
Rows: 51
Columns: 11
$ abb      <chr> "AK", "AL", "AR", "AZ", "CA", "CO", "CT", "DC", "DE", "FL~
$ union    <dbl> 0.219, 0.097, 0.048, 0.063, 0.165, 0.084, 0.153, 0.127, 0~
$ religion  <dbl> 0.1548637, 0.4104603, 0.4370625, 0.1438850, 0.0874711, 0.~
$ cst      <chr> "AK", "AL", "AR", "AZ", "CA", "CO", "CT", "DC", "DE", "FL~
$ cregion  <chr> "west", "south", "south", "west", "west", "west", "northe~
$ region_num <dbl> 4, 3, 3, 4, 4, 4, 2, 2, 3, 3, 3, 4, 1, 4, 1, 1, 1, 3, 3, ~
$ state_num <dbl> 50, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 51, 11, 12, 13, 14, 15~
$ pop      <dbl> 626932, 4447100, 2673400, 5130632, 33871648, 4301261, 340~
$ union_pct <dbl> 21.9, 9.7, 4.8, 6.3, 16.5, 8.4, 15.3, 12.7, 12.4, 6.0, 6.~
$ religion_pct <dbl> 15.48637, 41.04603, 43.70625, 14.38850, 8.74711, 12.77672~
$ region    <chr> "West", "South", "South", "West", "West", "West", "Northe~
```

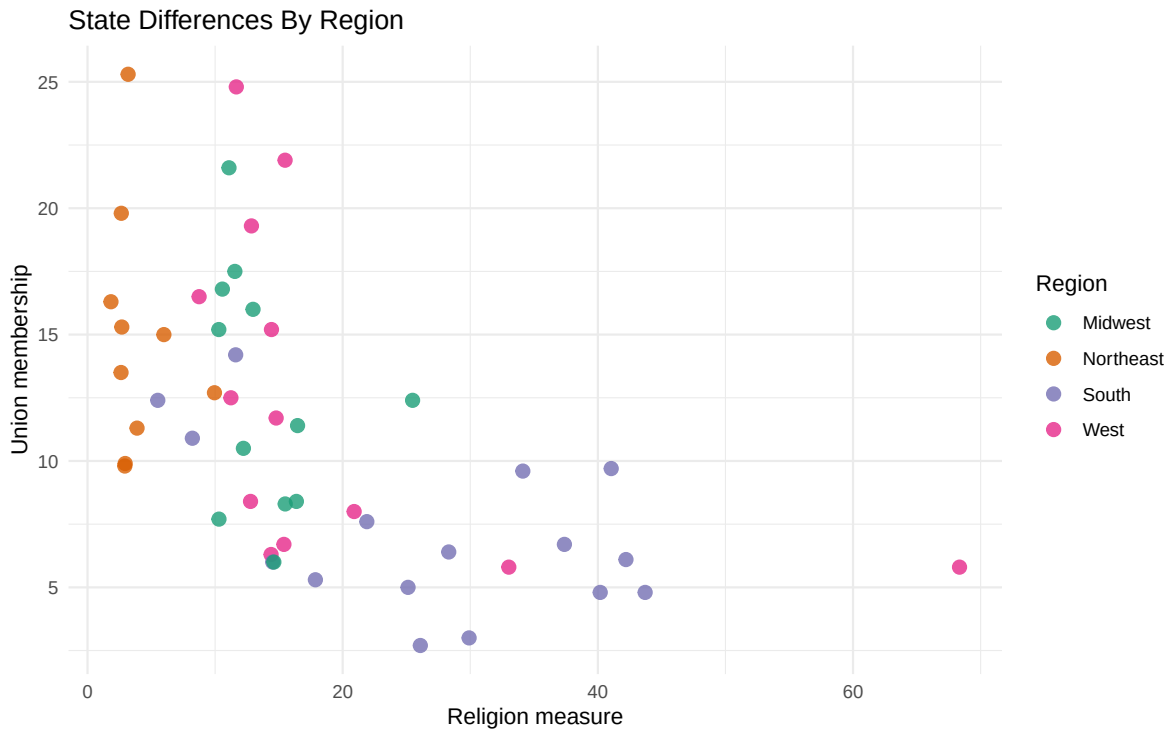
The two variables in the next plot are percentages. The point is not to make a causal claim. The point is to practice separating and comparing state-level observations.

```
ggplot(
  state_policy,
  aes(x = religion_pct, y = union_pct)
) +
  geom_point(size = 2.5, alpha = 0.75) +
  labs(
    title = "Religion And Union Membership Across States",
    subtitle = "Each point is one state",
    x = "Religion measure",
    y = "Union membership"
  ) +
  theme_minimal()
```



Color can separate regions without creating multiple panels. This is useful when the plot is still readable with all observations in one place.

```
ggplot(
  state_policy,
  aes(x = religion_pct, y = union_pct, color = region)
) +
  geom_point(size = 2.8, alpha = 0.8) +
  scale_color_brewer(palette = "Dark2") +
  labs(
    title = "State Differences By Region",
    x = "Religion measure",
    y = "Union membership",
    color = "Region"
  ) +
  theme_minimal()
```

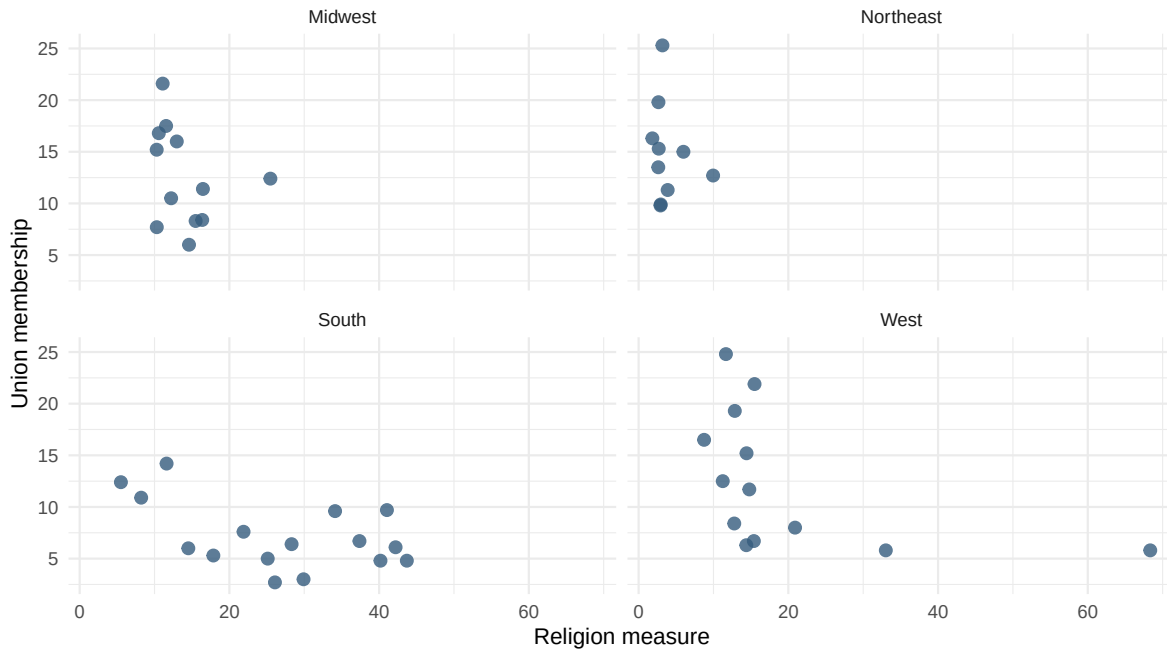


Facets split the same data into small multiples. This makes regional comparisons clearer when the color version starts to feel crowded.

```
ggplot(
  state_policy,
  aes(x = religion_pct, y = union_pct)
) +
  geom_point(color = "#355c7d", size = 2.5, alpha = 0.8) +
  facet_wrap(~ region) +
  labs(
    title = "State Differences By Region",
    subtitle = "Facets separate the same scatterplot into regional panels",
    x = "Religion measure",
    y = "Union membership"
  ) +
  theme_minimal()
```

State Differences By Region

Facets separate the same scatterplot into regional panels

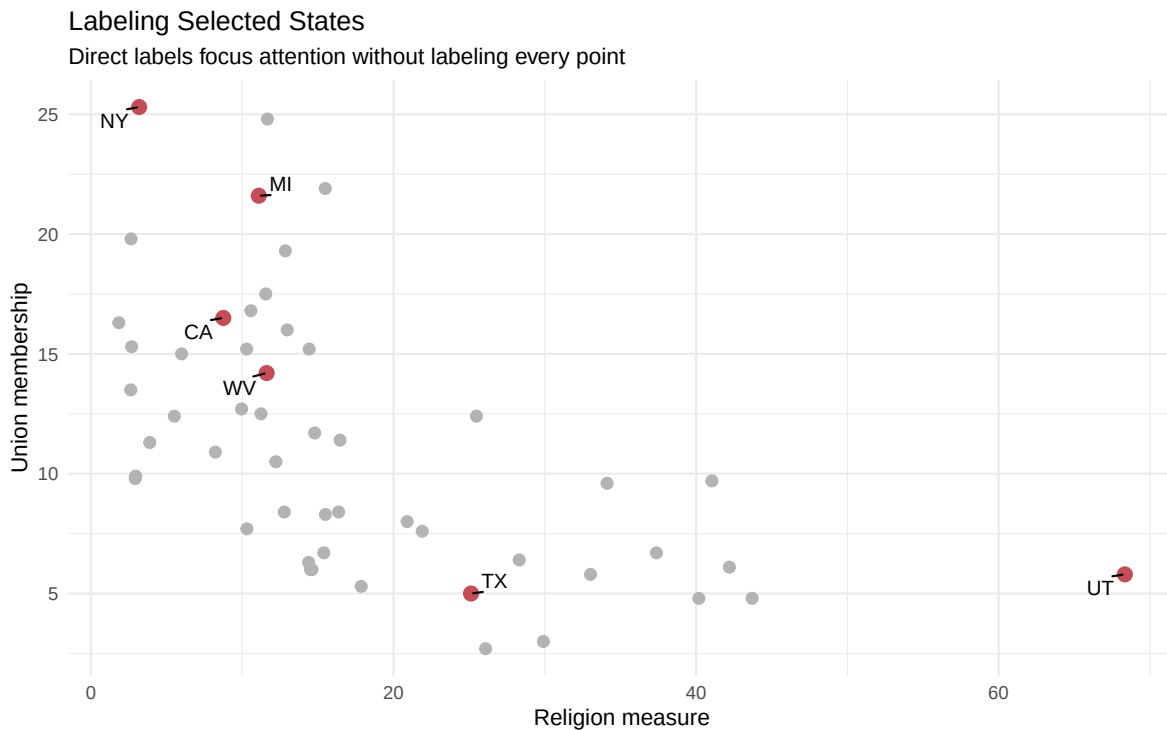


Direct labels are useful when a few cases matter. Labeling every state would crowd the figure, so the next example labels only selected states.

```
label_states <- c("CA", "MI", "NY", "TX", "UT", "WV")

ggplot(
  state_policy,
  aes(x = religion_pct, y = union_pct)
) +
  geom_point(color = "gray70", size = 2.3) +
  geom_point(
    data = filter(state_policy, abb %in% label_states),
    color = "#c44d58",
    size = 3
  ) +
  geom_text_repel(
    data = filter(state_policy, abb %in% label_states),
    aes(label = abb),
    size = 3.5,
    min.segment.length = 0
  ) +
```

```
labs(
  title = "Labeling Selected States",
  subtitle = "Direct labels focus attention without labeling every point",
  x = "Religion measure",
  y = "Union membership"
) +
theme_minimal()
```



5.11 Exercise

Use `gapminder` to make a comparison plot for 2007.

1. Filter the data to `year == 2007`.
2. Make a scatterplot of GDP per capita and life expectancy.
3. Separate continents using color.
4. Make a second version using `facet_wrap(~ continent)`.
5. Decide which version makes the comparison clearer and write one sentence explaining why.

6 Distributions

6.1 Why distributions matter

A distribution shows how values of a variable are spread out. Before comparing means, fitting models, or making polished charts, it is often useful to ask simpler questions:

- What values are common?
- Are there extreme values?
- Is the variable symmetric or skewed?
- Do groups have similar or different spreads?
- Are there gaps, clusters, or unusual cases?

Distribution plots are especially useful during exploratory data analysis.

6.2 Histograms

A histogram divides a continuous variable into bins and counts how many observations fall in each bin.

The `midwest` dataset is included with `ggplot2`. It has one row per county in five Midwestern states.

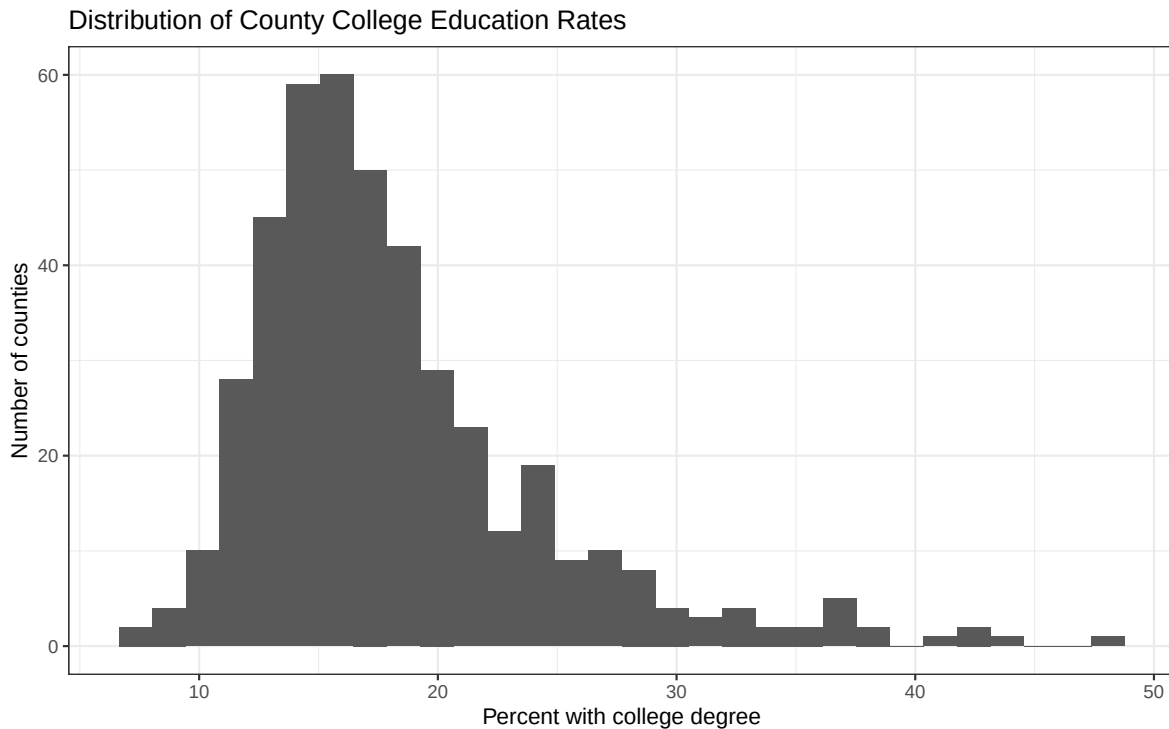
```
midwest |>
  select(state, county, percollege, poptotal, area) |>
  head(10)
```

```
# A tibble: 10 x 5
   state county    percollege poptotal  area
   <chr> <chr>         <dbl>    <int> <dbl>
1 IL    ADAMS         19.6     66090 0.052
2 IL    ALEXANDER      11.2     10626 0.014
3 IL    BOND           17.0     14991 0.022
4 IL    BOONE          17.3     30806 0.017
5 IL    BROWN         14.5      5836 0.018
```

6	IL	BUREAU	18.9	35688	0.05
7	IL	CALHOUN	11.9	5322	0.017
8	IL	CARROLL	16.2	16805	0.027
9	IL	CASS	14.1	13437	0.024
10	IL	CHAMPAIGN	41.3	173025	0.058

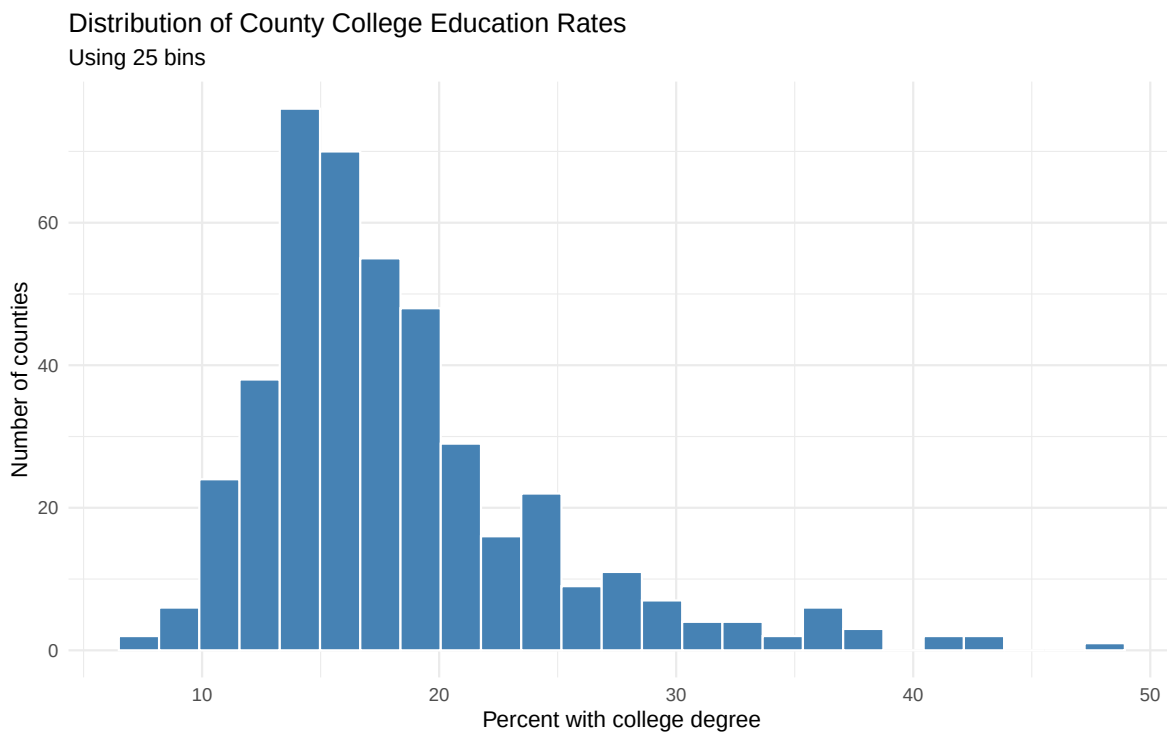
We will start with `percollege`, the percentage of adults in each county with a college degree.

```
ggplot(
  data = midwest |> filter(!is.na(percollege)),
  mapping = aes(x = percollege)
) +
  geom_histogram() +
  labs(
    title = "Distribution of County College Education Rates",
    x = "Percent with college degree",
    y = "Number of counties"
  ) +
  theme_bw()
```



When we do not set the number of bins, `ggplot2` chooses a default. That default is not always the best choice, so it is worth trying several values.

```
ggplot(
  data = midwest |> filter(!is.na(percollege)),
  mapping = aes(x = percollege)
) +
  geom_histogram(bins = 25, fill = "steelblue", color = "white") +
  labs(
    title = "Distribution of County College Education Rates",
    subtitle = "Using 25 bins",
    x = "Percent with college degree",
    y = "Number of counties"
  ) +
  theme_minimal()
```



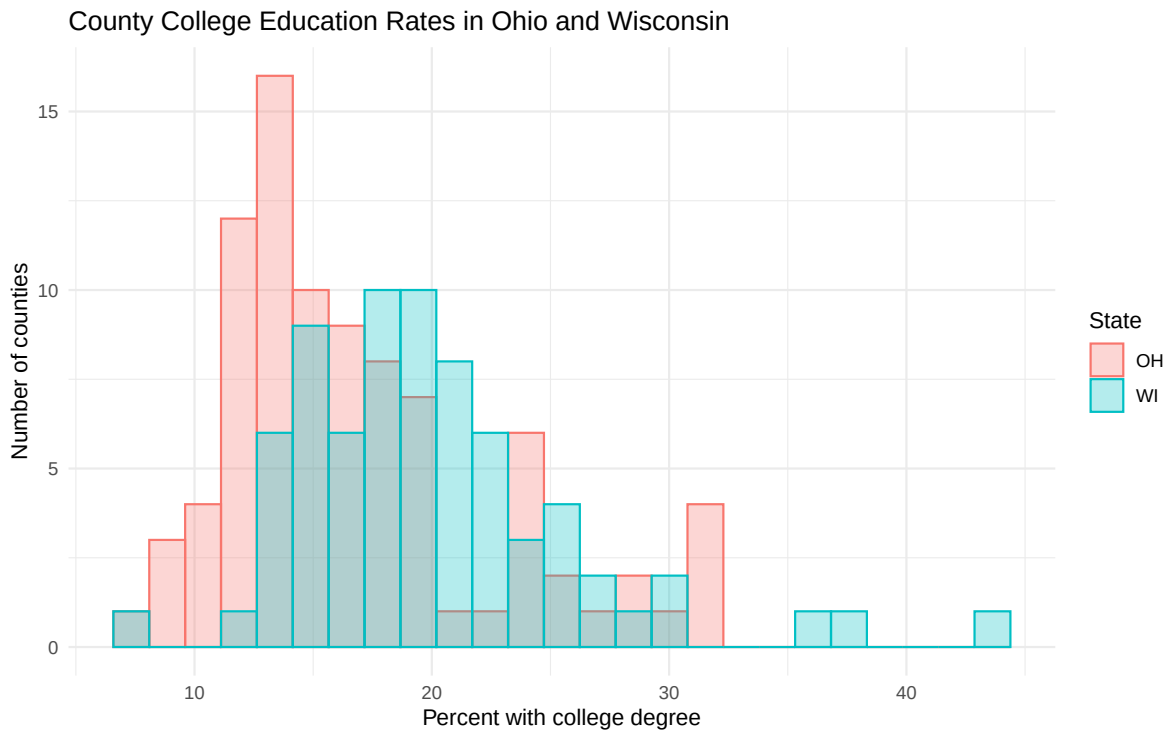
Changing the number of bins changes the visual summary. Very few bins can hide structure. Too many bins can make the plot look noisy. Histograms are useful for seeing skew, clustering, and unusual tails.

6.3 Comparing histograms

We can compare distributions across groups by mapping a group to fill and using transparency.

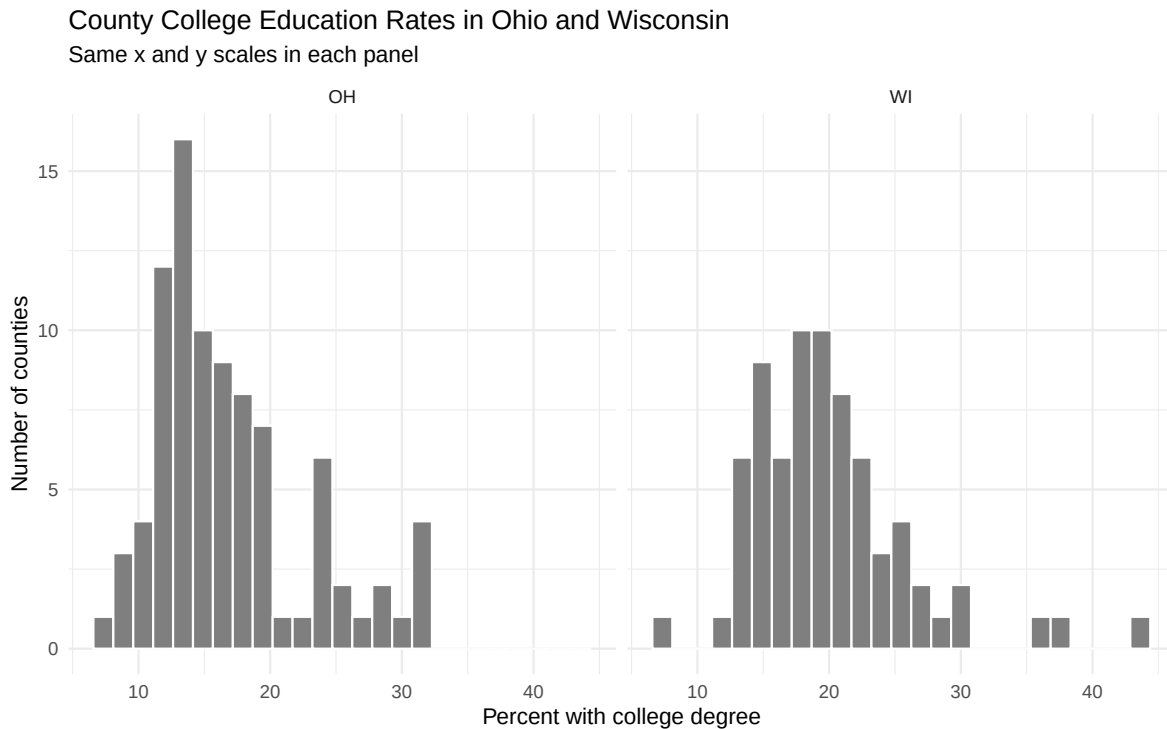
```
oh_wi <- midwest |>
  filter(state %in% c("OH", "WI"), !is.na(percollege))

ggplot(data = oh_wi, mapping = aes(x = percollege, fill = state, color = state)) +
  geom_histogram(alpha = 0.3, bins = 25, position = "identity") +
  labs(
    title = "County College Education Rates in Ohio and Wisconsin",
    x = "Percent with college degree",
    y = "Number of counties",
    fill = "State",
    color = "State"
  ) +
  theme_minimal()
```



Overlaid histograms are compact, but they can be difficult to read when the groups overlap. Small multiples are often clearer.

```
ggplot(data = oh_wi, mapping = aes(x = percollege)) +
  geom_histogram(bins = 25, fill = "gray50", color = "white") +
  facet_wrap(~ state) +
  labs(
    title = "County College Education Rates in Ohio and Wisconsin",
    subtitle = "Same x and y scales in each panel",
    x = "Percent with college degree",
    y = "Number of counties"
  ) +
  theme_minimal()
```

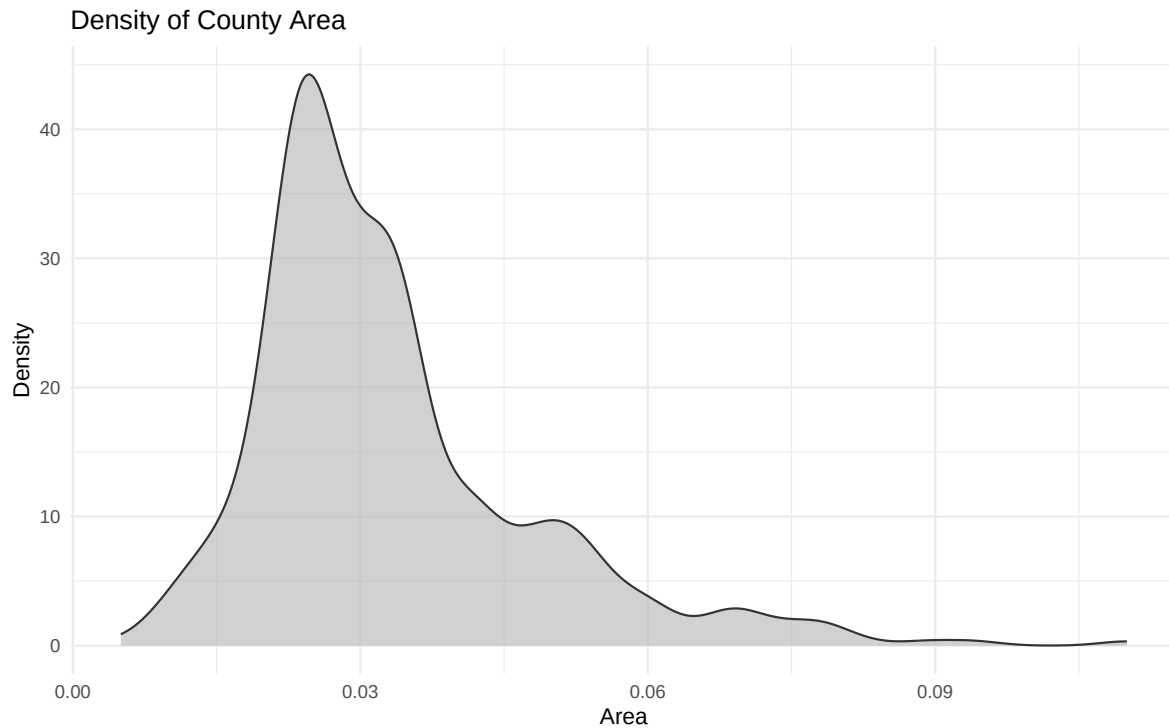


Because the panels share scales by default, the two histograms can be compared directly.

6.4 Density plots

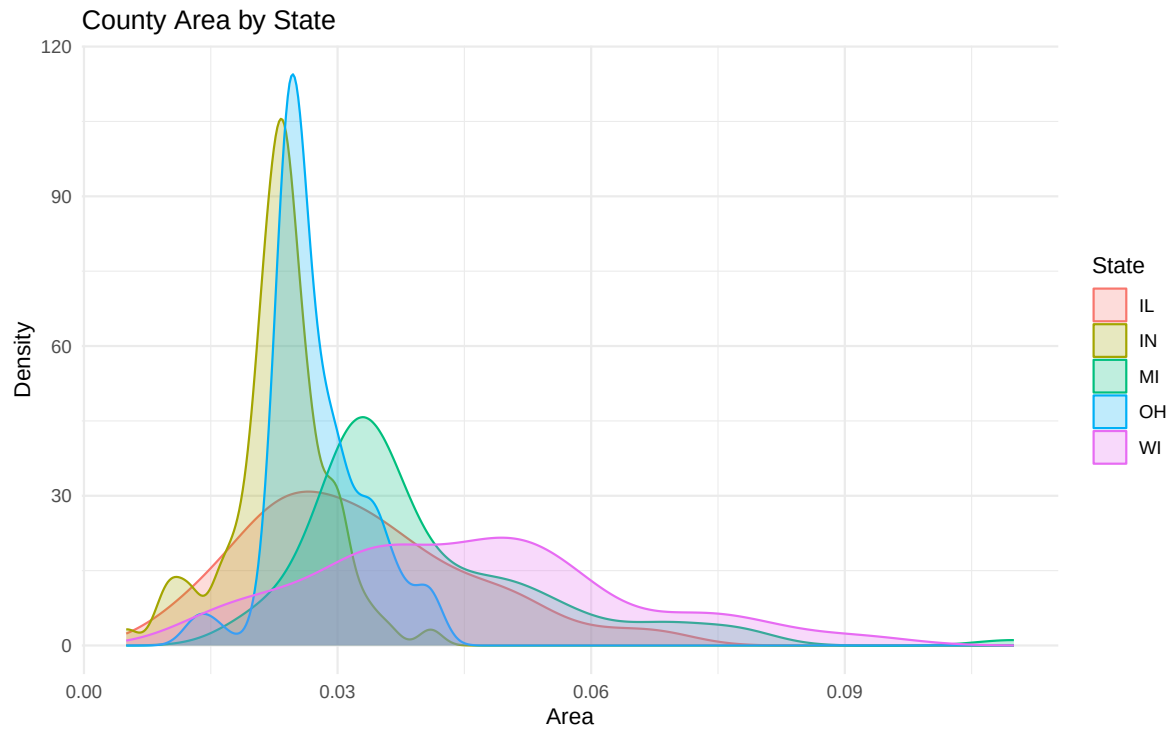
A density plot is a smoothed version of a distribution. Instead of showing counts in bins, it estimates the shape of the distribution.

```
ggplot(data = midwest, mapping = aes(x = area)) +
  geom_density(fill = "gray70", color = "gray20", alpha = 0.6) +
  labs(
    title = "Density of County Area",
    x = "Area",
    y = "Density"
  ) +
  theme_minimal()
```



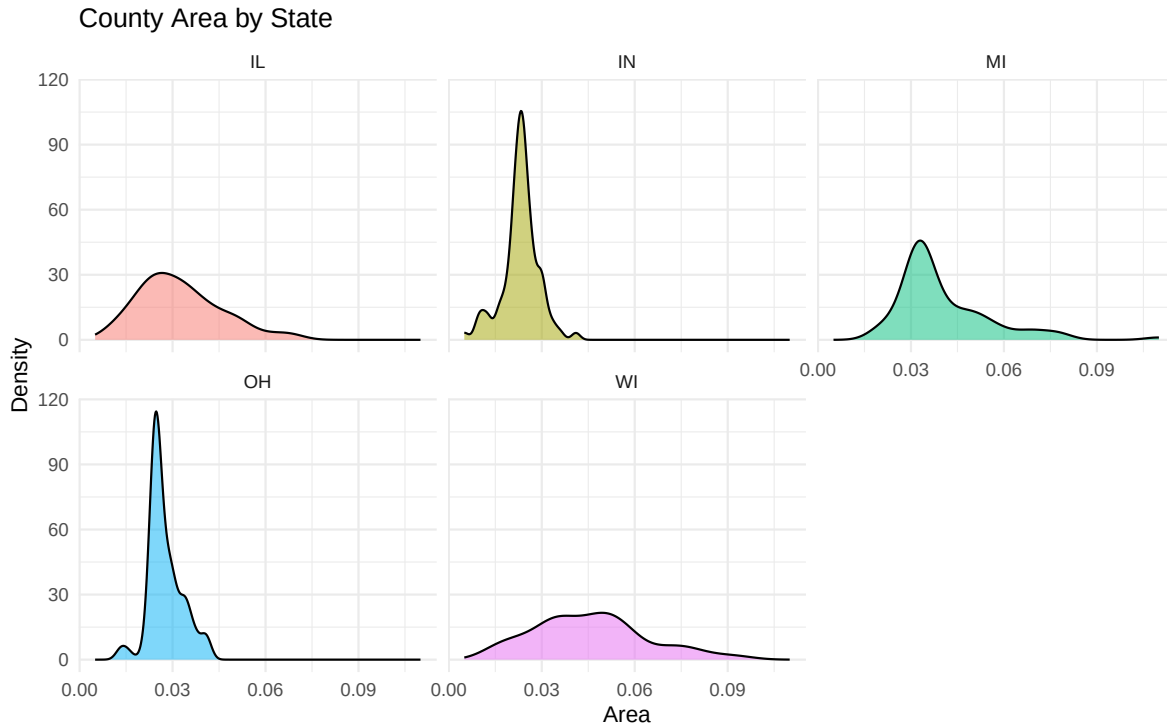
Density plots are useful for comparing shapes across groups.

```
ggplot(data = midwest, mapping = aes(x = area, fill = state, color = state)) +
  geom_density(alpha = 0.25) +
  labs(
    title = "County Area by State",
    x = "Area",
    y = "Density",
    fill = "State",
    color = "State"
  ) +
  theme_minimal()
```



When there are several groups, faceting can again reduce clutter.

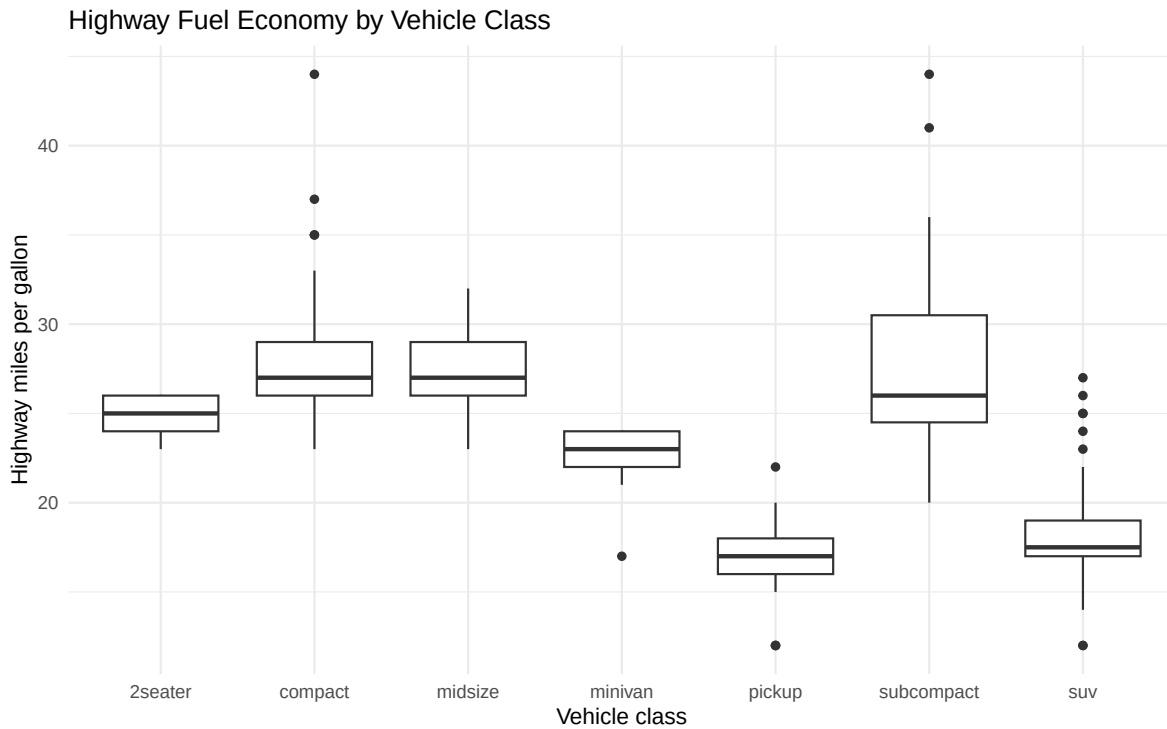
```
ggplot(data = midwest, mapping = aes(x = area, fill = state)) +  
  geom_density(alpha = 0.5, show.legend = FALSE) +  
  facet_wrap(~ state) +  
  labs(  
    title = "County Area by State",  
    x = "Area",  
    y = "Density"  
  ) +  
  theme_minimal()
```



6.5 Boxplots

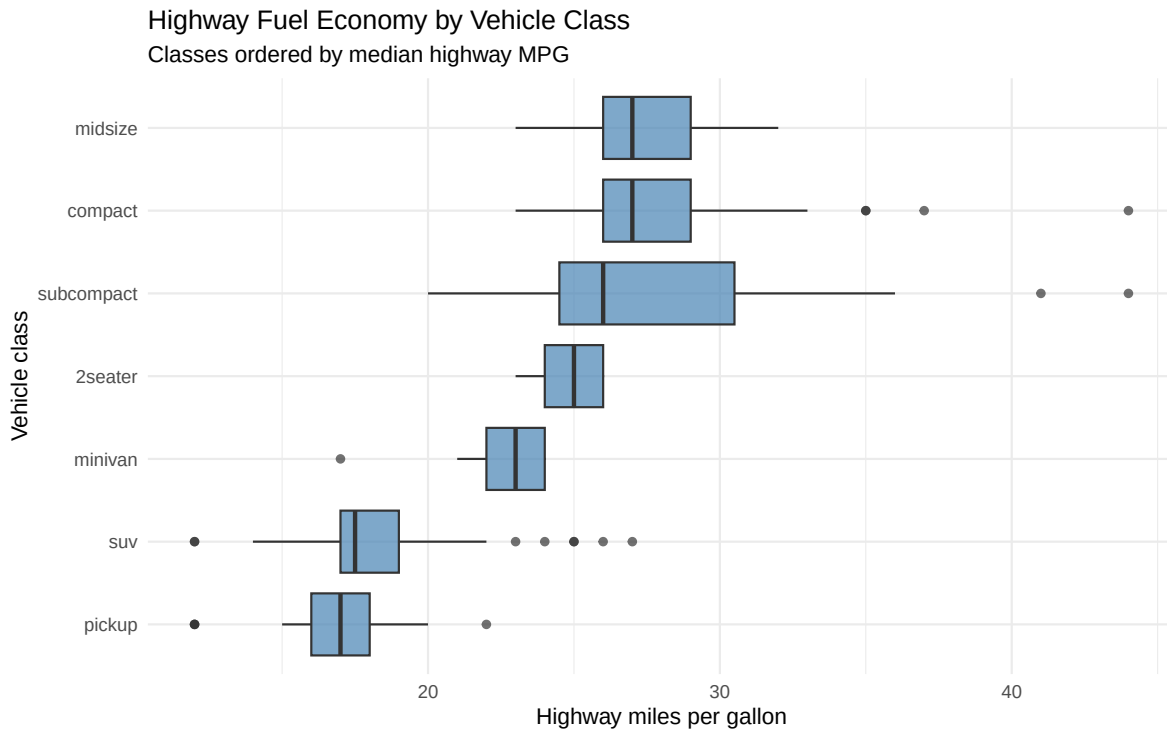
Boxplots summarize a distribution with a median, interquartile range, whiskers, and outliers. They are compact and good for comparing many groups.

```
ggplot(data = mpg, mapping = aes(x = class, y = hwy)) +
  geom_boxplot() +
  labs(
    title = "Highway Fuel Economy by Vehicle Class",
    x = "Vehicle class",
    y = "Highway miles per gallon"
  ) +
  theme_minimal()
```



The x-axis is crowded. We can reorder vehicle classes by median highway MPG and flip the coordinates.

```
ggplot(data = mpg, mapping = aes(x = reorder(class, hwy, FUN = median), y = hwy)) +
  geom_boxplot(fill = "steelblue", alpha = 0.7) +
  coord_flip() +
  labs(
    title = "Highway Fuel Economy by Vehicle Class",
    subtitle = "Classes ordered by median highway MPG",
    x = "Vehicle class",
    y = "Highway miles per gallon"
  ) +
  theme_minimal()
```

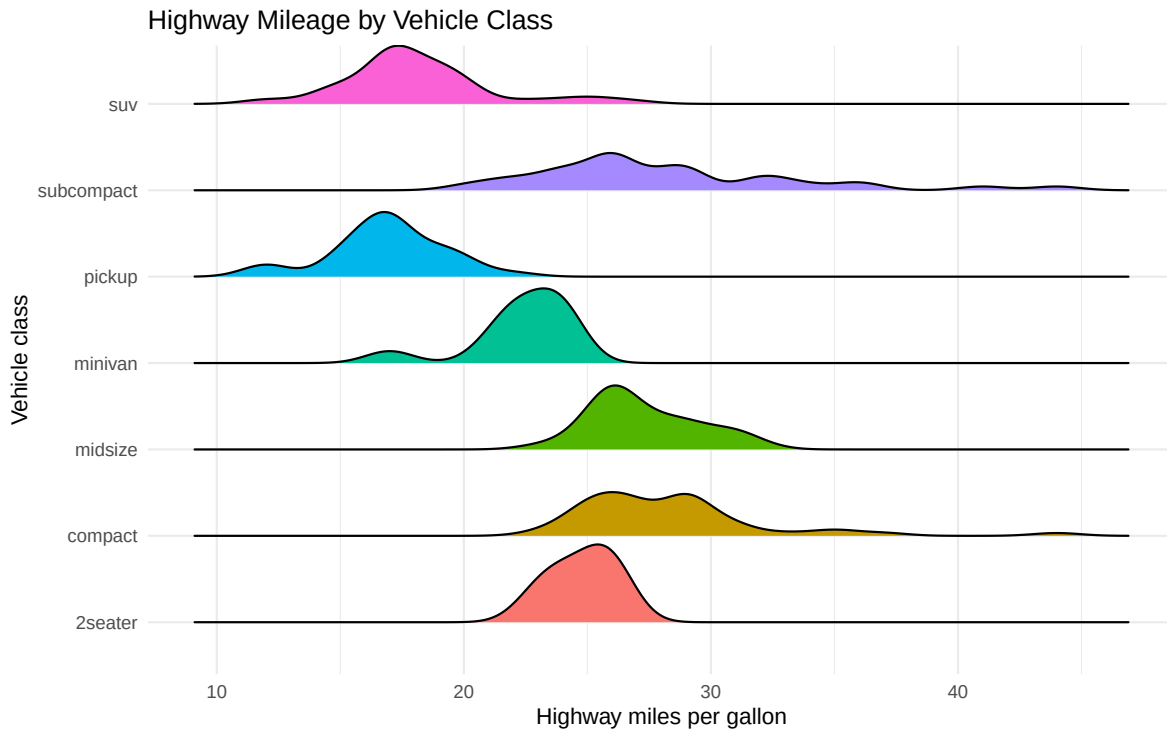


Boxplots hide individual observations, but they make the comparison of medians and spreads easy.

6.6 Ridge plots

Ridge plots stack density plots vertically. They are useful when you want to compare many distributions and still see the shape of each one. They use `geom_density_ridges()` from the `ggridges` package.

```
ggplot(mpg, aes(x = hwy, y = class, fill = class)) +
  ggridges::geom_density_ridges(scale = 0.9, show.legend = FALSE) +
  labs(
    title = "Highway Mileage by Vehicle Class",
    x = "Highway miles per gallon",
    y = "Vehicle class"
  ) +
  theme_minimal()
```



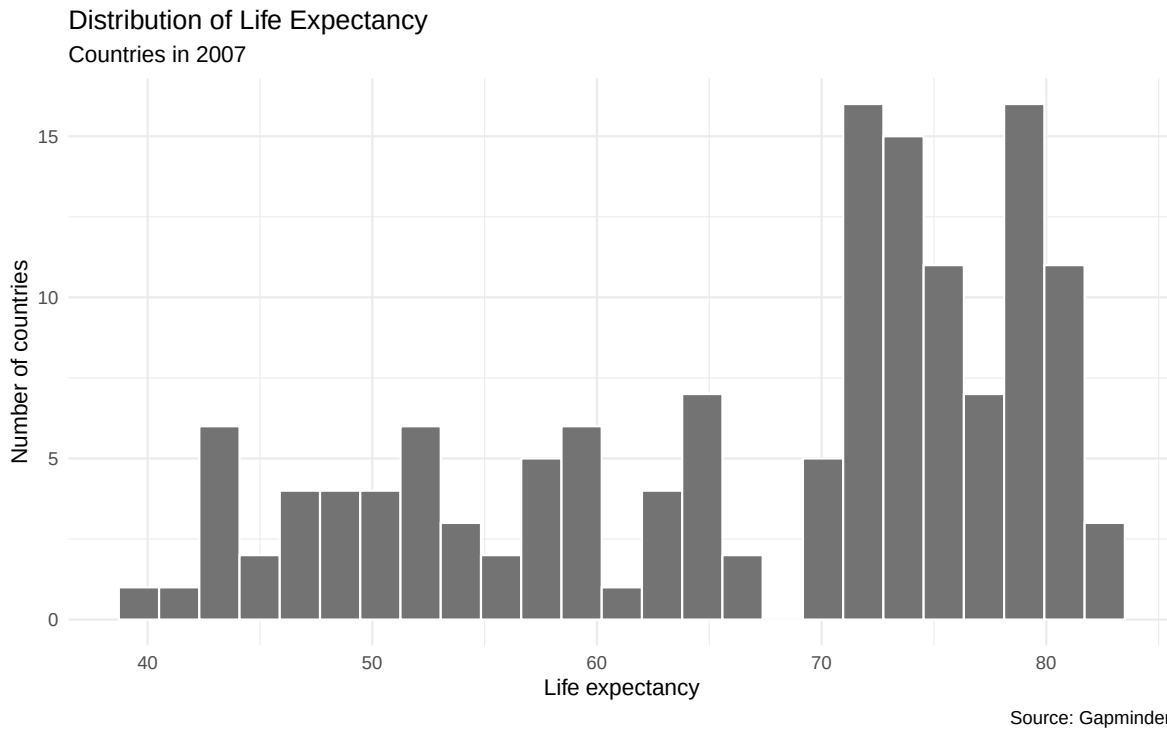
Ridge plots work best when there are enough categories to make a single boxplot crowded, but few enough that each density still has room to be readable.

6.7 Distributions in Gapminder

Distribution plots can also help before making cross-national comparisons.

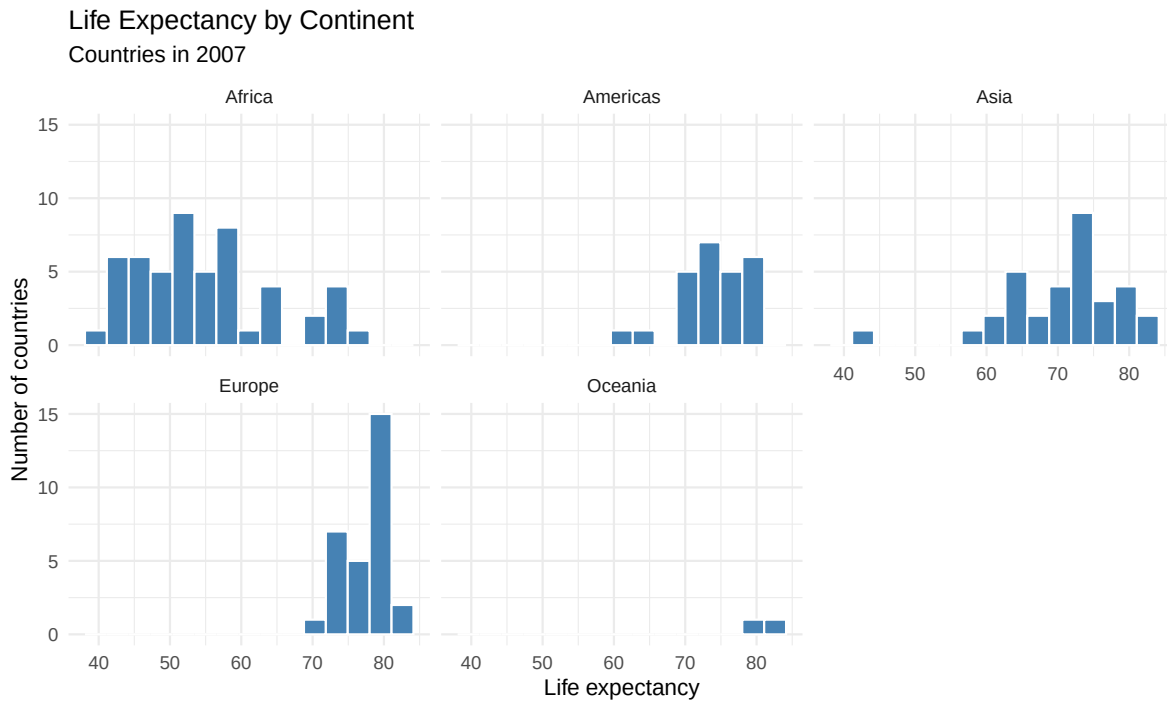
```
gap_2007 <- gapminder |>
  filter(year == 2007)

ggplot(data = gap_2007, mapping = aes(x = lifeExp)) +
  geom_histogram(bins = 25, fill = "gray45", color = "white") +
  labs(
    title = "Distribution of Life Expectancy",
    subtitle = "Countries in 2007",
    x = "Life expectancy",
    y = "Number of countries",
    caption = "Source: Gapminder."
  ) +
  theme_minimal()
```

Now separate by continent. Faceted histograms keep the same geom but split the data into panels.

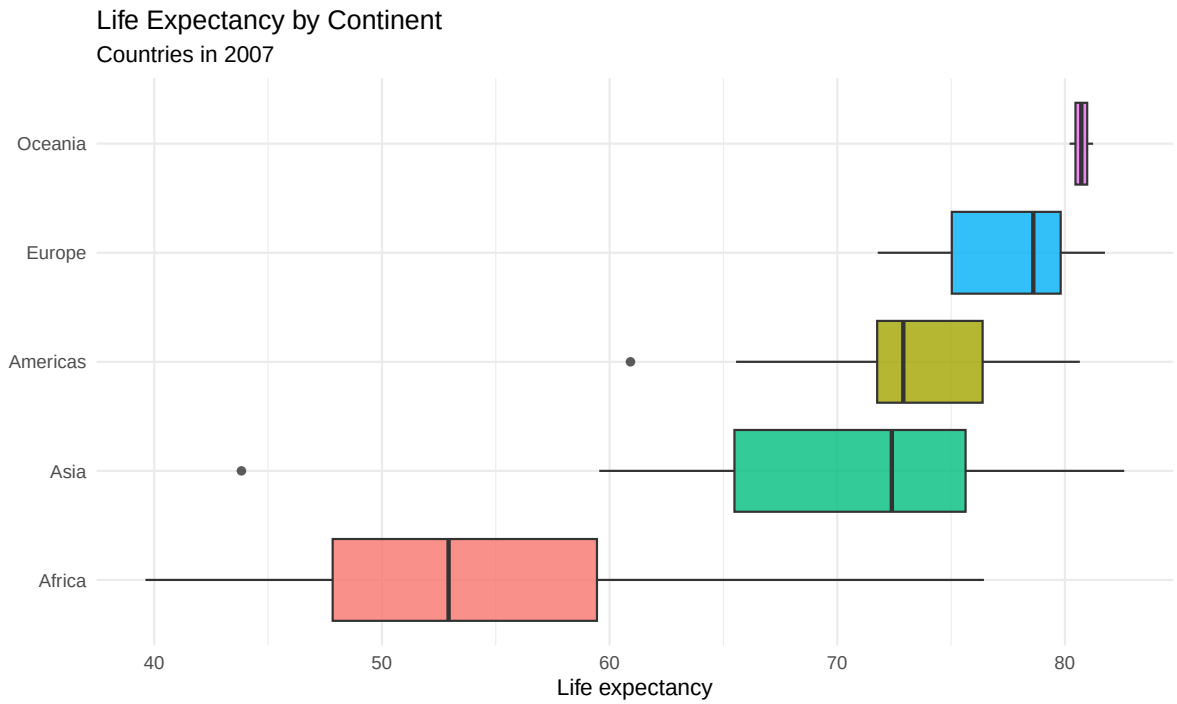
```
ggplot(data = gap_2007, mapping = aes(x = lifeExp)) +  
  geom_histogram(bins = 15, fill = "steelblue", color = "white") +  
  facet_wrap(~ continent) +  
  labs(  
    title = "Life Expectancy by Continent",  
    subtitle = "Countries in 2007",  
    x = "Life expectancy",  
    y = "Number of countries",  
    caption = "Source: Gapminder."  
  ) +  
  theme_minimal()
```



Source: Gapminder.

A boxplot gives a more compact comparison.

```
ggplot(data = gap_2007, mapping = aes(x = reorder(continent, lifeExp, FUN = median), y = lifeExp)) +
  geom_boxplot(aes(fill = continent), show.legend = FALSE, alpha = 0.8) +
  coord_flip() +
  labs(
    title = "Life Expectancy by Continent",
    subtitle = "Countries in 2007",
    x = NULL,
    y = "Life expectancy",
    caption = "Source: Gapminder."
  ) +
  theme_minimal()
```



6.8 Choosing among histograms, densities, and boxplots

Use a histogram when you want to show counts and binning is meaningful.

Use a density plot when you want to compare smooth distribution shapes.

Use a boxplot when you want compact comparisons across many groups.

No single distribution plot is always best. It is normal to make several versions while exploring the data.

6.9 Exercise 1: histogram

Use the built-in `iris` dataset.

1. Create a histogram of `Sepal.Length`.
2. Try `bins = 10`, `bins = 30`, and `bins = 50`.
3. Add a title and readable axis labels.
4. Write one sentence explaining which bin choice is easiest to interpret.

```
data(iris)
```

```
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

```
# Write your plot here.
```

6.10 Exercise 2: density plot

Use the built-in diamonds dataset.

1. Create a density plot of `carat`.
2. Map `cut` to fill.
3. Use `alpha` so the overlapping densities remain visible.
4. Try a faceted version with `facet_wrap(~ cut)`.

```
diamonds |>  
  select(carat, cut, price) |>  
  head(10)
```

```
# A tibble: 10 x 3  
  carat cut      price  
  <dbl> <ord>    <int>  
1  0.23 Ideal      326  
2  0.21 Premium    326  
3  0.23 Good       327  
4  0.29 Premium    334  
5  0.31 Good       335  
6  0.24 Very Good  336  
7  0.24 Very Good  336  
8  0.26 Very Good  337  
9  0.22 Fair       337  
10 0.23 Very Good  338
```

```
# Write your plot here.
```

6.11 Exercise 3: boxplot

Use the built-in `mpg` dataset.

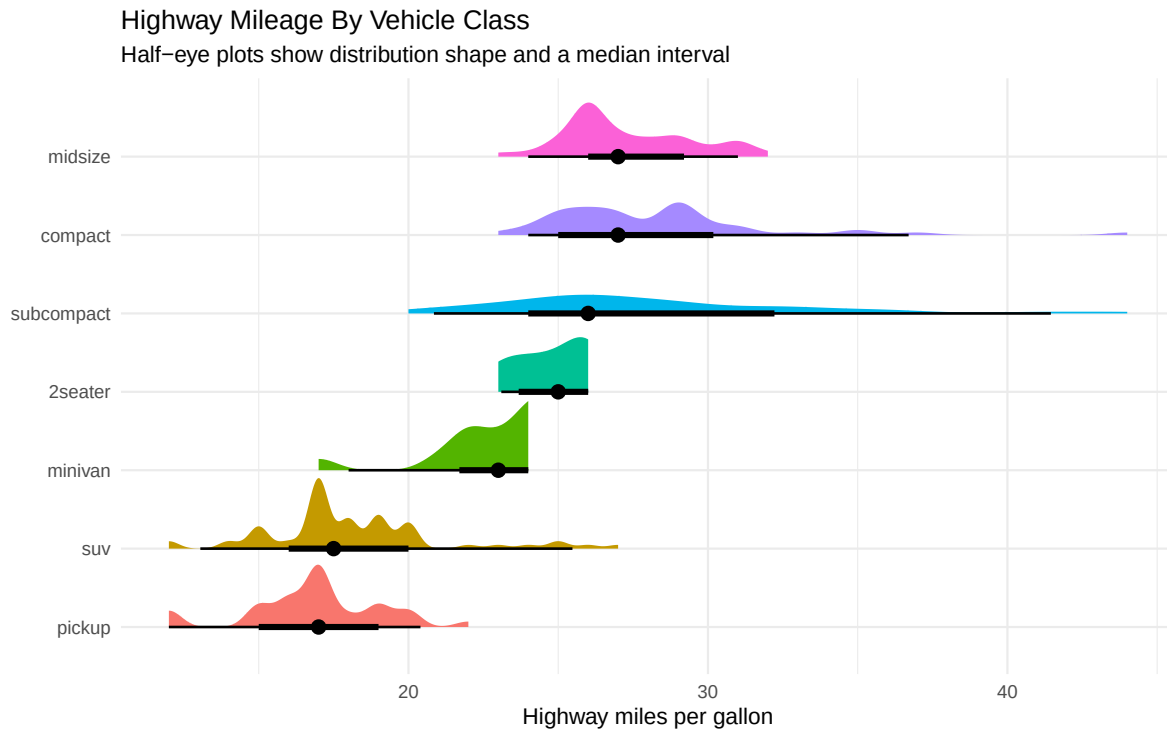
1. Compare `hwy` across vehicle `class`.
2. Reorder the classes by median `hwy`.
3. Flip the coordinates if the labels are hard to read.

```
# Write your plot here.
```

6.12 Extra: distribution intervals with `ggdist`

The `ggdist` package adds distribution geoms that combine several ideas at once. A half-eye plot shows the shape of a distribution while also marking a summary interval. It is useful when a boxplot feels too compressed but a full density plot takes too much space.

```
if (requireNamespace("ggdist", quietly = TRUE)) {  
  mpg |>  
    mutate(class = fct_reorder(class, hwy, .fun = median)) |>  
    ggplot(aes(x = hwy, y = class, fill = class)) +  
    ggdist::stat_halfeye(  
      adjust = 0.7,  
      width = 0.65,  
      point_interval = ggdist::median_qi,  
      show.legend = FALSE  
    ) +  
    labs(  
      title = "Highway Mileage By Vehicle Class",  
      subtitle = "Half-eye plots show distribution shape and a median interval",  
      x = "Highway miles per gallon",  
      y = NULL  
    ) +  
    theme_minimal()  
}
```

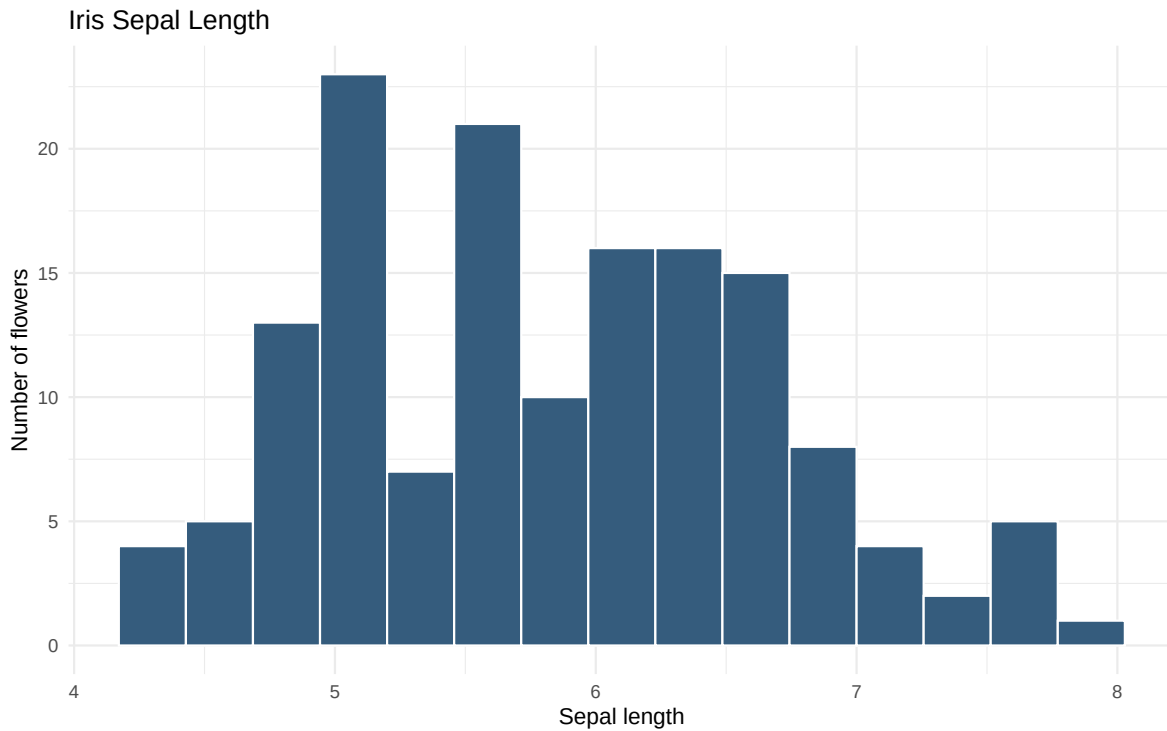


This is not a replacement for histograms, densities, or boxplots. It is another option when the goal is to show both distribution shape and uncertainty or spread in a compact display.

6.13 Extra: worked exercise variants

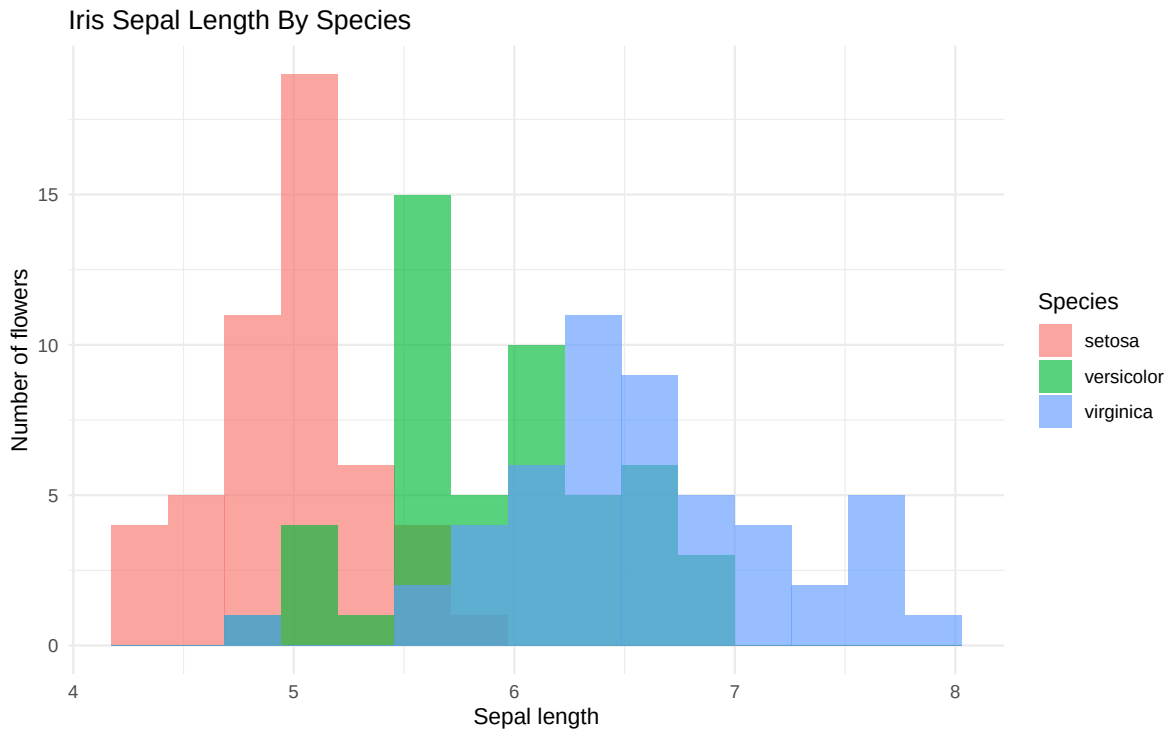
The next three chunks are short exercise-style examples. Each uses a built-in dataset and changes only one or two plotting decisions at a time.

```
iris |>
  ggplot(aes(x = Sepal.Length)) +
  geom_histogram(bins = 15, fill = "#355c7d", color = "white") +
  labs(
    title = "Iris Sepal Length",
    x = "Sepal length",
    y = "Number of flowers"
  ) +
  theme_minimal()
```



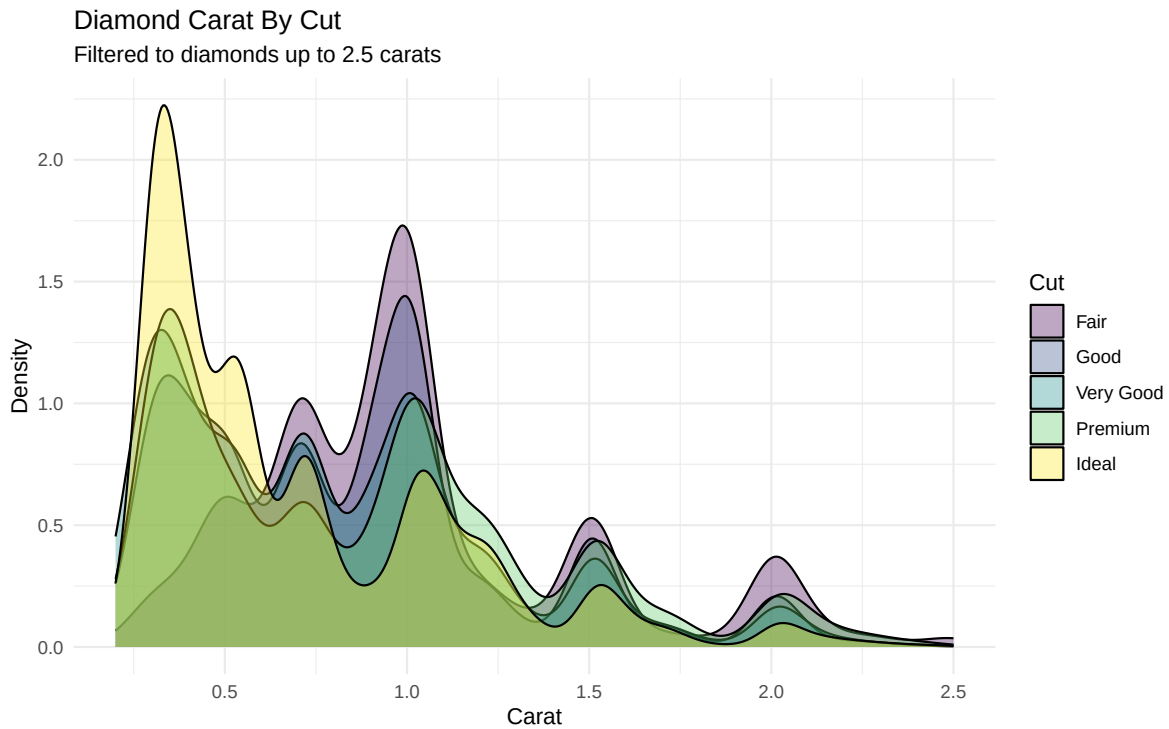
The same distribution can be separated by group. Here, `fill = Species` separates the histogram by species, while `alpha` makes overlapping bars partly transparent.

```
iris |>
  ggplot(aes(x = Sepal.Length, fill = Species)) +
  geom_histogram(bins = 15, alpha = 0.65, position = "identity") +
  labs(
    title = "Iris Sepal Length By Species",
    x = "Sepal length",
    y = "Number of flowers",
    fill = "Species"
  ) +
  theme_minimal()
```



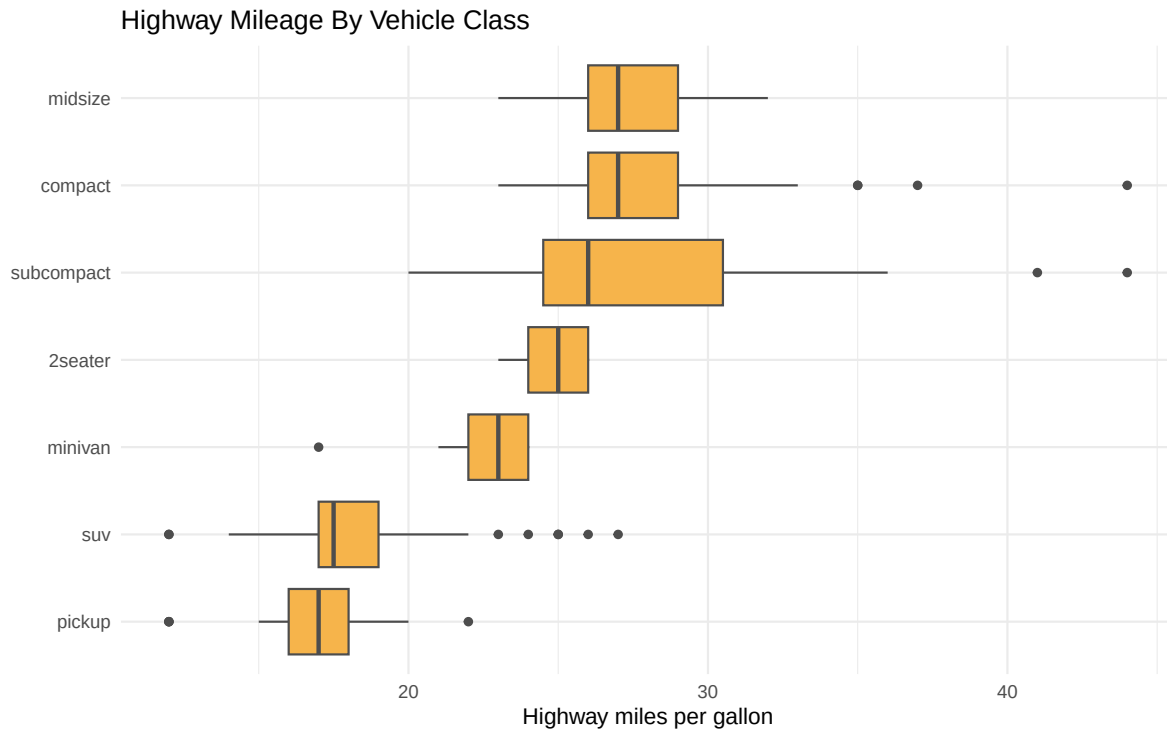
The `diamonds` dataset is larger, so density plots are often easier to compare than overlapping histograms.

```
diamonds |>
  filter(carat <= 2.5) |>
  ggplot(aes(x = carat, fill = cut)) +
  geom_density(alpha = 0.35) +
  labs(
    title = "Diamond Carat By Cut",
    subtitle = "Filtered to diamonds up to 2.5 carats",
    x = "Carat",
    y = "Density",
    fill = "Cut"
  ) +
  theme_minimal()
```

Boxplots are useful when the goal is a compact comparison of medians and spread across categories.

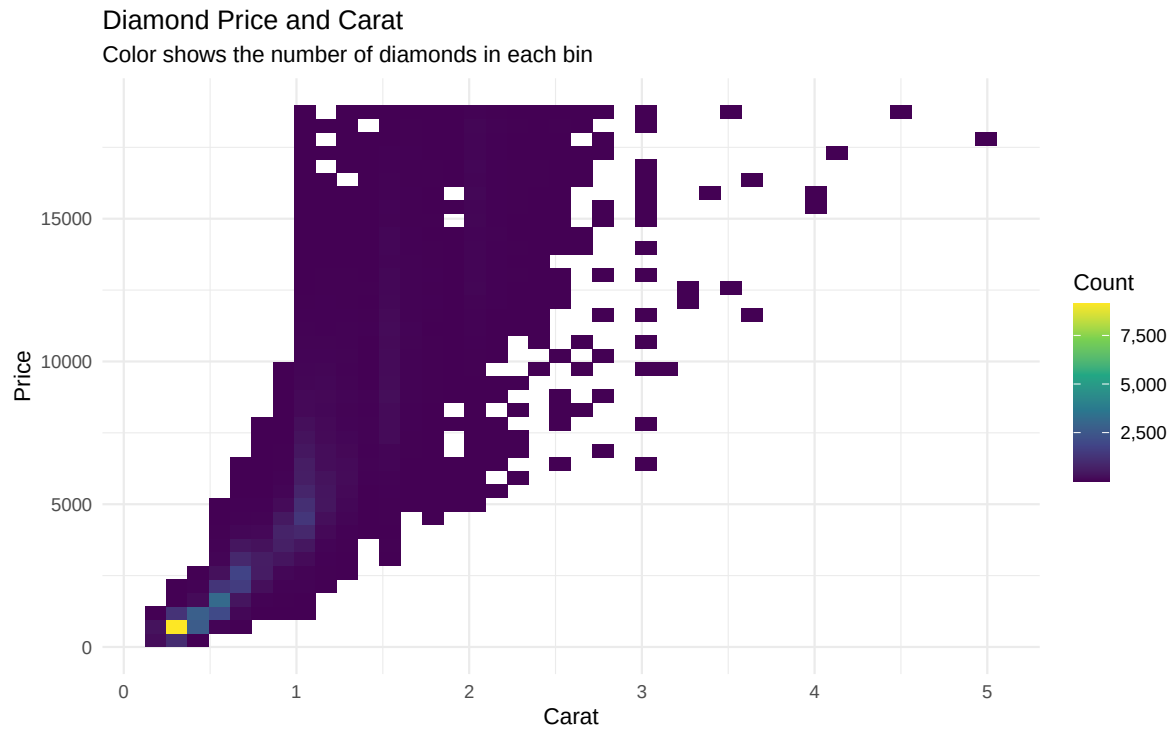
```
mpg |>
  ggplot(aes(x = fct_reorder(class, hwy, .fun = median), y = hwy)) +
  geom_boxplot(fill = "#f6b44b", color = "gray30") +
  coord_flip() +
  labs(
    title = "Highway Mileage By Vehicle Class",
    x = NULL,
    y = "Highway miles per gallon"
  ) +
  theme_minimal()
```



6.14 Extra: heatmaps for dense two-variable distributions

When there are too many points, a scatterplot can turn into a cloud of overplotting. A two-dimensional bin plot counts observations inside rectangular bins.

```
ggplot(data = diamonds, mapping = aes(x = carat, y = price)) +
  geom_bin2d(bins = 40) +
  scale_fill_viridis_c(labels = comma) +
  labs(
    title = "Diamond Price and Carat",
    subtitle = "Color shows the number of diamonds in each bin",
    x = "Carat",
    y = "Price",
    fill = "Count"
  ) +
  theme_minimal()
```



This is a distribution plot for two variables at once. It is useful when individual points would overlap too much.

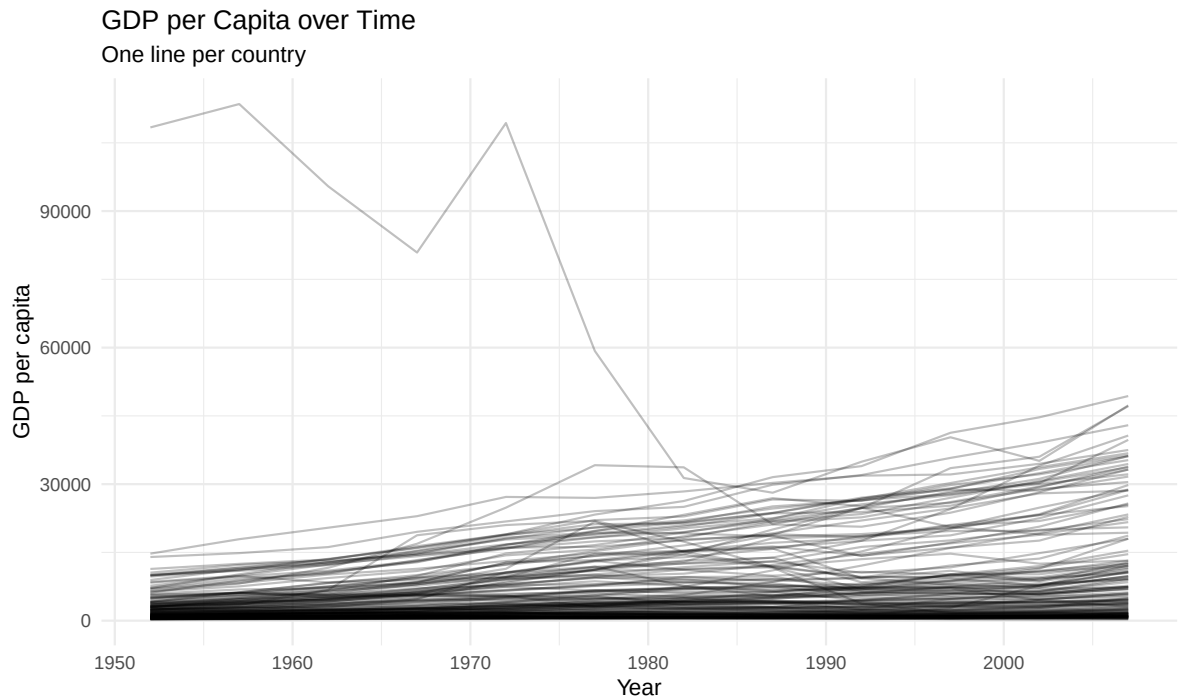
7 Lines, Bars, and Annotation

7.1 Line charts

Line charts are designed for ordered data, especially time. They work best when the x-axis has a meaningful sequence and the line connects observations that should be read as a path.

We will begin with GDP per capita over time in Gapminder.

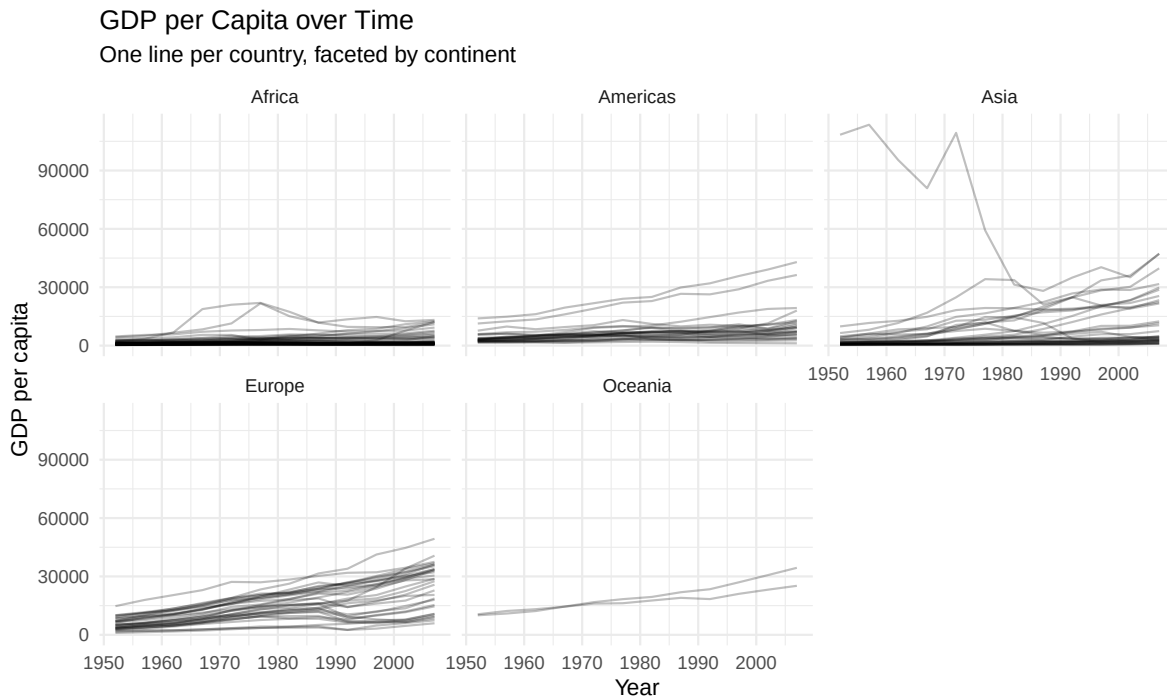
```
ggplot(data = gapminder, mapping = aes(x = year, y = gdpPercap, group = country)) +  
  geom_line(alpha = 0.25) +  
  labs(  
    title = "GDP per Capita over Time",  
    subtitle = "One line per country",  
    x = "Year",  
    y = "GDP per capita",  
    caption = "Source: Gapminder."  
  ) +  
  theme_minimal()
```



Source: Gapminder.

This plot has too many lines in one panel. We can separate continents with facets.

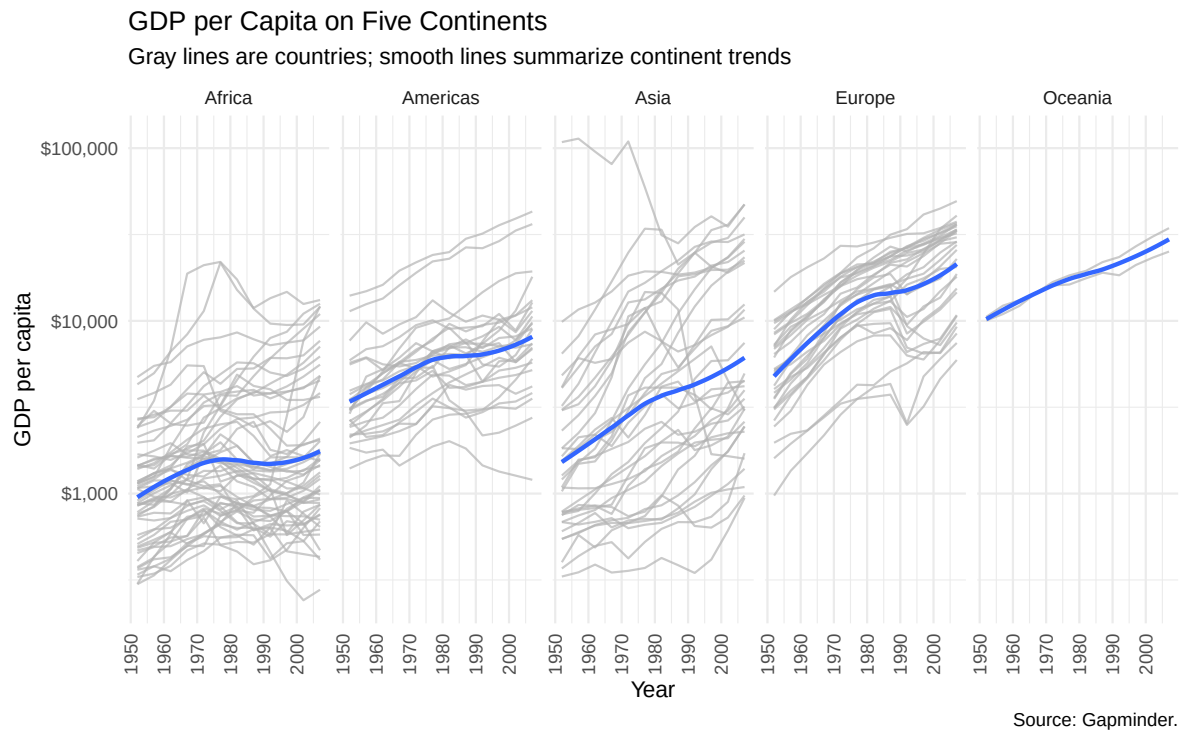
```
ggplot(data = gapminder, mapping = aes(x = year, y = gdpPercap, group = country)) +
  geom_line(alpha = 0.25) +
  facet_wrap(~ continent) +
  labs(
    title = "GDP per Capita over Time",
    subtitle = "One line per country, faceted by continent",
    x = "Year",
    y = "GDP per capita",
    caption = "Source: Gapminder."
  ) +
  theme_minimal()
```



Kuwait is an extreme case in this dataset. A log scale makes the vertical comparison easier.

```
ggplot(data = gapminder, mapping = aes(x = year, y = gdpPercap, group = country)) +
  geom_line(color = "gray70", alpha = 0.7) +
  geom_smooth(
    mapping = aes(group = continent),
    method = "loess",
    formula = y ~ x,
    se = FALSE,
    linewidth = 1
  ) +
  scale_y_log10(labels = dollar_format(accuracy = 1)) +
  facet_wrap(~ continent, ncol = 5) +
  labs(
    title = "GDP per Capita on Five Continents",
    subtitle = "Gray lines are countries; smooth lines summarize continent trends",
    x = "Year",
    y = "GDP per capita",
    caption = "Source: Gapminder."
  ) +
  theme_minimal() +
```

```
theme(axis.text.x = element_text(angle = 90, vjust = 0.5))
```



7.2 Working with dates

Some datasets store time as actual dates. Others store years as numbers. The `lubridate` package helps convert text or numeric values into date objects.

```
make_date(2026)
```

```
[1] "2026-01-01"
```

```
ymd(c("2009-01-02", "2009 01 03", "Created on 2009 1 6"))
```

```
[1] "2009-01-02" "2009-01-03" "2009-01-06"
```

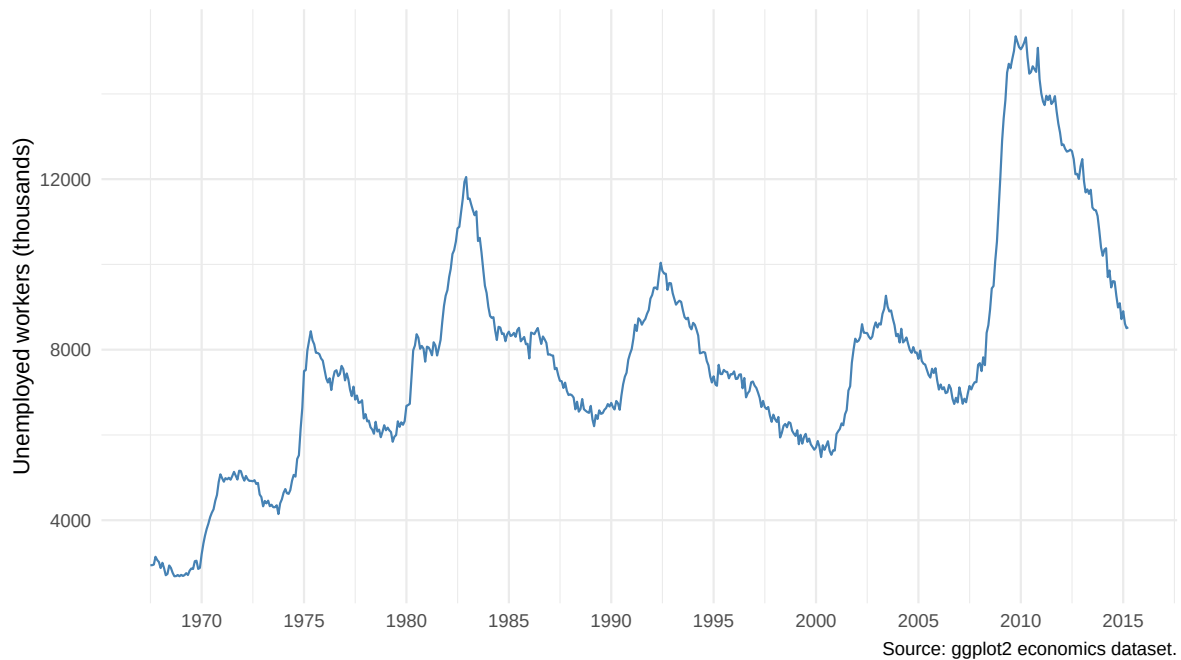
The built-in `economics` dataset has monthly dates.

```
economics |>
  select(date, unemploy, psavert) |>
  slice_head(n = 10)
```

```
# A tibble: 10 x 3
  date      unemploy psavert
  <date>      <dbl>   <dbl>
1 1967-07-01    2944    12.6
2 1967-08-01    2945    12.6
3 1967-09-01    2958    11.9
4 1967-10-01    3143    12.9
5 1967-11-01    3066    12.8
6 1967-12-01    3018    11.8
7 1968-01-01    2878    11.7
8 1968-02-01    3001    12.3
9 1968-03-01    2877    11.7
10 1968-04-01    2709    12.3
```

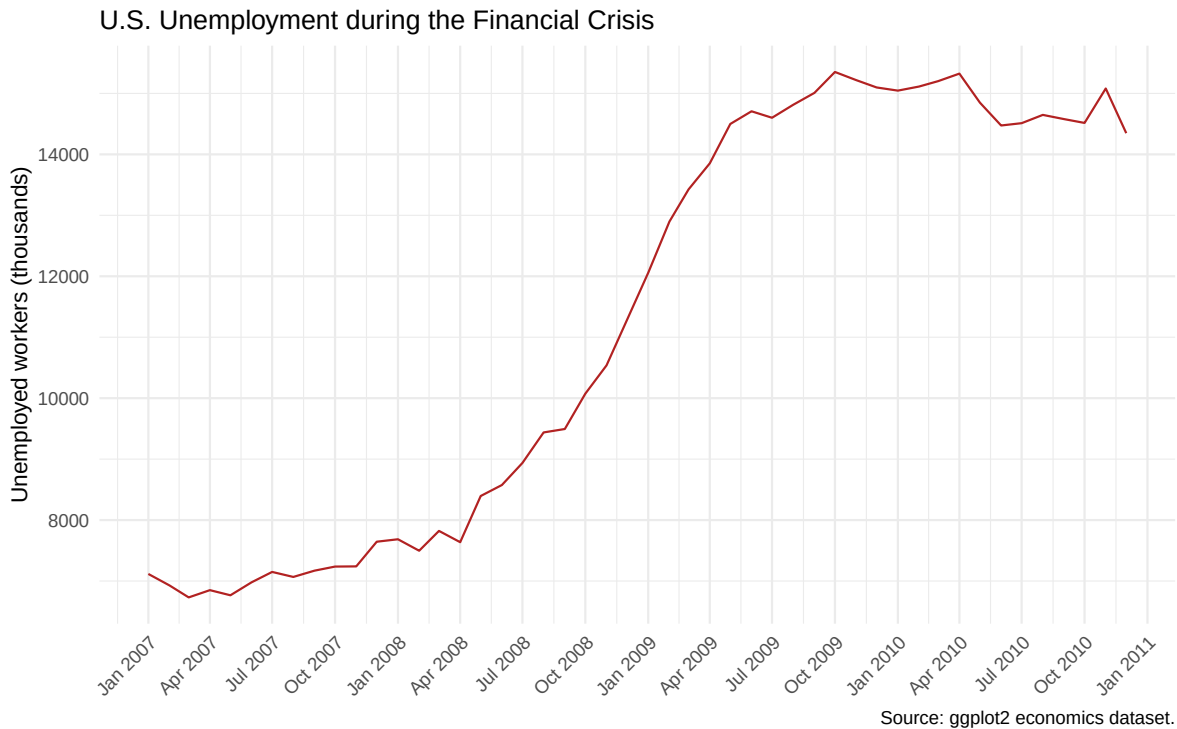
```
ggplot(data = economics, mapping = aes(x = date, y = unemploy)) +
  geom_line(color = "steelblue") +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y") +
  labs(
    title = "U.S. Unemployment",
    subtitle = "Monthly observations, 1967-2015",
    x = NULL,
    y = "Unemployed workers (thousands)",
    caption = "Source: ggplot2 economics dataset."
  ) +
  theme_minimal()
```


U.S. Unemployment
Monthly observations, 1967–2015



For a shorter period, use denser date labels.

```
economics |>
  filter(date >= "2007-01-01", date <= "2010-12-31") |>
  ggplot(mapping = aes(x = date, y = unemploy)) +
  geom_line(color = "firebrick") +
  scale_x_date(date_breaks = "3 months", date_labels = "%b %Y") +
  labs(
    title = "U.S. Unemployment during the Financial Crisis",
    x = NULL,
    y = "Unemployed workers (thousands)",
    caption = "Source: ggplot2 economics dataset."
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



7.3 Highlighting trends

Line charts often need highlighting. A common strategy is to show all lines in gray and draw the important line in a stronger color.

The state policy dataset is useful for this because it has repeated state-year observations. The examples below use unemployment rates by state.

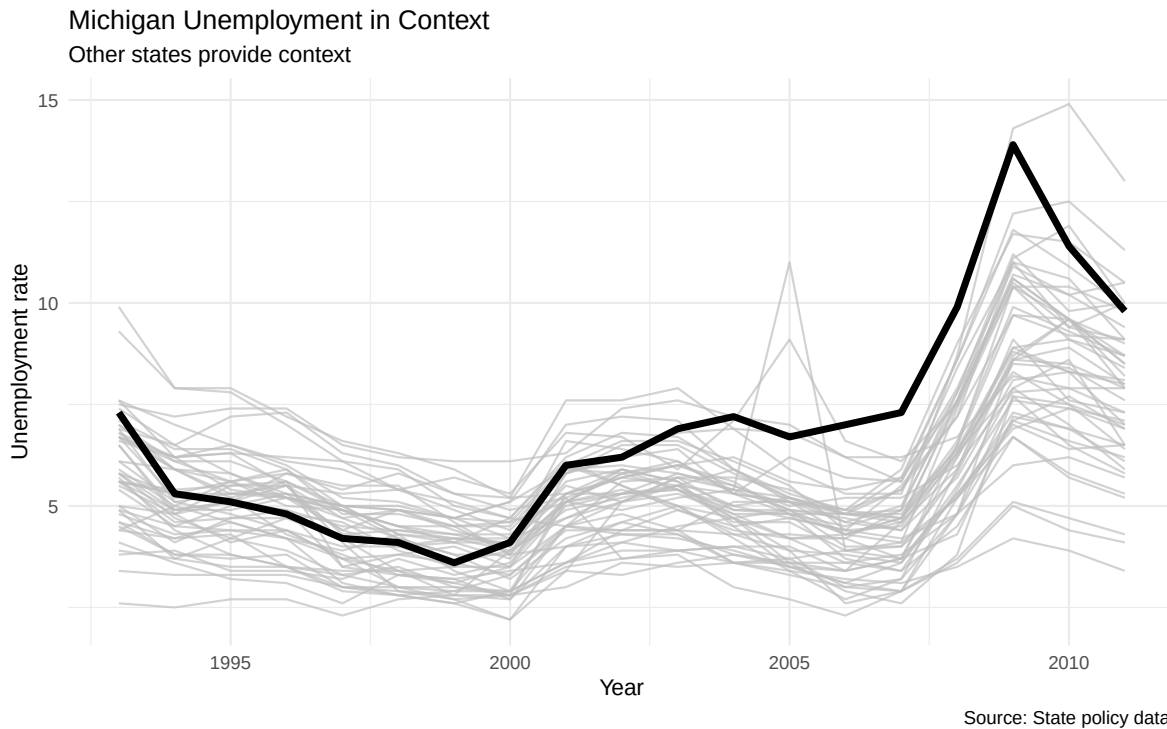
```
state_policy_panel <- haven::read_dta("Data/state_policy/state_data_1993_2016.dta") |>
  mutate(st = as.character(st)) |>
  filter(!is.na(unemploy))

state_policy_panel |>
  select(st, year, unemploy) |>
  head(10)
```

```
# A tibble: 10 x 3
  st      year unemploy
<chr> <dbl>   <dbl>
1 AK     1993     7.5
```

2	AK	1994	7.20
3	AK	1995	7.40
4	AK	1996	7.40
5	AK	1997	6.5
6	AK	1998	6.20
7	AK	1999	6.10
8	AK	2000	6.10
9	AK	2001	6.30
10	AK	2002	7.40

```
ggplot(data = state_policy_panel, mapping = aes(x = year, y = unemploy, group = st)) +
  geom_line(color = "gray75", alpha = 0.7) +
  geom_line(
    data = state_policy_panel |> filter(st == "MI"),
    color = "black",
    linewidth = 1.6
  ) +
  labs(
    title = "Michigan Unemployment in Context",
    subtitle = "Other states provide context",
    x = "Year",
    y = "Unemployment rate",
    caption = "Source: State policy data."
  ) +
  theme_minimal()
```



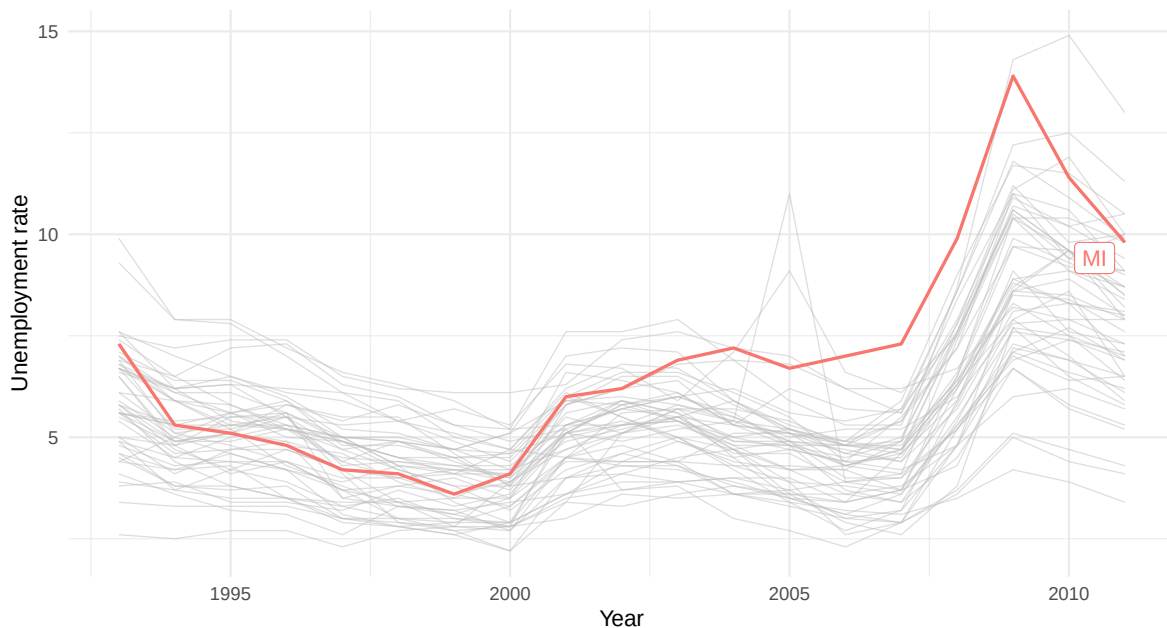
The plot above identifies Michigan visually but not by name. Chapter 8 returns to manual direct labels in a fuller workflow. Here, the shorter option is `gghighlight()`.

The `gghighlight` package automates highlighting and direct labeling when the highlighting rule can be stated as a logical condition.

```
ggplot(data = state_policy_panel, mapping = aes(x = year, y = unemploy, color = st)) +
  geom_line(linewidth = 0.75) +
  gghighlight(
    st == "MI",
    use_group_by = FALSE,
    label_key = st,
    unhighlighted_params = list(linewidth = 0.3, alpha = 0.5)
  ) +
  labs(
    title = "Michigan Unemployment in Context",
    subtitle = "Highlighted with gghighlight()",
    x = "Year",
    y = "Unemployment rate",
    caption = "Source: State policy data."
  ) +
```

```
theme_minimal() +
theme(legend.position = "none")
```

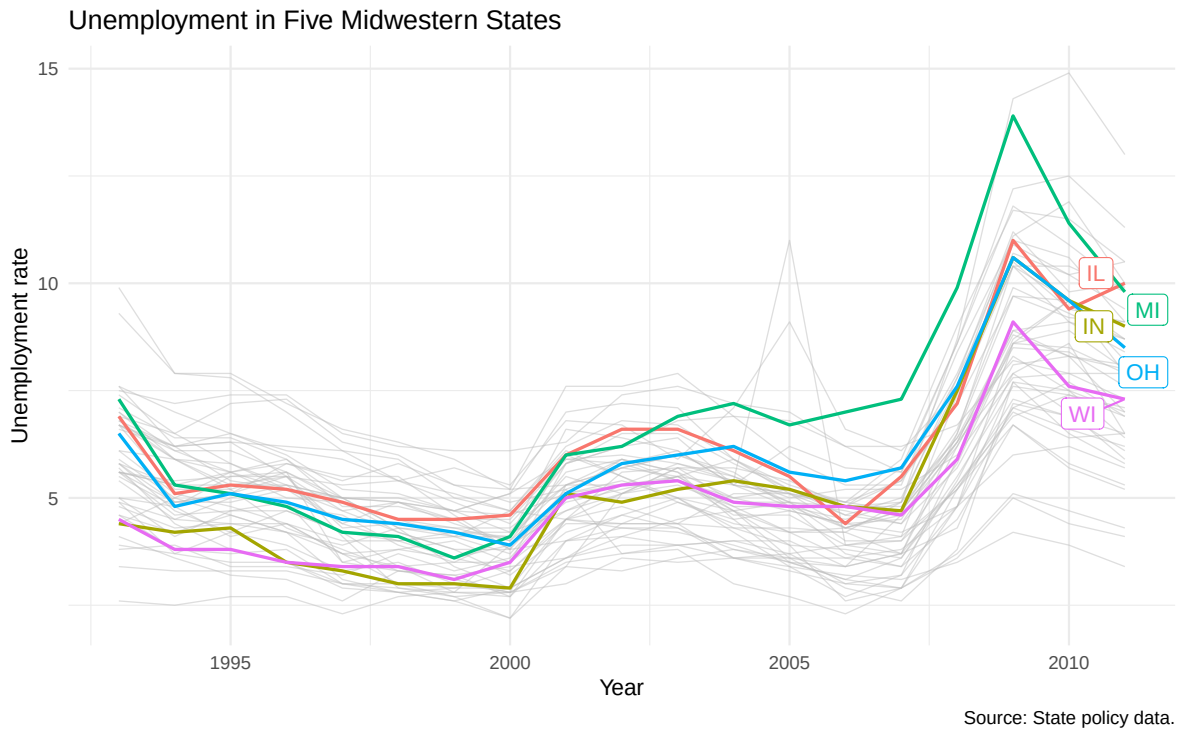
Michigan Unemployment in Context
Highlighted with gghighlight()



Source: State policy data.

We can also highlight a set of cases. The condition can name several states with `%in%`.

```
ggplot(data = state_policy_panel, mapping = aes(x = year, y = unemploy, color = st)) +
  geom_line(linewidth = 0.75) +
  gghighlight(
    st %in% c("IL", "IN", "MI", "OH", "WI"),
    use_group_by = FALSE,
    label_key = st,
    unhighlighted_params = list(linewidth = 0.3, alpha = 0.5)
  ) +
  labs(
    title = "Unemployment in Five Midwestern States",
    x = "Year",
    y = "Unemployment rate",
    caption = "Source: State policy data."
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```



7.4 Bar charts

Bar charts are useful for comparing quantities across categories. When the data is already summarized, use `geom_col()`.

The `midwest` dataset has county-level observations. To compare average college education by state, we first summarize.

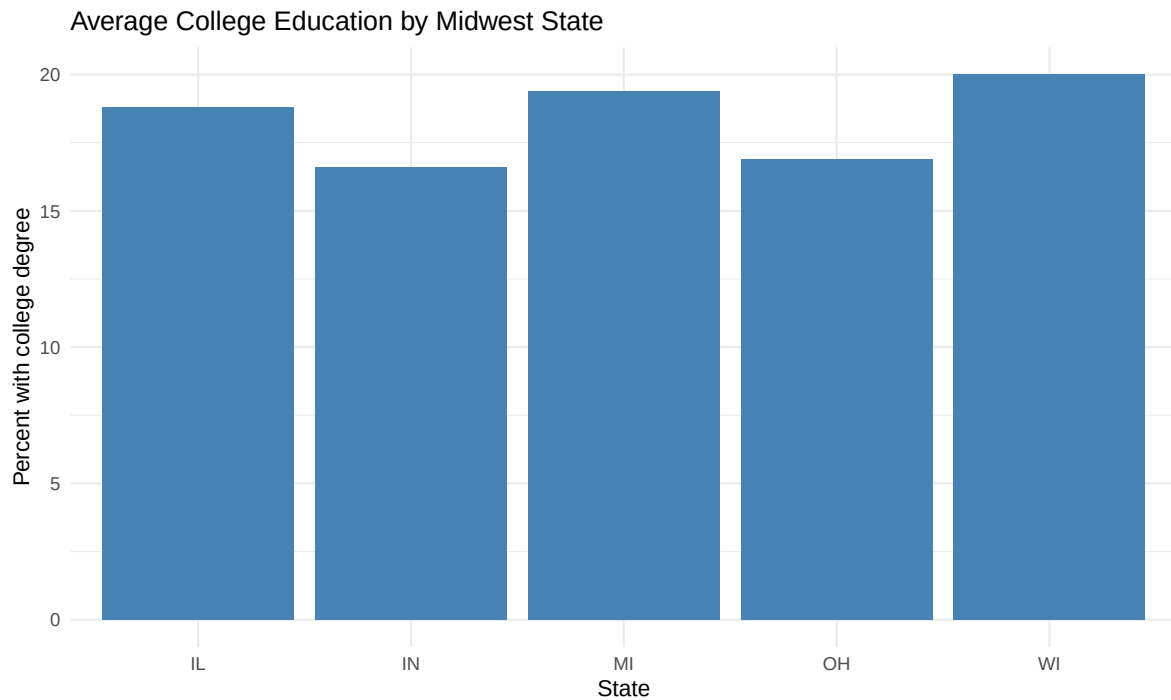
```
college_by_state <- midwest |>
  filter(!is.na(percollege)) |>
  group_by(state) |>
  summarize(
    avg_college = round(mean(percollege), 1)
  )
```

```
college_by_state
```

```
# A tibble: 5 x 2
  state avg_college
```

	<chr>	<dbl>
1	IL	18.8
2	IN	16.6
3	MI	19.4
4	OH	16.9
5	WI	20

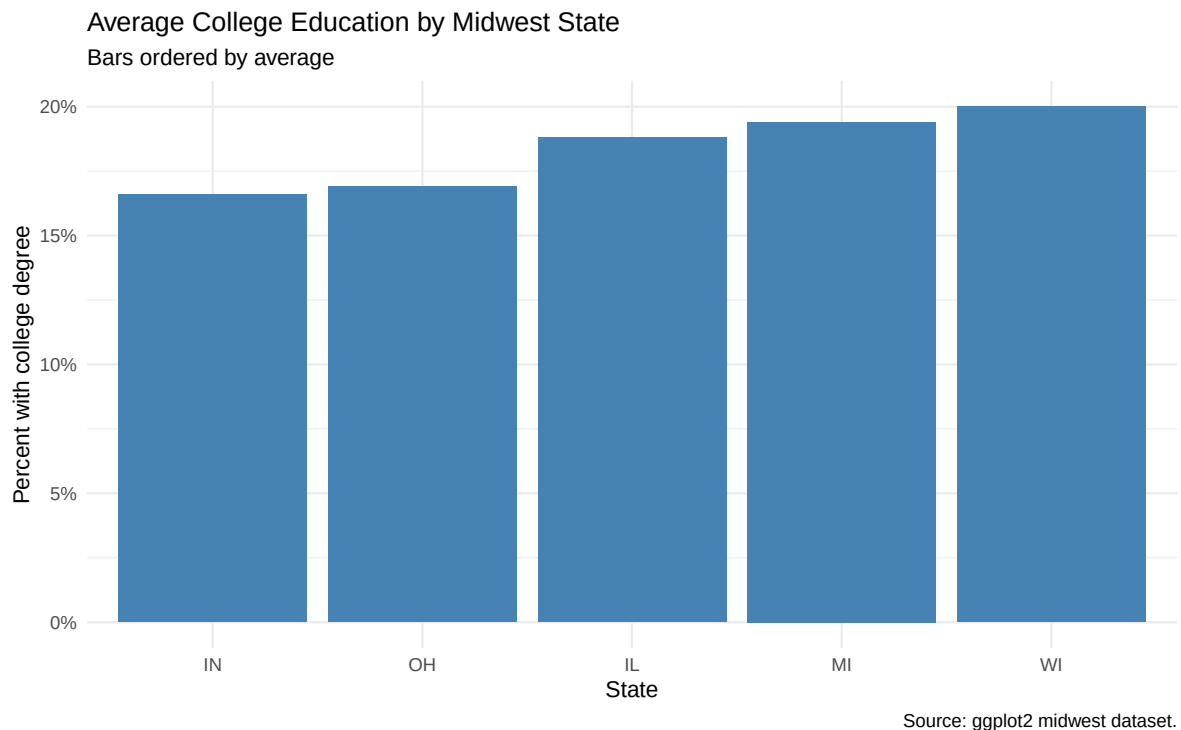
```
ggplot(data = college_by_state, mapping = aes(x = state, y = avg_college)) +
  geom_col(fill = "steelblue") +
  labs(
    title = "Average College Education by Midwest State",
    x = "State",
    y = "Percent with college degree",
    caption = "Source: ggplot2 midwest dataset."
  ) +
  theme_minimal()
```



Source: ggplot2 midwest dataset.

Ordering the bars by value makes comparison easier.

```
ggplot(data = college_by_state, mapping = aes(x = reorder(state, avg_college), y = avg_college)) +
  geom_col(fill = "steelblue") +
  scale_y_continuous(labels = percent_format(scale = 1)) +
  labs(
    title = "Average College Education by Midwest State",
    subtitle = "Bars ordered by average",
    x = "State",
    y = "Percent with college degree",
    caption = "Source: ggplot2 midwest dataset."
  ) +
  theme_minimal()
```

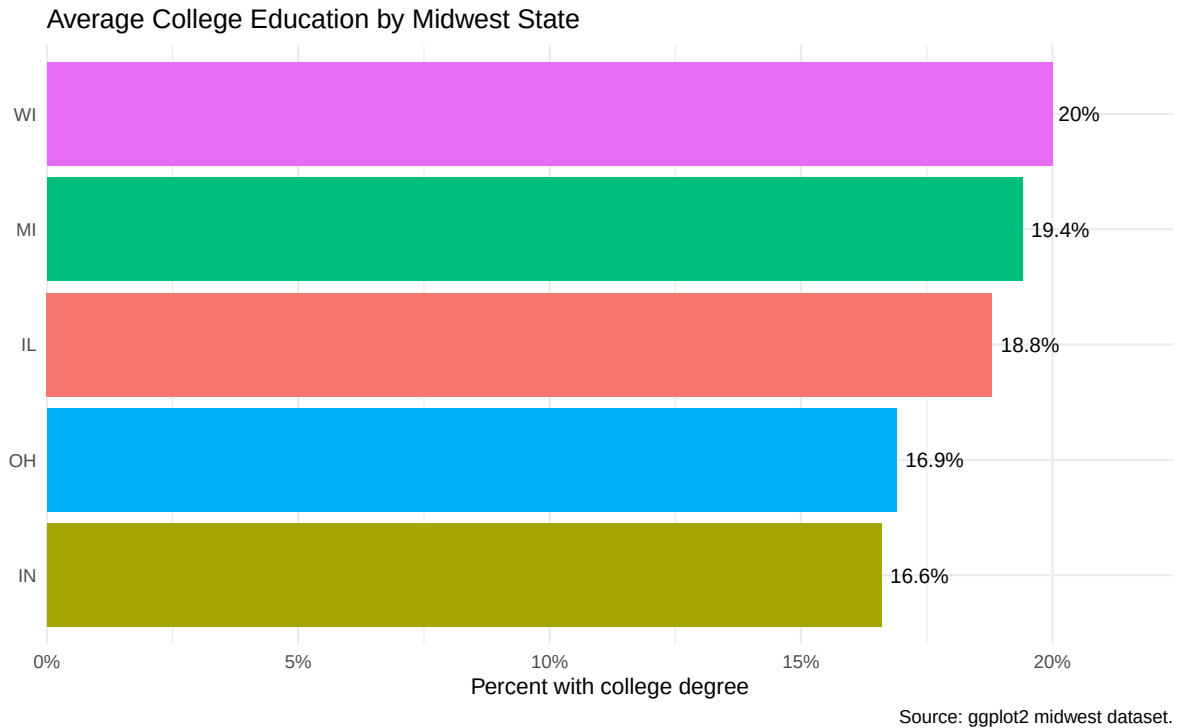


If category labels are long, `coord_flip()` often helps.

```
ggplot(data = college_by_state, mapping = aes(x = reorder(state, avg_college), y = avg_college)) +
  geom_col(aes(fill = state), show.legend = FALSE) +
  geom_text(aes(label = str_c(avg_college, "%")), hjust = -0.15, size = 3.5) +
  coord_flip() +
  scale_y_continuous(labels = percent_format(scale = 1), expand = expansion(mult = c(0, 0.12))) +
  labs(
    title = "Average College Education by Midwest State",
```



```
x = NULL,
y = "Percent with college degree",
caption = "Source: ggplot2 midwest dataset."
) +
theme_minimal()
```



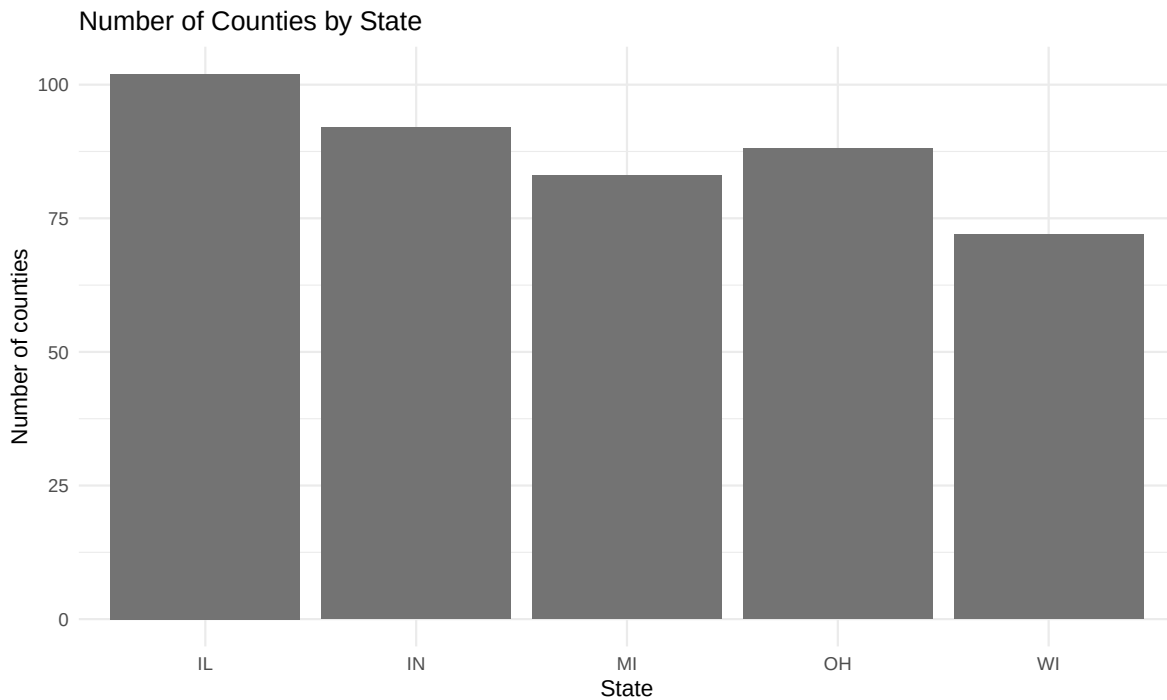
7.5 geom_col() versus geom_bar()

Use `geom_col()` when you already have the heights of the bars.

Use `geom_bar()` when you want ggplot2 to count rows.

```
ggplot(data = midwest, mapping = aes(x = state)) +
  geom_bar(fill = "gray45") +
  labs(
    title = "Number of Counties by State",
    x = "State",
    y = "Number of counties",
    caption = "Source: ggplot2 midwest dataset."
  )
```

```
) +  
theme_minimal()
```



Source: ggplot2 midwest dataset.

Here, no y variable is mapped. `geom_bar()` counts the number of rows in each state.

7.6 Dodged bars

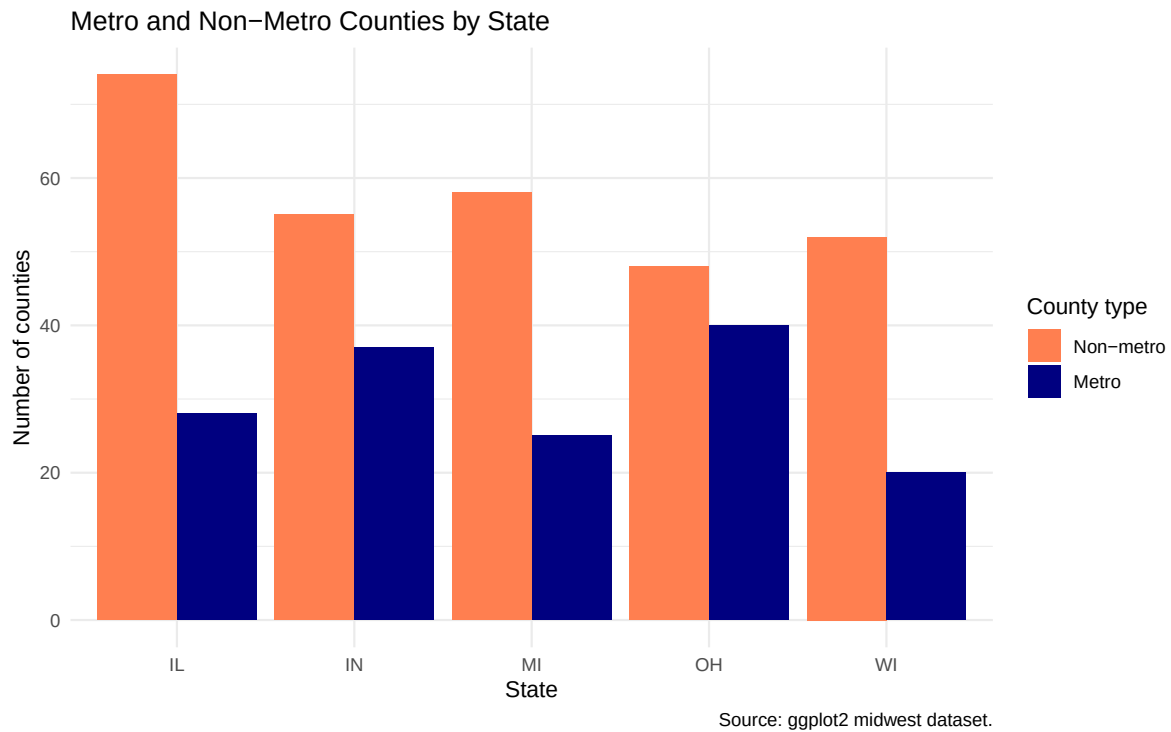
Dodged bars place bars side by side within each category. Here we compare metro and non-metro counties within states.

```
ggplot(data = midwest, mapping = aes(x = state, fill = factor(inmetro))) +  
  geom_bar(position = "dodge") +  
  scale_fill_manual(  
    values = c("0" = "coral", "1" = "navy"),  
    labels = c("0" = "Non-metro", "1" = "Metro")  
  ) +  
  labs(  
    title = "Metro and Non-Metro Counties by State",  
    x = "State",
```

```

y = "Number of counties",
fill = "County type",
caption = "Source: ggplot2 midwest dataset."
) +
theme_minimal()

```



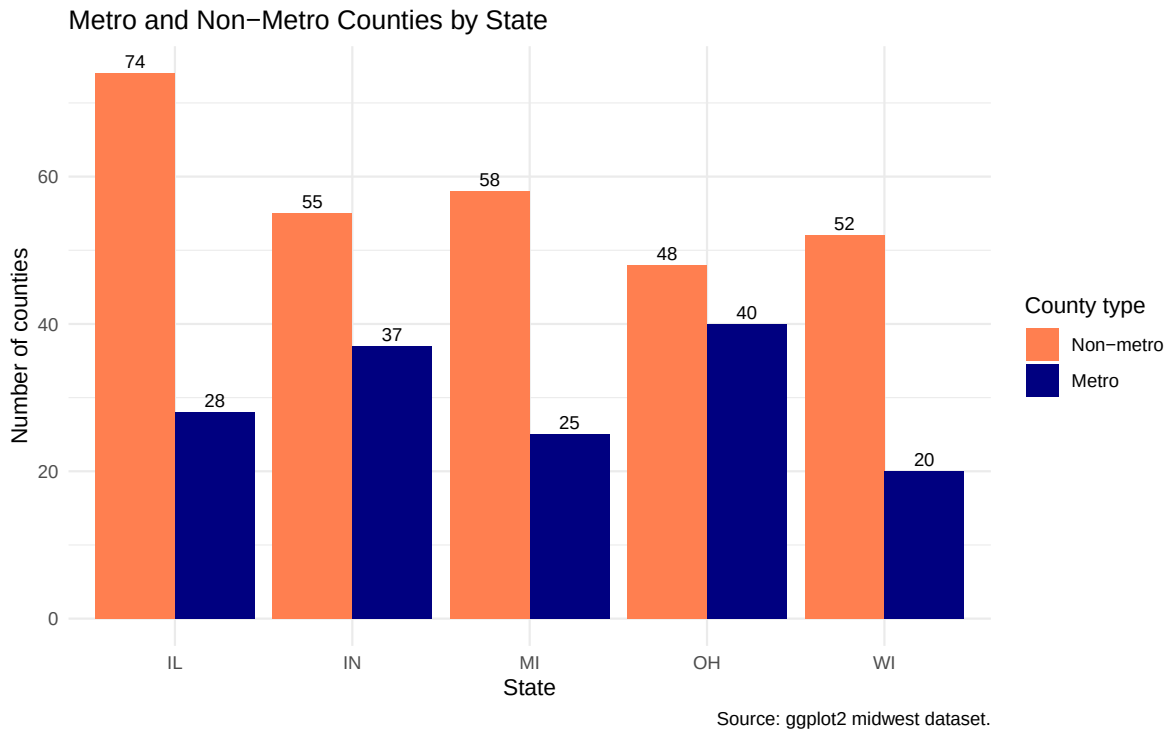
Adding labels to dodged bars requires the text to use the same dodging.

```

ggplot(data = midwest, mapping = aes(x = state, fill = factor(inmetro))) +
  geom_bar(position = position_dodge(width = 0.9)) +
  geom_text(
    mapping = aes(label = after_stat(count), group = factor(inmetro)),
    stat = "count",
    position = position_dodge(width = 0.9),
    vjust = -0.4,
    size = 3
  ) +
  scale_fill_manual(
    values = c("0" = "coral", "1" = "navy"),
    labels = c("0" = "Non-metro", "1" = "Metro")
  ) +

```

```
labs(
  title = "Metro and Non-Metro Counties by State",
  x = "State",
  y = "Number of counties",
  fill = "County type",
  caption = "Source: ggplot2 midwest dataset."
) +
theme_minimal()
```



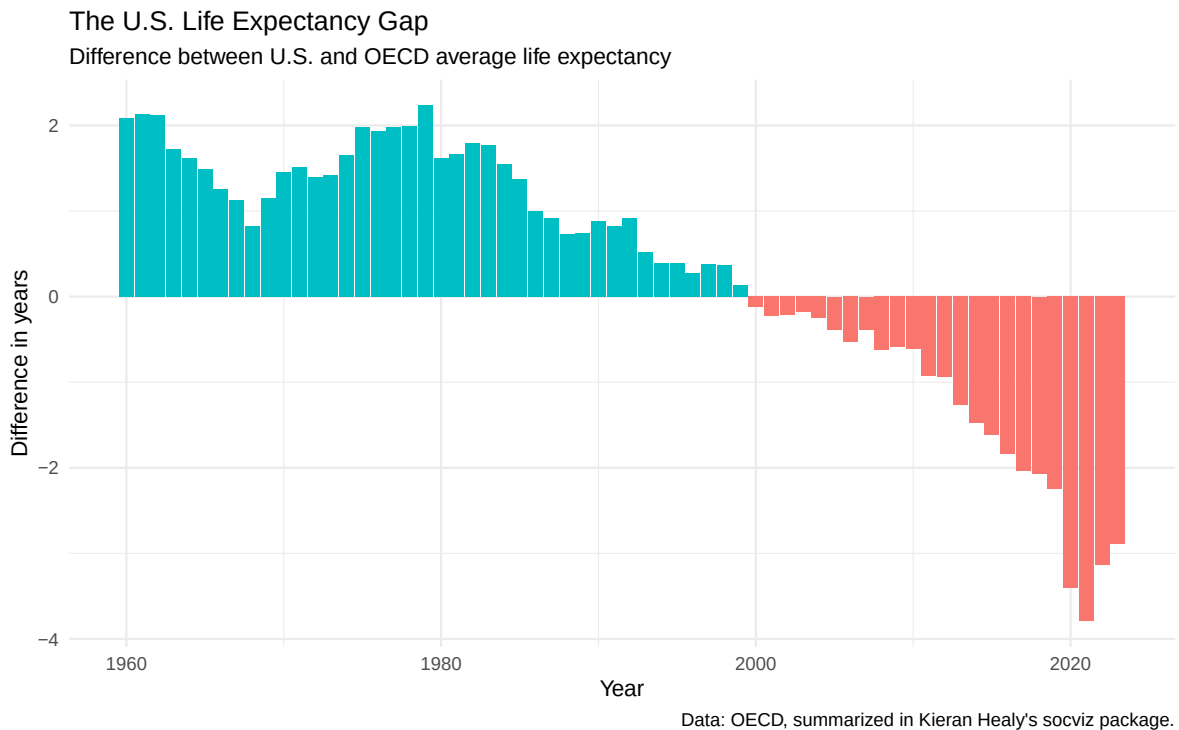
7.7 Bar chart example: OECD life expectancy gap

A local copy of the `oecd_sum` summary lives at `Data/oecd/oecd_sum.csv`. Each row is one year, with columns for U.S. life expectancy, the OECD non-U.S. average, the difference, and a `hi_lo` indicator for whether the U.S. is above or below the average.

```
oecd_sum <- read_csv("Data/oecd/oecd_sum.csv", show_col_types = FALSE)

ggplot(data = oecd_sum, mapping = aes(x = year, y = diff, fill = hi_lo)) +
  geom_col() +
```

```
guides(fill = "none") +
labs(
  title = "The U.S. Life Expectancy Gap",
  subtitle = "Difference between U.S. and OECD average life expectancy",
  x = "Year",
  y = "Difference in years",
  caption = "Data: OECD, summarized in Kieran Healy's socviz package."
) +
theme_minimal()
```



This is a bar chart because the main comparison is the size and direction of a yearly difference.

7.8 Exercise

Create an ordered bar chart from a summarized dataset.

1. Use `midwest`.
2. Group by `state`.
3. Calculate the median of `percollege`.
4. Make an ordered `geom_col()` chart.

5. Add value labels.
6. Remove any redundant legend.

```
# Write your plot here.
```

7.9 Extra: slopegraphs

Slopegraphs compare the same units at a small number of time points. They are especially useful when the start and end values matter more than the intermediate path. The form was popularized by Edward Tufte in *The Visual Display of Quantitative Information*, but the design predates him by a century.

A generic Tufte-style slopegraph:

Ben Fry's slopegraph of baseball salary per win:

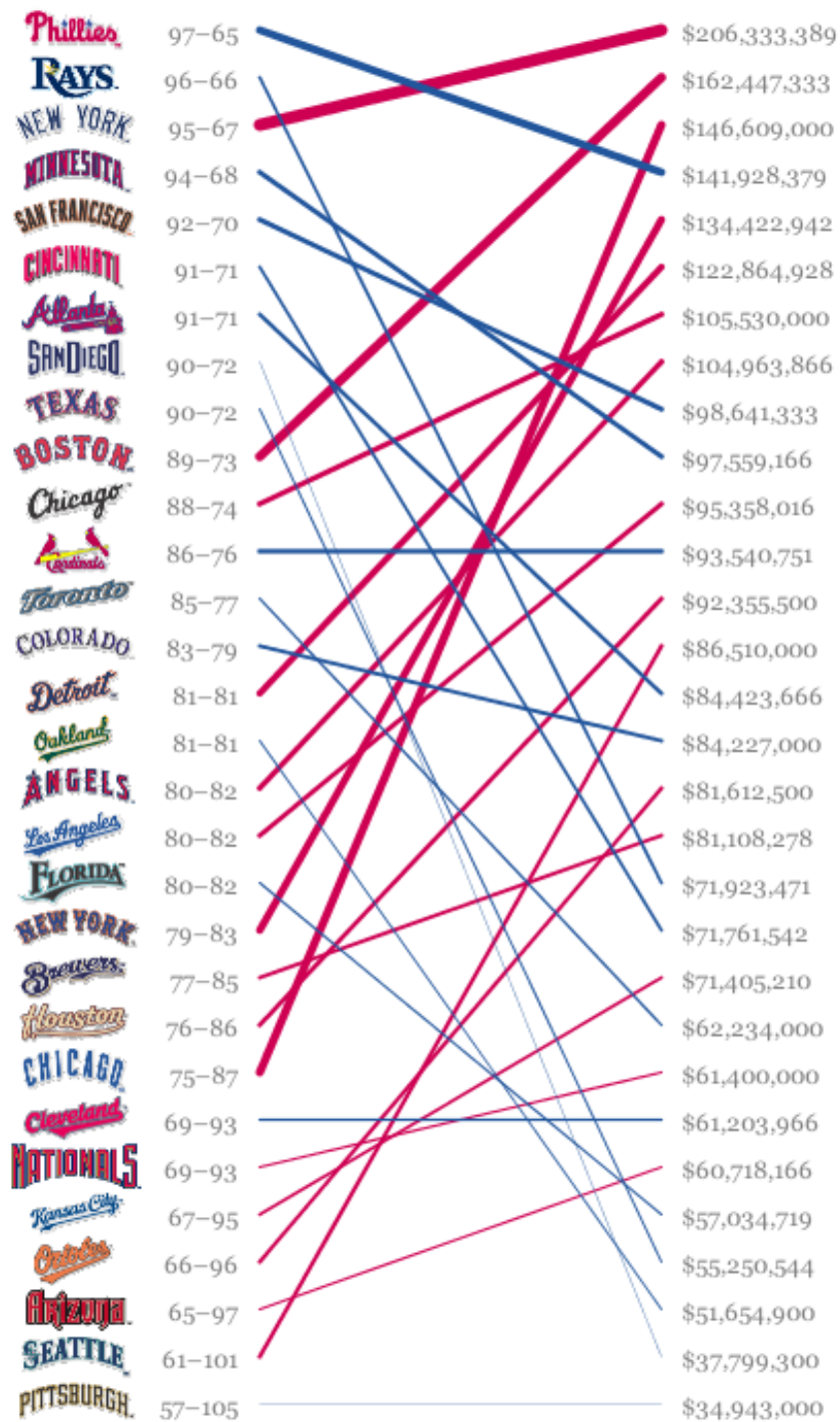


Figure 7.1: Baseball slopegraph by Ben Fry

And one from *Scribner's Statistical Atlas of the United States* in 1883, well before the modern term existed:

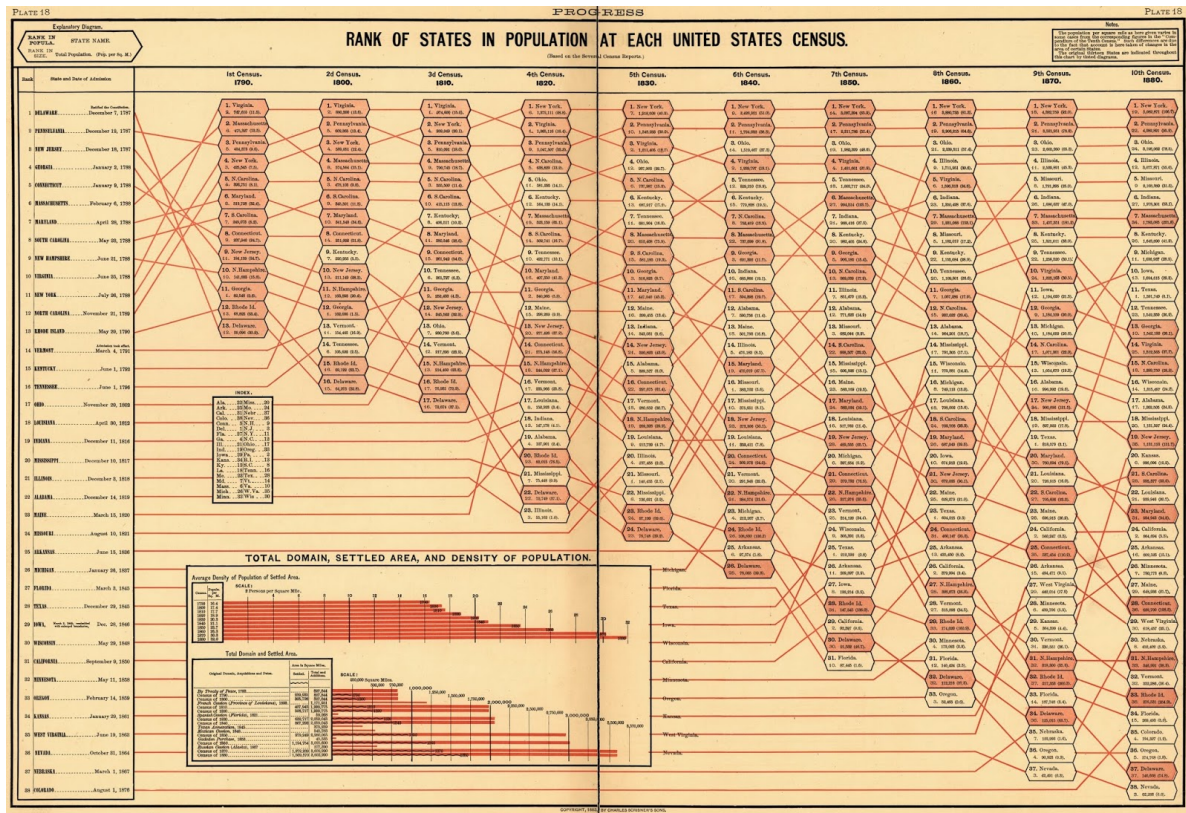


Figure 7.2: Scribner's Statistical Atlas, 1883

Unlike a traditional line chart, a slopegraph emphasizes the start and end values of each unit, so change is read as the slope of a line rather than as a path through many points.

The states table is a local snapshot of Correlates of State Policy data, originally fetched with `cspp::get_cspp_data()`. `polconserv` is policy conservatism (the negation of `pollib_median`), so larger values mean a more conservative policy environment.

```
states <- read_csv("Data/state_policy/cspp_states.csv", show_col_types = FALSE)

library(ggslopegraph)

ggslopegraph(
  dataframe = states |>
  mutate(year = as.character(year)) |>
```



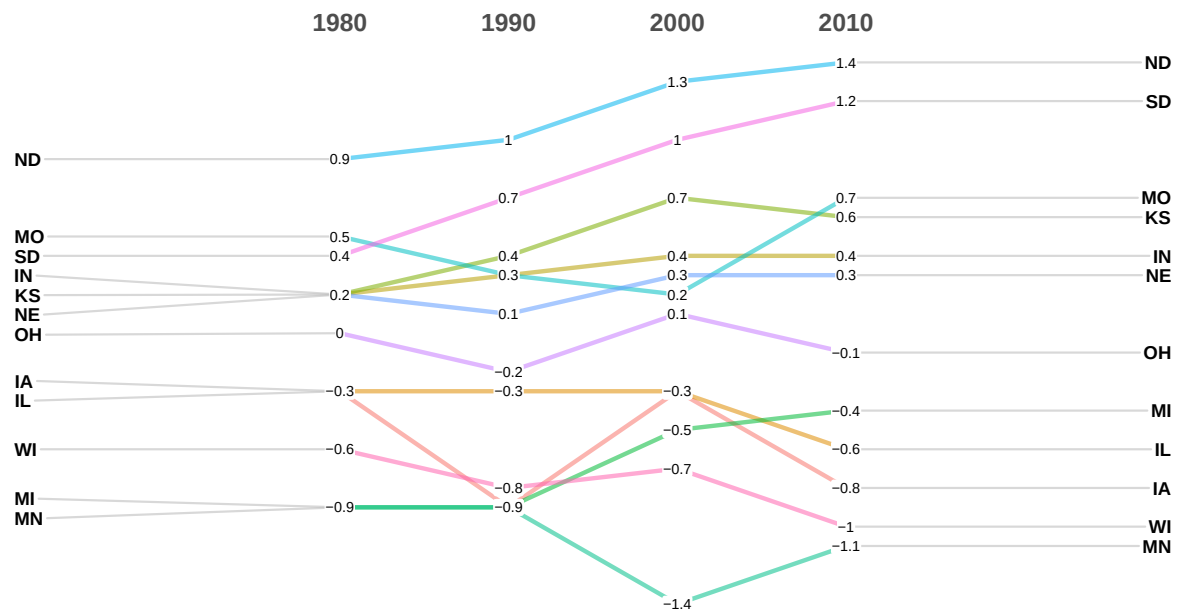
```

  filter(year %in% c("1980", "1990", "2000", "2010"), region.name == "midwest"),
  Times = year,
  Measurement = polconserv,
  Grouping = st,
  Title = "Tufte-style Slopegraph of Policy Conservatism",
  SubTitle = "Midwest, 1980-2010",
  Caption = "Source: Correlates of State Policy via the cspp package."
)

```

Tufte-style Slopegraph of Policy Conservatism

Midwest, 1980-2010



Source: Correlates of State Policy via the cspp package.

`ggslopegraph()` requires the `Times` column to be a character or factor, which is why `year` is converted before filtering. If `ggslopegraph` is not available, the same plot can be built with plain `geom_line()` and `geom_text()`.

8 From Question to Final Figure

8.1 The workflow

A finished figure usually comes from a sequence of smaller decisions. This chapter walks through one applied workflow:

1. State a question.
2. Inspect the data.
3. Make a rough plot.
4. Identify what is unclear.
5. Revise the data transformation.
6. Revise the plot.
7. Add labels, scales, and annotation.
8. Export the result when it is ready.

The goal is not to jump directly to a polished chart. The goal is to make each revision solve a specific problem.

Several of the individual techniques in this workflow — highlighting one line against gray context, adding a direct label, and building a benchmark — were introduced in Chapter 7. They appear here inside a sustained applied lab so the decisions accumulate into one finished figure.

8.2 Question

We will use state-level cigarette consumption data.

The working question:

How did cigarette consumption change over time across U.S. states, and where does Michigan fit relative to the broader pattern?

That question suggests a line chart because we are comparing change over time.

8.3 Inspect the data

The function `glimpse()` comes from `dplyr`, which is loaded as part of the `tidyverse`.

```
glimpse(cigarettes)
```

Rows: 96

Columns: 12

```
$ state      <chr> "AL", "AR", "AZ", "CA", "CO", "CT", "DE", "FL", "GA", "IA", ~
$ year       <dbl> 1985, 1985, 1985, 1985, 1985, 1985, 1985, 1985, 1985, 1985, ~
$ cpi        <dbl> 1.076, 1.076, 1.076, 1.076, 1.076, 1.076, 1.076, 1.076, 1.0~
$ population <dbl> 3973000, 2327000, 3184000, 26444000, 3209000, 3201000, 6180~
$ packpc     <dbl> 116.4863, 128.5346, 104.5226, 100.3630, 112.9635, 109.2784, ~
$ income     <dbl> 46014968, 26210736, 43956936, 447102816, 49466672, 60063368~
$ tax        <dbl> 32.50000, 37.00000, 31.00000, 26.00000, 31.00000, 42.00000, ~
$ price      <dbl> 102.18167, 101.47500, 108.57875, 107.83734, 94.26666, 128.0~
$ taxes      <dbl> 33.34834, 37.00000, 36.17042, 32.10400, 31.00000, 51.48333, ~
$ real_price <dbl> 94.96438, 94.30762, 100.90962, 100.22058, 87.60842, 118.982~
$ real_tax   <dbl> 30.20447, 34.38662, 28.81041, 24.16357, 28.81041, 39.03346, ~
$ real_taxes <dbl> 30.99288, 34.38662, 33.61563, 29.83643, 28.81041, 47.84697, ~
```

```
cigarettes |>
  select(any_of(c("state", "year", "packpc", "region"))) |>
  head(12)
```

A tibble: 12 x 3

	state	year	packpc
	<chr>	<dbl>	<dbl>
1	AL	1985	116.
2	AR	1985	129.
3	AZ	1985	105.
4	CA	1985	100.
5	CO	1985	113.
6	CT	1985	109.
7	DE	1985	144.
8	FL	1985	122.
9	GA	1985	127.
10	IA	1985	114.
11	ID	1985	103.
12	IL	1985	123.

For this workflow, we need at least three variables:

- **state**: the unit being tracked.
- **year**: the time variable.
- **packpc**: packs per capita.

We will create a date variable so that **ggplot2** treats time as a date scale.

```
cigarettes <- cigarettes |>
  mutate(date = make_date(year))

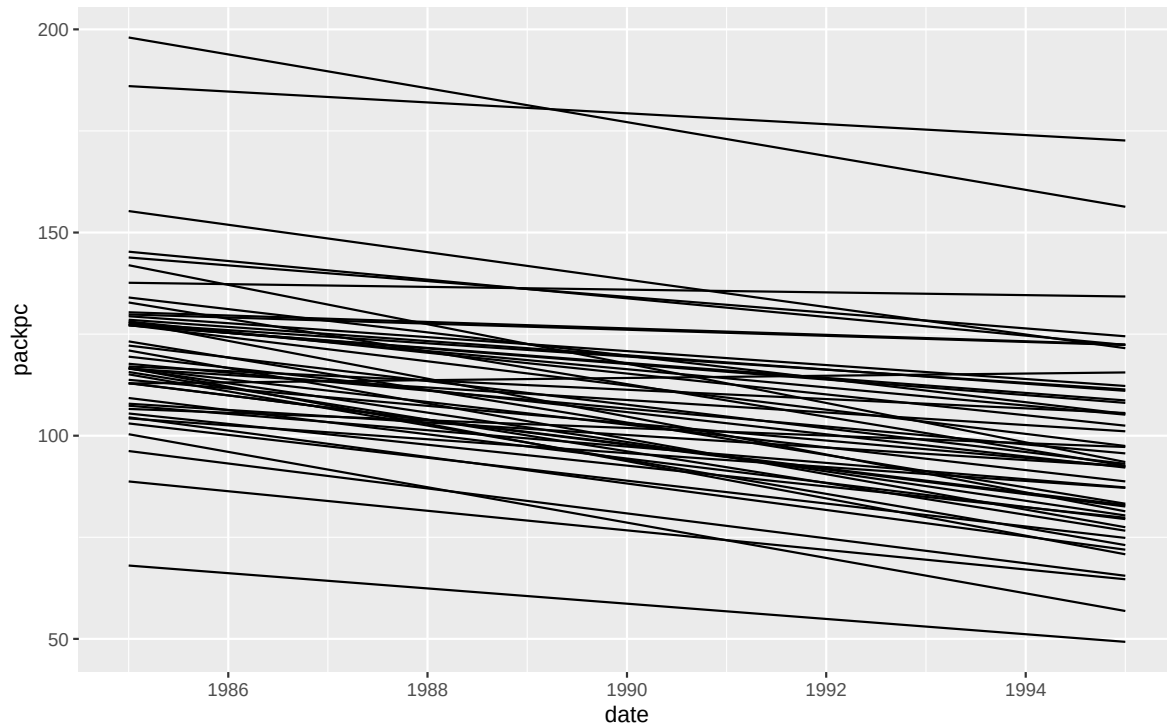
cigarettes |>
  summarize(
    first_year = min(year, na.rm = TRUE),
    last_year = max(year, na.rm = TRUE),
    states = n_distinct(state),
    observations = n()
  )
```

```
# A tibble: 1 x 4
  first_year last_year states observations
    <dbl>      <dbl>   <int>         <int>
1   1985      1995     48           96
```

8.4 First rough plot

Start with the simplest plot that matches the question.

```
ggplot(data = cigarettes, mapping = aes(x = date, y = packpc, group = state)) +
  geom_line()
```

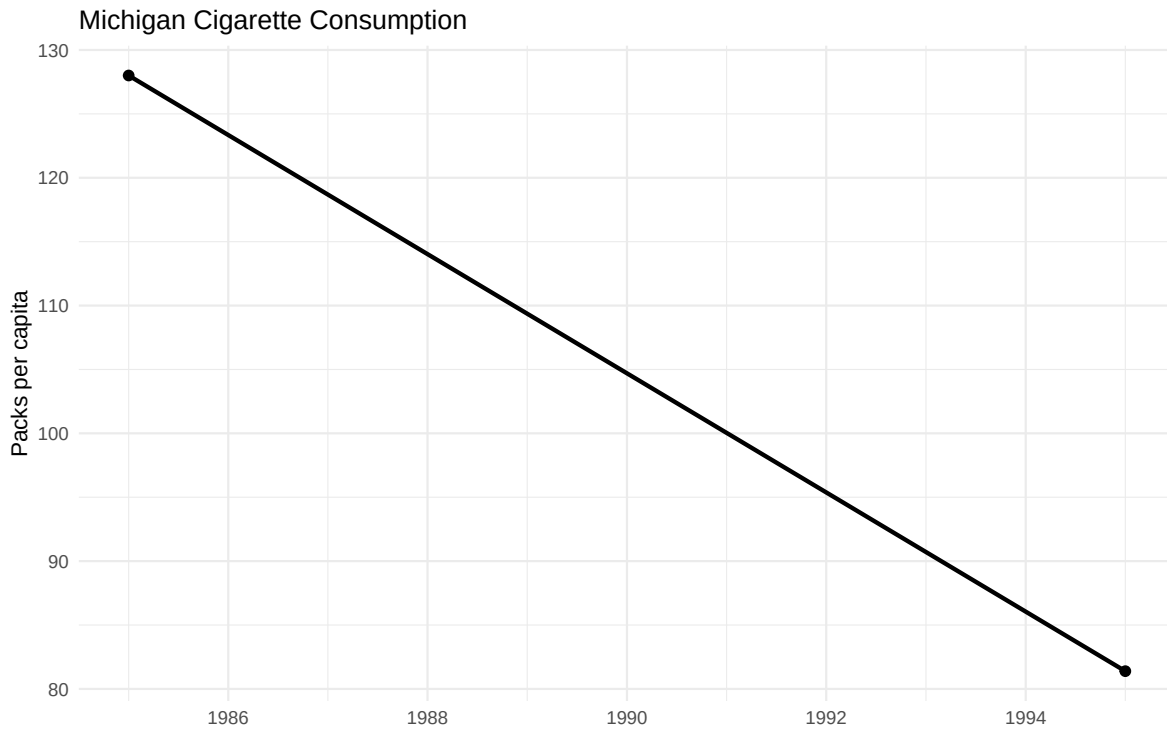


This plot is useful because it confirms that the data can be drawn as state-level time series. It is not yet a good final figure. It has no labels, too much visual weight on every state, and no clear focal comparison.

8.5 Check one state first

Before improving the full plot, isolate one state. A one-state plot is not the final answer, but it is easier to inspect.

```
cigarettes |>
  filter(state == "MI") |>
  ggplot(aes(x = date, y = packpc)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  labs(
    title = "Michigan Cigarette Consumption",
    x = NULL,
    y = "Packs per capita"
  ) +
  theme_minimal()
```



This confirms that Michigan declines across the observed years. The next question is whether that decline is unusual.

8.6 Compare a small set of states

A small comparison set is often a good intermediate step. It is more informative than one state but less crowded than all states.

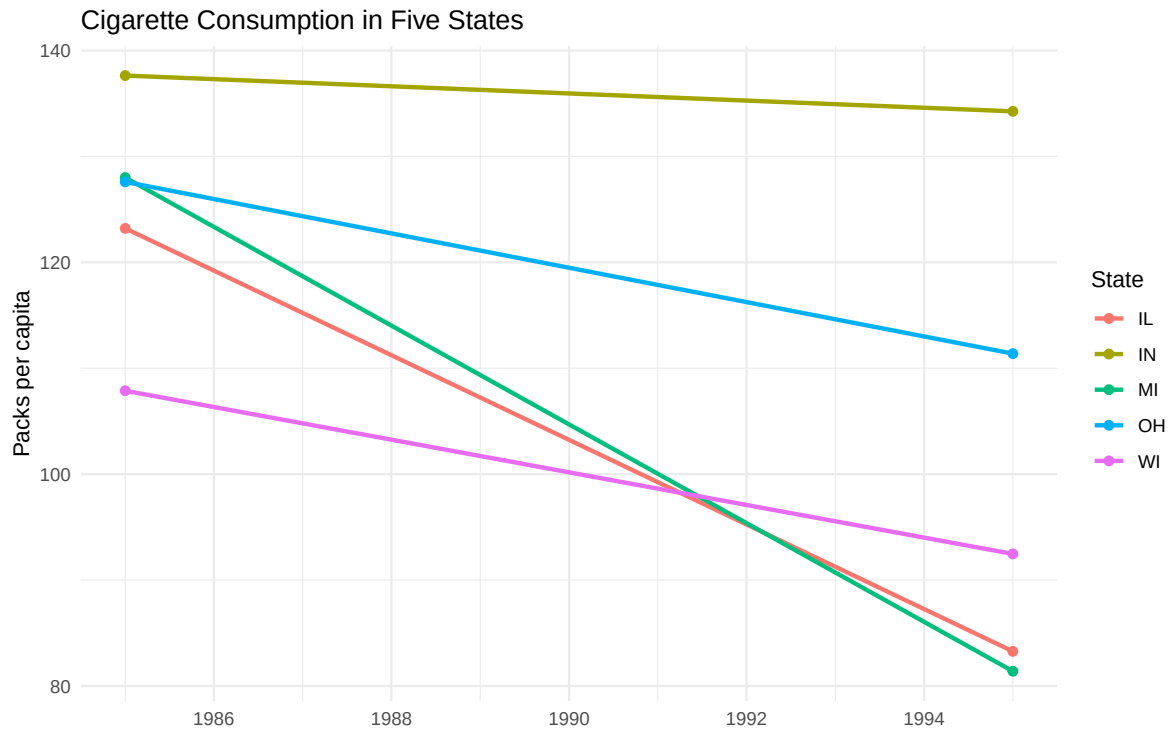
```
comparison_states <- c("MI", "IL", "IN", "OH", "WI")

cigarettes |>
  filter(state %in% comparison_states) |>
  ggplot(aes(x = date, y = packpc, color = state)) +
  geom_line(linewidth = 1) +
  geom_point(size = 1.8) +
  scale_x_date(date_breaks = "2 years", date_labels = "%Y") +
  labs(
    title = "Cigarette Consumption in Five States",
    x = NULL,
    y = "Packs per capita",
```

```

    color = "State"
  ) +
  theme_minimal()

```



This version makes individual states easier to follow, but it does not show how these states compare with the entire country.

8.7 Try color on all states

Coloring every state is usually a bad idea, but trying it is instructive. It shows why the final figure should not use a 48-state legend.

```

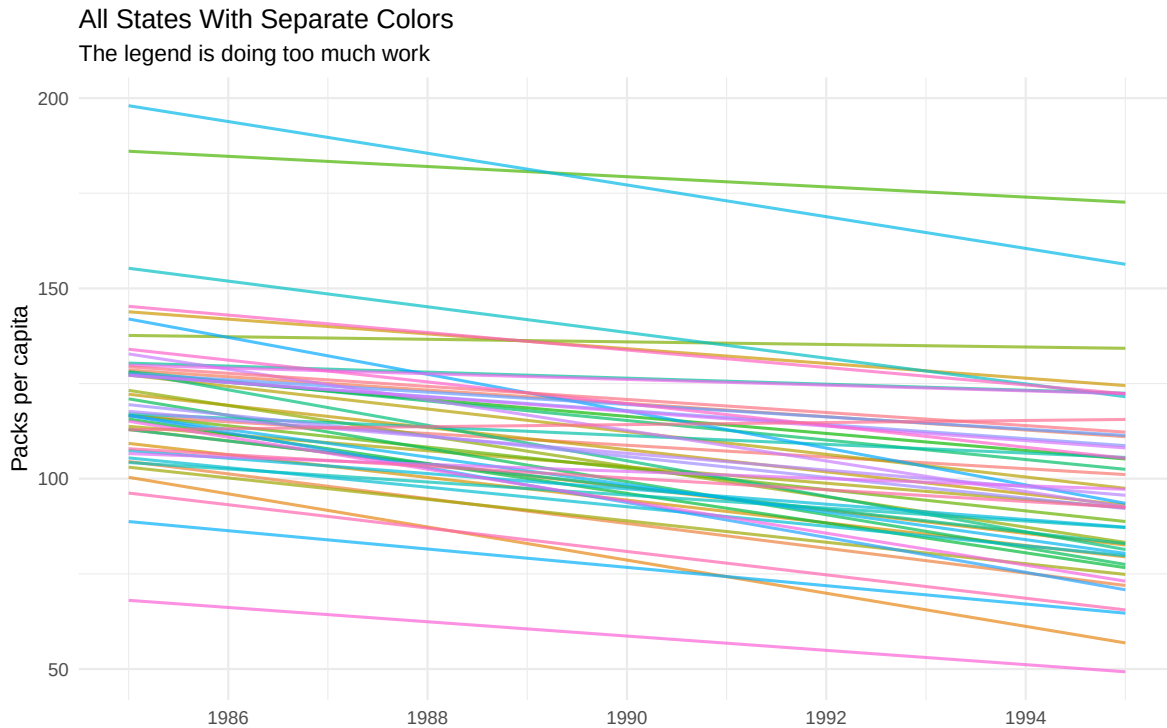
ggplot(data = cigarettes, mapping = aes(x = date, y = packpc, color = state)) +
  geom_line(alpha = 0.7, linewidth = 0.7) +
  scale_x_date(date_breaks = "2 years", date_labels = "%Y") +
  labs(
    title = "All States With Separate Colors",
    subtitle = "The legend is doing too much work",
    x = NULL,

```

```

y = "Packs per capita",
color = "State"
) +
theme_minimal() +
theme(legend.position = "none")

```



The colors make the plot noisier without making the comparison clearer. This is a useful failed draft.

8.8 Build gray context with labels

If every state is context, the context should be visible but quiet. Thin gray lines are a good default. At the same time, the plot needs enough labels to be readable outside the code file.

```

p_all_states <- ggplot(data = cigarettes, mapping = aes(x = date, y = packpc, group = state)) +
  geom_line(color = "gray82", alpha = 0.7, linewidth = 0.45) +
  scale_x_date(date_breaks = "2 years", date_labels = "%Y") +
  labs(
    title = "Cigarette Consumption Across States",

```

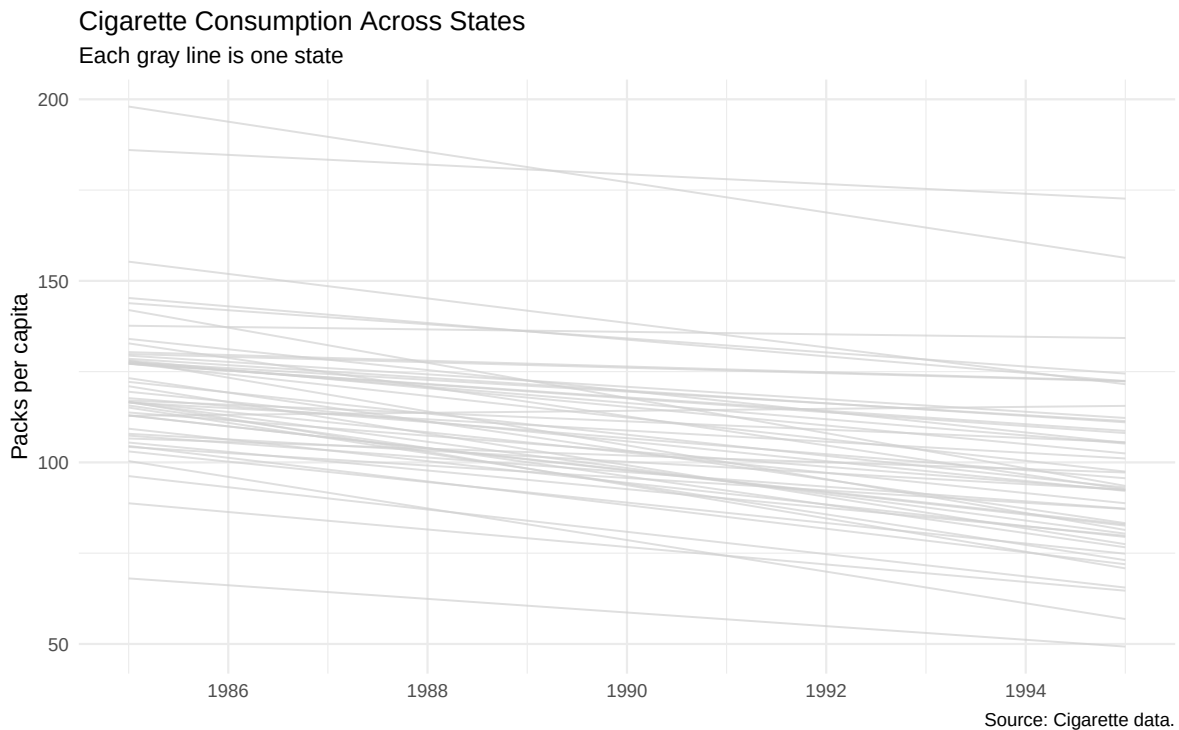


```

    subtitle = "Each gray line is one state",
    x = NULL,
    y = "Packs per capita",
    caption = "Source: Cigarette data."
  ) +
  theme_minimal()

p_all_states

```



This version is readable enough for exploration, and it creates room for emphasis. It still does not answer the Michigan comparison clearly.

8.9 Highlight one state

Apply the manual highlight pattern from Chapter 7 (gray context lines plus a heavier focal line) and save the result as a plot object so later revisions can be layered on top of it.

```

p_michigan <- ggplot(data = cigarettes, mapping = aes(x = date, y = packpc, group = state)) +
  geom_line(color = "gray78", alpha = 0.75) +

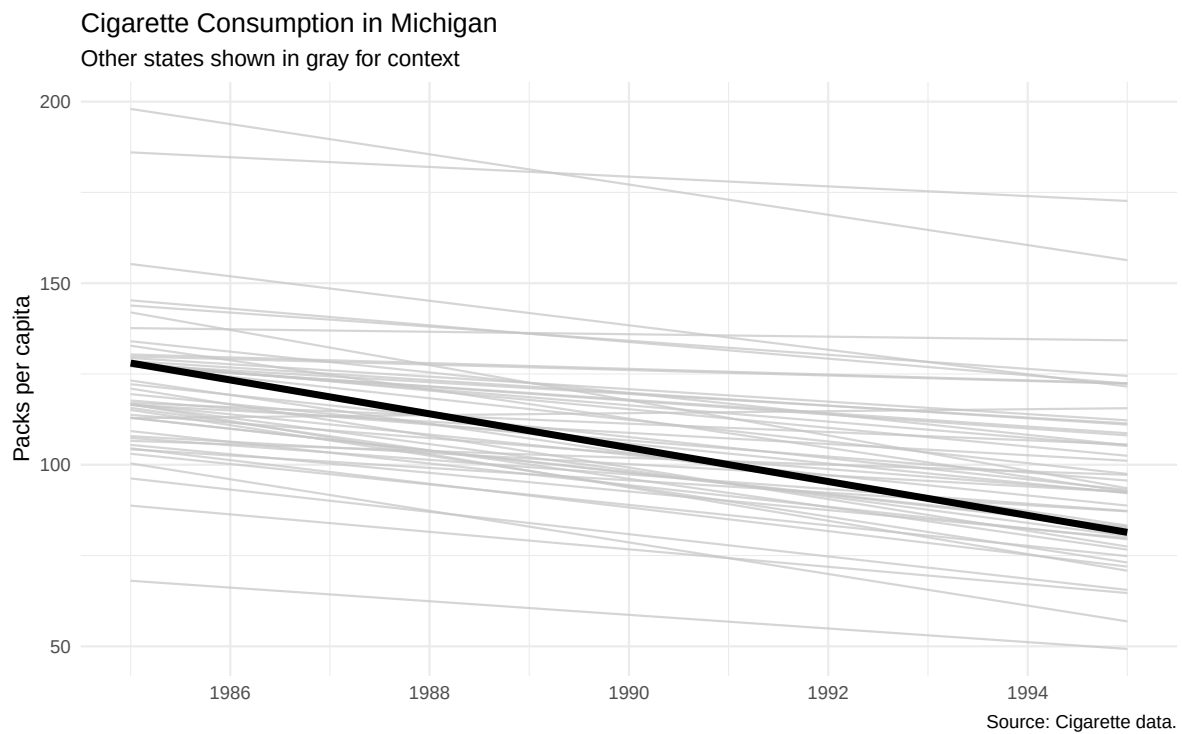
```

```

geom_line(
  data = cigarettes |> filter(state == "MI"),
  color = "black",
  linewidth = 1.6
) +
scale_x_date(date_breaks = "2 years", date_labels = "%Y") +
labs(
  title = "Cigarette Consumption in Michigan",
  subtitle = "Other states shown in gray for context",
  x = NULL,
  y = "Packs per capita",
  caption = "Source: Cigarette data."
) +
theme_minimal()

p_michigan

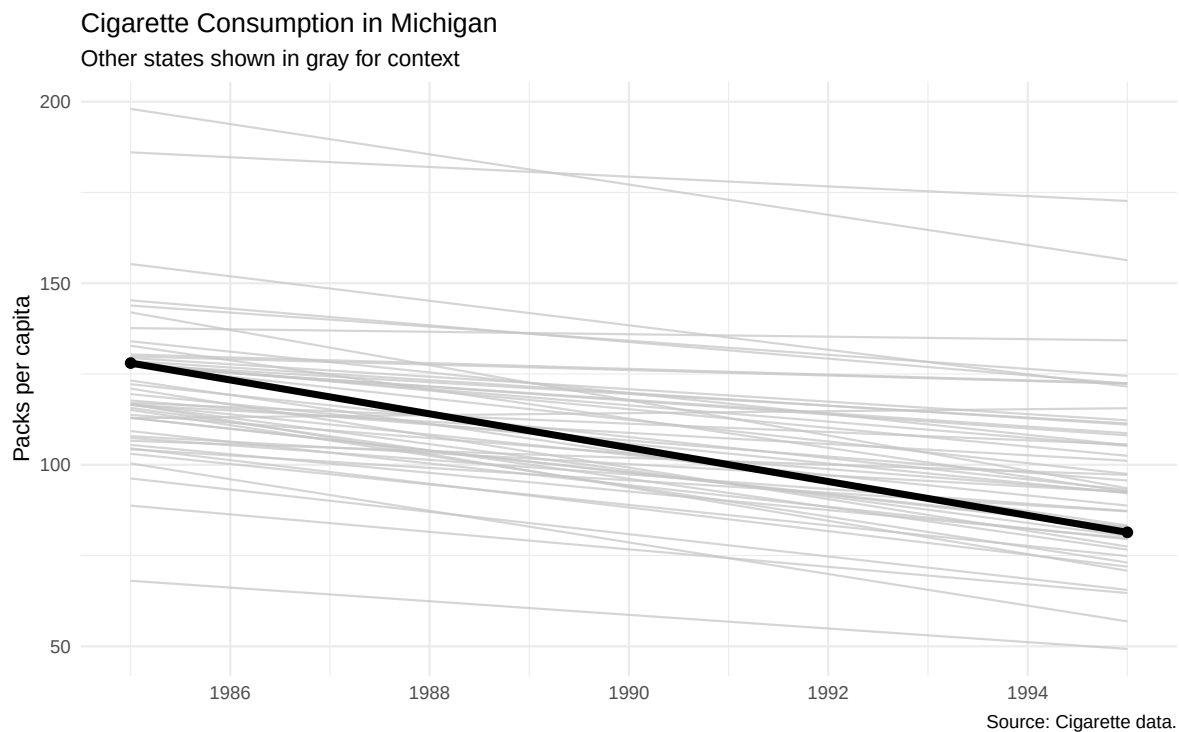
```



8.10 Add points only where they help

Points can show the actual observations, but adding points to every state creates clutter. Add them only to the focal state.

```
p_michigan_points <- p_michigan +  
  geom_point(  
    data = cigarettes |> filter(state == "MI"),  
    mapping = aes(x = date, y = packpc),  
    inherit.aes = FALSE,  
    color = "black",  
    size = 2  
  )  
  
p_michigan_points
```



This is a judgment call. The points make the Michigan measurements visible, but they may be unnecessary once the line and label are clear.

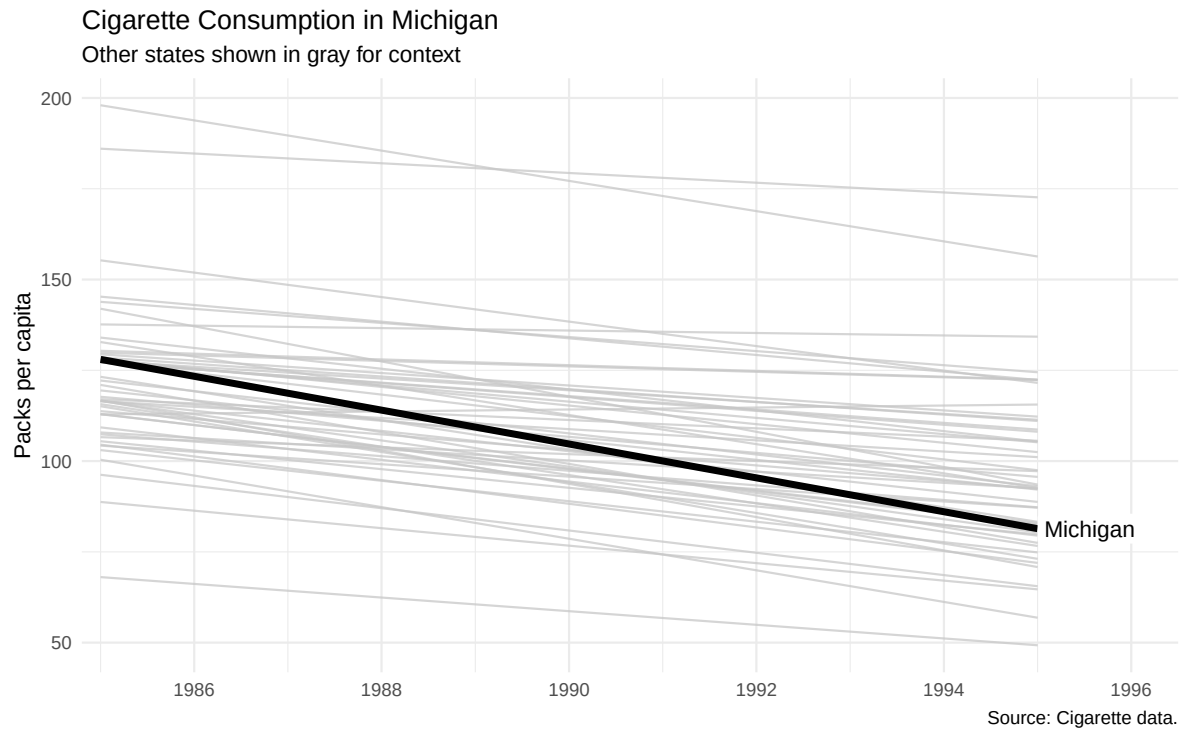
8.11 Add a direct label

Layer a Michigan label onto the saved plot, and widen the right margin so the label has room.

```
mi_last <- cigarettes |>
  filter(state == "MI") |>
  slice_max(year, n = 1, with_ties = FALSE)

p_michigan_labeled <- p_michigan +
  geom_label(
    data = mi_last,
    mapping = aes(x = date, y = packpc, label = "Michigan"),
    inherit.aes = FALSE,
    hjust = 0,
    label.size = 0,
    fill = "white"
  ) +
  scale_x_date(
    date_breaks = "2 years",
    date_labels = "%Y",
    expand = expansion(mult = c(0.02, 0.15))
  )

p_michigan_labeled
```



8.12 Compare Michigan to the average state

Another way to answer the question is to compare Michigan to a yearly average.

```
average_by_year <- cigarettes |>
  group_by(year, date) |>
  summarize(avg_packpc = mean(packpc, na.rm = TRUE))

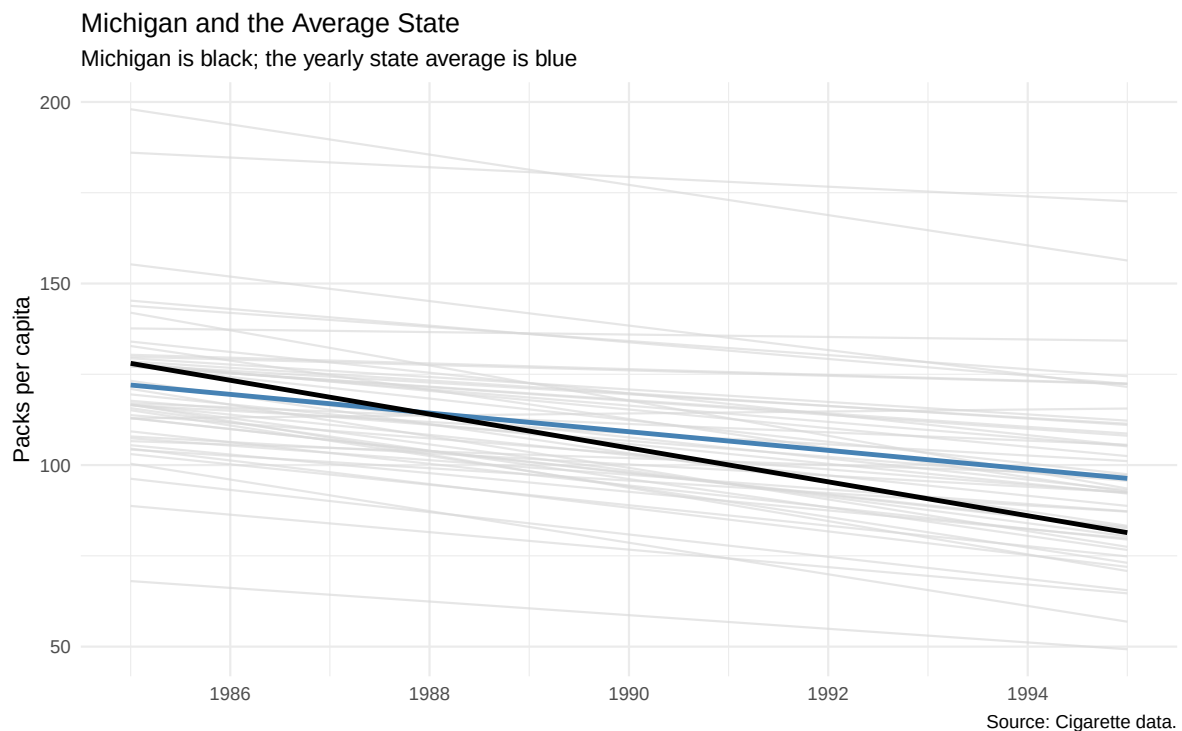
mi_data <- cigarettes |>
  filter(state == "MI")

ggplot() +
  geom_line(
    data = cigarettes,
    mapping = aes(x = date, y = packpc, group = state),
    color = "gray85",
    alpha = 0.65
  ) +
  geom_line(
```

```

data = average_by_year,
mapping = aes(x = date, y = avg_packpc),
color = "steelblue",
linewidth = 1.1
) +
geom_line(
  data = mi_data,
  mapping = aes(x = date, y = packpc),
  color = "black",
  linewidth = 1.1
) +
scale_x_date(date_breaks = "2 years", date_labels = "%Y") +
labs(
  title = "Michigan and the Average State",
  subtitle = "Michigan is black; the yearly state average is blue",
  x = NULL,
  y = "Packs per capita",
  caption = "Source: Cigarette data."
) +
theme_minimal()

```



This version introduces a benchmark. The benchmark makes the comparison more specific than “Michigan versus a cloud of states.”

8.13 Final line figure

This version keeps the key comparison and reduces visual clutter.

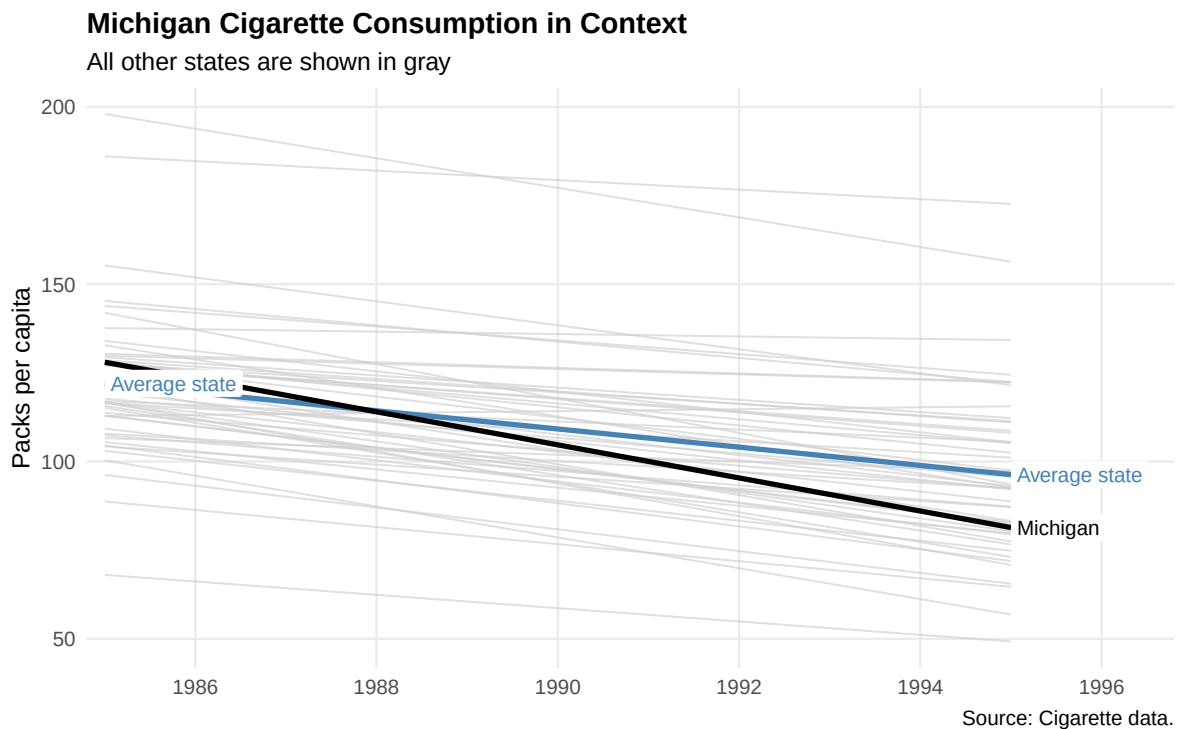
```
final_line <- ggplot(data = cigarettes, mapping = aes(x = date, y = packpc, group = state)) +
  geom_line(color = "gray82", alpha = 0.7, linewidth = 0.45) +
  geom_line(
    data = average_by_year,
    mapping = aes(x = date, y = avg_packpc),
    inherit.aes = FALSE,
    color = "steelblue",
    linewidth = 1.2
  ) +
  geom_line(
    data = mi_data,
    mapping = aes(x = date, y = packpc),
    inherit.aes = FALSE,
    color = "black",
    linewidth = 1.2
  ) +
  geom_label(
    data = mi_last,
    mapping = aes(x = date, y = packpc, label = "Michigan"),
    inherit.aes = FALSE,
    hjust = 0,
    label.size = 0,
    fill = "white",
    size = 3.5
  ) +
  geom_label(
    data = average_by_year |> slice_max(year, n = 1, with_ties = FALSE),
    mapping = aes(x = date, y = avg_packpc, label = "Average state"),
    inherit.aes = FALSE,
    hjust = 0,
    label.size = 0,
    fill = "white",
    color = "steelblue",
    size = 3.5
  )
```

```

) +
scale_x_date(
  date_breaks = "2 years",
  date_labels = "%Y",
  expand = expansion(mult = c(0.02, 0.18))
) +
labs(
  title = "Michigan Cigarette Consumption in Context",
  subtitle = "All other states are shown in gray",
  x = NULL,
  y = "Packs per capita",
  caption = "Source: Cigarette data."
) +
theme_minimal(base_size = 12) +
theme(
  panel.grid.minor = element_blank(),
  plot.title = element_text(face = "bold")
)

final_line

```



This figure has a clear message:

- Michigan is the focal state.
- Other states provide context.
- The average state provides a benchmark.
- Direct labels remove the need for a legend.

8.14 Exporting

Use `ggsave()` to export a plot object. The file path below writes to an `Output` folder. Create that folder first if you use this chunk.

```
ggsave(  
  filename = "Output/michigan-cigarette-consumption.png",  
  plot = final_line,  
  width = 9,  
  height = 5,  
  dpi = 300  
)
```

The important arguments are the filename, plot object, dimensions, and resolution. For a paper or slide deck, decide the intended size before exporting.

8.15 Checklist for final figures

Before treating a figure as final, check:

- Does the title describe the main comparison?
- Are the axes labeled in ordinary language?
- Are units visible?
- Is the source named?
- Is color doing data work, emphasis work, or unnecessary decoration?
- Is a legend necessary, or would direct labels be clearer?
- Would a reader know what to look at first?

8.16 Exercise

Build a final figure from the same data.

1. Choose a state other than Michigan.
2. Create a line chart that highlights that state.
3. Add all other states in gray.
4. Add the average state line as a benchmark.
5. Directly label the highlighted state.
6. Write a two-sentence caption that explains the comparison.

```
# Write your workflow here.
```

8.17 Extra: summarize total change

Sometimes a line chart is not enough. A related question is which states changed the most from the first year to the last year.

The function `pivot_wider()` comes from `tidyr`, which is loaded as part of the tidyverse. Here it puts the first and last years into separate columns so the change can be calculated directly.

```
first_last <- cigarettes |>
  filter(year %in% range(year, na.rm = TRUE)) |>
  select(state, year, packpc) |>
  pivot_wider(names_from = year, values_from = packpc, names_prefix = "year_")

first_year_name <- str_c("year_", min(cigarettes$year, na.rm = TRUE))
last_year_name <- str_c("year_", max(cigarettes$year, na.rm = TRUE))

change_by_state <- first_last |>
  mutate(
    change = .data[[last_year_name]] - .data[[first_year_name]],
    state_label = if_else(state == "MI", "Michigan", state)
  )

change_by_state |>
  arrange(change) |>
  head(10)
```

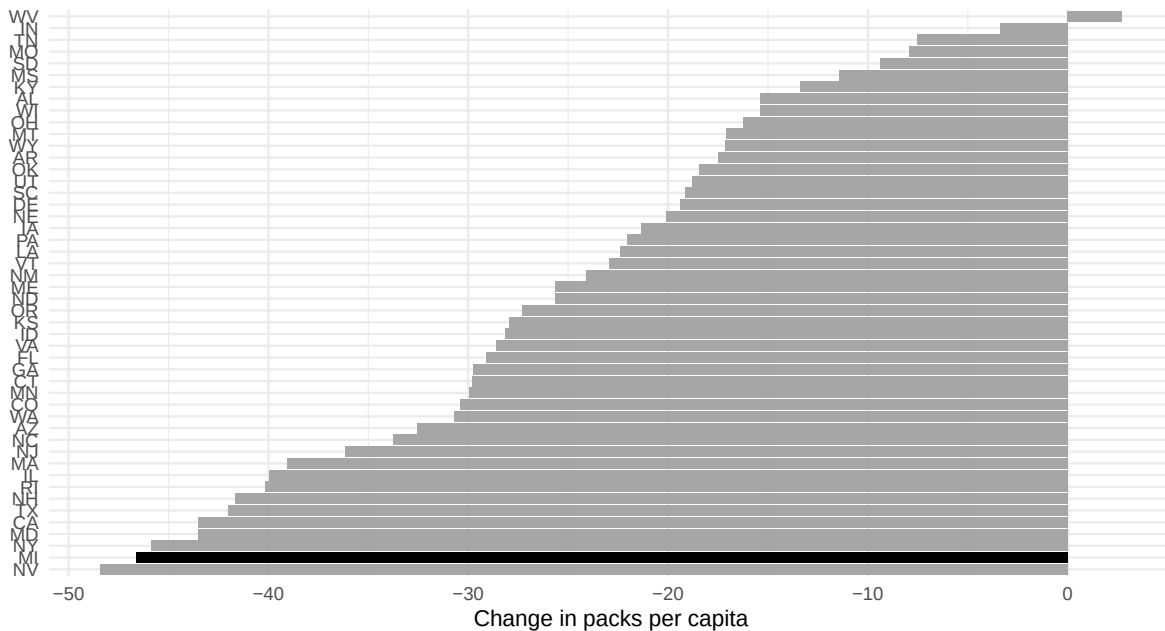
```
# A tibble: 10 x 5
  state year_1985 year_1995 change state_label
```

	<chr>	<dbl>	<dbl>	<dbl>	<chr>
1	NV	142.	93.5	-48.4	NV
2	MI	128.	81.4	-46.6	Michigan
3	NY	117.	70.8	-45.8	NY
4	MD	121.	77.5	-43.5	MD
5	CA	100.	56.9	-43.5	CA
6	TX	115.	73.1	-42.0	TX
7	NH	198.	156.	-41.7	NH
8	RI	133.	92.6	-40.2	RI
9	IL	123.	83.3	-39.9	IL
10	MA	116.	76.6	-39.1	MA

Now make an ordered bar chart of change.

```
ggplot(data = change_by_state, mapping = aes(x = reorder(state, change), y = change)) +
  geom_col(aes(fill = state == "MI"), show.legend = FALSE) +
  coord_flip() +
  scale_fill_manual(values = c("FALSE" = "gray65", "TRUE" = "black")) +
  labs(
    title = "Change in Cigarette Consumption by State",
    subtitle = str_glue("{min(cigarettes$year, na.rm = TRUE)} to {max(cigarettes$year, na.rm = TRUE)}",
    x = NULL,
    y = "Change in packs per capita",
    caption = "Source: Cigarette data."
  ) +
  theme_minimal()
```

Change in Cigarette Consumption by State
1985 to 1995



Source: Cigarette data.

This is a different figure for a related question. The line chart shows trajectories. The bar chart shows total change.

8.18 Extra: alternative question

The same dataset can answer a regional question if a `region` column is available.

```
if ("region" %in% names(cigarettes)) {
  region_year <- cigarettes |>
    group_by(region, year, date) |>
    summarize(avg_packpc = mean(packpc, na.rm = TRUE))

  ggplot(data = region_year, mapping = aes(x = date, y = avg_packpc, color = region)) +
    geom_line(linewidth = 1.1) +
    scale_x_date(date_breaks = "2 years", date_labels = "%Y") +
    labs(
      title = "Average Cigarette Consumption by Region",
      x = NULL,
      y = "Average packs per capita",
      color = "Region",
    )
}
```

```
    caption = "Source: Cigarette data."  
  ) +  
  theme_minimal()  
}
```

This is not the same figure with a different color scheme. It is a different question: regional averages rather than one focal state.

9 Interactivity And Dashboards

9.1 Learning Goals

By the end of this chapter, the main goals are:

- convert a static `ggplot2` figure to an interactive HTML figure with `ggplotly()`
- control hover text so interactive output shows useful information
- recognize when a dashboard is a good output format
- understand the basic structure of a Quarto dashboard file

9.2 Where This Fits

The earlier chapters focused on data import, data wrangling, static plots, labels, scales, annotations, and complete figure revision. This chapter keeps those skills in place and changes the output form.

Interactivity is most useful when readers need to inspect details, zoom into dense areas, or compare individual cases. A dashboard is most useful when several related outputs belong together.

The underlying workflow is still the same:

1. load the data
2. inspect the variables
3. wrangle the data into the shape needed for the plot
4. build a clear static version
5. add interactivity or package the output only after the basic figure works

9.3 Setup

This chapter uses the local cigarette data copied into the 2026 course project.

```

library(tidyverse)
library(plotly)
library(scales)

cigarettes <- read_csv(
  "Data/cigarettes/cigarettes.csv",
  show_col_types = FALSE
) |>
  rename(
    packpc = any_of("packs"),
    avgprs = any_of("price"),
    pop = any_of("population")
  ) |>
  mutate(
    year_date = make_date(year),
    real_tax = tax / cpi,
    real_price = avgprs / cpi,
    income_per_capita = income / pop
  )

```

9.4 From Static To Interactive

The easiest route into interactivity is to begin with a normal `ggplot2` plot. The plot below uses the final year in the cigarette data and compares state cigarette taxes with annual packs per capita.

```

cigarettes_1995 <- cigarettes |>
  filter(year == max(year))

tax_plot <- ggplot(
  cigarettes_1995,
  aes(x = real_tax, y = packpc)
) +
  geom_point(color = "gray35", alpha = 0.75, size = 2.4) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE, color = "firebrick") +
  scale_x_continuous(labels = label_number(suffix = " cents")) +
  labs(
    title = "Cigarette Taxes And Consumption",
    subtitle = "U.S. states in 1995",
    x = "Real cigarette tax per pack",

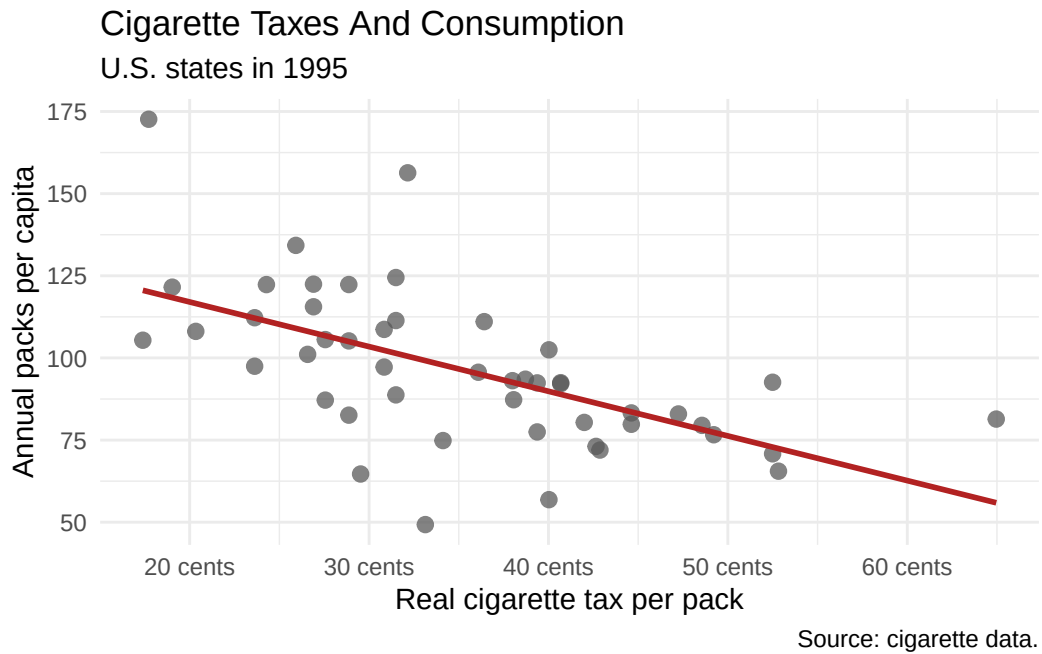
```

```

y = "Annual packs per capita",
caption = "Source: cigarette data."
) +
theme_minimal()

```

```
tax_plot
```

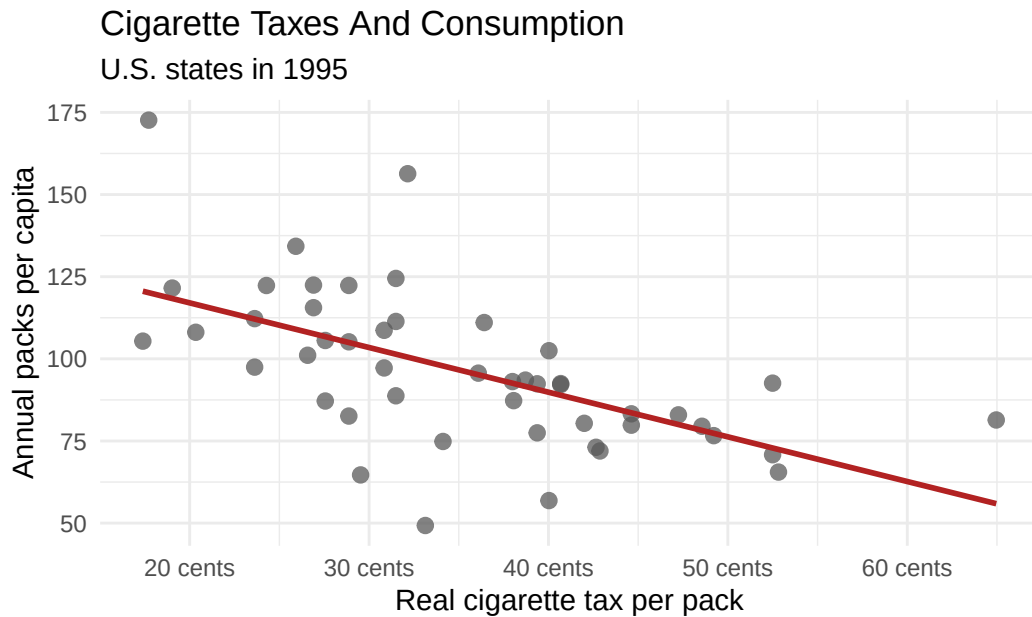


The static plot should come first because it is easier to check. The next step wraps the completed plot in `ggplotly()`.

```

if (knitr::is_html_output()) {
  ggplotly(tax_plot)
} else {
  tax_plot
}

```

That one call adds hover, zoom, and pan behavior in HTML output. It does not create an interactive PDF. If the document is rendered to PDF, Quarto will use a static version of the plot.

9.5 Custom Hover Text

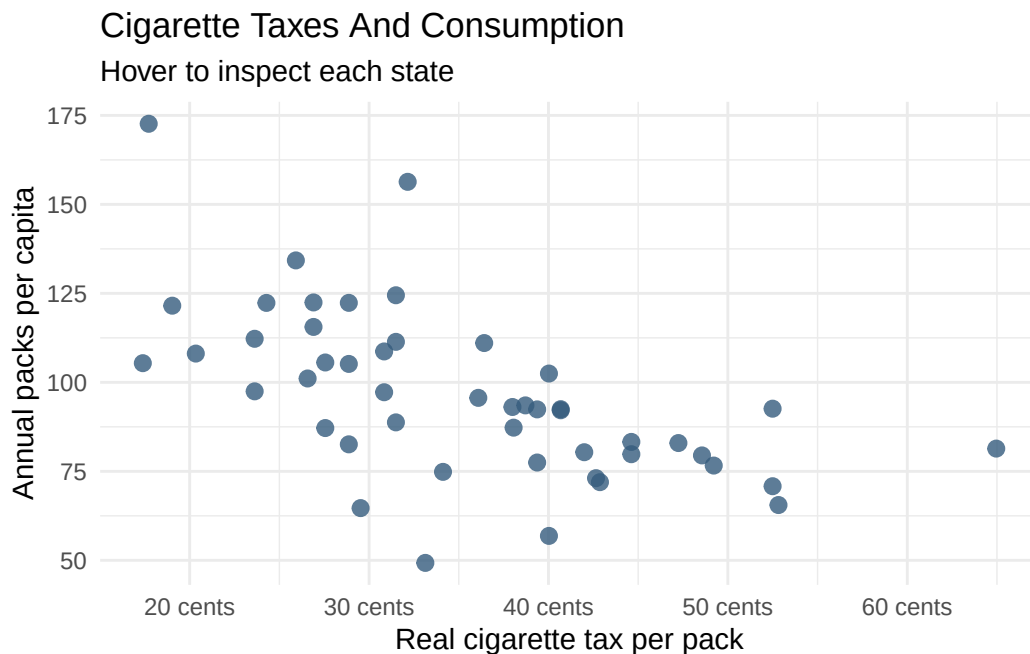
The default hover text is often too mechanical. It usually reports mapped variables, but it may not show the case name or format numbers cleanly.

Add a `text` aesthetic when building the plot. Then tell `ggplotly()` to use only that text for the tooltip.

```
cigarettes_1995_hover <- cigarettes_1995 |>
  mutate(
    hover_text = paste0(
      state,
      "<br>Real tax: ", number(real_tax, accuracy = 0.1), " cents",
      "<br>Packs per capita: ", number(packpc, accuracy = 0.1),
      "<br>Real price: ", number(real_price, accuracy = 0.1), " cents"
    )
  )
```

```
tax_hover_plot <- ggplot(
  cigarettes_1995_hover,
  aes(x = real_tax, y = packpc, text = hover_text)
) +
  geom_point(color = "#355c7d", alpha = 0.8, size = 2.5) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE, color = "#c44d58") +
  scale_x_continuous(labels = label_number(suffix = " cents")) +
  labs(
    title = "Cigarette Taxes And Consumption",
    subtitle = "Hover to inspect each state",
    x = "Real cigarette tax per pack",
    y = "Annual packs per capita"
  ) +
  theme_minimal()

if (knitr::is_html_output()) {
  ggplotly(tax_hover_plot, tooltip = "text")
} else {
  tax_hover_plot
}
```



The `<br>` tags create line breaks inside the HTML tooltip. This is one place where HTML syntax appears inside R code because the output is an HTML widget.

9.6 Interactive Time Series

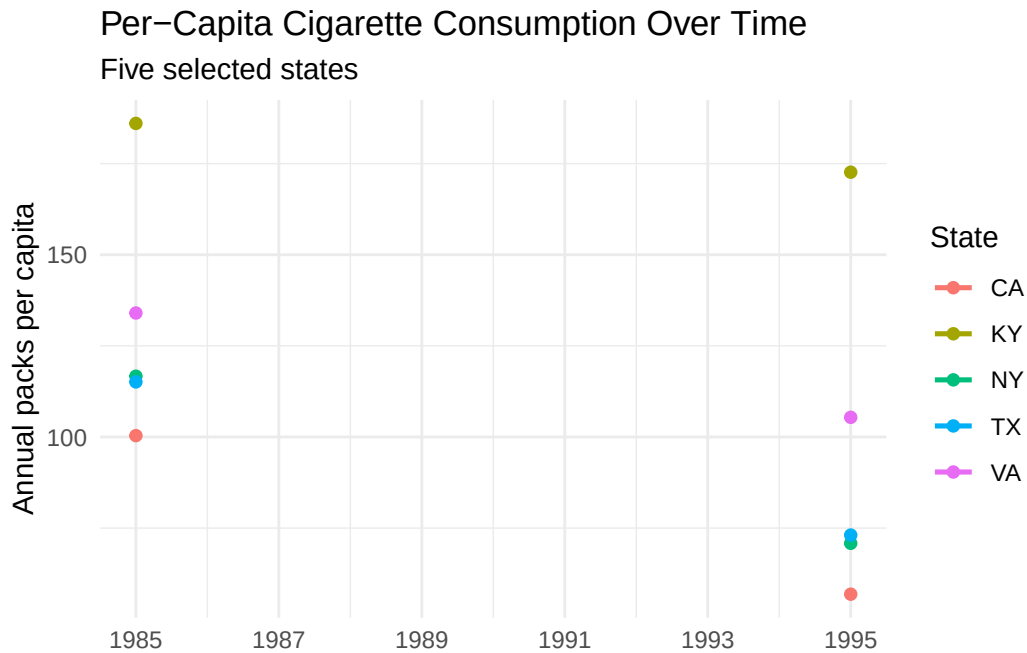
Interactivity is also useful for line charts. Hover text can identify the state, year, and exact value without putting every label directly on the static figure.

```
focus_states <- c("CA", "KY", "NY", "TX", "VA")

focus_trends <- cigarettes |>
  filter(state %in% focus_states) |>
  mutate(
    hover_text = paste0(
      state,
      "<br>Year: ", year,
      "<br>Packs per capita: ", number(packpc, accuracy = 0.1),
      "<br>Real tax: ", number(real_tax, accuracy = 0.1), " cents"
    )
  )

trend_plot <- ggplot(
  focus_trends,
  aes(x = year, y = packpc, color = state, text = hover_text)
) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 1.7) +
  scale_x_continuous(breaks = seq(1985, 1995, by = 2)) +
  labs(
    title = "Per-Capita Cigarette Consumption Over Time",
    subtitle = "Five selected states",
    x = NULL,
    y = "Annual packs per capita",
    color = "State"
  ) +
  theme_minimal()

if (knitr::is_html_output()) {
  ggplotly(trend_plot, tooltip = "text")
} else {
  trend_plot
}
```



9.7 Linked Highlighting

plotly can also dim unselected lines when the pointer moves across a figure. The `highlight_key()` function attaches an interactive key to the data. Here the key is the state abbreviation.

```
trend_key <- highlight_key(focus_trends, ~state)

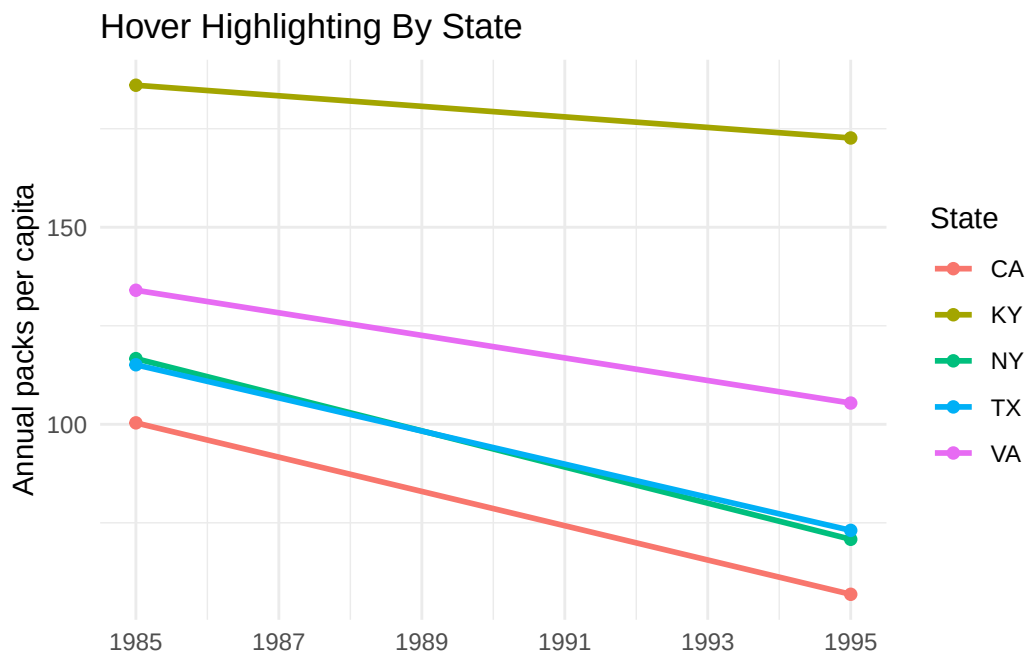
highlight_plot <- ggplot(
  trend_key,
  aes(x = year, y = packpc, color = state)
) +
  geom_line(linewidth = 1) +
  geom_point(aes(text = hover_text), size = 1.7) +
  scale_x_continuous(breaks = seq(1985, 1995, by = 2)) +
  labs(
    title = "Hover Highlighting By State",
    x = NULL,
    y = "Annual packs per capita",
    color = "State"
  ) +
```

```

theme_minimal()

if (knitr::is_html_output()) {
  ggplotly(highlight_plot, tooltip = "text") |>
    highlight(
      on = "plotly_hover",
      dynamic = TRUE,
      opacityDim = 0.18,
      selected = attrs_selected(line = list(width = 4))
    )
} else {
  highlight_plot
}

```



This is useful when a line chart contains several series. It is less useful when a plot is already simple enough to read without interaction.

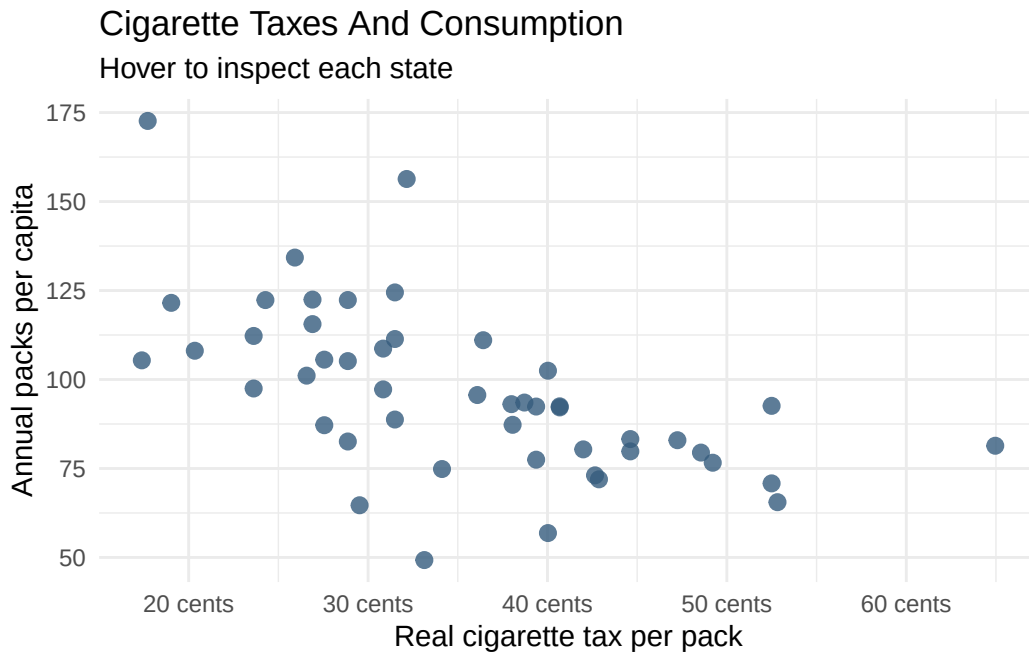
9.8 Toolbar Control

By default, plotly shows a toolbar when the pointer enters the figure. `config(displayModeBar = FALSE)` hides it entirely, which produces a cleaner embed in a report. Set it to `TRUE` to keep the toolbar always visible when zoom or download buttons are useful.

```

if (knitr::is_html_output()) {
  ggplotly(tax_hover_plot, tooltip = "text") |>
    config(displayModeBar = FALSE)
} else {
  tax_hover_plot
}

```



Plotly's `layout()` function exposes many more options — tooltip styling, margins, legend placement, axis formatting — but most of that work is better done in `ggplot2` before the conversion, where the tools are more familiar.

9.9 Highcharter

`highcharter` is an R interface to the Highcharts JavaScript library. It is another route to interactive HTML figures. The syntax is different from `ggplot2`: instead of building a static plot and converting it with `ggplotly()`, `highcharter` builds the interactive widget directly.

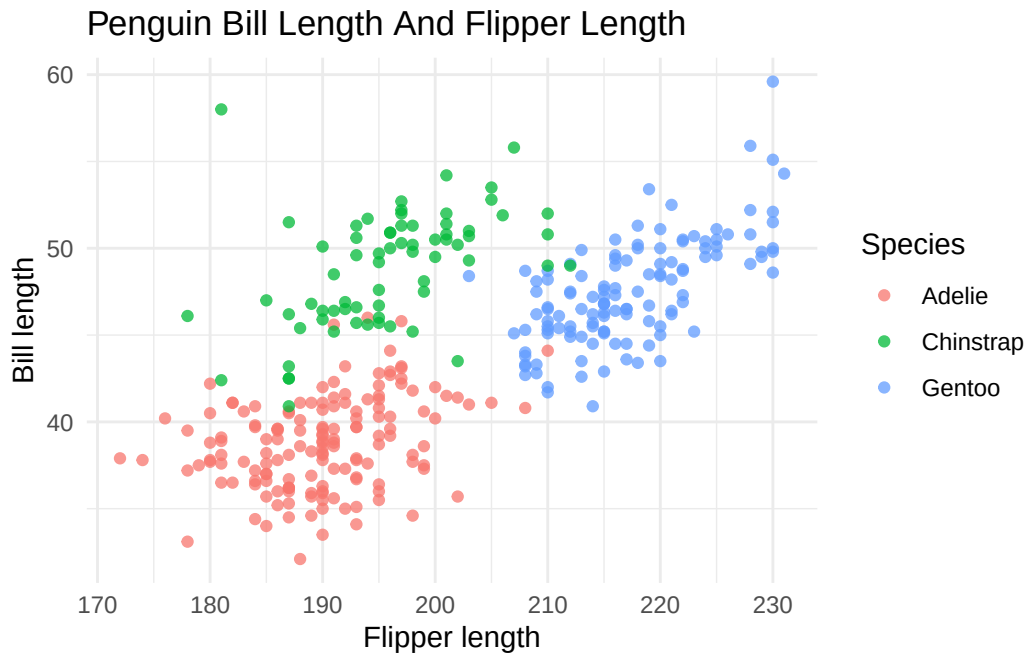
The examples in this section use local or built-in data. They render as interactive widgets in HTML. In PDF output, the chunks show static `ggplot2` versions because JavaScript widgets are HTML output.

```

penguins <- read_csv("Data/penguins/penguins.csv", show_col_types = FALSE) |>
  filter(!is.na(flipper_length_mm), !is.na(bill_length_mm), !is.na(species))

if (knitr::is_html_output()) {
  highcharter::hchart(
    penguins,
    "scatter",
    highcharter::hcaes(
      x = flipper_length_mm,
      y = bill_length_mm,
      group = species
    )
  ) |>
  highcharter::hc_title(text = "Penguin Bill Length And Flipper Length") |>
  highcharter::hc_xAxis(title = list(text = "Flipper length")) |>
  highcharter::hc_yAxis(title = list(text = "Bill length"))
} else {
  ggplot(
    penguins,
    aes(x = flipper_length_mm, y = bill_length_mm, color = species)
  ) +
    geom_point(alpha = 0.75) +
    labs(
      title = "Penguin Bill Length And Flipper Length",
      x = "Flipper length",
      y = "Bill length",
      color = "Species"
    ) +
    theme_minimal()
}

```



This example restores a second path into interactive scatterplots. It also reinforces a broader pattern: interactive charts still need clear variables, readable labels, and data cleaning before the widget is created.

9.10 Highcharter Line Charts

Line charts are a natural place to use interactivity because hover text can identify the exact date and value without crowding the static figure.

```
economics_long2 <- economics_long |>
  filter(variable %in% c("pop", "uempmed", "unemploy"))

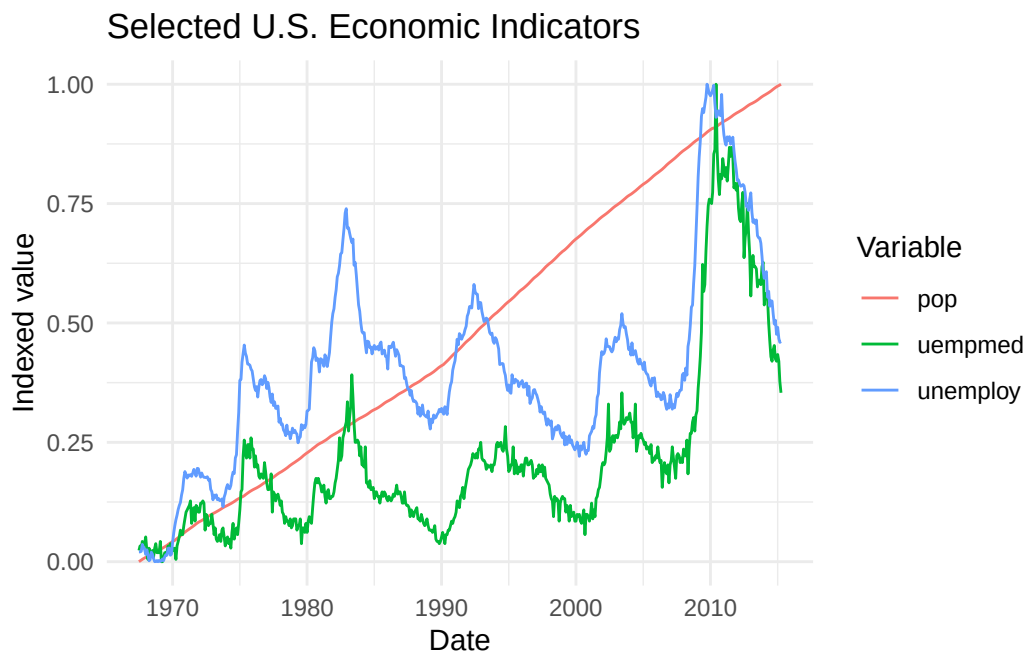
if (knitr::is_html_output()) {
  highcharter::hchart(
    economics_long2,
    "line",
    highcharter::hcaes(x = date, y = value01, group = variable)
  ) |>
  highcharter::hc_title(text = "Selected U.S. Economic Indicators") |>
  highcharter::hc_xAxis(title = list(text = "Date")) |>
  highcharter::hc_yAxis(title = list(text = "Indexed value")) |>
  highcharter::hc_tooltip(valueDecimals = 2)
```



```

} else {
  ggplot(economics_long2, aes(x = date, y = value01, color = variable)) +
    geom_line() +
    labs(
      title = "Selected U.S. Economic Indicators",
      x = "Date",
      y = "Indexed value",
      color = "Variable"
    ) +
    theme_minimal()
}

```



9.11 Highcharter Range Selector

Some Highcharts outputs include interface controls that go beyond ordinary hover text. A range selector lets the reader zoom to common time windows.

```

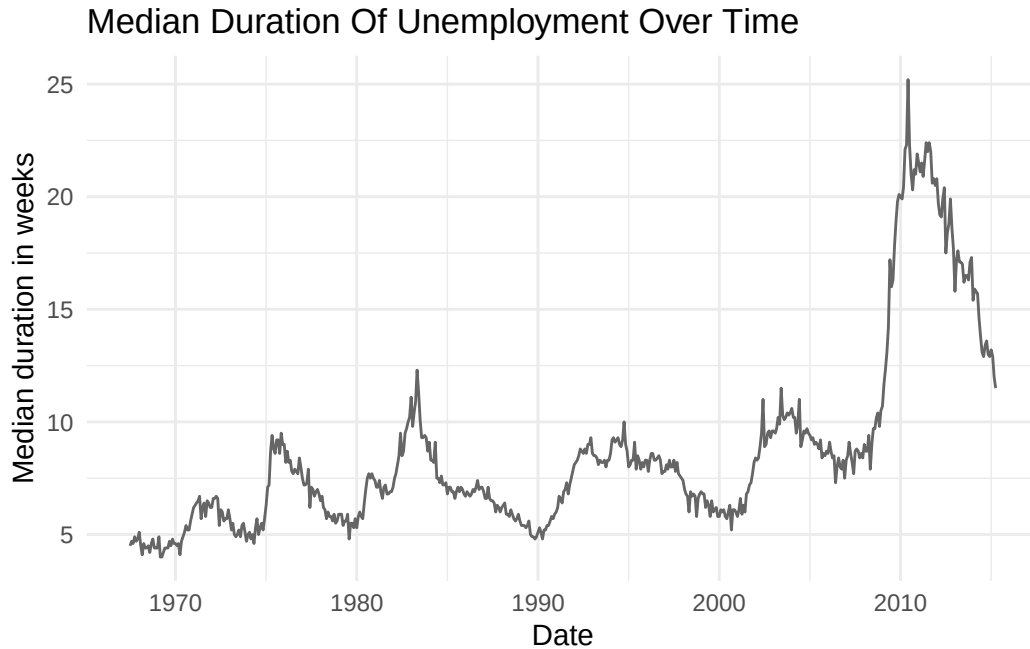
if (knitr::is_html_output()) {
  highcharter::hchart(
    economics,
    "line",

```

```

    highcharter::hcaes(x = date, y = uempmed)
  ) |>
  highcharter::hc_title(
    text = "Median Duration Of Unemployment Over Time"
  ) |>
  highcharter::hc_xAxis(type = "datetime", title = list(text = "Date")) |>
  highcharter::hc_yAxis(title = list(text = "Median duration in weeks")) |>
  highcharter::hc_rangeSelector(
    enabled = TRUE,
    buttons = list(
      list(type = "year", count = 1, text = "1y"),
      list(type = "year", count = 5, text = "5y"),
      list(type = "year", count = 10, text = "10y"),
      list(type = "all", text = "All")
    ),
    inputEnabled = TRUE
  )
} else {
  ggplot(economics, aes(x = date, y = uempmed)) +
    geom_line(color = "gray40") +
    labs(
      title = "Median Duration Of Unemployment Over Time",
      x = "Date",
      y = "Median duration in weeks"
    ) +
    theme_minimal()
}

```



The range selector is a good example of why HTML output can matter. The same underlying data can become a static figure in PDF or an exploratory widget in HTML.

9.12 Highcharter Column Chart

Interactive bar and column charts are useful when exact values should appear on hover but do not need to be printed on every bar.

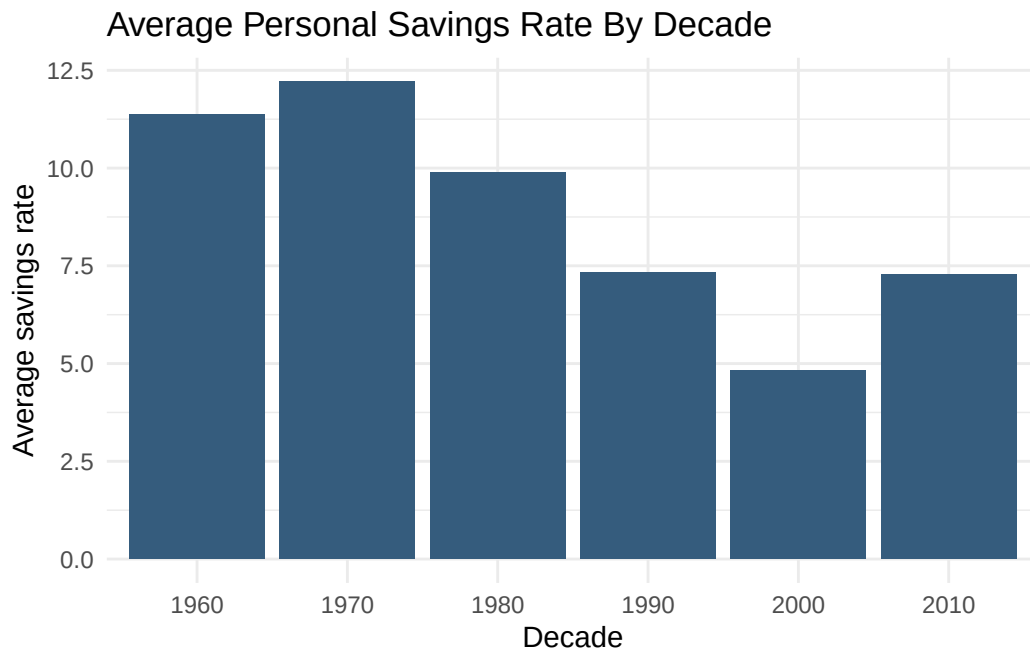
```
economics_decade <- economics |>
  mutate(decade = floor(year(date) / 10) * 10) |>
  group_by(decade) |>
  summarize(avg_psavert = mean(psavert, na.rm = TRUE))

if (knitr::is_html_output()) {
  highcharter::hchart(
    economics_decade,
    "column",
    highcharter::hcaes(x = decade, y = avg_psavert)
  ) |>
  highcharter::hc_title(text = "Average Personal Savings Rate By Decade") |>
  highcharter::hc_xAxis(title = list(text = "Decade")) |>
  highcharter::hc_yAxis(title = list(text = "Average savings rate")) |>
```

```

    highcharter::hc_tooltip(
      pointFormat = "Decade: {point.x}<br>Average savings rate: {point.y:.1f}%"
    )
  } else {
    ggplot(economics_decade, aes(x = factor(decade), y = avg_psavert)) +
      geom_col(fill = "#355c7d") +
      labs(
        title = "Average Personal Savings Rate By Decade",
        x = "Decade",
        y = "Average savings rate"
      ) +
      theme_minimal()
  }
}

```



9.13 Highcharter Heatmap

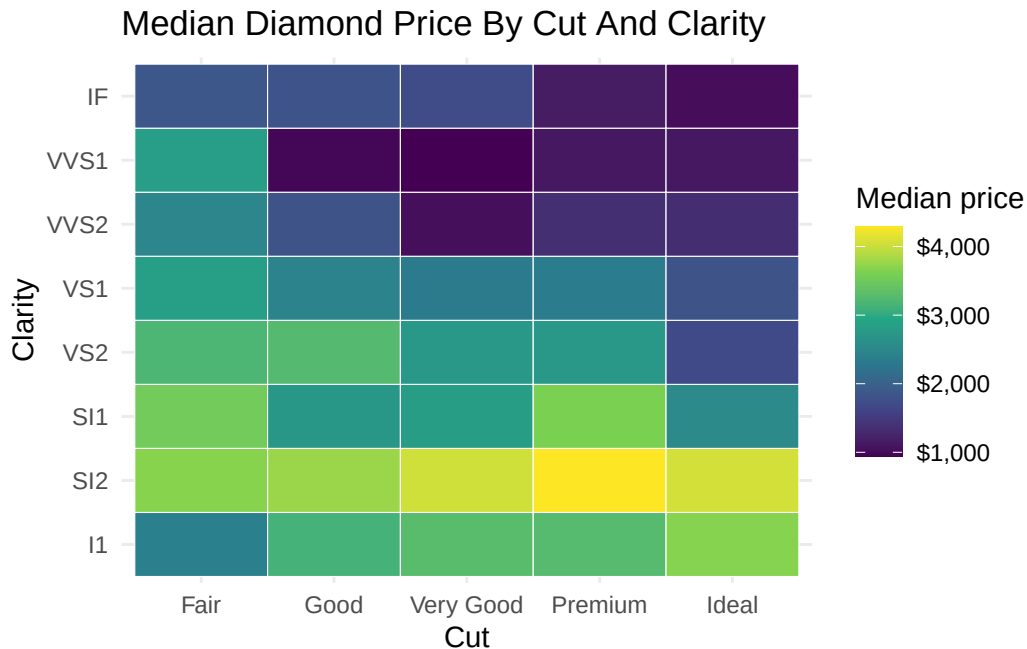
Heatmaps are another common interactive form. The hover text can show the exact value inside each cell.

```

diamonds_heatmap <- diamonds |>
  group_by(cut, clarity) |>
  summarize(median_price = median(price))

if (knitr::is_html_output()) {
  highcharter::hchart(
    diamonds_heatmap,
    "heatmap",
    highcharter::hcaes(x = cut, y = clarity, value = median_price),
    name = "Median price"
  ) |>
  highcharter::hc_title(text = "Median Diamond Price By Cut And Clarity") |>
  highcharter::hc_tooltip(
    pointFormat = "Cut: {point.x}<br>Clarity: {point.y}<br>Median price: ${point.value:.0f}"
  )
} else {
  ggplot(diamonds_heatmap, aes(x = cut, y = clarity, fill = median_price)) +
    geom_tile(color = "white") +
    scale_fill_viridis_c(labels = dollar) +
    labs(
      title = "Median Diamond Price By Cut And Clarity",
      x = "Cut",
      y = "Clarity",
      fill = "Median price"
    ) +
    theme_minimal()
}

```



The practical choice is not `plotly` versus `highcharter` in the abstract. The choice is which tool produces the clearest result for a specific data task and output format.

9.14 Dashboards As Output

A dashboard is a layout for several related outputs. It is not a substitute for choosing good measures or building readable plots.

A useful dashboard usually contains a small set of components:

- one or two headline quantities
- one main relationship or trend
- one supporting distribution or ranking
- one table or lookup view when individual cases matter

The course includes two standalone dashboard demos.

9.14.1 Baby Names Dashboard

`Demos/babynames-dashboard-demo.qmd` uses U.S. Social Security name data from 1880 to 2017. It includes overview cards, a trend chart for selected names, a top-15 ranking for the most recent year, a sex-split trend for a single name, and an all-time table.

9.14.2 Cigarette Tax Dashboard

Demos/dashboard-demo.qmd uses state-level cigarette consumption and tax data. It includes summary cards, an interactive scatterplot of taxes against consumption, a ranking of highest-tax states, a trend chart for selected states, and a change table.

Both files use the same dashboard format. The YAML header tells Quarto to use it:

```
---
title: "Dashboard Demo"
format:
  dashboard:
    orientation: columns
---
```

Inside the file, headings create sections, columns, and cards. The plots and tables are ordinary R code chunks. The source files contain comments explaining the data preparation and plotting choices.

9.15 Exercise

Take one static plot from an earlier chapter and make it interactive.

1. Save the static plot as a named object.
2. Add a `text` aesthetic with at least three lines of hover text.
3. Convert the plot with `ggplotly(plot_name, tooltip = "text")`.
4. Decide whether the interactive version adds useful detail or only adds novelty.

9.16 Closing

Interactivity and dashboards work best after the static figure is already understandable. The durable sequence is to build the plot, check the plot, then choose whether interaction or dashboard packaging helps the audience use it.

10 Tables

Counts, summaries, publication tables, and export

Tables are part of a visualization workflow. A plot is often the fastest way to show a pattern, but a table is often the clearest way to show exact values, counts, summaries, and compact descriptions of a dataset.

This chapter starts with quick exploratory tables and then moves toward tables that belong in reports, slides, papers, and appendices.

```
library(tidyverse)
library(janitor)
library(tinytable)
library(modelsummary)
library(corr)
```

10.1 A first frequency table

Base R has a built-in function called `table()`. It counts how often each value appears.

```
table(mtcars$cyl)
```

```
 4  6  8
11  7 14
```

This table counts the number of cars in `mtcars` with 4, 6, and 8 cylinders.

A two-way table counts combinations of two variables.

```
table(mtcars$cyl, mtcars$gear)
```


	3	4	5
4	1	8	2
6	2	4	1
8	12	0	2

The output is useful, but the labels are not very descriptive. We can improve the row and column labels before printing the table.

```
cylinders_gear <- table(mtcars$cyl, mtcars$gear)

dimnames(cylinders_gear) <- list(
  "Cylinders" = paste(rownames(cylinders_gear), "cyl"),
  "Gears" = paste(colnames(cylinders_gear), "gears")
)

cylinders_gear
```

		Gears		
Cylinders		3 gears	4 gears	5 gears
4 cyl		1	8	2
6 cyl		2	4	1
8 cyl		12	0	2

Base R tables are dependable and fast. They are especially useful when you are exploring data in the Console. For tables that will appear in a report, other packages give us cleaner formatting and easier percentages.

10.2 Cleaner crosstabs with janitor

The `janitor` package has a function called `tabyl()`. It behaves like a modern version of `table()`: it counts values, keeps the output as a data frame, and works well in tidyverse pipelines.

We will use a local copy of the Gapminder data.

The function `glimpse()` comes from `dplyr`, which is loaded as part of the tidyverse.

```
gapminder <- readr::read_csv("Data/gapminder/gapminder.csv")

glimpse(gapminder)
```

```

Rows: 1,704
Columns: 6
$ country   <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", ~
$ continent <chr> "Asia", "Asia", "Asia", "Asia", "Asia", "Asia", "Asia", "Asi~
$ year      <dbl> 1952, 1957, 1962, 1967, 1972, 1977, 1982, 1987, 1992, 1997, ~
$ lifeExp   <dbl> 28.801, 30.332, 31.997, 34.020, 36.088, 38.438, 39.854, 40.8~
$ pop       <dbl> 8425333, 9240934, 10267083, 11537966, 13079460, 14880372, 12~
$ gdpPercap <dbl> 779.4453, 820.8530, 853.1007, 836.1971, 739.9811, 786.1134, ~

```

A one-variable `tabyl()` gives counts and proportions.

```

gapminder |>
  tabyl(continent)

```

continent	n	percent
Africa	624	0.36619718
Americas	300	0.17605634
Asia	396	0.23239437
Europe	360	0.21126761
Oceania	24	0.01408451

A two-variable `tabyl()` gives a crosstab.

```

gapminder |>
  tabyl(continent, year)

```

continent	1952	1957	1962	1967	1972	1977	1982	1987	1992	1997	2002	2007
Africa	52	52	52	52	52	52	52	52	52	52	52	52
Americas	25	25	25	25	25	25	25	25	25	25	25	25
Asia	33	33	33	33	33	33	33	33	33	33	33	33
Europe	30	30	30	30	30	30	30	30	30	30	30	30
Oceania	2	2	2	2	2	2	2	2	2	2	2	2

This table counts country-year observations by continent and year. A different kind of crosstab uses a category we create ourselves. Here we classify countries by whether life expectancy in 2007 was at least 70 years.

```
gapminder_2007 <- gapminder |>
  filter(year == 2007) |>
  mutate(
    life_expectancy_group = if_else(lifeExp >= 70, "70 years or more", "Under 70 years")
  )

gapminder_2007 |>
  tabyl(continent, life_expectancy_group)
```

continent	70 years or more	Under 70 years
Africa	7	45
Americas	22	3
Asia	22	11
Europe	30	0
Oceania	2	0

The `adorn_*()` functions add totals, percentages, and count labels.

```
gapminder_2007 |>
  tabyl(continent, life_expectancy_group) |>
  adorn_totals(c("row", "col")) |>
  adorn_percentages("row") |>
  adorn_pct_formatting(digits = 1) |>
  adorn_ns()
```

continent	70 years or more	Under 70 years	Total
Africa	13.5% (7)	86.5% (45)	100.0% (52)
Americas	88.0% (22)	12.0% (3)	100.0% (25)
Asia	66.7% (22)	33.3% (11)	100.0% (33)
Europe	100.0% (30)	0.0% (0)	100.0% (30)
Oceania	100.0% (2)	0.0% (0)	100.0% (2)
Total	58.5% (83)	41.5% (59)	100.0 (142)

Use row percentages when the row categories are your comparison groups. In this case, the question is: within each continent, what share of countries had life expectancy of at least 70 years?

Column percentages answer a different question.

```
gapminder_2007 |>
  tabyl(continent, life_expectancy_group) |>
  adorn_totals(c("row", "col")) |>
  adorn_percentages("col") |>
  adorn_pct_formatting(digits = 1) |>
  adorn_ns()
```

continent	70 years or more	Under 70 years	Total
Africa	8.4% (7)	76.3% (45)	36.6% (52)
Americas	26.5% (22)	5.1% (3)	17.6% (25)
Asia	26.5% (22)	18.6% (11)	23.2% (33)
Europe	36.1% (30)	0.0% (0)	21.1% (30)
Oceania	2.4% (2)	0.0% (0)	1.4% (2)
Total	100.0% (83)	100.0% (59)	100.0 (142)

Here the question is: within each life expectancy group, what share of countries came from each continent?

10.3 Three-way tables

`tabyl()` can also make a three-way table. The result is a list of tables, one for each value of the third variable.

```
gapminder |>
  filter(year %in% c(1952, 2007)) |>
  mutate(
    life_expectancy_group = if_else(lifeExp >= 70, "70 years or more", "Under 70 years")
  ) |>
  tabyl(continent, life_expectancy_group, year)
```

```
$`1952`
continent 70 years or more Under 70 years
  Africa              0             52
Americas              0             25
  Asia                0             33
  Europe              5             25
Oceania              0              2
```

```
$`2007`
```

car	mpg	cyl	disp	hp	wt
Fiat X1-9	27.3	4	79.0	66	1.935
Lincoln Continental	10.4	8	460.0	215	5.424
Merc 240D	24.4	4	146.7	62	3.190
Valiant	18.1	6	225.0	105	3.460
Duster 360	14.3	8	360.0	245	3.570
Merc 450SE	16.4	8	275.8	180	4.070

continent	70 years or more	Under 70 years
Africa	7	45
Americas	22	3
Asia	22	11
Europe	30	0
Oceania	2	0

Three-way tables are useful when the third variable is central to the comparison. They can also become dense quickly. If a three-way table feels hard to read, consider splitting it into separate tables or making a plot instead.

10.4 Publication-style data tables with `tinytable`

`tinytable` is useful when the table needs to look good in a document. It starts with a data frame and returns a formatted table.

```
vehicle_sample <- mtcars |>
  rownames_to_column("car") |>
  slice_sample(n = 6) |>
  select(car, mpg, cyl, disp, hp, wt)

tt(vehicle_sample)
```

Add a caption and format the numbers.

```
tt(
  vehicle_sample,
  caption = "A Small Sample of Vehicle Characteristics",
  digits = 1
)
```

Table 10.1: A Small Sample of Vehicle Characteristics

car	mpg	cyl	disp	hp	wt
Fiat X1-9	27	4	79	66	2
Lincoln Continental	10	8	460	215	5
Merc 240D	24	4	147	62	3
Valiant	18	6	225	105	3
Duster 360	14	8	360	245	4
Merc 450SE	16	8	276	180	4

Table 10.2: A Small Sample of Vehicle Characteristics

car	mpg	cyl	disp	hp	wt
Fiat X1-9	27	4	79	66	2
Lincoln Continental	10	8	460	215	5
Merc 240D	24	4	147	62	3
Valiant	18	6	225	105	3
Duster 360	14	8	360	245	4
Merc 450SE	16	8	276	180	4

Data are from the built-in mtcars dataset.

You can add notes to explain the data source or a measurement choice.

```
tt(
  vehicle_sample,
  caption = "A Small Sample of Vehicle Characteristics",
  digits = 1,
  notes = "Data are from the built-in mtcars dataset."
)
```

You can also style headers and selected cells.

```
tt(vehicle_sample, digits = 1) |>
  style_tt(i = 0, color = "white", background = "steelblue", fontweight = "bold") |>
  style_tt(i = 2, j = 4, background = "lightyellow")
```

Column groups help when several variables belong together.

car	mpg	cyl	disp	hp	wt
Fiat X1-9	27	4	79	66	2
Lincoln Continental	10	8	460	215	5
Merc 240D	24	4	147	62	3
Valiant	18	6	225	105	3
Duster 360	14	8	360	245	4
Merc 450SE	16	8	276	180	4

Identity	Engine and Performance				
car	mpg	cyl	disp	hp	wt
Fiat X1-9	27	4	79	66	2
Lincoln Continental	10	8	460	215	5
Merc 240D	24	4	147	62	3
Valiant	18	6	225	105	3
Duster 360	14	8	360	245	4
Merc 450SE	16	8	276	180	4

```
tt(vehicle_sample, digits = 1) |>
  group_tt(
    j = list(
      "Identity" = 1,
      "Engine and Performance" = 2:6
    )
  )
```

10.5 Summary tables with modelsummary

The `modelsummary` package includes useful functions for descriptive statistics. The function `datasummary_skim()` comes from `modelsummary`.

```
gapminder_2007 |>
  select(continent, lifeExp, gdpPercap) |>
  datasummary_skim()
```

	Unique	Missing Pct.	Mean	SD	Min	Median	Max	Histogram
lifeExp	142	0	67.0	12.1	39.6	71.9	82.6	
gdpPercap	142	0	11 680.1	12 859.9	277.6	6124.4	49 357.2	
continent	N	%						
Africa	52	36.6						
Americas	25	17.6						
Asia	33	23.2						
Europe	30	21.1						
Oceania	2	1.4						

		N	%
continent	Africa	52	36.6
	Americas	25	17.6
	Asia	33	23.2
	Europe	30	21.1
	Oceania	2	1.4
life_expectancy_group	70 years or more	83	58.5
	Under 70 years	59	41.5

The default skim table is good for numeric variables. Categorical variables can be summarized separately.

```
gapminder_2007 |>
  select(continent, life_expectancy_group) |>
  datasummary_skim(type = "categorical")
```

`datasummary_crosstab()` creates crosstabs using a formula. The left side of the formula becomes the rows. The right side becomes the columns.

```
datasummary_crosstab(
  continent ~ life_expectancy_group,
  data = gapminder_2007
)
```


continent		70 years or more	Under 70 years	All
Africa	N	7	45	52
	% row	13.5	86.5	100.0
Americas	N	22	3	25
	% row	88.0	12.0	100.0
Asia	N	22	11	33
	% row	66.7	33.3	100.0
Europe	N	30	0	30
	% row	100.0	0.0	100.0
Oceania	N	2	0	2
	% row	100.0	0.0	100.0
All	N	83	59	142
	% row	58.5	41.5	100.0

10.6 Colored correlation tables with corrr

A correlation table is useful when you need to inspect linear relationships among numeric variables. The `corrr` package is designed for correlation workflows. Its `correlate()` function calculates correlations, and `stretch()` turns the result into a tidy table that can be plotted.

```
correlation_data <- gapminder_2007 |>
  transmute(
    `Life expectancy` = lifeExp,
    `Log GDP per capita` = log10(gdpPercap),
    `Log population` = log10(pop)
  )

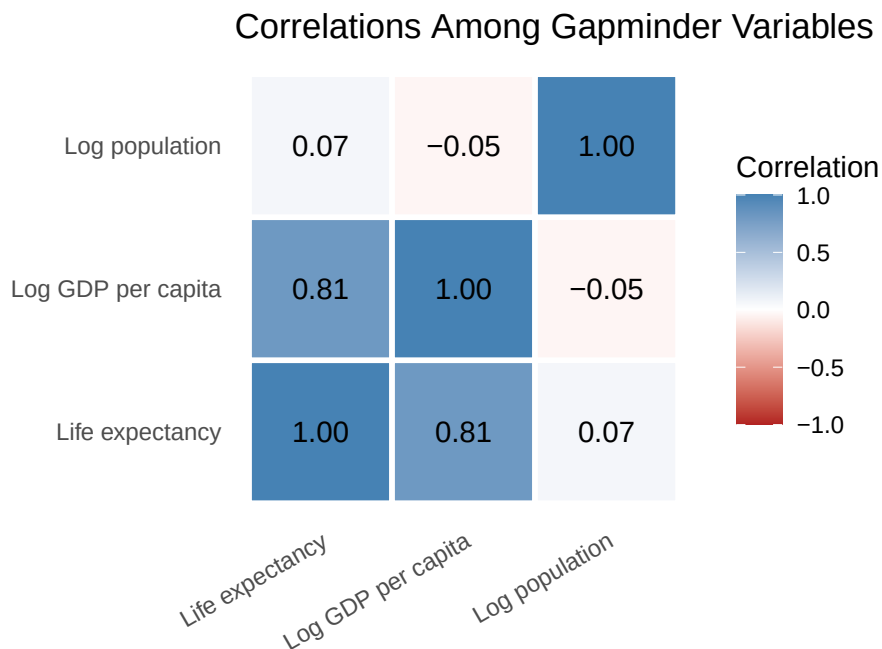
correlation_table <- correlation_data |>
  correlate(quiet = TRUE, diagonal = 1) |>
  stretch()

ggplot(correlation_table, aes(x = x, y = y, fill = r)) +
  geom_tile(color = "white", linewidth = 1) +
  geom_text(aes(label = sprintf("%.2f", r)), size = 4) +
  scale_fill_gradient2(
    low = "firebrick",
    mid = "white",
    high = "steelblue",
```

```

limits = c(-1, 1),
name = "Correlation"
) +
labs(
  title = "Correlations Among Gapminder Variables",
  x = NULL,
  y = NULL
) +
coord_equal() +
theme_minimal() +
theme(
  panel.grid = element_blank(),
  axis.text.x = element_text(angle = 30, hjust = 1)
)

```



The printed values are correlations. The color makes direction and strength easier to scan: blue values are positive, red values are negative, and paler cells are closer to zero.

10.7 Exporting tables

When a table belongs in another file, save it deliberately. The examples below are not run during rendering because they write files. If you use them, make sure the output folder exists.

```

vehicle_table <- tt(
  vehicle_sample,
  caption = "A Small Sample of Vehicle Characteristics",
  digits = 1
)

vehicle_table |> save_tt("Tables/vehicle_characteristics.html", overwrite = TRUE)
vehicle_table |> save_tt("Tables/vehicle_characteristics.docx", overwrite = TRUE)
vehicle_table |> save_tt("Tables/vehicle_characteristics.png", overwrite = TRUE)

```

In a Quarto project, keep exported tables in a clear output folder such as **Tables/** or **Output/**. Avoid saving tables into the **Data/** folder, which should hold source data.

10.8 Short exercise

Use `gapminder_2007`.

1. Create a new variable that classifies countries as above or below the median GDP per capita.
2. Make a `janitor::tabyl()` table of `continent` by your new GDP group.
3. Add row percentages and counts.
4. Make a `tinytable` version of a six-row sample containing `country`, `continent`, `lifeExp`, and `gdpPercap`.

10.9 Extra: table package choices

R has several strong table packages. You do not need to use all of them in one project.

- Use base R `table()` for quick Console checks.
- Use `janitor::tabyl()` for quick counts, percentages, totals, and exploratory crosstabs.
- Use `tinytable` for compact publication-style tables that work across Quarto output formats.
- Use `modelsummary` for descriptive summaries and crosstabs.
- Use `corr` for correlation tables and correlation displays.
- Consider `gt` when you want highly designed HTML tables.
- Consider `flextable` when Word output is the main target.

The main goal is not to memorize every option. The main goal is to know which kind of table you need: quick exploration, descriptive summary, publication display, correlation display, or interactive browsing.

country	continent	lifeExp	gdpPercap
Reunion	Africa	76	7670
Libya	Africa	74	12057
Tunisia	Africa	74	7093
Mauritius	Africa	73	10957
Algeria	Africa	72	6223
Egypt	Africa	71	5581
Morocco	Africa	71	3820
Sao Tome and Principe	Africa	66	1598
Comoros	Africa	65	986
Mauritania	Africa	64	1803
Senegal	Africa	63	1712
Ghana	Africa	60	1328

10.10 Extra: interactive tables with DT

Interactive tables are useful when readers need to search, sort, or inspect more rows than would fit comfortably on a page. The DT package creates HTML tables with built-in search and pagination.

```
table_for_browsing <- gapminder_2007 |>
  select(country, continent, lifeExp, gdpPercap) |>
  mutate(gdpPercap = round(gdpPercap)) |>
  arrange(continent, desc(lifeExp))

if (knitr::is_html_output() && requireNamespace("DT", quietly = TRUE)) {
  DT::datatable(
    table_for_browsing,
    rownames = FALSE,
    options = list(
      pageLength = 8,
      autoWidth = TRUE
    )
  )
} else {
  tt(table_for_browsing |> slice_head(n = 12), digits = 1)
}
```

Interactive tables work best for exploration or web appendices. For a printed report, a smaller static table is usually better.

11 Visualizing Models

Coefficient plots, predictions, marginal effects, and interactions

Regression tables are useful, but they are not always the best way to understand a model. A coefficient plot can make uncertainty easier to see. A prediction plot can translate model estimates into the scale of the outcome. For logit models, prediction plots are especially helpful because log-odds are hard to interpret directly.

This chapter uses Chicago Airbnb listings from 2016 as its running example. The data covers 1,561 listings with information on price, room type, guest capacity, and a handful of other listing attributes.

```
library(tidyverse)
library(modelsummary) # modelsummary creates regression and summary tables
library(marginaleffects) # marginaleffects creates prediction and effect plots
library(broom) # broom turns model output into tidy data frames
library(patchwork) # patchwork combines multiple ggplot figures
```

11.1 A first model: predicting price directly

The natural starting point is to model price in dollars. We will use guest capacity and room type as predictors.

```
airbnb_raw <- read_csv("Data/airbnb/airbnb.csv", show_col_types = FALSE)

airbnb <- airbnb_raw |>
  mutate(
    log_price = log(price),
    log_reviews = log(reviews),
    room_type = factor(
      room_type,
      levels = c("Entire home/apt", "Private room", "Shared room")
    )
  )
```

```

    ),
    expensive = price >= 150
  )

glimpse(airbnb)

```

```

Rows: 1,561
Columns: 16
$ rownames      <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 1~
$ price         <dbl> 85, 35, 175, 75, 160, 60, 39, 65, 47, 100, 150, 38, 87, ~
$ rating        <dbl> 5.0, 5.0, 4.5, 5.0, 5.0, 5.0, 5.0, 5.0, 4.5, 4.5, 4.5, 5~
$ reviews       <dbl> 6, 8, 7, 52, 3, 5, 8, 25, 7, 24, 8, 3, 10, 15, 3, 6, 5, ~
$ room_type     <fct> Private room, Private room, Entire home/apt, Private roo~
$ accommodates  <dbl> 2, 1, 4, 2, 6, 1, 2, 2, 2, 6, 6, 1, 4, 3, 4, 6, 2, 4, 4,~
$ bedrooms      <dbl> 1, 1, 2, 1, 2, 1, 1, 1, 1, 2, 2, 1, 1, 1, 2, 3, 1, 0, 1,~
$ minimum_stay  <dbl> 1, 1, 2, 1, 1, 1, 3, 2, 1, NA, 5, 10, 2, 1, 2, 2, 1, NA,~
$ neighborhood  <chr> "Albany Park", "Albany Park", "Albany Park", "Albany Par~
$ district      <chr> "Far North", "Far North", "Far North", "Far North", "Far~
$ walk_score    <dbl> 87, 87, 87, 87, 87, 87, 87, 87, 87, 87, 87, 87, 87, 87, ~
$ transit_score <dbl> 62, 62, 62, 62, 62, 62, 62, 62, 62, 62, 62, 62, 62, 62, ~
$ bike_score    <dbl> 68, 68, 68, 68, 68, 68, 68, 68, 68, 68, 68, 68, 68, 68, ~
$ log_price     <dbl> 4.442651, 3.555348, 5.164786, 4.317488, 5.075174, 4.0943~
$ log_reviews   <dbl> 1.791759, 2.079442, 1.945910, 3.951244, 1.098612, 1.6094~
$ expensive     <lgl> FALSE, FALSE, TRUE, FALSE, TRUE, FALSE, FALSE, FALSE, FA~

```

Entire home/apt is the reference category for room type.

```
airbnb |> count(room_type)
```

```

# A tibble: 3 x 2
  room_type      n
  <fct>        <int>
1 Entire home/apt 826
2 Private room    685
3 Shared room     50

```

Shared rooms are rare (about 50 listings). This will matter later when fitting an interaction model.

The function `lm()` is part of base R. It fits ordinary least squares regression models.

```
price_raw <- lm(price ~ accommodates + room_type, data = airbnb)
summary(price_raw)
```

Call:

```
lm(formula = price ~ accommodates + room_type, data = airbnb)
```

Residuals:

Min	1Q	Median	3Q	Max
-246.83	-29.43	-5.04	20.04	640.27

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	64.1480	4.7551	13.490	<2e-16 ***
accommodates	18.4739	0.8705	21.221	<2e-16 ***
room_typePrivate room	-41.1361	4.1146	-9.998	<2e-16 ***
room_typeShared room	-85.6087	9.8368	-8.703	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 66.75 on 1557 degrees of freedom

Multiple R-squared: 0.4305, Adjusted R-squared: 0.4294

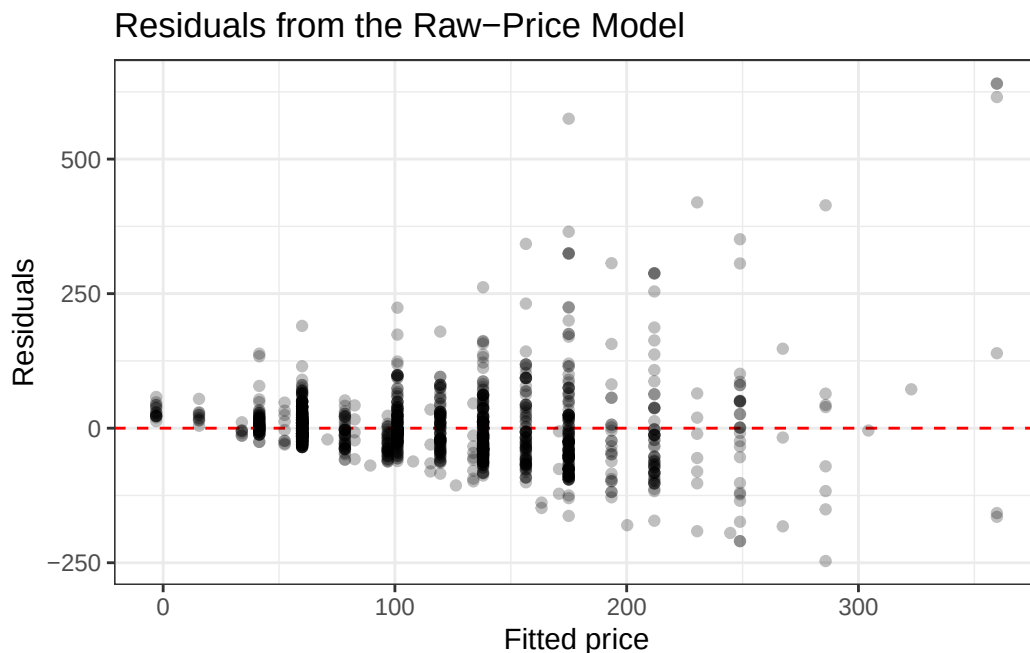
F-statistic: 392.3 on 3 and 1557 DF, p-value: < 2.2e-16

The coefficients are easy to read: each additional guest the listing accommodates is associated with about \$18 more per night, and private rooms cost about \$41 less than entire homes after holding capacity constant.

But the model itself has problems. We can see them by looking at the fitted values and residuals.

```
price_raw_aug <- augment(price_raw)

ggplot(price_raw_aug, aes(x = .fitted, y = .resid)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  geom_point(alpha = 0.25) +
  labs(
    title = "Residuals from the Raw-Price Model",
    x = "Fitted price",
    y = "Residuals"
  ) +
  theme_bw()
```

The residual cloud fans out. At low fitted prices the spread is tight; at high fitted prices the spread is huge. That is heteroskedasticity, and it means standard errors and prediction intervals from this model are not trustworthy.

There is a second problem. Price cannot be negative, but a linear model on dollars can produce negative predictions for small, cheap listings.

```
sum(predict(price_raw) < 0)
```

```
[1] 17
```

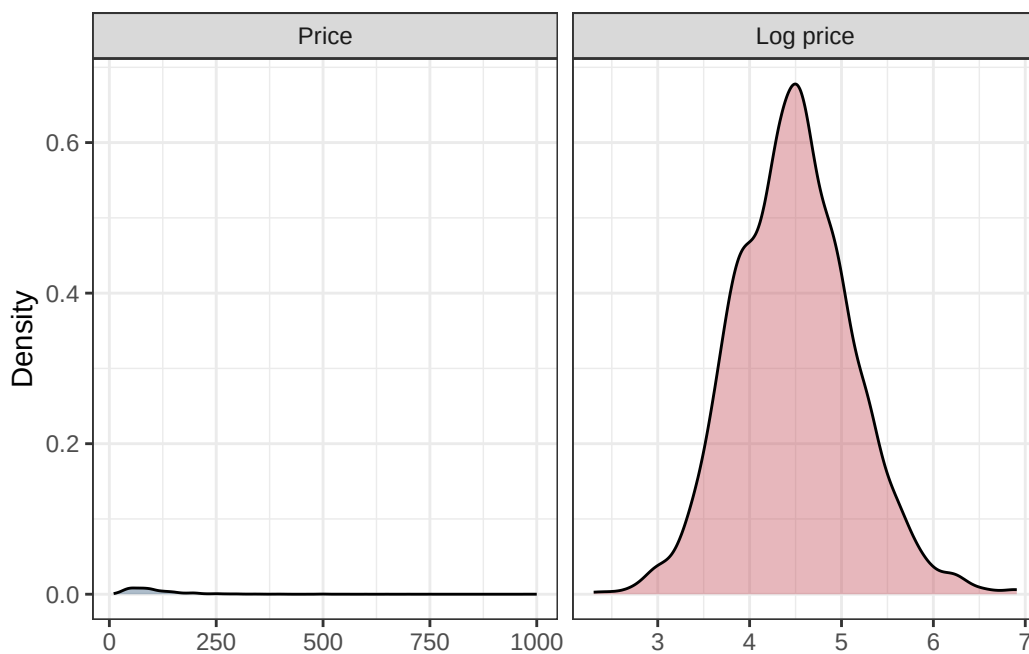
```
range(predict(price_raw))
```

```
[1] -2.986744 359.730343
```

Seventeen listings have negative predicted prices. The model can produce nonsense values because nothing in OLS knows that price has a floor.

Both problems share a root cause: prices are right-skewed. We can see that directly by plotting the price distribution.

```
airbnb |>
  select(Price = price, `Log price` = log_price) |>
  pivot_longer(everything(), names_to = "scale", values_to = "value") |>
  mutate(scale = factor(scale, levels = c("Price", "Log price"))) |>
  ggplot(aes(x = value, fill = scale)) +
  geom_density(alpha = 0.4) +
  facet_wrap(~ scale, scales = "free_x") +
  scale_fill_manual(values = c("Price" = "#355c7d", "Log price" = "#c44d58")) +
  labs(x = NULL, y = "Density") +
  theme_bw() +
  theme(legend.position = "none")
```



Raw price has a long right tail running into the high hundreds. Log price is close to symmetric and roughly bell-shaped. A linear model assumes that the variation around the conditional mean is roughly even in spread and not strongly skewed; the log-price distribution lines up with that assumption much better than the raw-price one.

The fix, then, is to model log price instead. On the log scale, an additional guest adds a *proportional* amount to the price rather than a flat dollar amount. Coefficients on a log outcome read as approximate percent changes, which is the natural unit for price.

11.2 OLS on log price

```
price_model <- lm(log_price ~ accommodates + room_type, data = airbnb)
```

A regular model summary gives the numerical details.

```
summary(price_model)
```

Call:

```
lm(formula = log_price ~ accommodates + room_type, data = airbnb)
```

Residuals:

Min	1Q	Median	3Q	Max
-2.50769	-0.25335	-0.01587	0.25889	1.62747

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	4.467905	0.030019	148.84	<2e-16 ***
accommodates	0.087449	0.005496	15.91	<2e-16 ***
room_typePrivate room	-0.584210	0.025975	-22.49	<2e-16 ***
room_typeShared room	-1.304344	0.062100	-21.00	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.4214 on 1557 degrees of freedom

Multiple R-squared: 0.557, Adjusted R-squared: 0.5562

F-statistic: 652.7 on 3 and 1557 DF, p-value: < 2.2e-16

modelsummary() comes from the modelsummary package. It gives a cleaner table.

```
modelsummary(  
  price_model,  
  stars = TRUE,  
  coef_rename = c("accommodates" = "Guest capacity"),  
  gof_omit = "AIC|BIC|Log.Lik|F|RMSE"  
)
```

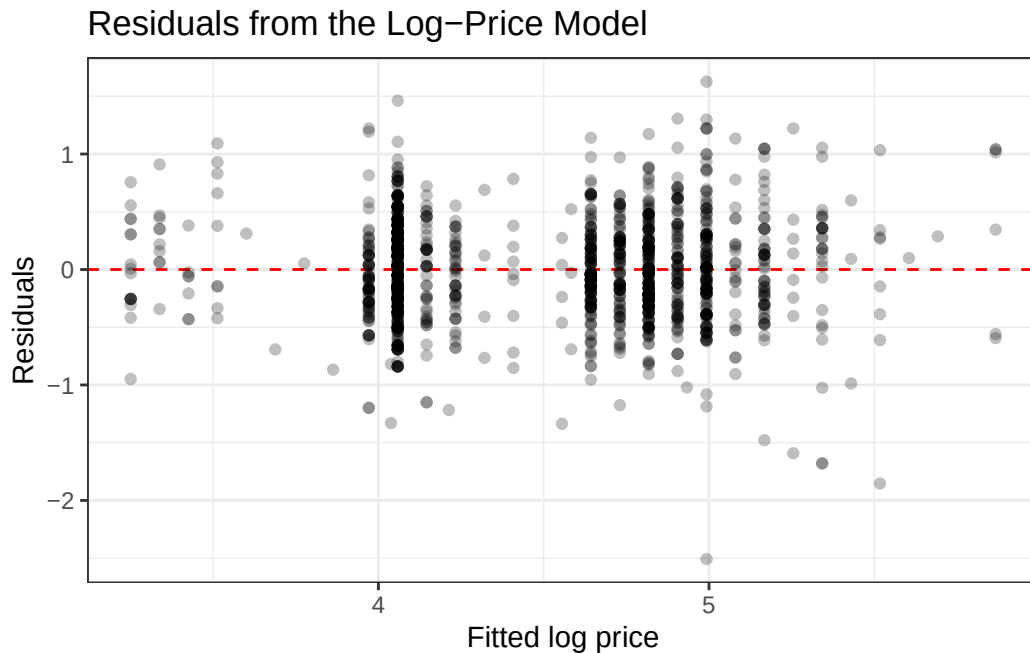
The accommodates coefficient is about 0.09 on the log scale, so each additional guest the listing accommodates is associated with roughly 9% higher price. The Private room coefficient is

	(1)
(Intercept)	4.468*** (0.030)
Guest capacity	0.087*** (0.005)
room_typePrivate room	-0.584*** (0.026)
room_typeShared room	-1.304*** (0.062)
Num.Obs.	1561
R2	0.557
R2 Adj.	0.556
+ p <0.1, * p <0.05, ** p <0.01, *** p <0.001	

around -0.58 , meaning private rooms cost about 44% less than entire homes after holding capacity constant ($\exp(-0.58) = 0.56$). Shared rooms are about 73% cheaper. The model's R^2 is around 0.56, much higher than typical for cross-sectional pricing data — these two predictors carry most of the structure.

```
log_aug <- augment(price_model)

ggplot(log_aug, aes(x = .fitted, y = .resid)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  geom_point(alpha = 0.25) +
  labs(
    title = "Residuals from the Log-Price Model",
    x = "Fitted log price",
    y = "Residuals"
  ) +
  theme_bw()
```



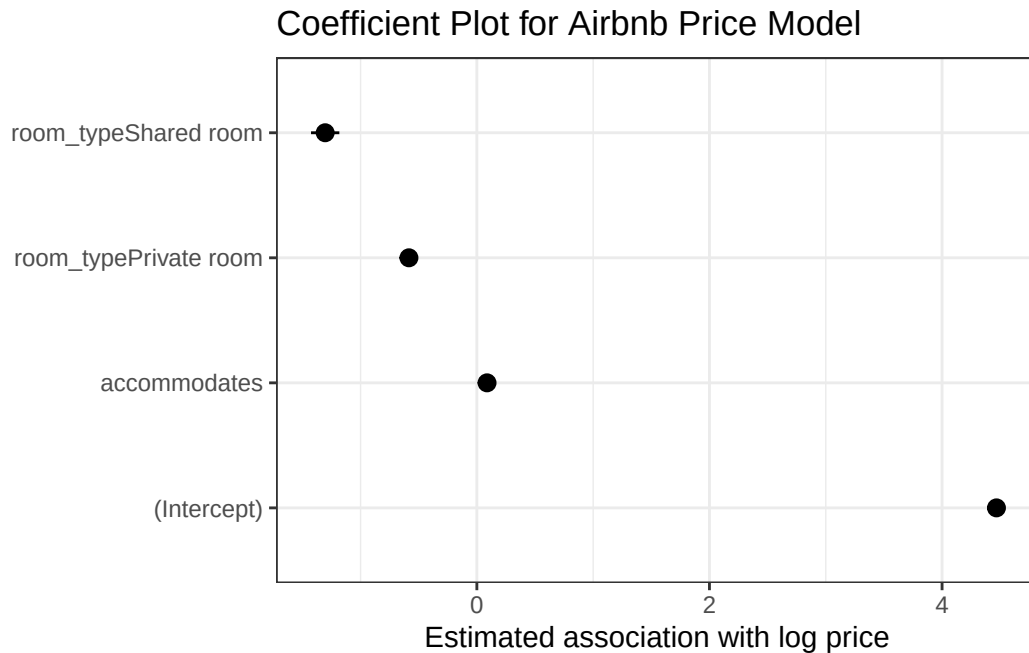
The residual cloud is roughly even in width across the fitted range. The vertical bands at the lower end correspond to Private and Shared rooms, which have fewer distinct fitted values. This is the kind of residual plot you want to see before trusting standard errors and predictions.

11.3 Coefficient plots

A coefficient plot shows each estimate as a point and the uncertainty around it as an interval.

The function `modelplot()` comes from the `modelsummary` package.

```
modelplot(price_model) +
  labs(
    title = "Coefficient Plot for Airbnb Price Model",
    x = "Estimated association with log price",
    y = NULL
  ) +
  theme_bw()
```



The vertical line at zero is the reference point. If an interval crosses zero, the estimate is less clearly distinguishable from no association at conventional levels. Here all three intervals are far from zero. The Private room and Shared room coefficients are large and negative, which is the expected ordering of property types.

Coefficient plots are useful when the coefficients are on a meaningful scale. They are less useful when coefficients are transformations or log-odds that readers will struggle to interpret.

11.4 A manual coefficient plot

`modelplot()` is convenient, but it is useful to see what it is doing conceptually. The function `tidy()` comes from the `broom` package. It turns model estimates into a data frame that can be filtered, renamed, and plotted like any other dataset.

```
price_coef <- tidy(price_model, conf.int = TRUE) |>
  filter(term != "(Intercept)") |>
  mutate(
    term = recode(
      term,
      "accommodates" = "Guest capacity",
      "room_typePrivate room" = "Private room",
      "room_typeShared room" = "Shared room"
```

```

    )
  )

price_coef

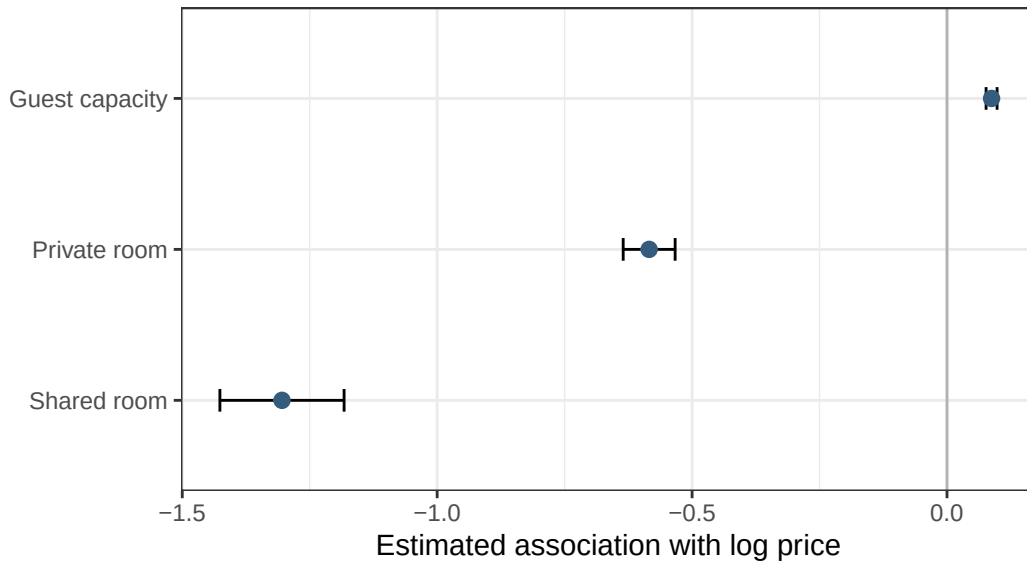
# A tibble: 3 x 7
  term          estimate std.error statistic  p.value conf.low conf.high
<chr>         <dbl>     <dbl>     <dbl>   <dbl>   <dbl>   <dbl>
1 Guest capacity  0.0874    0.00550     15.9 6.15e-53  0.0767  0.0982
2 Private room   -0.584    0.0260     -22.5 3.12e-97 -0.635 -0.533
3 Shared room    -1.30     0.0621     -21.0 1.94e-86 -1.43  -1.18

ggplot(price_coef, aes(x = estimate, y = reorder(term, estimate))) +
  geom_vline(xintercept = 0, color = "gray70") +
  geom_errorbar(aes(xmin = conf.low, xmax = conf.high), width = 0.15) +
  geom_point(size = 2.4, color = "#355c7d") +
  labs(
    title = "Manual Coefficient Plot",
    subtitle = "Same model output, now treated as data",
    x = "Estimated association with log price",
    y = NULL
  ) +
  theme_bw()

```

Manual Coefficient Plot

Same model output, now treated as data



This manual version is useful when the default plot needs more control over labels, ordering, colors, or annotations.

11.5 Tidy model output with broom

The **broom** package turns model output into data frames. This makes model output easier to inspect, filter, and plot with **ggplot2**.

```
tidy(price_model)
```

```
# A tibble: 4 x 5
```

term	estimate	std.error	statistic	p.value
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1 (Intercept)	4.47	0.0300	149.	0
2 accommodates	0.0874	0.00550	15.9	6.15e-53
3 room_typePrivate room	-0.584	0.0260	-22.5	3.12e-97
4 room_typeShared room	-1.30	0.0621	-21.0	1.94e-86

glance() also comes from **broom**. It gives a one-row model summary.


```
glance(price_model)
```

```
# A tibble: 1 x 12
  r.squared adj.r.squared sigma statistic p.value    df logLik   AIC   BIC
    <dbl>      <dbl> <dbl>    <dbl>    <dbl> <dbl> <dbl> <dbl> <dbl>
1     0.557        0.556 0.421      653. 1.16e-274     3 -864. 1738. 1765.
# i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

`augment()` also comes from `broom`. It adds fitted values and residuals to the data.

```
price_augmented <- augment(price_model)

price_augmented |>
  select(log_price, .fitted, .resid, room_type) |>
  slice_head(n = 8)
```

```
# A tibble: 8 x 4
  log_price .fitted .resid room_type
    <dbl>    <dbl>   <dbl> <fct>
1     4.44     4.06  0.384 Private room
2     3.56     3.97 -0.416 Private room
3     5.16     4.82  0.347 Entire home/apt
4     4.32     4.06  0.259 Private room
5     5.08     4.99  0.0826 Entire home/apt
6     4.09     3.97  0.123 Private room
7     3.66     4.06 -0.395 Private room
8     4.17     4.06  0.116 Private room
```

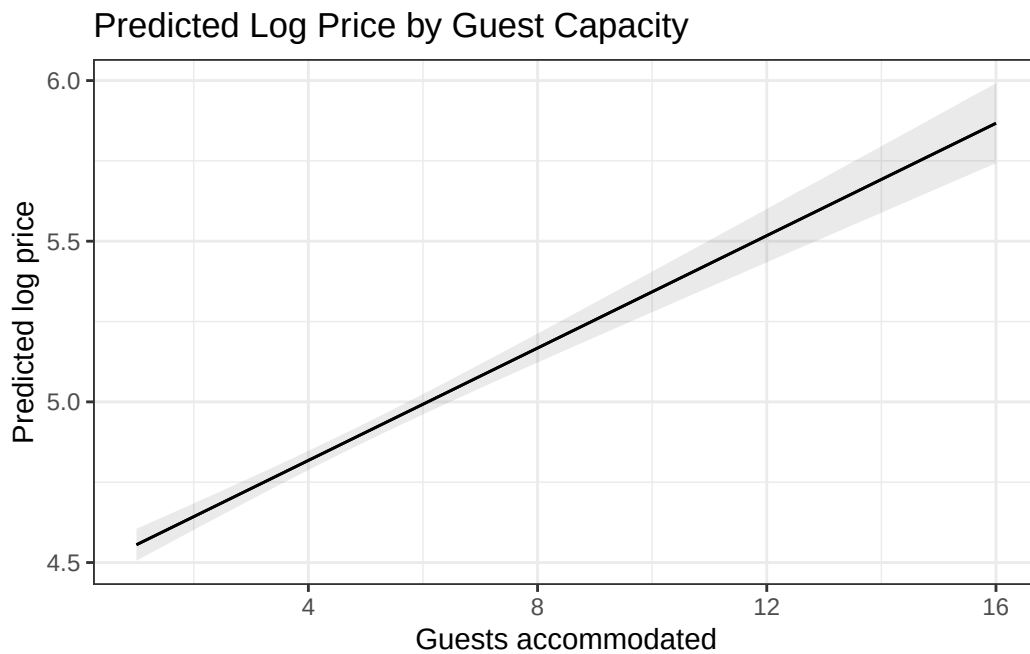
These columns are what made the diagnostic plot above possible.

11.6 Prediction plots

Prediction plots translate the model back into the outcome scale. Here we show predicted log price across guest capacity.

The function `plot_predictions()` comes from the `marginalEffects` package.

```
plot_predictions(price_model, condition = "accommodates") +
  labs(
    title = "Predicted Log Price by Guest Capacity",
    x = "Guests accommodated",
    y = "Predicted log price"
  ) +
  theme_bw()
```



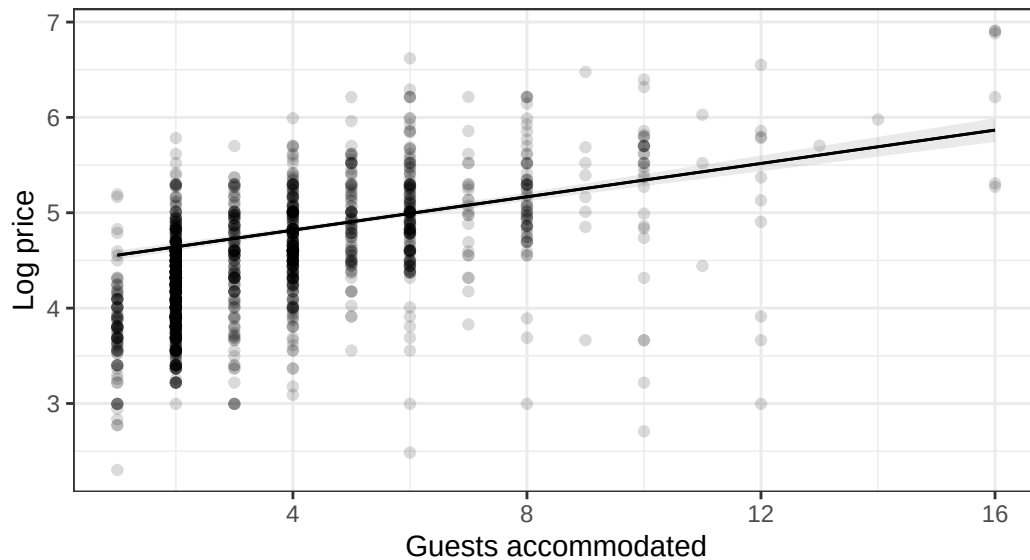
Because `plot_predictions()` returns a `ggplot` object, we can add layers.

```
plot_predictions(price_model, condition = "accommodates") +
  geom_point(
    data = airbnb,
    aes(x = accommodates, y = log_price),
    alpha = 0.15,
    inherit.aes = FALSE
  ) +
  labs(
    title = "Predicted Log Price with Raw Data",
    subtitle = "Line is the model prediction; points are listings",
    x = "Guests accommodated",
    y = "Log price"
  )
```

```
) +  
theme_bw()
```

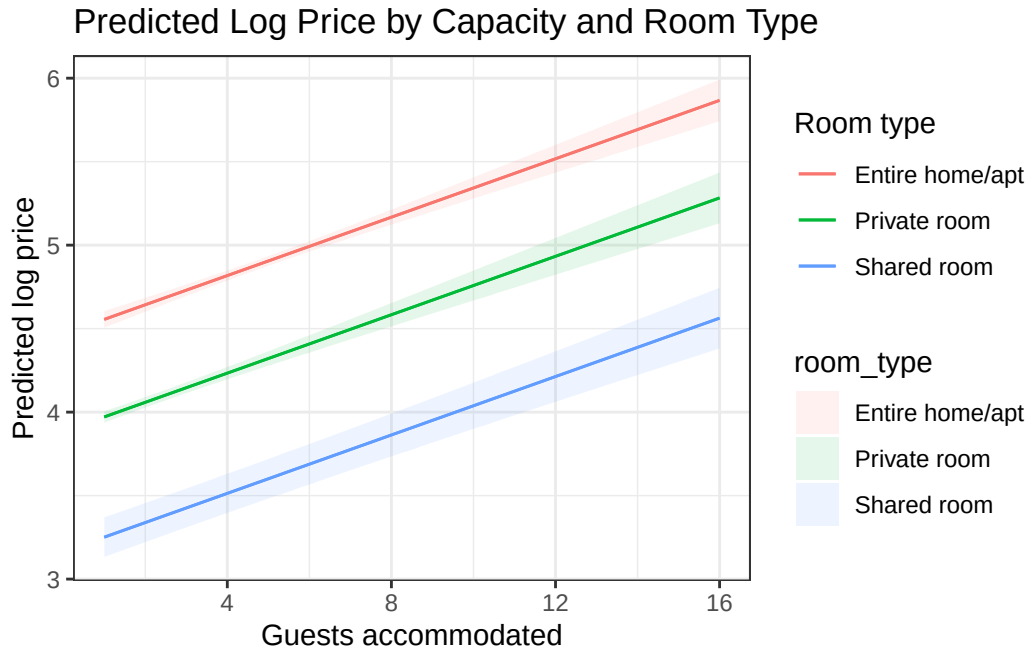
Predicted Log Price with Raw Data

Line is the model prediction; points are listings



Now condition on two variables.

```
plot_predictions(price_model, condition = c("accommodates", "room_type")) +  
  labs(  
    title = "Predicted Log Price by Capacity and Room Type",  
    x = "Guests accommodated",  
    y = "Predicted log price",  
    color = "Room type"  
  ) +  
  theme_bw()
```



This is a conditional prediction plot. It shows model predictions over a grid of values. The three room-type lines are parallel because this is an additive model: each room type has its own intercept but shares the same capacity slope. The vertical gaps between the lines are the room-type coefficients made visible — Entire homes at the top, Private rooms below, Shared rooms lowest. The interaction model below relaxes the parallel-slopes assumption.

11.7 Comparing nested models

Model visualization is often comparative. A single model may answer a question, but two or three related models show how an estimate changes when controls are added.

The next step is to add two more predictors: the number of guest reviews the listing has accumulated (a rough pre-booking signal of popularity) and `minimum_stay` (the host-set policy on the shortest allowed booking).

The raw review count is heavily right-skewed — most listings have a handful of reviews, a few have hundreds. The same logic that motivated logging price applies here: on the raw scale, the model would treat the difference between 5 and 50 reviews as much smaller than the difference between 250 and 295, when substantively the first is the more meaningful change. We use `log_reviews` (created in the data-prep step above) for the same reason. On a logged predictor, the coefficient describes the effect of a *proportional* change in reviews, not a one-unit change.

	Capacity only	+ Room type	+ Controls
accommodates	0.158*** (0.006)	0.087*** (0.005)	0.091*** (0.006)
room_typePrivate room		-0.584*** (0.026)	-0.572*** (0.027)
room_typeShared room		-1.304*** (0.062)	-1.273*** (0.064)
log_reviews			-0.021* (0.010)
minimum_stay			0.000 (0.009)
Num.Obs.	1561	1561	1423
R2	0.335	0.557	0.558
R2 Adj.	0.335	0.556	0.556

+ p <0.1, * p <0.05, ** p <0.01, *** p <0.001

```
price_model_simple <- lm(log_price ~ accommodates, data = airbnb)
price_model_rooms <- lm(log_price ~ accommodates + room_type, data = airbnb)
price_model_controls <- lm(
  log_price ~ accommodates + room_type + log_reviews + minimum_stay,
  data = airbnb
)

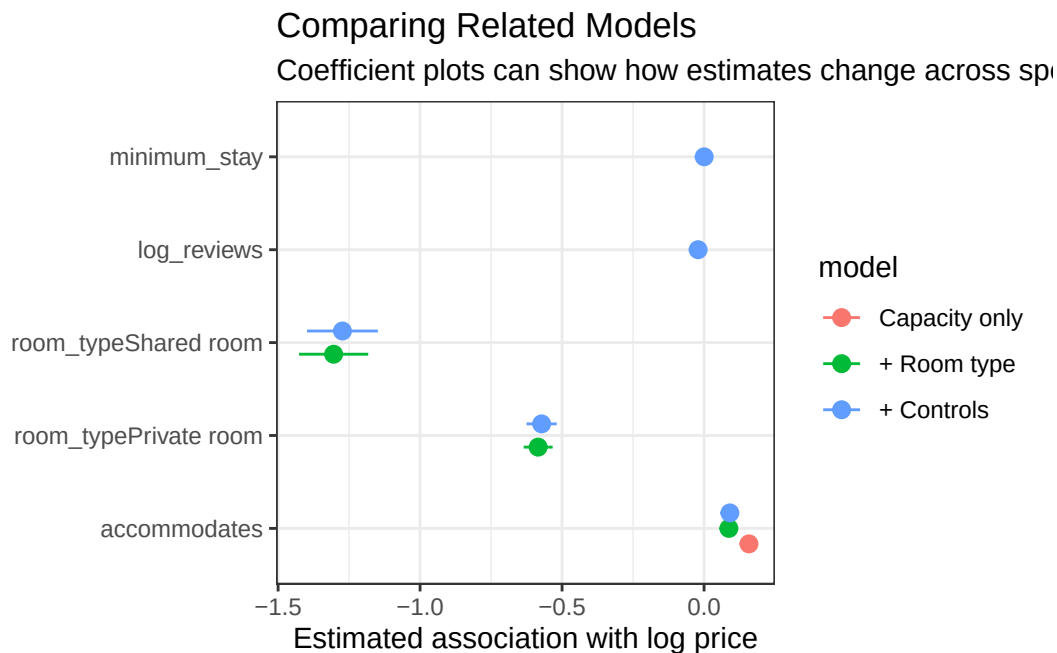
price_models <- list(
  "Capacity only" = price_model_simple,
  "+ Room type" = price_model_rooms,
  "+ Controls" = price_model_controls
)

modelsummary(
  price_models,
  stars = TRUE,
  coef_omit = "Intercept",
  gof_omit = "AIC|BIC|Log.Lik|F|RMSE"
)
```

```

modelplot(
  price_models,
  coef_omit = "Intercept"
) +
  labs(
    title = "Comparing Related Models",
    subtitle = "Coefficient plots can show how estimates change across specifications",
    x = "Estimated association with log price",
    y = NULL
  ) +
  theme_bw()

```



This plot is not a substitute for thinking carefully about the model. It is a compact way to inspect whether estimates are stable across reasonable specifications.

Two things stand out. First, the capacity coefficient drops sharply once room type enters the model — from about 0.16 in the capacity-only model to about 0.09 with room type included. The simple capacity effect was partly capturing the fact that larger groups stay in entire homes; once room type is in the model, the remaining capacity effect is the within-room-type effect.

Second, the controls barely move the headline estimates. The capacity and room-type coefficients are nearly identical with or without `log_reviews` and `minimum_stay`, and R^2 rises by less than half a percentage point. The `log_reviews` coefficient is about -0.021 and marginally significant: doubling the number of reviews is associated with roughly 1–2% lower price. The

sign is the opposite of a clean popularity-pricing story. Several stories are consistent with this: older listings accumulate more reviews and may not have raised their prices, or popular listings book up regardless of price and so review count ends up uncorrelated with — or even slightly negative on — price in equilibrium. The `minimum_stay` coefficient is essentially zero and clearly non-significant.

This is a useful teaching outcome on its own: not every plausible predictor moves the headline estimates, and a good comparative table is what makes that visible.

11.8 Interactions

An interaction allows a relationship to differ across groups. In this model, the association between capacity and price can vary by room type.

Shared rooms have only about 50 listings in the data, and within that group the price range is very narrow. That is too little information for a room-type-specific capacity slope, so this example drops Shared rooms before fitting the interaction model. This keeps the example focused on interaction interpretation rather than on an extreme uncertainty band from a tiny group.

```
airbnb_interaction <- airbnb |>
  filter(room_type != "Shared room") |>
  mutate(room_type = droplevels(room_type))

price_interaction <- lm(
  log_price ~ accommodates * room_type + log_reviews + minimum_stay,
  data = airbnb_interaction
)

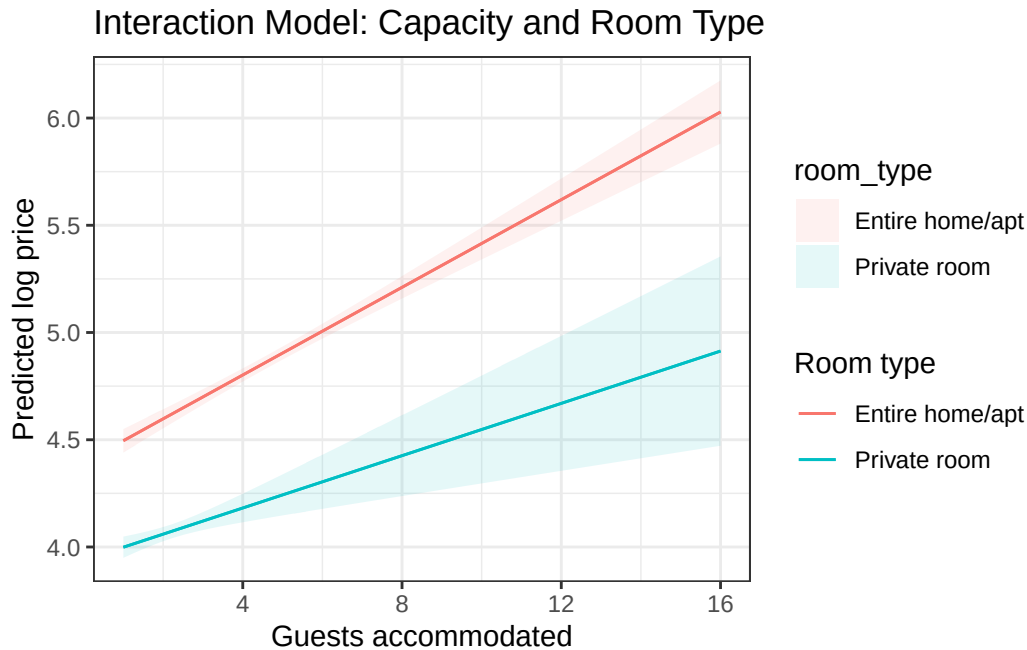
modelsummary(
  list(
    "Additive model" = price_model_controls,
    "Interaction model" = price_interaction
  ),
  stars = TRUE,
  coef_omit = "Intercept",
  gof_omit = "AIC|BIC|Log.Lik|F|RMSE"
)
```

The table is detailed, but the interaction is easier to understand with predictions.

	Additive model	Interaction model
accommodates	0.091*** (0.006)	0.102*** (0.006)
room_typePrivate room	−0.572*** (0.027)	−0.455*** (0.051)
room_typeShared room	−1.273*** (0.064)	
log_reviews	−0.021* (0.010)	−0.016 (0.010)
minimum_stay	0.000 (0.009)	0.001 (0.009)
accommodates × room_typePrivate room		−0.041* (0.017)
Num.Obs.	1423	1376
R2	0.558	0.533
R2 Adj.	0.556	0.531

+ p <0.1, * p <0.05, ** p <0.01, *** p <0.001


```
plot_predictions(price_interaction, condition = c("accommodates", "room_type")) +
  labs(
    title = "Interaction Model: Capacity and Room Type",
    x = "Guests accommodated",
    y = "Predicted log price",
    color = "Room type"
  ) +
  theme_bw()
```



When presenting an interaction, a prediction plot is usually clearer than a table of interaction coefficients.

The interaction plot tells us that the capacity-price relationship is steeper for entire homes than for private rooms. Each additional guest adds roughly 10% to the price of an entire home but only about 6% to a private room (the main capacity coefficient is about 0.10, and the interaction term is about -0.04 , so the private-room slope is roughly 0.06). The interaction coefficient is significant at conventional levels. The substantive direction is sensible: the marginal value of accommodating one more person is larger when the whole property is yours.

	(1)
accommodates	0.390*** (0.042)
room_typePrivate room	-2.842*** (0.378)
log_reviews	-0.247** (0.081)
minimum_stay	0.073 (0.060)
Num.Obs.	1376
F	46.531
+ p <0.1, * p <0.05, ** p <0.01, *** p <0.001	

11.9 Logit: binary outcomes

A logistic regression model is used when the outcome is binary. We will define a binary outcome: whether a listing is expensive (price of \$150 or more per night, the top quartile in this data).

The function `glm()` is part of base R. With `family = binomial`, it fits a logistic regression model.

```
price_logit <- glm(
  expensive ~ accommodates + room_type + log_reviews + minimum_stay,
  data = airbnb_interaction,
  family = binomial
)
```

The model estimates log-odds. Log-odds are not intuitive for most readers.

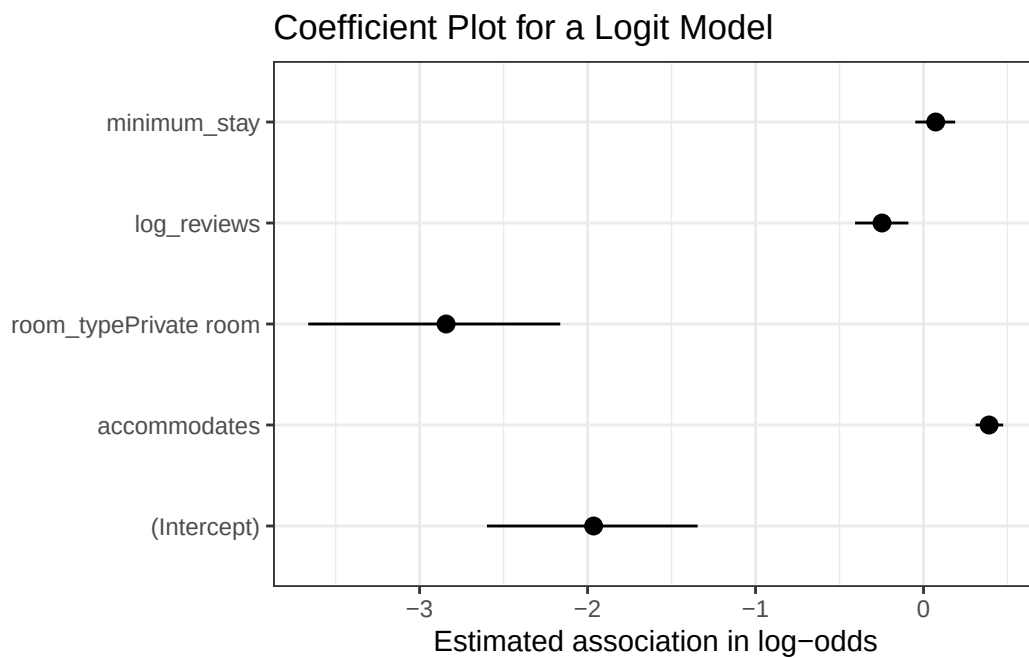
```
modelsummary(
  price_logit,
  stars = TRUE,
  coef_omit = "Intercept",
  gof_omit = "AIC|BIC|Log.Lik|RMSE"
)
```

A coefficient plot is still possible, but the x-axis is in log-odds.

```

modelplot(price_logit) +
  labs(
    title = "Coefficient Plot for a Logit Model",
    x = "Estimated association in log-odds",
    y = NULL
  ) +
  theme_bw()

```

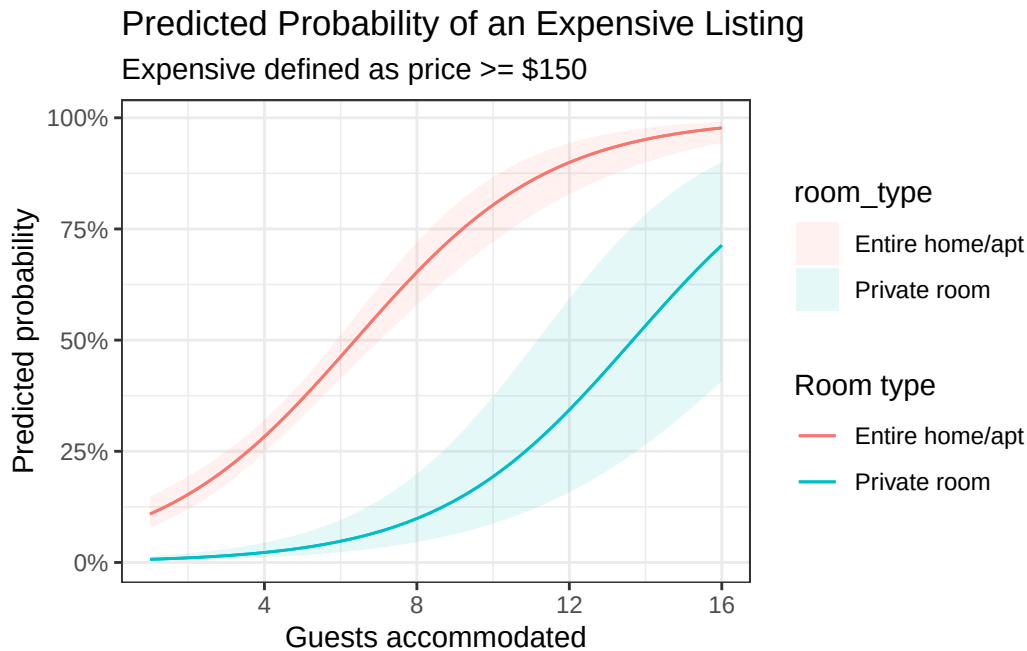


For interpretation, predicted probabilities are better.

```

plot_predictions(price_logit, condition = c("accommodates", "room_type")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(
    title = "Predicted Probability of an Expensive Listing",
    subtitle = "Expensive defined as price $150",
    x = "Guests accommodated",
    y = "Predicted probability",
    color = "Room type"
  ) +
  theme_bw()

```



This plot returns the model to a scale people understand: probability. The story is stark. An entire home that accommodates eight guests has roughly a 90% predicted probability of being expensive. A private room at the same capacity has well under 10%. The two curves separate quickly even at low capacity. The natural reading is that room type matters far more than capacity for whether a listing ends up in the top quartile.

11.10 Marginal and adjusted predictions

There are two related ideas that often appear in model visualization:

- Conditional predictions hold variables at selected values and show predictions across a grid.
- Marginal or adjusted predictions compute predictions for observed cases and average across a group.

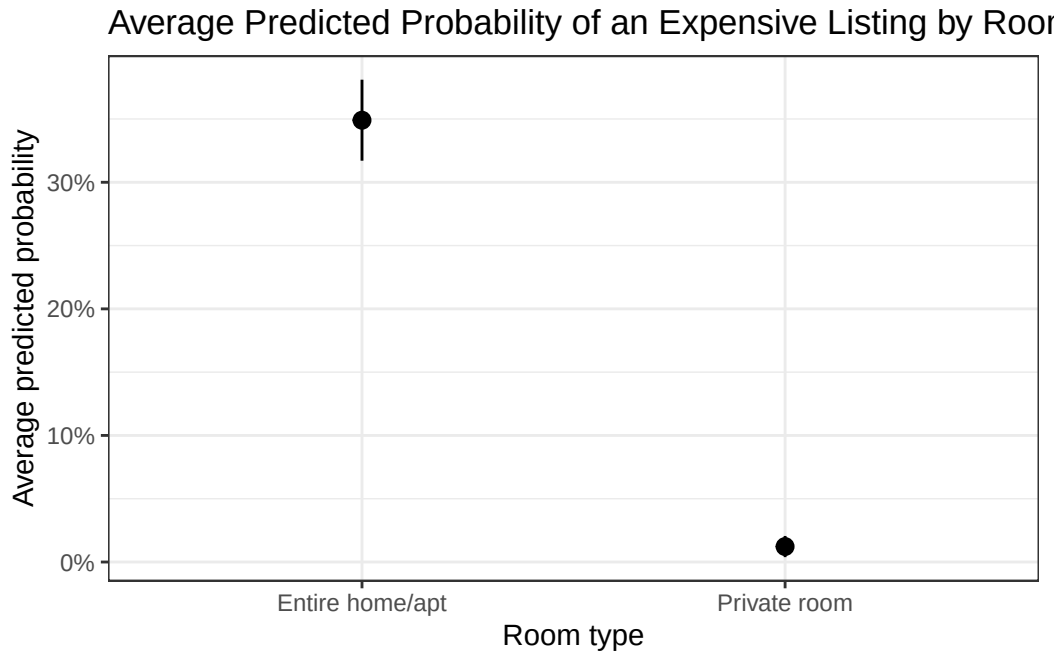
The `by` argument gives averaged predictions by group.

```
plot_predictions(price_logit, by = "room_type") +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(
    title = "Average Predicted Probability of an Expensive Listing by Room Type",
    x = "Room type",
```

```

y = "Average predicted probability"
) +
theme_bw()

```



Use conditional predictions when you want to show a relationship across values. Use averaged predictions when you want a compact group comparison.

The averaged predictions make the gap concrete: about 36% of entire homes are predicted to be expensive, versus roughly 1% of private rooms. This is averaged across the observed distribution of capacity in each group, so it reflects the kinds of listings that actually exist rather than predictions at arbitrary capacity values.

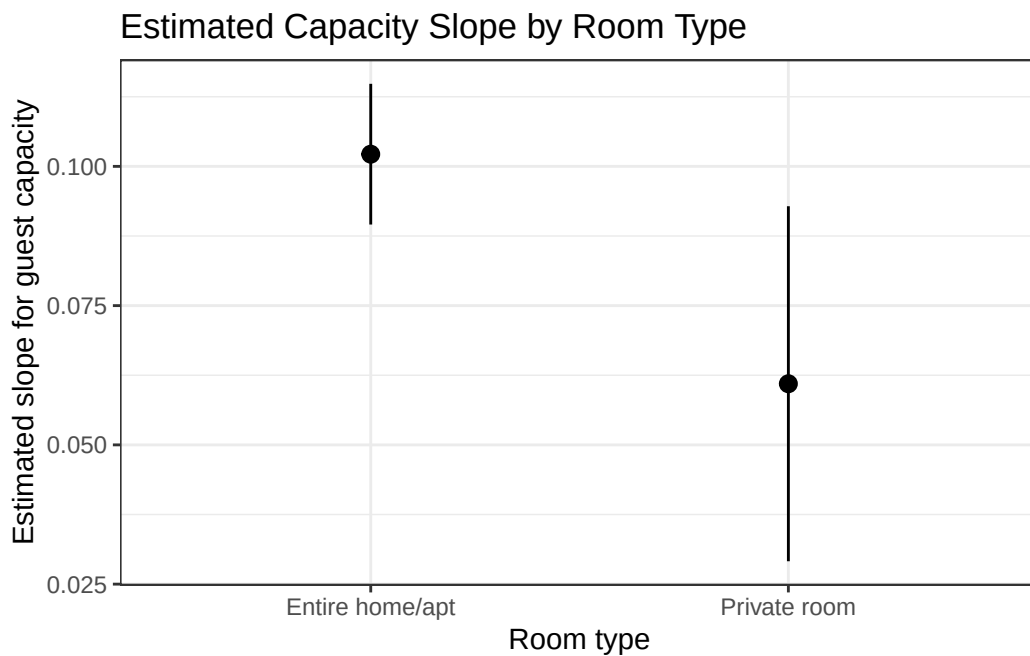
11.11 Slopes and marginal effects

Prediction plots show predicted outcomes. Sometimes the quantity of interest is the slope: how much the predicted outcome changes when a predictor changes.

For the interaction model, the capacity slope is allowed to differ by room type. `plot_slopes()` visualizes those room-type-specific slopes directly.

`plot_slopes()` comes from the `marginalEffects` package.

```
plot_slopes(
  price_interaction,
  variables = "accommodates",
  condition = "room_type"
) +
  labs(
    title = "Estimated Capacity Slope by Room Type",
    x = "Room type",
    y = "Estimated slope for guest capacity"
  ) +
  theme_bw()
```



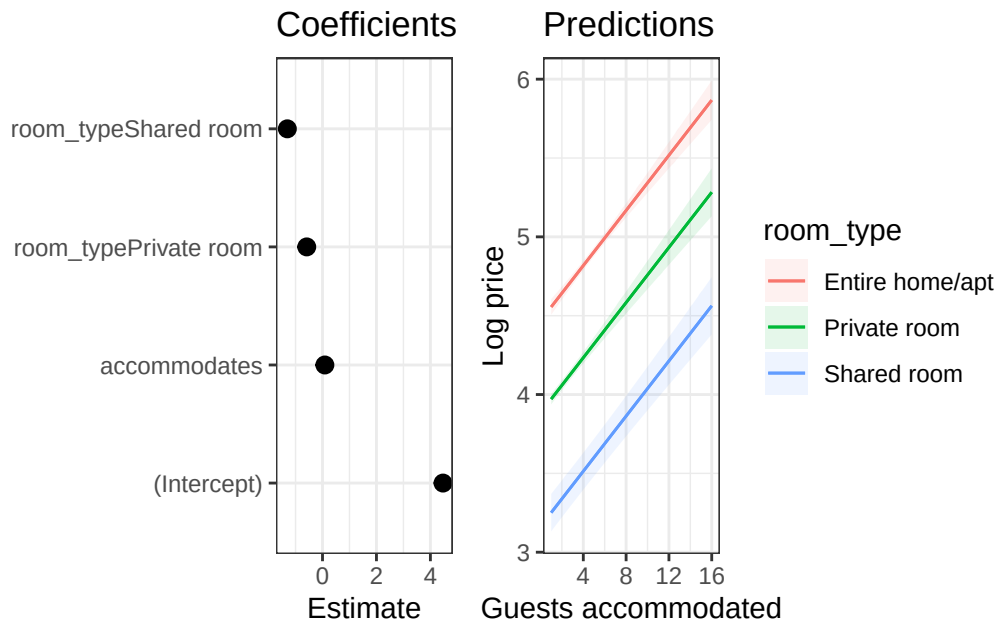
The slopes plot makes the interaction story numerical: the estimated capacity slope is about 0.10 for entire homes and about 0.06 for private rooms. The two intervals are close but the entire-home slope is clearly higher. The same finding could be read out of the interaction-model table, but the plot makes the size, direction, and uncertainty visible in one frame.

This is a model-based comparison. It should be read as an estimate from the fitted model, not as a raw descriptive difference.

11.12 Comparing plots

The `patchwork` package can place related model plots side by side.

```
p_coef <- modelplot(price_model) +  
  labs(title = "Coefficients", x = "Estimate", y = NULL) +  
  theme_bw()  
  
p_pred <- plot_predictions(price_model, condition = c("accommodates", "room_type")) +  
  labs(title = "Predictions", x = "Guests accommodated", y = "Log price") +  
  theme_bw()  
  
p_coef + p_pred
```



The coefficient plot emphasizes estimates and uncertainty. The prediction plot emphasizes substantive interpretation.

12 Mapping

US maps, FIPS codes, choropleths, and optional spatial extensions

Maps are useful when geography is part of the question. They are not automatically better than tables or charts. Use a map when location, region, distance, or spatial clustering matters.

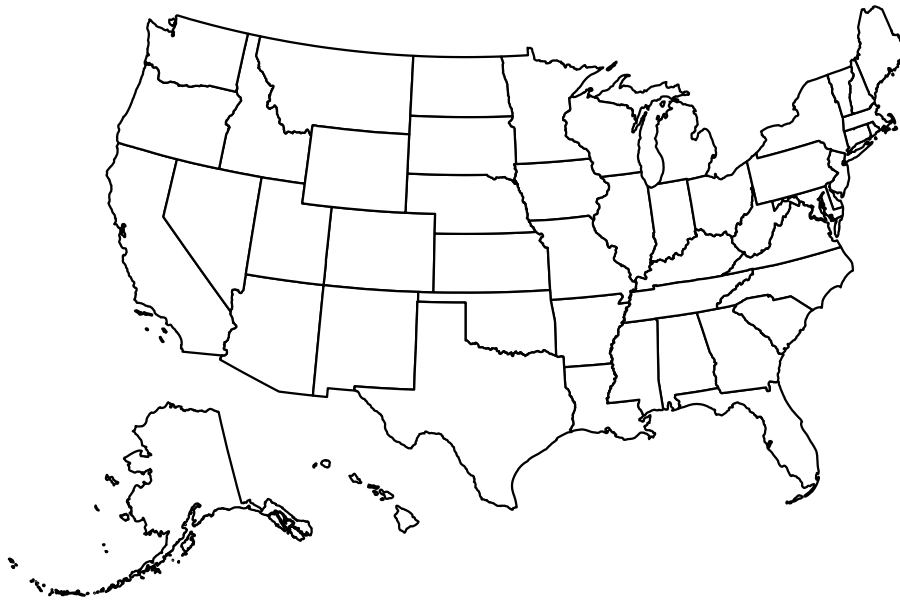
This chapter covers `usmap` for United States maps and `tmap` for world maps. The extras introduce `tidycensus` for Census data and `statebins` for equal-area tile maps. Lower-level spatial tools (`sf`, `rnatrualearth`, `leaflet`) are in the companion experiment file.

```
library(tidyverse)
library(usmap)
library(ggplot2)
```

12.1 A blank US map

`plot_usmap()` creates a map with very little code.

```
plot_usmap()
```

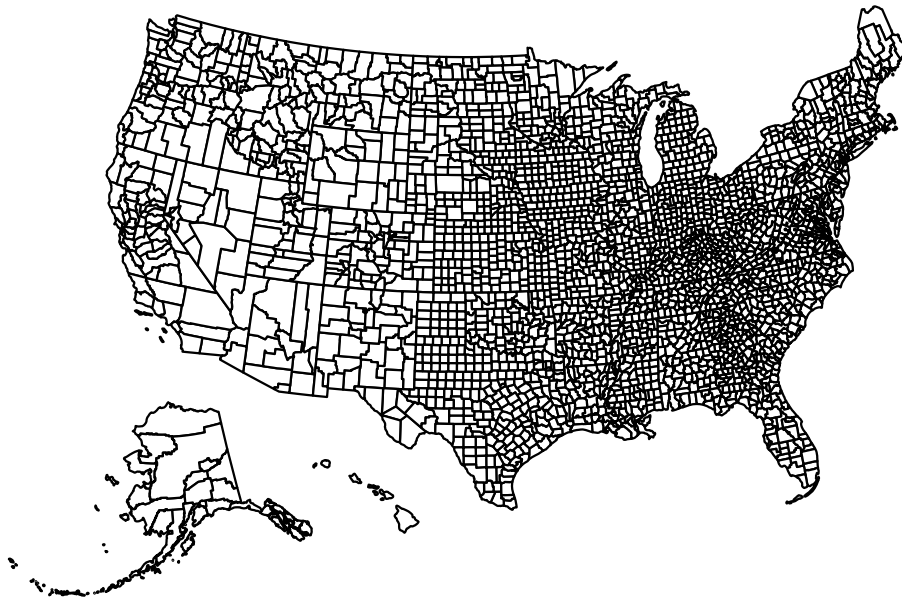



The package handles the map geometry and projection for us. This is a good first tool when the goal is a United States state or county map.

12.2 State and county maps

The default map shows states. County maps use `regions = "counties"`.

```
plot_usmap(regions = "counties")
```



County maps contain many small shapes. They can be useful, but they can also become visually busy. Use county maps when county-level variation is central to the question.

12.3 Understanding map data

Map packages store geometry data in data frames. `us_map()` lets us inspect the data that `usmap` uses.

```
states_df <- usmap::us_map()
counties_df <- usmap::us_map(regions = "counties")

states_df |>
  sf::st_drop_geometry() |>
  slice_sample(n = 10)
```

	fips	abbr	full
1	38	ND	North Dakota
2	12	FL	Florida
3	17	IL	Illinois
4	34	NJ	New Jersey
5	49	UT	Utah

6	09	CT	Connecticut
7	53	WA	Washington
8	48	TX	Texas
9	50	VT	Vermont
10	32	NV	Nevada

```
counties_df |>
  sf::st_drop_geometry() |>
  slice_sample(n = 10)
```

	fips	abbr	full	county
1	01125	AL	Alabama	Tuscaloosa County
2	39155	OH	Ohio	Trumbull County
3	08053	CO	Colorado	Hinsdale County
4	47117	TN	Tennessee	Marshall County
5	01087	AL	Alabama	Macon County
6	21067	KY	Kentucky	Fayette County
7	72061	PR	Puerto Rico	Guaynabo Municipio
8	30053	MT	Montana	Lincoln County
9	55045	WI	Wisconsin	Green County
10	06089	CA	California	Shasta County

The key variables include state names, abbreviations, and FIPS codes.

12.4 FIPS codes

FIPS codes are standardized geographic identifiers. State FIPS codes have two digits. County FIPS codes have five digits: two for the state and three for the county.

```
fips(state = "Massachusetts")
```

```
[1] "25"
```

```
fips_info(c("25", "26", "27"))
```

	abbr	fips	full
1	MA	25	Massachusetts
2	MI	26	Michigan
3	MN	27	Minnesota

County FIPS lookups work the same way.

```
fips_info(c("25025", "26163", "27053"))
```

	full	abbr	county	fips
1	Massachusetts	MA	Suffolk County	25025
2	Michigan	MI	Wayne County	26163
3	Minnesota	MN	Hennepin County	27053

FIPS codes matter because many government and social science datasets use them as join keys. A map will not know that “Massachusetts”, “MA”, and “25” refer to the same place unless you create a common identifier.

12.5 A state choropleth

A choropleth map fills geographic units by a data value. `usmap` includes a state population dataset.

```
data(statepop, package = "usmap")

statepop |>
  select(full, abbr, fips, pop_2022) |>
  slice_head(n = 8)
```

```
# A tibble: 8 x 4
  full      abbr fips pop_2022
  <chr>    <chr> <chr>   <dbl>
1 Alabama  AL     01    5074296
2 Alaska   AK     02     733583
3 Arizona  AZ     04    7359197
4 Arkansas AR     05    3045637
5 California CA     06   39029342
6 Colorado CO     08    5839926
7 Connecticut CT     09    3626205
8 Delaware DE     10    1018396
```

Now map 2022 population.

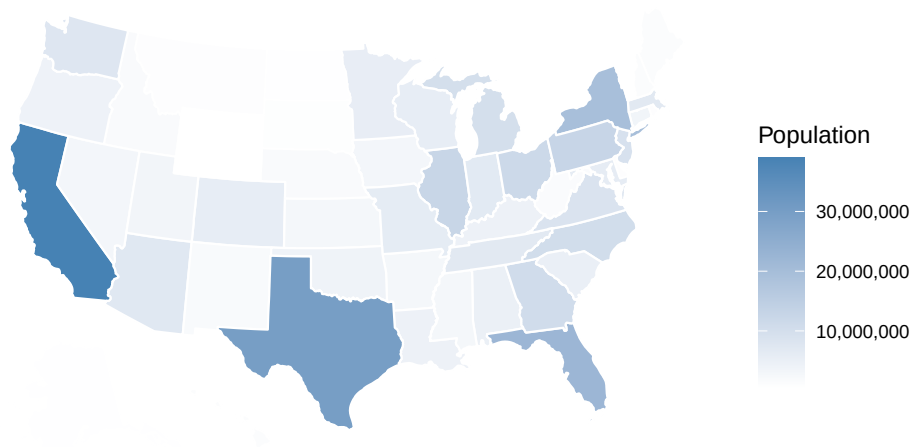
```

plot_usmap(data = statepop, values = "pop_2022", color = "white") +
  scale_fill_continuous(
    low = "white",
    high = "steelblue",
    name = "Population",
    labels = scales::comma
  ) +
  labs(
    title = "US State Populations",
    subtitle = "2022 estimates"
  ) +
  theme(legend.position = "right")

```

US State Populations

2022 estimates



Population maps often emphasize large states. That may be the point, but it may also hide other patterns. Always ask whether the mapped value should be a total, a rate, a percentage, or a change.

12.6 Focused regional maps

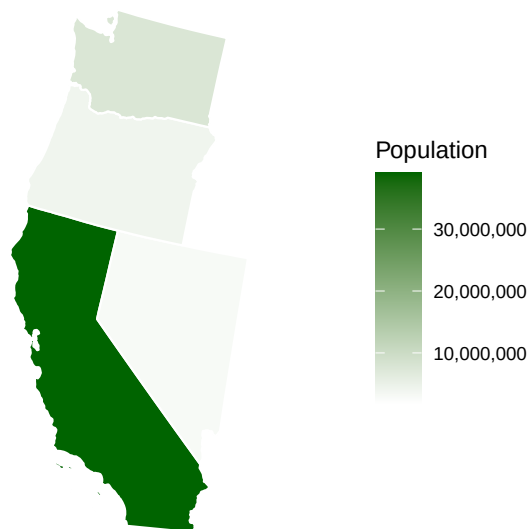
The `include` argument limits the map to selected states.

```

plot_usmap(
  data = statepop,
  values = "pop_2022",
  include = c("CA", "ID", "NV", "OR", "WA"),
  color = "white"
) +
  scale_fill_continuous(
    low = "white",
    high = "darkgreen",
    name = "Population",
    labels = scales::comma
  ) +
  labs(
    title = "Population in Selected Western States",
    subtitle = "2022 estimates"
  ) +
  theme(legend.position = "right")

```

Population in Selected Western States
2022 estimates



usmap also has predefined region objects.

```

plot_usmap(include = .south_region)

```



You can combine inclusion and exclusion.

```
plot_usmap(include = .south_region, exclude = .east_south_central)
```



12.7 County choropleths

usmap includes county poverty data.

```
data(countypov, package = "usmap")

countypov |>
  select(fips, abbr, county, pct_pov_2021) |>
  slice_head(n = 8)
```

```
# A tibble: 8 x 4
  fips  abbr  county      pct_pov_2021
  <chr> <chr> <chr>      <dbl>
1 01000 AL    Alabama      16.3
2 01001 AL    Autauga County  10.7
3 01003 AL    Baldwin County 10.8
4 01005 AL    Barbour County  23
5 01007 AL    Bibb County    20.6
6 01009 AL    Blount County   12
7 01011 AL    Bullock County  32.1
8 01013 AL    Butler County   22.7
```

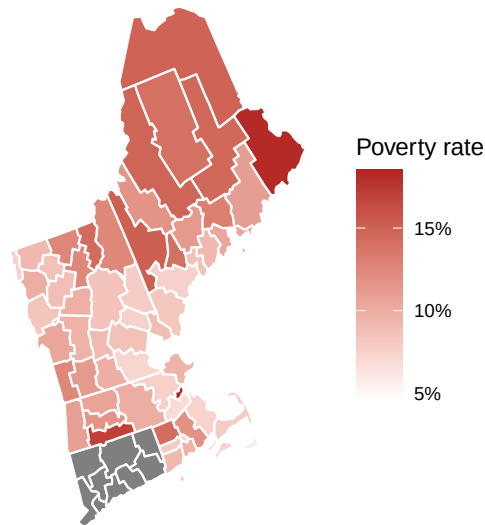
County maps require `regions = "counties"`.

```
plot_usmap(
  data = countypov,
  values = "pct_pov_2021",
  regions = "counties",
  include = c("CT", "ME", "MA", "NH", "RI", "VT"),
  color = "white",
  size = 0.1
) +
  scale_fill_continuous(
    low = "white",
    high = "firebrick",
    name = "Poverty rate",
    labels = scales::label_percent(scale = 1)
  ) +
  labs(
    title = "County Poverty Rates in New England",
    subtitle = "2021 estimates"
```



```
) +  
theme(legend.position = "right")
```

County Poverty Rates in New England
2021 estimates



County maps work best when the color scale and the legend are easy to read. Thin borders and focused regions can help.

12.8 Joining your own data

Most mapping projects follow the same pattern:

1. Get a data frame with a geographic identifier such as a state abbreviation or FIPS code.
2. Clean the identifier so it matches the map data.
3. Join or pass the data to a mapping function.
4. Choose a color scale and labels that match the substance of the variable.

Here is a small state-level example.

```
state_example <- tibble(  
  state = c("CA", "TX", "FL", "NY", "IL", "PA", "OH", "GA"),  
  example_rate = c(12.3, 10.8, 14.1, 11.7, 13.4, 12.8, 13.1, 15.0)  
)
```

```
state_example
```

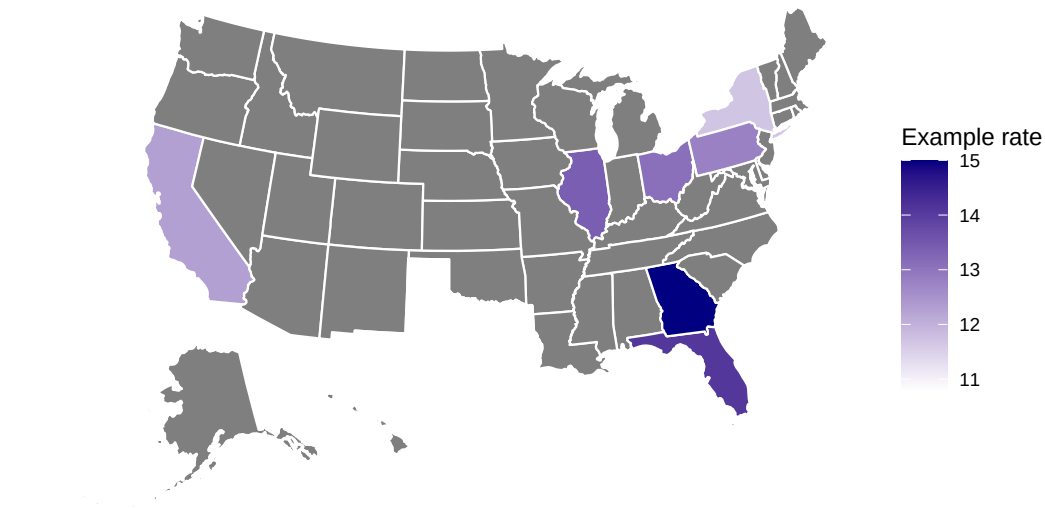
```
# A tibble: 8 x 2
  state example_rate
  <chr>      <dbl>
1 CA         12.3
2 TX         10.8
3 FL         14.1
4 NY         11.7
5 IL         13.4
6 PA         12.8
7 OH         13.1
8 GA         15
```

`plot_usmap()` can use state abbreviations directly when the data have a `state` column.

```
plot_usmap(data = state_example, values = "example_rate", color = "white") +
  scale_fill_continuous(
    low = "white",
    high = "navy",
    name = "Example rate"
  ) +
  labs(
    title = "Example State-Level Data",
    subtitle = "Only states in the example data are filled"
  ) +
  theme(legend.position = "right")
```

Example State-Level Data

Only states in the example data are filled



For county data, make sure FIPS codes remain character strings. Do not let leading zeroes disappear.

```
county_example <- tibble(  
  fips = c("25025", "26163", "17031"),  
  value = c(41, 38, 44)  
)
```

```
county_example
```

```
# A tibble: 3 x 2
```

```
  fips value  
  <chr> <dbl>
```

```
1 25025    41
```

```
2 26163    38
```

```
3 17031    44
```

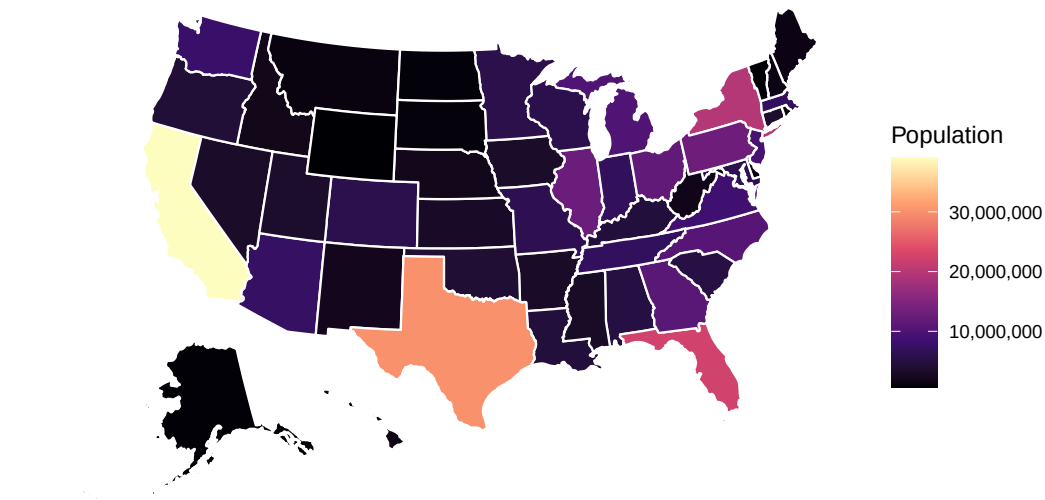
12.9 Choosing map colors

Sequential color scales work well for ordered values such as population, income, or percentages. `viridis` scales are a good default because they are readable and colorblind-friendlier than many rainbow palettes.

```
plot_usmap(data = statepop, values = "pop_2022", color = "white") +
  scale_fill_viridis_c(
    option = "magma",
    name = "Population",
    labels = scales::comma
  ) +
  labs(
    title = "US State Populations",
    subtitle = "A viridis sequential color scale"
  ) +
  theme(legend.position = "right")
```

US State Populations

A viridis sequential color scale



Use diverging color scales when the variable has a meaningful midpoint, such as change around zero or an index centered at an average.

12.10 Short exercise

Use `statepop`.

1. Make a state population map with a different sequential color scale.
2. Focus the map on one region or a custom set of states.
3. Change the title, subtitle, and legend title.

4. Write one sentence explaining whether a map is better than a sorted bar chart for this variable.

12.11 Extra: ACS data with tidycensus

The `tidycensus` package can download American Community Survey data. This is optional because it may require a Census API key and internet access.

```
library(tidycensus)

county_income <- get_acs(
  geography = "county",
  variables = "B19013_001",
  year = 2022,
  geometry = FALSE
) |>
  transmute(
    fips = GEOID,
    income = estimate
  )

plot_usmap(
  data = county_income,
  values = "income",
  regions = "counties",
  include = .midwest_region,
  color = "white",
  size = 0.1
) +
  scale_fill_viridis_c(
    name = "Median household income",
    labels = scales::dollar
  ) +
  labs(
    title = "Median Household Income by County",
    subtitle = "ACS estimates"
  ) +
  theme(legend.position = "right")
```

This workflow is useful for public policy, sociology, political science, and demography projects because Census data use standard geographic identifiers.

12.12 Extra: equal-area state tile maps with statebins

Traditional maps give more visual space to geographically large states. A tile map gives every state the same visual size.

```
library(statebins)

USArrests |>
  rownames_to_column("state") |>
  statebins(value_col = "Assault", name = "Assault", round = TRUE) +
  theme_statebins(legend_position = "right") +
  labs(title = "Assault Arrests by State")
```

State bins are not geographically precise, but they are useful when equal visual weight is more important than exact shape.

12.13 Extra: world maps with tmap

The **tmap** package is built specifically for thematic maps. Its grammar resembles **ggplot2** — you start with a shape layer and add styled layers on top — but the defaults are tuned for cartography: sensible projections, automatic legends, and built-in support for switching between static and interactive maps with a single line.

tmap ships with a **World** dataset (an **sf** object covering 177 countries with indicators like population, life expectancy, and GDP per capita). That makes it a good entry point for country-level visualization without separately loading natural-earth data.

```
library(tmap)
library(sf)

data(World)

tmap_mode("plot") # static maps

tm_shape(World) +
  tm_polygons(fill = "life_exp", fill.scale = tm_scale_continuous(values = "viridis")) +
  tm_title("Life Expectancy by Country") +
  tm_layout(legend.outside = TRUE)
```

`tm_shape()` declares the geometry. `tm_polygons()` fills the countries by a variable. `tm_scale_continuous()` controls the color ramp. The structure parallels `ggplot() + geom_sf() + scale_fill_*`, but the components are map-specific.

A categorical fill works the same way.

```
tm_shape(World) +
  tm_polygons(
    fill = "continent",
    fill.scale = tm_scale_categorical(values = "brewer.set3")
  ) +
  tm_title("Countries by Continent") +
  tm_layout(legend.outside = TRUE)
```

World maps benefit from a thoughtful projection. The default Mercator-style projection inflates high latitudes; Equal Earth (EPSG:8857) preserves area, which is usually what you want for global comparisons.

```
World_ea <- st_transform(World, crs = "+proj=eqearth")

tm_shape(World_ea) +
  tm_polygons(
    fill = "gdp_cap_est",
    fill.scale = tm_scale_continuous(values = "plasma"),
    fill.legend = tm_legend(title = "GDP per capita")
  ) +
  tm_title("GDP Per Capita on an Equal-Earth Projection") +
  tm_layout(legend.outside = TRUE)
```

`tmap` can also switch into interactive mode with one line. The same map code is reused; the renderer changes.

```
tmap_mode("view") # interactive

tm_shape(World) +
  tm_polygons(
    fill = "life_exp",
    fill.scale = tm_scale_continuous(values = "viridis"),
    popup.vars = c("Country" = "name", "Life expectancy" = "life_exp", "GDP/capita" = "gdp_c")
  )

tmap_mode("plot") # switch back
```

That dual-mode behavior is the main reason to use `tmap` over plain `ggplot2 + geom_sf` for world maps. The same syntax produces a static figure for a paper and an interactive map for a webpage. You can also join your own country-level data to `World` by `iso_a3` (a three-letter country code) and map any variable that way.

12.13.1 Zooming into a region

To show a specific continent, filter the `World` `sf` object before passing it to `tm_shape()`. This removes all other countries from the frame entirely.

```
# Russia spans the antimeridian (full globe width) and France includes French
# Guiana (far southwest). Excluding Russia and cropping to the European extent
# keeps the map focused on the European landmass.
europe <- World[World$continent == "Europe" & World$name != "Russia", ] |>
  sf::st_crop(xmin = -25, ymin = 34, xmax = 45, ymax = 72)

tm_shape(europe) +
  tm_polygons(
    fill = "gdp_cap_est",
    fill.scale = tm_scale_continuous(values = "plasma"),
    fill.legend = tm_legend(title = "GDP per capita")
  ) +
  tm_text("name", size = 0.45) +
  tm_crs("auto") +
  tm_title("GDP Per Capita in Europe") +
  tm_layout(legend.outside = TRUE)
```

`tm_crs("auto")` lets `tmap` choose a suitable projection for the region — for Europe it typically selects something like ETRS89-LAEA, which minimises distortion at European latitudes. `tm_text()` adds country name labels; keep `size` small for regions with many small countries.

The same pattern works for any continent. Replace "Europe" with "Africa", "Asia", "South America", or "North America".

```
africa <- World[World$continent == "Africa", ]

tm_shape(africa) +
  tm_polygons(
    fill = "well_being",
    fill.scale = tm_scale_continuous(values = "brewer.yl_or_rd"),
    fill.legend = tm_legend(title = "Well-being index")
  ) +
```



```
tm_crs("auto") +  
tm_title("Well-Being Index in Africa") +  
tm_layout(legend.outside = TRUE)
```

13 Agentic AI and Next Steps

Debugging, critique, revision, reproducibility, and continued learning

Large language models can help with coding, data analysis, visualization, writing, and debugging. They can also produce confident mistakes. The goal is not to replace your judgment. The goal is to use AI as an assistant while keeping you responsible for the analysis.

This chapter focuses on practical workflows: how to ask for help, how to review answers, how to protect privacy, and how to use agentic tools without losing control of your project.

13.1 What AI can help with

AI tools are most useful when the task has enough structure for you to check the answer.

Common visualization uses include:

- explaining unfamiliar R code
- finding the source of an error message
- suggesting a cleaner `ggplot2` pipeline
- proposing labels, captions, or alternative chart types
- converting a rough plot into a more polished plot
- reviewing a table or figure for readability
- writing short helper functions
- drafting Quarto prose around code output
- checking whether a project depends on files that are not included

AI is less reliable when the task requires facts that may have changed, private data it cannot see, or judgment without enough context.

13.2 Web chatbots

Web chatbots are the most familiar AI interface. You paste a question, code, data description, or error message into a browser and get a response.

This works well for short, self-contained questions.

Example prompt:

```
I am using R and ggplot2. This code produces a plot, but the legend title is unclear.
Please explain what each line does and suggest a more readable version.
```

```
<paste code here>
```

For debugging, include the error message and the smallest code example that reproduces the problem.

```
I get this error in R:
```

```
<paste exact error>
```

```
Here is the code that triggered it:
```

```
<paste code>
```

```
Please identify the likely cause. Do not rewrite the whole script unless needed.
```

The main limitation of web chat is context. If the model cannot see your full project, it may guess incorrectly about data names, packages, or earlier steps.

13.3 IDE integration

Some AI tools work inside the coding environment. GitHub Copilot is one common example. In RStudio, Copilot can suggest completions while you write code. It can be useful for repetitive syntax, common transformations, and quick examples.

IDE integration has two advantages:

- It can use nearby code as context.
- It reduces copying and pasting between the editor and a browser.

It also has risks:

- Suggested code may look plausible but use the wrong variable.
- It may silently introduce a package you did not plan to use.
- It may choose a more complicated approach than needed.

Treat inline suggestions as drafts. Read them before accepting them, and run the code in small pieces.

13.4 RStudio-focused AI with gander

The **gander** package is an RStudio-oriented way to send selected code and surrounding context to an LLM. It can be configured to use different model providers through packages such as **ellmer**.

The exact setup depends on the model provider and your local environment, so the following examples are not executed.

```
options(.gander_chat = ellmer::chat_openai(model = "gpt-4.1-mini"))
```

```
options(.gander_chat = ellmer::chat_google_gemini(model = "gemini-2.5-flash"))
```

The important workflow is:

1. Select the code you want help with.
2. Ask a focused question.
3. Review the response before applying it.
4. Run the revised code.
5. Compare the output to what you expected.

Useful requests include:

```
Explain this pipeline one step at a time.
```

```
Suggest a clearer version of this plot, but keep the same data and variables.
```

```
Find likely bugs in this code. Do not change the statistical model.
```

13.5 Programmatic AI with `ellmer`

`ellmer` gives R users a more programmatic interface to language models. Instead of using a web page manually, you can make model calls from R code.

This can be useful for research workflows such as:

- summarizing text fields
- classifying open-ended survey responses
- generating draft labels for qualitative coding
- extracting structured information from documents
- comparing the behavior of multiple prompts

Programmatic AI needs extra care. Save prompts, model names, settings, dates, and outputs when they are part of the research process. If an LLM contributes to an analysis, the workflow should be documented like any other computational step.

```
library(ellmer)

chat <- chat_openai(model = "gpt-4.1-mini")

chat$chat(
  "In one sentence, explain why direct labels can be easier to read than legends."
)
```

Do not send confidential, identifiable, or restricted data to a cloud model unless your data agreement and institutional rules allow it.

13.6 Local and cloud models

Cloud models run on servers provided by companies or institutions. Local models run on your own computer or a controlled server.

Cloud models often provide stronger performance and easier setup. They can be accessed through web interfaces, IDE tools, or APIs.

Local models can improve privacy and reproducibility because your data do not need to leave your machine and you can pin a specific model version. Tools such as Ollama make local models easier to run, but performance depends on hardware. Larger models may require a powerful GPU or a recent machine with enough memory.

The practical choice depends on the project:

- Use cloud models for general learning, public examples, and low-sensitivity code help.

- Consider local models for sensitive data, restricted research material, or workflows where you need control over the exact model.
- Avoid sending private data, unpublished research data, student records, or confidential text to a public chatbot.

13.7 Agentic coding tools

Agentic tools go beyond chat. They can inspect files, run commands, edit code, run tests, and iterate. Examples include terminal or codebase agents such as Claude Code, Codex, Gemini CLI, and similar tools.

The difference from a chatbot is scope. A chatbot usually answers from pasted context. An agentic tool can work inside a project.

This makes agents useful for:

- finding where a function or variable is defined
- updating several related files
- running checks after edits
- comparing source and rendered output
- writing tests or validation scripts
- reviewing a project for missing files or missing packages

The same power creates risk. An agent can make broad changes quickly. Use boundaries.

Good instructions for an agent are explicit:

```
Edit only 07-lines-bars-and-annotation.qmd.
Do not modify data files or setup files.
Use files from the course project, especially files in Data/.
After editing, report exactly which files changed.
```

For code review:

```
Review this Quarto chapter for runnable R code, missing files, missing packages, and unclear
Do not edit files. List findings by severity with file and line references.
```

For debugging:

```
The render fails in 12-mapping.qmd.
Find the smallest change that fixes the render.
Do not change other chapters.
Explain the cause and the verification command.
```

13.8 A safe AI workflow

Use AI in small loops.

1. State the task and constraints.
2. Provide only the context needed.
3. Ask for a small change or diagnosis.
4. Inspect the answer before applying it.
5. Run the code.
6. Compare the output to the goal.
7. Commit or save only after verification.

This loop is slower than accepting the first suggestion, but it prevents many common errors.

13.9 How to ask better questions

A strong AI request includes the goal, context, constraints, and desired output.

Weak request:

```
Make this plot better.
```

Stronger request:

```
This plot compares life expectancy by continent in 2007 using Gapminder.  
Suggest improvements that make the comparison clearer for a general audience.  
Keep the data and chart type the same.  
Return revised ggplot2 code and explain the changes briefly.
```

Weak request:

```
Fix my code.
```

Stronger request:

```
This Quarto chunk should read Data/gapminder/gapminder.csv and make a scatterplot.  
It fails with the error below.  
Identify the likely cause and propose the smallest code change.  
Do not introduce new packages.  
  
<paste error>  
<paste code>
```

13.10 Reviewing AI output

Before using AI-generated code, check:

- Does it use variables that exist?
- Does it use packages that are loaded or installed?
- Does it preserve the original research question?
- Does it change the model, filter, or grouping without saying so?
- Does it depend on files that are not included with the project?
- Does it silently drop missing data?
- Does the plot or table support the intended interpretation?
- Can you explain every line?

If you cannot explain the code, ask the model to explain it or simplify it before using it.

13.11 Privacy, attribution, and reproducibility

AI use is part of the research workflow. Treat it like other research assistance.

Protect privacy:

- Do not paste sensitive data into public tools.
- Remove names, IDs, free-text responses, and confidential details unless sharing is allowed.
- Check institutional, funder, journal, and IRB rules.

Support reproducibility:

- Save important prompts when they affect analysis decisions.
- Record model names when AI output is part of a method.
- Keep AI-generated code in scripts or Quarto documents where it can be reviewed.
- Prefer deterministic code over one-off manual AI interactions for repeated tasks.

Use attribution when appropriate. If AI contributed materially to text, code, or analysis, follow the disclosure rules for the course, journal, employer, or institution.

13.12 AI for visualization critique

AI can be useful as a first-pass critic of a figure. Ask it to evaluate the plot against specific criteria rather than asking whether it is “good.”

Review this visualization for a social science audience.

Focus on:

1. whether the chart type matches the comparison,
2. whether labels and legends are clear,
3. whether the color scale is appropriate,
4. whether the caption supports the main claim.

Do not invent facts beyond what is visible in the plot.

When possible, provide the code and the rendered figure. The code shows how the chart was made. The image shows what the reader sees.

13.13 AI for debugging Quarto projects

Quarto errors often come from one of a few causes:

- a missing package
- a data file that does not exist
- a chunk that depends on an object created in a different chapter
- a typo in a variable name
- output written to a missing folder
- internet-dependent code running during render
- YAML indentation errors

A useful AI debugging prompt is specific:

This Quarto render fails.

Here is the error message and the chunk before it.

Please identify the most likely cause and propose one minimal fix.

If the problem is a missing data file, suggest how to check it in R.

Then verify the answer in R:

```
file.exists("Data/cigarettes/cigarettes.csv")
```

```
renv::status()
```

```
quarto::quarto_render("12-mapping.qmd")
```

13.14 Next steps after the course

The course has covered a full workflow:

- import data
- tidy and transform data
- build plots with `ggplot2`
- separate and compare groups
- choose chart types
- annotate and highlight
- create tables
- visualize model output
- make maps
- use interactivity and Quarto output formats
- use AI assistance critically

The next step is to apply the workflow to one project you care about.

Choose a dataset and make a small portfolio:

1. One exploratory table.
2. One polished static plot.
3. One model visualization or map, if appropriate.
4. One short Quarto report explaining the question, data, code, and result.

Keep the project portable. Keep data inside the project, put code in scripts or Quarto files, and render the report from a clean session.

13.15 Short exercise

Choose one plot or table from an earlier chapter.

1. Write a prompt asking an AI tool to critique it.
2. Include at least three constraints in the prompt.
3. Ask for suggestions, not a full rewrite.
4. Decide which suggestions you would accept, reject, or test.
5. Write two sentences explaining your decision.

13.16 Final checklist

Before sharing a visualization project, check:

- The project renders from a clean session.
- Data files are included with the project.
- Required packages are listed.
- Data sources are documented.
- Tables and figures have clear titles, labels, and captions.
- Optional internet/API steps are marked clearly.
- AI-assisted code has been reviewed and tested.
- The final report explains both what you found and how you produced it.

The purpose of the workflow is not just to make a figure. It is to make a figure that another person can understand, evaluate, and reproduce.